THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms. **UDHAYAVANI M(20524UCSC050)** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO: 01 Creation of DDL Operations Using Book Store Table

DATE: _____

Aim

To create a book store table, insert book records, display all records, and retrieve books whose price is 500 and above.

Software Used

Oracle / MySQL / SQLite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects in a database.

DDL COMMANDS

- Create
- Alter
- Drop
- Truncate
- Rename

CREATE

The CREATE command is used to create a new table and define its columns and constraints.

Syntax

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
CREATE TABLE books (
   book_id INTEGER PRIMARY KEY,
   book_name TEXT,
   price INTEGER
```

);

Table Created

ALTER

The ALTER command is used to alter a table or make changes to an existing table. It can be used to add, modify, or drop columns.

ADD:

The ADD command is used to add a new attribute/column with a data type.

Syntax:

ALTER TABLE table_name ADD column_name datatype;

Query:

ALTER TABLE books ADD author VARCHAR(100);

Table Altered

MODIFY:

The MODIFY command is used to change the data type or size of an existing column.

Syntax:

ALTER TABLE table_name MODIFY column_name datatype;

Query:

ALTER TABLE books MODIFY book_name VARCHAR(150);

For MySQL:

ALTER TABLE books MODIFY COLUMN book_name VARCHAR(150);**DROP COLUMN**

The DROP COLUMN command is used to delete an attribute/column from a table.

Syntax

ALTER TABLE table_name DROP COLUMN column_name;

Query

ALTER TABLE books DROP COLUMN author;

RENAME

The RENAME command is used to change the name of a table.

Query

RENAME TABLE books TO book_store;

Table Renamed

Alternatively:

ALTER TABLE books RENAME TO book_store;

Column Rename

ALTER TABLE books RENAME COLUMN book_name TO title;

TRUNCATE:The TRUNCATE command is used to delete all records from a table while keeping the table structure.

Syntax

TRUNCATE TABLE table_name;

Query

TRUNCATE TABLE books;

DROP

The DROP command is used to delete the table and all its data permanently.

Syntax

DROP TABLE table_name;

Query

DROP TABLE books;

Explanation

- Create a table named books.
- Insert book details into the table.
- Display all book records.
- Retrieve books whose price is 500 and above.
- Sort the books according to their price.

Create Table

```
CREATE TABLE books (
    book_id INTEGER PRIMARY KEY,
    book_name TEXT,
    price INTEGER
);
```

Sample Data

```
INSERT INTO books VALUES (1, 'Database Systems', 450);

INSERT INTO books VALUES (2, 'Python Programming', 650);

INSERT INTO books VALUES (3, 'Java Basics', 550);

INSERT INTO books VALUES (4, 'Web Development', 750);

INSERT INTO books VALUES (5, 'Computer Networks', 400);

INSERT INTO books VALUES (6, 'Operating Systems', 600);
```

SQL Query

Query 1: Display All Book Records

```
SELECT * FROM books;
```

Resultant Query

book_id	book_name	price
1	Database Systems	450
2	Python Programming	650
3	Java Basics	550
4	Web Development	750
5	Computer Networks	400
6	Operating Systems	600

Query 2: Display Books Whose Price is 500 and Above

```
SELECT *
FROM books
WHERE price >= 500
ORDER BY price;
```

Resultant Query

book_id	book_name	price
3	Java Basics	550
6	Operating Systems	600
2	Python Programming	650
4	Web Development	750

RESULT

Thus, the books table was created successfully, book records were inserted, all book details were displayed, and the books whose price was 500 and above were retrieved successfully.

EX. NO: 02 Creation of DML Operations Using Employee Table

DATE: _____

Aim

To create an **Employee** table, insert employee records, display all records, update employee details, delete an employee record, and retrieve employees based on their salary.

Software Used

Oracle / MySQL / SQLite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query

SQL> INSERT INTO employees VALUES (1, 'Arun', 35000);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

Query

SQL> UPDATE employees

SET salary = 40000

WHERE id = 1;

1 row updated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM employees

WHERE id = 5;

1 row deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM employees;

Explanation

- Create a table named **employees**.
- Insert employee details into the table.
- Display all employee records.
- Update the salary of an employee.
- Delete an employee record.
- Retrieve employees whose salary is **40000 or above** and sort them by salary.

Create Table

```
CREATE TABLE employees (
   id INTEGER PRIMARY KEY,
   name TEXT,
   salary INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO employees VALUES (1, 'Arun', 35000);

INSERT INTO employees VALUES (2, 'Priya', 42000);

INSERT INTO employees VALUES (3, 'Kavin', 38000);

INSERT INTO employees VALUES (4, 'Divya', 30000);

INSERT INTO employees VALUES (5, 'Rahul', 45000);

INSERT INTO employees VALUES (6, 'Meena', 50000);
```

Records Inserted

SQL Query

Query 1: Display all employee records

```
SQL> SELECT * FROM employees;
```

Resultant Query

id	name	salary
1	Arun	35000
2	Priya	42000
3	Kavin	38000
4	Divya	30000
5	Rahul	45000
6	Meena	50000

Query 2: Update the salary of Arun

SQL> UPDATE employees

SET salary = 40000

WHERE id = 1;

1 row updated.

Updated Record

id name salary

1 Arun 40000

Query 3: Delete Rahul's record

SQL> DELETE FROM employees

WHERE id = 5;

1 row deleted.

Remaining Records

id	name	salary
1	Arun	40000
2	Priya	42000
3	Kavin	38000
4	Divya	30000
6	Meena	50000

Query 4: Display employees whose salary is 40000 or above

```
SQL> SELECT *
FROM employees
WHERE salary >= 40000
ORDER BY salary;
```

Resultant Query

id	name	salary
1	Arun	40000
2	Priya	42000
6	Meena	50000

RESULT

Thus, the **Employees** table was created successfully, records were inserted, employee details were displayed, the employee salary was updated, an employee record was deleted, and the employees whose salary was **40000 or above** were retrieved successfully.

EX. NO:03 Creation of DQL Operations using Employees Table

DATE:

Aim

To create an Employees table, insert employee records, display all records, update employee details, delete an employee record, and retrieve employees based on their salary.

Software Used

Oracle / MySQL / sqlite3

Description

DQL (Data Query Language) is used to retrieve and display data stored in database tables. DQL mainly uses the SELECT command to query data from tables.

DQL COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query

SQL> INSERT INTO employees VALUES (1, 'Arun', 35000);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

Query

SQL> UPDATE employees

SET salary = 40000

WHERE id = 1;

1 row updated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM employees

WHERE id = 5;

1 row deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM employees;

Explanation

- Create a table named employees.
- Insert employee details into the table.
- Display all employee records.
- Update the salary of an employee.
- Delete an employee record.
- Retrieve employees whose salary is 40000 or above and sort them by salary.

Create Table

CREATE TABLE employees (

id INTEGER PRIMARY KEY,

```
    name TEXT,
    salary INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO employees VALUES (1, 'Arun', 35000);

INSERT INTO employees VALUES (2, 'Priya', 42000);

INSERT INTO employees VALUES (3, 'Kavin', 38000);

INSERT INTO employees VALUES (4, 'Divya', 30000);

INSERT INTO employees VALUES (5, 'Rahul', 45000);

INSERT INTO employees VALUES (6, 'Meena', 50000);
```

Records Inserted

SQL Query

Query 1: Display all employee records

```
SQL> SELECT * FROM employees;
```

Resultant Query`

id	name	salary
1	Arun	35000
2	Priya	42000
3	Kavin	38000
4	Divya	30000
5	Rahul	45000
6	Meena	50000

Query 2: Update the salary of Arun

```
SQL> UPDATE employees
SET salary = 40000
```

WHERE id = 1;

1 row updated.

Updated Record

id	name	salary
1	Arun	40000

Query 3: Delete Rahul's record

SQL> DELETE FROM employees

WHERE id = 5;

1 row deleted.

Remaining Records

id	name	salary
1	Arun	40000
2	Priya	42000
3	Kavin	38000
4	Divya	30000
6	Meena	50000

Query 4: Display employees whose salary is 40000 or above

SQL> SELECT *

FROM employees

WHERE salary >= 40000

ORDER BY salary;

Resultant Query

id	name	salary
1	Arun	40000
2	Priya	42000
6	Meena	50000

RESULT:

Thus, the Employees table was created successfully, records were inserted, employee details were displayed, the employee salary was updated, an employee record was deleted, and the employees who earned 40000 or above were retrieved successfully.

EX. NO:04 Creation of Transaction Operations using Library Books Table

DATE:

Aim

To create a Library Books table, insert book records, perform transaction operations using COMMIT, ROLLBACK, and SAVEPOINT, and verify the changes made to the database.

Software Used

Oracle / MySQL / sqlite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database. It helps to save or undo the changes made to the database.

TRANSACTION COMMANDS:

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT

COMMIT command is used to permanently save all the changes made during a transaction.

Syntax

COMMIT;

Query

SQL> COMMIT;

Transaction committed.

ROLLBACK

ROLLBACK command is used to undo the changes made during a transaction before the changes are committed.

Syntax

ROLLBACK;

Query

SQL> ROLLBACK;

Transaction rolled back.

SAVEPOINT

SAVEPOINT command is used to create a temporary point within a transaction. The transaction can be rolled back to that specific point.

Syntax

SAVEPOINT savepoint_name;

Query

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to undo the changes made after a particular savepoint without undoing the entire transaction.

Syntax

ROLLBACK TO SAVEPOINT savepoint_name;

Query

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

Explanation

- Create a table named library_books.
- Insert book details into the table.
- Use COMMIT to permanently save the inserted records.
- Update a book price and create a SAVEPOINT.
- Make another change after the savepoint.
- Use ROLLBACK TO SAVEPOINT to undo the changes made after the savepoint.
- Use ROLLBACK to undo uncommitted changes.

Create Table

```
CREATE TABLE library_books (
   book_id INTEGER PRIMARY KEY,
   book_name TEXT,
   price INTEGER
);
```

Table Created

Sample Data

INSERT INTO library_books VALUES (201, 'Database Systems', 500);

INSERT INTO library_books VALUES (202, 'Python Programming', 650);

INSERT INTO library_books VALUES (203, 'Java Programming', 550);

INSERT INTO library_books VALUES (204, 'Web Development', 700);

INSERT INTO library_books VALUES (205, 'Computer Networks', 600);

Records Inserted

SQL Query

Query 1: Display all library book records

SQL> SELECT * FROM library_books;

Resultant Query

book_id	book_name	price
201	Database Systems	500
202	Python Programming	650
203	Java Programming	550
204	Web Development	700
205	Computer Networks	600

Query 2: Commit the inserted records

SQL> COMMIT;

Transaction committed.

The inserted library book records are now permanently saved.

Query 3: Update Database Systems book price

SQL> UPDATE library_books

SET price = 550

WHERE book_id = 201;

1 row updated.

Updated Record

book_id	book_name	price
201	Database Systems	550

Query 4: Create a SAVEPOINT

SQL> SAVEPOINT sp1;

Savepoint created.

Query 5: Update Python Programming book price

SQL> UPDATE library_books

SET price = 700

WHERE book_id = 202;

1 row updated.

Updated Record

book_id	book_name	price
202	Python Programming	700

Query 6: Rollback to SAVEPOINT

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

The changes made after SAVEPOINT sp1 are undone.

Resultant Query

book_id	book_name	price
201	Database Systems	550
202	Python Programming	650

Query 7: Rollback an uncommitted change

SQL> UPDATE library_books

SET price = 450

WHERE book_id = 203;

1 row updated.

SQL> ROLLBACK;

Transaction rolled back.

The price of Java Programming is restored to its previous value.

Resultant Query

book_id	book_name	price
203	Java Programming	550

RESULT:

Thus, the Library Books table was created successfully, book records were inserted, and the transaction operations COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT were performed successfully.

EX. NO:05 Creation of Join Operations Using Students and Courses Table

DATE: _____

AIM

To create Students and Courses tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used

Oracle / MySQL / sqlite3

Description

Joins are used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

• Inner Join

• Left Join

• Right Join

• Full Join

• Self Join

INNER JOIN

Inner Join returns only the records that have matching values in both tables.

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

LEFT JOIN

Left Join returns all records from the left table and the matching records from the right table.

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

RIGHT JOIN

Right Join returns all records from the right table and the matching records from the left table.

Syntax:

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.column = table2.column;

FULL JOIN

Full Join returns all records from both tables. It includes matching and non-matching records.

Syntax:

SELECT columns

FROM table1

FULL JOIN table2

ON table1.column = table2.column;

SELF JOIN

Self Join is used to join a table with itself. It is useful when records in the same table are related to each other.

Syntax:

SELECT A.column, B.column

FROM table_name A

JOIN table_name B

ON A.column = B.column;

Explanation

• Create Students and Courses tables.

• Insert student and course details into the tables.

• Display matching student and course records using Inner Join.

• Display all students and their matching courses using Left Join.

• Display all courses and their matching students using Right Join.

• Display all matching and non-matching records using Full Join.

• Use Self Join to display students and their mentors.

Create Tables

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT,
    department TEXT
);

CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_name TEXT,
    marks INTEGER
);
```

Tables Created

Sample Data

```
INSERT INTO students VALUES (1, 'Kavin', 'Computer Science');
INSERT INTO students VALUES (2, 'Meena', 'Computer Science');
INSERT INTO students VALUES (3, 'Arun', 'Information Technology');
INSERT INTO students VALUES (4, 'Divya', 'Computer Science');
INSERT INTO students VALUES (5, 'Priya', 'Information Technology');

INSERT INTO courses VALUES (101, 1, 'Database Management', 85);
INSERT INTO courses VALUES (102, 2, 'Python Programming', 78);
INSERT INTO courses VALUES (103, 1, 'Web Development', 90);
INSERT INTO courses VALUES (104, 3, 'Computer Networks', 75);
INSERT INTO courses VALUES (105, 6, 'Java Programming', 80);
```

Records Inserted

SQL Query

Query 1: Inner Join

```
SQL> SELECT students.student_name, courses.course_name, courses.marks
FROM students
INNER JOIN courses
ON students.student_id = courses.student_id;
```

Resultant Query

student_name	course_name	marks
Kavin	Database Management	85
Meena	Python Programming	78
Kavin	Web Development	90
Arun	Computer Networks	75

Query 2: Left Join

```
SQL> SELECT students.student_name, courses.course_name, courses.marks
FROM students
LEFT JOIN courses
ON students.student_id = courses.student_id;
```

Resultant Query

student_name	course_name	marks
Kavin	Database Management	85
Meena	Python Programming	78
Arun	Computer Networks	75
Divya	NULL	NULL
Priya	NULL	NULL
Kavin	Web Development	90

Query 3: Right Join

```
SQL> SELECT students.student_name, courses.course_name, courses.marks
FROM students
RIGHT JOIN courses
ON students.student_id = courses.student_id;
```

Resultant Query

student_name	course_name	marks
Kavin	Database Management	85
Meena	Python Programming	78
Kavin	Web Development	90
Arun	Computer Networks	75
NULL	Java Programming	80

Query 4: Full Join

```
SQL> SELECT students.student_name, courses.course_name, courses.marks
FROM students
FULL JOIN courses
ON students.student_id = courses.student_id;
```

Resultant Query

student_name	course_name	marks
Kavin	Database Management	85
Meena	Python Programming	78
Kavin	Web Development	90
Arun	Computer Networks	75
Divya	NULL	NULL
Priya	NULL	NULL
NULL	Java Programming	80

Query 5: Self Join

Create Student Table

```
CREATE TABLE student_mentor (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT,
    mentor_id INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO student_mentor VALUES (1, 'Kavin', NULL);
INSERT INTO student_mentor VALUES (2, 'Meena', 1);
```

INSERT INTO student_mentor VALUES (3, 'Arun', 1);

INSERT INTO student_mentor VALUES (4, 'Divya', 2);

INSERT INTO student_mentor VALUES (5, 'Priya', 2);

Records Inserted

SQL> SELECT A.student_name AS Student,

B.student_name AS Mentor

FROM student_mentor A

LEFT JOIN student_mentor B

ON A.mentor_id = B.student_id;

Resultant Query

Student	Mentor
Kavin	NULL
Meena	Kavin
Arun	Kavin
Divya	Meena
Priya	Meena

RESULT:

Thus, the Students and Courses tables were created successfully, and the required records were retrieved successfully using Inner Join, Left Join, Right Join, Full Join, and Self Join operations.

EX. NO:06 Creation of SQL Functions Using Employee Table

DATE: _____

Aim

To create an Employees table, insert employee records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Syntax

SELECT aggregate_function(column_name)

FROM table_name;

STRING FUNCTIONS

String functions are used to perform operations on text or character values.

Common string functions are:

- UPPER()

- LOWER()
- LENGTH()

Syntax

SELECT string_function(column_name)

FROM table_name;

DATE FUNCTIONS

Date functions are used to perform operations on date values.

Common date functions are:

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

SELECT date_function(date_column)

FROM table_name;

Explanation

- Create a table named employees.
- Insert employee details into the table.
- Display all employee records.
- Calculate the total number of employees using COUNT().
- Calculate the total and average salary using SUM() and AVG().
- Find the highest and lowest salary using MAX() and MIN().
- Display employee names in uppercase and lowercase using UPPER() and LOWER().
- Find the length of employee names using LENGTH().
- Retrieve employee details based on the joining date.

Create Table

```
CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT,
```

department TEXT,

salary INTEGER,

joining_date DATE

);

Table Created

Sample Data

INSERT INTO employees VALUES (1, 'Kavin', 'IT', 35000, '2026-01-10');

INSERT INTO employees VALUES (2, 'Meena', 'HR', 30000, '2026-02-15');

INSERT INTO employees VALUES (3, 'Arun', 'Finance', 40000, '2026-03-08');

INSERT INTO employees VALUES (4, 'Divya', 'IT', 45000, '2026-04-20');

INSERT INTO employees VALUES (5, 'Priya', 'Sales', 32000, '2026-05-12');

INSERT INTO employees VALUES (6, 'Rahul', 'IT', 50000, '2026-06-25');

Records Inserted

SQL Query

Query 1: Display all employee records

SQL> SELECT * FROM employees;

Resultant Query

Employee_id	employee_name	department	salary	joining_date
1	Kavin	IT	35000	2026-01-10
2	Meena	HR	30000	2026-02-15
3	Arun	Finance	40000	2026-03-08
4	Divya	IT	45000	2026-04-20
5	Priya	Sales	32000	2026-05-12
6	Rahul	IT	50000	2026-06-25

AGGREGATE FUNCTIONS

Query 2: Count the total number of employees

SQL> SELECT COUNT(*) AS Total_Employees

FROM employees;

Resultant Query

Total_Employees
6

Query 3: Calculate the total salary of all employees

SQL> SELECT SUM(salary) AS Total_Salary

FROM employees;

Resultant Query

Total_Salary
232000

Query 4: Calculate the average salary of employees

SQL> SELECT AVG(salary) AS Average_Salary

FROM employees;

Resultant Query

Average_Salary
38666.67

Query 5: Find the highest and lowest employee salary

SQL> SELECT MAX(salary) AS Highest_Salary,

MIN(salary) AS Lowest_Salary

FROM employees;

Resultant Query

Highest_Salary	Lowest_Salary
50000	30000

STRING FUNCTIONS

Query 6: Display employee names in uppercase

SQL> SELECT employee_name, UPPER(employee_name) AS Upper_Name

FROM employees;

Resultant Query

employee_name	Upper_Name
Kavin	KAVIN
Meena	MEENA
Arun	ARUN
Divya	DIVYA
Priya	PRIYA
Rahul	RAHUL

Query 7: Display employee names in lowercase

SQL> SELECT employee_name, LOWER(employee_name) AS Lower_Name

FROM employees;

Resultant Query

employee_name	Lower_Name
Kavin	kavin
Meena	meena
Arun	arun
Divya	divya
Priya	priya
Rahul	rahul

Query 8: Display the length of each employee name

SQL> SELECT employee_name, LENGTH(employee_name) AS Name_Length

FROM employees;

Resultant Query

employee_name	Name_Length
Kavin	5
Meena	5
Arun	4
Divya	5
Priya	5
Rahul	5

DATE FUNCTIONS

Query 9: Display the current date

SQL> SELECT CURRENT_DATE AS Current_Date;

Resultant Query

Current_Date
Current system date

Query 10: Display employees who joined after March 2026

SQL> SELECT employee_name, joining_date

FROM employees

WHERE joining_date > '2026-03-31'; **Resultant Query**

Resultant Query

employee_name	joining_date
Divya	2026-04-20
Priya	2026-05-12
Rahul	2026-06-25

RESULT:

Thus, the Employees table was created successfully, employee records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO:07 Creation of Subqueries and Nested Queries Using Products Table

DATE: _____

Aim

To create a Products table, insert product records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / sqlite3

Description

A Subquery is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES:

• Single Row Subquery

• Multiple Row Subquery

• Nested Subquery

SINGLE ROW SUBQUERY

A Single Row Subquery returns only one value or one row and is used with comparison operators such as =, >, <, >= and <=.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name > (SELECT column_name
                     FROM table_name
                     WHERE condition);
```

MULTIPLE ROW SUBQUERY

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL operators.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name IN (SELECT column_name
        FROM table_name
        WHERE condition);
```

NESTED SUBQUERY

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT column_name
        FROM table_name
        WHERE condition);
```

Explanation

- Create a table named products.
- Insert product details into the table.
- Display all product records.
- Find products whose price is greater than the average price.
- Find products that belong to Electronics or Accessories categories.
- Find the product that has the highest price.
- Retrieve products based on the result of another query.

Create Table

```
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT,
    category TEXT,
    price INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO products VALUES (1, 'Laptop', 'Electronics', 55000);
INSERT INTO products VALUES (2, 'Mobile', 'Electronics', 25000);
INSERT INTO products VALUES (3, 'Keyboard', 'Accessories', 2000);
INSERT INTO products VALUES (4, 'Monitor', 'Electronics', 15000);
INSERT INTO products VALUES (5, 'Mouse', 'Accessories', 1000);
INSERT INTO products VALUES (6, 'Printer', 'Office', 12000);
```

Records Inserted

SQL Query

Query 1: Display all product records

```
SQL> SELECT * FROM products;
```

Resultant Query

product_id	product_name	category	price
1	Laptop	Electronics	55000
2	Mobile	Electronics	25000
3	Keyboard	Accessories	2000
4	Monitor	Electronics	15000
5	Mouse	Accessories	1000
6	Printer	Office	12000

Query 2: Display products whose price is greater than the average price

```
SQL> SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price)
```

FROM products);

Resultant Query

product_name	price
Laptop	55000
Mobile	25000

Query 3: Display products that belong to Electronics or Accessories category

```
SQL> SELECT product_name, category
FROM products
WHERE category IN (SELECT category
        FROM products
        WHERE category IN ('Electronics', 'Accessories'));
```

Resultant Query

product_name	category
Laptop	Electronics
Mobile	Electronics
Keyboard	Accessories
Monitor	Electronics
Mouse	Accessories

Query 4: Display the product with the highest price

```
SQL> SELECT product_name, price
FROM products
WHERE price = (SELECT MAX(price)
        FROM products);
```

Resultant Query

product_name	price
Laptop	55000

Query 5: Display products whose price is greater than Mobile's price

```
SQL> SELECT product_name, price
FROM products
```

```
WHERE price > (SELECT price
        FROM products
        WHERE product_name = 'Mobile');
```

Resultant Query

product_name	price
Laptop	55000

RESULT:

Thus, the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected library records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO:08 Creation of Views, Indexes and Sequences Using Library Table

DATE: _____

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View:
A View is a virtual table created from one or more tables using a SELECT query. It does not store the actual data separately.

Index:
An Index is used to improve the speed of data retrieval from a table.

Sequence:
A Sequence is a database object used to generate unique numeric values automatically.

1. VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Create Table

```
CREATE TABLE library_books (
   book_id NUMBER PRIMARY KEY,
   book_name VARCHAR2(50),
   category VARCHAR2(30),
   price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO library_books VALUES (201, 'Database Systems', 'Computer', 500);
INSERT INTO library_books VALUES (202, 'Python Programming', 'Programming', 650);
INSERT INTO library_books VALUES (203, 'Java Basics', 'Programming', 550);
INSERT INTO library_books VALUES (204, 'Web Development', 'Computer', 700);
INSERT INTO library_books VALUES (205, 'Computer Networks', 'Computer', 600);

COMMIT;
```

Records Inserted

Create View

```
CREATE VIEW computer_books AS
SELECT book_id, book_name, price
FROM library_books
WHERE category = 'Computer';
```

View Created

Display View

```
SELECT * FROM computer_books;
```

Resultant Query

book_id	book_name	price
201	Database Systems	500
204	Web Development	700
205	Computer Networks	600

Explanation

The computer_books view displays only the books that belong to the Computer category.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

CREATE INDEX index_name

ON table_name(column_name);

Create Index

CREATE INDEX idx_book_name

ON library_books(book_name);

Index Created

Use Index in Search

SELECT *

FROM library_books

WHERE book_name = 'Python Programming';

Resultant Query

book_id	book_name	category	price
202	Python Programming	Programming	650

Explanation

The index idx_book_name is created on the book_name column. It helps improve the performance of searches based on book names.

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY increment_value;

Create Sequence

CREATE SEQUENCE book_seq

START WITH 206

INCREMENT BY 1;

Sequence Created

Insert Record Using Sequence

```
INSERT INTO library_books
VALUES (book_seq.NEXTVAL, 'Operating Systems', 'Computer', 750);

COMMIT;
```

Record Inserted

Display Sequence Value

```
SELECT book_seq.CURRVAL AS current_sequence
FROM dual;
```

Resultant Query

current_sequence
206

Insert Another Record Using Sequence

```
INSERT INTO library_books
VALUES (book_seq.NEXTVAL, 'C Programming', 'Programming', 450);

COMMIT;
```

Record Inserted

Display All Library Book Records

```
SELECT * FROM library_books
ORDER BY book_id;
```

Resultant Query

book_id	book_name	category	price
201	Database Systems	Computer	500
202	Python Programming	Programming	650
203	Java Basics	Programming	550
204	Web Development	Computer	700
205	Computer Networks	Computer	600
206	Operating Systems	Computer	750
207	C Programming	Programming	450

Important Commands

Drop View

DROP VIEW computer_books;

View Dropped

Drop Index

DROP INDEX idx_book_name;

Index Dropped

Drop Sequence

DROP SEQUENCE book_seq;

Sequence Dropped

Result

Thus, the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected library records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO:09 Creation of Stored Procedures and Functions

DATE: _____

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

Description

Stored Procedure:
A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:
A Function is a named PL/SQL program that performs a specific operation and always returns a single value.

1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can modify or display data.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
(parameter_name IN datatype)
IS
BEGIN
   statements;
END;
/
```

Create Table

```
CREATE TABLE employee_salary (
   employee_id NUMBER PRIMARY KEY,
   employee_name VARCHAR2(50),
   salary NUMBER
```

);

Table Created

Insert Records

INSERT INTO employee_salary VALUES (201, 'Kavin', 35000);

INSERT INTO employee_salary VALUES (202, 'Meena', 32000);

INSERT INTO employee_salary VALUES (203, 'Arun', 45000);

INSERT INTO employee_salary VALUES (204, 'Divya', 50000);

INSERT INTO employee_salary VALUES (205, 'Priya', 42000);

COMMIT;

Records Inserted

Display Employee Records

SELECT * FROM employee_salary;

Resultant Query

employee_id	employee_name	salary
201	Kavin	35000
202	Meena	32000
203	Arun	45000
204	Divya	50000
205	Priya	42000

Create Stored Procedure

The following procedure displays the details of an employee based on the employee ID.

```
CREATE OR REPLACE PROCEDURE display_employee (
    p_employee_id IN NUMBER
)
IS
    v_name employee_salary.employee_name%TYPE;
    v_salary employee_salary.salary%TYPE;
```

```
BEGIN
   SELECT employee_name, salary
   INTO v_name, v_salary
   FROM employee_salary
   WHERE employee_id = p_employee_id;

   DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
END;
/
```

Procedure Created

Execute Stored Procedure

```
SET SERVEROUTPUT ON;

EXEC display_employee(203);
```

Resultant Query

Employee Name	Salary
Arun	45000

Output

Employee Name: Arun

Salary: 45000

Explanation

The procedure display_employee accepts the employee ID as an input parameter and retrieves the corresponding employee name and salary from the employee_salary table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

```
CREATE OR REPLACE FUNCTION function_name
(parameter_name IN datatype)
RETURN datatype
IS
BEGIN
   statements;
   RETURN value;
END;
/
```

Create Function

The following function returns the salary category based on the employee's salary.

```
CREATE OR REPLACE FUNCTION get_salary_category (
   p_salary IN NUMBER
)
RETURN VARCHAR2
IS
   v_category VARCHAR2(20);
BEGIN
   IF p_salary >= 45000 THEN
      v_category := 'High';
   ELSIF p_salary >= 35000 THEN
      v_category := 'Medium';
   ELSE
      v_category := 'Low';
   END IF;

   RETURN v_category;
END;
```

/

Function Created

Execute Function

```
SELECT employee_id, employee_name, salary,
    get_salary_category(salary) AS category
FROM employee_salary;
```

Resultant Query

employee_id	employee_name	salary	category
201	Kavin	35000	Medium
202	Meena	32000	Low
203	Arun	45000	High
204	Divya	50000	High
205	Priya	42000	Medium

Explanation

The function get_salary_category accepts the employee's salary as an input and returns the corresponding salary category. The function is called inside the SELECT statement for each employee.

Important Commands

Display Stored Procedure

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'DISPLAY_EMPLOYEE';
```

Resultant Query

object_name	object_type
DISPLAY_EMPLOYEE	PROCEDURE

Display Stored Function

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'GET_SALARY_CATEGORY';
```

Resultant Query

object_name	object_type
GET_SALARY_CATEGORY	FUNCTION

Drop Procedure

DROP PROCEDURE display_employee;

Procedure Dropped

Drop Function

DROP FUNCTION get_salary_category;

Function Dropped

Result

Thus, the Stored Procedure and Function were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve employee details, and the Function was used to determine the salary category based on the employee's salary.

EX. NO:10 Creation of Triggers Method Using Bank Account Table

DATE: _____

Aim

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT, UPDATE, or DELETE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically perform operations such as inserting, updating, or maintaining records when a particular event takes place.

1. CREATE TABLE

```
CREATE TABLE bank_accounts (
   account_id NUMBER PRIMARY KEY,
   account_holder VARCHAR2(50),
   balance NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO bank_accounts VALUES (301, 'Kavin', 25000);
INSERT INTO bank_accounts VALUES (302, 'Meena', 18000);
INSERT INTO bank_accounts VALUES (303, 'Arun', 30000);
INSERT INTO bank_accounts VALUES (304, 'Divya', 45000);
INSERT INTO bank_accounts VALUES (305, 'Priya', 22000);

COMMIT;
```

Records Inserted

Display Bank Account Records

```
SELECT * FROM bank_accounts;
```

Resultant Query

account_id	account_holder	balance
301	Kavin	25000
302	Meena	18000
303	Arun	30000
304	Divya	45000
305	Priya	22000

2. BEFORE INSERT TRIGGER

Definition

A BEFORE INSERT Trigger is executed automatically before a new record is inserted into a table.

Syntax

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
   statements;
END;
/
```

Create Trigger

The following trigger checks whether the account balance is negative before inserting a bank account record.

```
CREATE OR REPLACE TRIGGER check_account_balance
BEFORE INSERT ON bank_accounts
FOR EACH ROW
BEGIN
   IF :NEW.balance < 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Balance cannot be negative');
   END IF;
```

```
END;
/
```

Trigger Created

Execute Trigger

```
INSERT INTO bank_accounts
VALUES (306, 'Rahul', -5000);
```

Output

ORA-20001: Balance cannot be negative

The trigger prevents the insertion of an invalid balance.

3. BEFORE UPDATE TRIGGER

Definition

A BEFORE UPDATE Trigger is executed automatically before an existing record is updated.

Create Trigger

The following trigger prevents the account balance from being updated to a negative value.

```
CREATE OR REPLACE TRIGGER check_account_update
BEFORE UPDATE OF balance ON bank_accounts
FOR EACH ROW
BEGIN
   IF :NEW.balance < 0 THEN
      RAISE_APPLICATION_ERROR(-20002, 'Balance cannot be negative');
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
UPDATE bank_accounts
SET balance = -1000
WHERE account_id = 303;
```

Output

ORA-20002: Balance cannot be negative

The trigger prevents the invalid balance from being updated.

4. AFTER INSERT TRIGGER

Definition

An AFTER INSERT Trigger is executed automatically after a new record is successfully inserted into a table.

Create Audit Table

```
CREATE TABLE account_audit (
   audit_id NUMBER,
   account_id NUMBER,
   action_type VARCHAR2(20)
);
```

Table Created

Create Trigger

```
CREATE OR REPLACE TRIGGER account_insert_audit
AFTER INSERT ON bank_accounts
FOR EACH ROW
BEGIN
   INSERT INTO account_audit
   VALUES (account_audit_seq.NEXTVAL, :NEW.account_id, 'INSERT');
END;
/
```

Trigger Created

Before executing the above trigger, create the required sequence:

```
CREATE SEQUENCE account_audit_seq
START WITH 1
INCREMENT BY 1;
```

Sequence Created

Insert New Account

```
INSERT INTO bank_accounts
VALUES (306, 'Rahul', 20000);


COMMIT;
```

Record Inserted

Display Audit Records

```
SELECT * FROM account_audit;
```

Resultant Query

audit_id	account_id	action_type
1	306	INSERT

5. DISPLAY FINAL BANK ACCOUNT RECORDS

```
SELECT * FROM bank_accounts
ORDER BY account_id;
```

Resultant Query

account_id	account_holder	balance
301	Kavin	25000
302	Meena	18000
303	Arun	30000
304	Divya	45000
305	Priya	22000
306	Rahul	20000

Important Commands

Display Triggers

```
SELECT trigger_name, status
FROM user_triggers;
```

Resultant Query

trigger_name	status
CHECK_ACCOUNT_BALANCE	ENABLED
CHECK_ACCOUNT_UPDATE	ENABLED
ACCOUNT_INSERT_AUDIT	ENABLED

Drop Trigger

DROP TRIGGER check_account_balance;

Trigger Dropped

DROP TRIGGER check_account_update;

Trigger Dropped

DROP TRIGGER account_insert_audit;

Trigger Dropped

Result

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers automatically validated account balance before insertion and updating and recorded new account insertion details in the audit table.

EX. NO:11 Creation of cursor implementation using course table

DATE:

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

There are two types of cursors:

1. Implicit Cursor
2. Explicit Cursor

In this practical, an Explicit Cursor is implemented.

1. CREATE TABLE

```
CREATE TABLE course_details (
   course_id NUMBER PRIMARY KEY,
   course_name VARCHAR2(50),
   instructor_name VARCHAR2(50),
   fees NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO course_details VALUES (401, 'Python Programming', 'Arun', 5000);

INSERT INTO course_details VALUES (402, 'Database Management', 'Priya', 6500);

INSERT INTO course_details VALUES (403, 'Web Development', 'Kavin', 4500);
```

```
INSERT INTO course_details VALUES (404, 'Java Programming', 'Meena', 7000);

INSERT INTO course_details VALUES (405, 'Computer Networks', 'Rahul', 6000);

COMMIT;
```

Records Inserted

Display Course Records

```
SELECT * FROM course_details;
```

Resultant Query

course_id	course_name	instructor_name	fees
401	Python Programming	Arun	5000
402	Database Management	Priya	6500
403	Web Development	Kavin	4500
404	Java Programming	Meena	7000
405	Computer Networks	Rahul	6000

2. EXPLICIT CURSOR

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and process them one by one.

Syntax

```
DECLARE
   CURSOR cursor_name IS
      SELECT statement;

BEGIN
   OPEN cursor_name;

   LOOP
      FETCH cursor_name INTO variables;

      EXIT WHEN cursor_name%NOTFOUND;
```

```
        statements;

    END LOOP;

    CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves each course name and fees from the course_details table.

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR course_cursor IS
        SELECT course_name, fees
        FROM course_details;

    v_course_name course_details.course_name%TYPE;
    v_fees course_details.fees%TYPE;

BEGIN
    OPEN course_cursor;

    LOOP
        FETCH course_cursor INTO v_course_name, v_fees;

        EXIT WHEN course_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
```

```
        'Course: ' || v_course_name || ' | Fees: ₹' || v_fees
    );

  END LOOP;

  CLOSE course_cursor;
END;
/
```

Output

Course: Python Programming | Fees: ₹5000

Course: Database Management | Fees: ₹6500

Course: Web Development | Fees: ₹4500

Course: Java Programming | Fees: ₹7000

Course: Computer Networks | Fees: ₹6000

3. CURSOR WITH CONDITION

The following cursor displays only the courses whose fees are greater than ₹5000.

```
SET SERVEROUTPUT ON;

DECLARE
  CURSOR course_cursor IS
    SELECT course_id, course_name, fees
    FROM course_details
    WHERE fees > 5000;

  v_course_id course_details.course_id%TYPE;
  v_course_name course_details.course_name%TYPE;
  v_fees course_details.fees%TYPE;

BEGIN
```

```
    OPEN course_cursor;

    LOOP
        FETCH course_cursor INTO v_course_id, v_course_name, v_fees;

        EXIT WHEN course_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Course ID: ' || v_course_id ||
            ' | Course: ' || v_course_name ||
            ' | Fees: ₹' || v_fees
        );

    END LOOP;

    CLOSE course_cursor;
END;
/
```

Resultant Query

course_id	course_name	fees
402	Database Management	6500
404	Java Programming	7000
405	Computer Networks	6000

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Important Cursor Attributes

Cursor Attribute	Description
%FOUND	Returns TRUE if the last fetch returned a row
%NOTFOUND	Returns TRUE if the last fetch did not return a row

Cursor Attribute	Description
%ROWCOUNT	Returns the number of rows processed
%ISOPEN	Returns TRUE if the cursor is currently open

5. CURSOR USING %ROWCOUNT

The following program displays the total number of courses processed by the cursor.

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR course_cursor IS
      SELECT course_name, fees
      FROM course_details;

   v_course_name course_details.course_name%TYPE;
   v_fees course_details.fees%TYPE;

BEGIN
   OPEN course_cursor;

   LOOP
      FETCH course_cursor INTO v_course_name, v_fees;

      EXIT WHEN course_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         'Course: ' || v_course_name ||
         ' | Fees: ₹' || v_fees
      );

   END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE(
        'Total Courses Processed: ' || course_cursor%ROWCOUNT
    );

    CLOSE course_cursor;
END;
/
```

Output

Course: Python Programming | Fees: ₹5000

Course: Database Management | Fees: ₹6500

Course: Web Development | Fees: ₹4500

Course: Java Programming | Fees: ₹7000

Course: Computer Networks | Fees: ₹6000

Total Courses Processed: 5

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process course records one by one, apply conditions, and count the number of processed records.

EX. NO:12 Creation of PL/SQL Exception Handling using customer table

DATE:

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```
DECLARE
   declarations;

BEGIN
   statements;

EXCEPTION
   WHEN exception_name THEN
      statements;

END;
/
```

Types of Exceptions

Type	Description
Predefined Exception	Built-in exceptions provided by Oracle
User-Defined Exception	Exception created by the programmer

1. CREATE TABLE

```
CREATE TABLE customer_details (
   customer_id NUMBER PRIMARY KEY,
   customer_name VARCHAR2(50),
   purchase_amount NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO customer_details VALUES (501, 'Kavin', 25000);

INSERT INTO customer_details VALUES (502, 'Meena', 30000);

INSERT INTO customer_details VALUES (503, 'Arun', 20000);

INSERT INTO customer_details VALUES (504, 'Divya', 35000);

INSERT INTO customer_details VALUES (505, 'Priya', 40000);

COMMIT;
```

Records Inserted

Display Customer Records

```
SELECT * FROM customer_details;
```

Resultant Query

customer_id	customer_name	purchase_amount
501	Kavin	25000
502	Meena	30000
503	Arun	20000
504	Divya	35000
505	Priya	40000

2. NO_DATA_FOUND EXCEPTION

Definition

NO_DATA_FOUND occurs when a SELECT INTO statement does not return any record.

Program

```
SET SERVEROUTPUT ON;


DECLARE
   v_name customer_details.customer_name%TYPE;
   v_amount customer_details.purchase_amount%TYPE;


BEGIN
   SELECT customer_name, purchase_amount
   INTO v_name, v_amount
   FROM customer_details
   WHERE customer_id = 599;


   DBMS_OUTPUT.PUT_LINE('Customer Name: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Purchase Amount: ₹' || v_amount);


EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Customer not found.');
END;
/
```

Output

Exception: Customer not found.

Exception Handled Successfully

3. TOO_MANY_ROWS EXCEPTION

Definition

TOO_MANY_ROWS occurs when a SELECT INTO statement returns more than one record.

Program

```
SET SERVEROUTPUT ON;


DECLARE
   v_name customer_details.customer_name%TYPE;


BEGIN
   SELECT customer_name
   INTO v_name
   FROM customer_details
   WHERE purchase_amount > 20000;


   DBMS_OUTPUT.PUT_LINE('Customer Name: ' || v_name);


EXCEPTION
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: More than one customer found.'
      );
END;
/
```

Output

Exception: More than one customer found.

Exception Handled Successfully

4. ZERO_DIVIDE EXCEPTION

Definition

ZERO_DIVIDE occurs when a number is divided by zero.

Program

```
SET SERVEROUTPUT ON;
```

```
DECLARE
   v_amount NUMBER := 25000;
   v_divisor NUMBER := 0;
   v_result NUMBER;

BEGIN
   v_result := v_amount / v_divisor;

   DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);

EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Cannot divide by zero.'
      );
END;
/
```

Output

Exception: Cannot divide by zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, a purchase is allowed only when the purchase amount is within the permitted limit.

Program

```
SET SERVEROUTPUT ON;
```

```
DECLARE
    v_amount customer_details.purchase_amount%TYPE;
    v_purchase NUMBER := 50000;

    purchase_limit_exceeded EXCEPTION;

BEGIN
    SELECT purchase_amount
    INTO v_amount
    FROM customer_details
    WHERE customer_id = 501;

    IF v_purchase > v_amount THEN
        RAISE purchase_limit_exceeded;
    END IF;

    UPDATE customer_details
    SET purchase_amount = purchase_amount + v_purchase
    WHERE customer_id = 501;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Purchase Successful.');

EXCEPTION
    WHEN purchase_limit_exceeded THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Purchase amount exceeds the limit.'
        );
```

```
END;
/
```

Output

Exception: Purchase amount exceeds the limit.

User-Defined Exception Handled Successfully

6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program performs a valid purchase when the purchase amount is within the available limit.

```
SET SERVEROUTPUT ON;

DECLARE
   v_amount customer_details.purchase_amount%TYPE;
   v_purchase NUMBER := 5000;

BEGIN
   SELECT purchase_amount
   INTO v_amount
   FROM customer_details
   WHERE customer_id = 501;

   IF v_purchase > v_amount THEN
      RAISE_APPLICATION_ERROR(
         -20001,
         'Purchase amount exceeds the limit'
      );
   END IF;

   UPDATE customer_details
   SET purchase_amount = purchase_amount + v_purchase
```

```
    WHERE customer_id = 501;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Purchase Successful.');

    DBMS_OUTPUT.PUT_LINE(
        'Purchase Amount: ₹' || v_purchase
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```

Output

Purchase Successful.

Purchase Amount: ₹5000

Display Updated Customer

```
SELECT * FROM customer_details
WHERE customer_id = 501;
```

Resultant Query

customer_id	customer_name	purchase_amount
501	Kavin	30000

Important Predefined Exceptions

Exception	Description
NO_DATA_FOUND	No record is found
TOO_MANY_ROWS	More than one record is returned
ZERO_DIVIDE	Division by zero occurs
DUP_VAL_ON_INDEX	Duplicate value is inserted into a unique/primary key column
VALUE_ERROR	Invalid value or conversion error occurs



Result

Thus, PL/SQL Exception Handling was successfully implemented. The NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, and user-defined exceptions were handled successfully using appropriate exception-handling mechanisms.

EX. NO:13 Creation of Database Normalization using employee project table

DATE:

Aim

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and improve data integrity.

Software Used

Oracle SQL / SQL Command Line

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

The major normal forms covered are:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. UNNORMALIZED TABLE

Consider the following Employee Project table.

Unnormalized Table

The table contains repeated information. For example, the project manager name is repeated for employees working on the same project.

employee_ id	employee_ name	project_ id	project_ name	Manager _id	manager_ name
601	Kavin	P01	Website	M01	Arun
602	Meena	P02	Banking App	M02	Priya
603	Arun	P01	Website	M01	Arun
604	Divya	P03	Mobile App	M03	Ravi

2. FIRST NORMAL FORM (1NF)

Definition

A relation is in 1NF when:

- Each column contains atomic values.
- There are no repeating groups.

- Each row is uniquely identifiable.

The above table already contains atomic values, so it can be considered as the 1NF table.

1NF Table

Employee_id	Employee_name	project_id	Project_name	manager_id	Manager_name
601	Kavin	P01	Website	M01	Arun
602	Meena	P02	Banking App	M02	Priya
603	Arun	P01	Website	M01	Arun
604	Divya	P03	Mobile App	M03	Ravi

Primary Key: employee_id, project_id

3. SECOND NORMAL FORM (2NF)

Definition

A relation is in 2NF when:

1. It is already in 1NF.
2. There is no partial dependency.
3. Every non-key attribute must depend on the complete primary key.

In the 1NF table, some attributes depend only on part of the composite key.

For example:

- employee_name depends only on employee_id.
- project_name depends only on project_id.
- manager_id depends on project_id.

Therefore, the table is divided into separate tables.

Employee Table

employee_id	employee_name
601	Kavin
602	Meena
603	Arun
604	Divya

Project Table

project_id	project_name	manager_id
P01	Website	M01
P02	Banking App	M02
P03	Mobile App	M03

Employee_Project Table

employee_id	project_id
601	P01
602	P02
603	P01
604	P03

The partial dependencies are removed.

Therefore, the tables are in 2NF.

4. THIRD NORMAL FORM (3NF)

Definition

A relation is in 3NF when:

1. It is already in 2NF.
2. There is no transitive dependency.
3. Non-key attributes must depend only on the primary key.

In the Project table:

project_id → manager_id

and

manager_id → manager_name

Therefore, manager_name has a transitive dependency on project_id.

To remove this dependency, the manager information is separated into another table.

Project Table

project_id	project_name	manager_id
P01	Website	M01
P02	Banking App	M02
P03	Mobile App	M03

Manager Table

manager_id	manager_name
M01	Arun
M02	Priya
M03	Ravi

Employee_Project Table

employee_id	project_id
601	P01
602	P02
603	P01
604	P03

Therefore, the tables are in 3NF.

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in BCNF when, for every non-trivial functional dependency:

The determinant must be a candidate key.

BCNF is a stronger version of 3NF.

Example

Consider an Employee_Project_Manager table.

employee_id	project_id	manager_id
601	P01	M01
602	P02	M02
603	P01	M01
604	P03	M03

Assume that each project has only one manager.

Functional dependency:

project_id → manager_id

Here, project_id determines manager_id.

If project_id is not a candidate key in the given relationship, the relation violates BCNF.

Therefore, the relation can be decomposed into separate tables.

Project Manager Table

project_id	manager_id
P01	M01
P02	M02
P03	M03

Employee Project Table

employee_id	project_id
601	P01
602	P02
603	P01
604	P03

After decomposition, project_id becomes a key in the Project Manager relation.

Therefore, the relation is decomposed to satisfy BCNF.

Result

Thus, the Database Normalization process was studied and implemented up to 1NF, 2NF, 3NF, and BCNF. The normalization process reduces data redundancy, removes dependency-related anomalies, and improves database consistency and integrity.

MINI PROJECT

EX. NO: 14 ONLINE SHOPPING DELIVERY AND RETURNS TRACKING

DATE: 08.10.2026

Aim

To develop an Online Shopping Delivery and Returns Tracking system using SQL and PL/SQL to store, manage, and analyze order status, delivery time, returns, and revenue of an e-commerce store.

Software Used

- GitHub Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

Description

The Online Shopping Delivery and Returns Tracking system is developed to examine how customer orders move from purchase to delivery and how many of them are cancelled or returned. The system stores the order ID, customer ID, order date, delivery date, estimated delivery date, order amount, and order status.

SQL queries are used to retrieve and analyze order information, while PL/SQL procedures and functions are used to retrieve the delivery status and delivery duration of an order, classify delivery performance as On Time, Delayed, or Not Delivered, generate a returns and delivery performance report, and handle exceptions such as invalid order IDs.

Dataset Source

- Dataset: Olist Brazilian E-Commerce (file: olist_orders_dataset.csv, with payment values from olist_order_payments_dataset.csv)
- Author: Olist, Kaggle
- URL: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
- Records used: 25 orders
- Added fields: ORDER_AMOUNT (total payment value of the order, taken from the payments file) and ORDER_STATUS value 'returned' (marked on selected delivered orders, since the original dataset has no return status)
- Order IDs and customer IDs were shortened to 8 characters; dates are stored as DATE; amounts rounded to 2 decimal places to match NUMBER(10,2).

Database **Table**

ECOMMERCE_ORDERS

Column Name	Data Type	Constraint
ORDER_ID	VARCHAR2(32)	PRIMARY KEY

CUSTOMER_ID	VARCHAR2(32)	NOT NULL
ORDER_DATE	DATE	NOT NULL
DELIVERY_DATE	DATE	NULL until delivered, >= ORDER_DATE
ESTIMATED_DATE	DATE	NOT NULL
ORDER_AMOUNT	NUMBER(10,2)	NOT NULL, > 0
ORDER_STATUS	VARCHAR2(20)	NOT NULL, IN (delivered, shipped, canceled, processing, returned)

TABLE CREATION

Oracle

```
CREATE TABLE ecommerce_orders (
    order_id        VARCHAR2(32)   PRIMARY KEY,
    customer_id     VARCHAR2(32)  NOT
    NULL, order_date      DATE  NOT
    NULL, delivery_date  DATE,
    estimated_date  DATE          NOT
    NULL,                         order_amount
    NUMBER(10,2)       NOT       NULL,
    order_status    VARCHAR2(20)  NOT
    NULL,
    CONSTRAINT chk_amount_pos       CHECK (order_amount > 0),
    CONSTRAINT chk_status_valid CHECK (order_status IN
        ('delivered','shipped','canceled','processing','returned')),
    CONSTRAINT chk_deliv_date         CHECK (delivery_date IS NULL
                                        OR delivery_date >= order_date)
);
```

SQLite

```
CREATE TABLE
    ecommerce_orders ( order_id
                        TEXT
    PRIMARY KEY, customer_id
                        TEXT  NOT
    NULL, order_date        TEXT
    NOT NULL, delivery_date
                        TEXT,
    estimated_date TEXT NOT
    NULL,
    order_amount  REAL  NOT NULL CHECK
    (order_amount > 0), order_status        TEXT NOT
    NULL CHECK (order_status IN
        ('delivered','shipped','canceled','processing','returned')),
    CHECK (delivery_date IS NULL OR delivery_date >=
```

```
    order_date)
);
```

Table Created

The ECOMMERCE_ORDERS table was created successfully with primary key, CHECK, and NOT NULL constraints.

DATASET INSERTION

```
INSERT INTO ecommerce_orders VALUES ('f1584b99', 'a9f7e979', DATE
    '2017-10-02', DATE '2017-10-10', DATE '2017-10-18', 141.90,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('2d387ab9', '9ab62b5e', DATE
    '2017-07-24', DATE '2017-08-07', DATE '2017-08-13', 86.22,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('417c4fa3', '0a3d7213', DATE
    '2018-08-08', DATE '2018-08-17', DATE '2018-09-04', 72.19,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('500dbb50', 'cb7524d7', DATE
    '2017-11-18', DATE '2017-12-02', DATE '2017-12-15', 259.83,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('46fa5a23', '25ea1682', DATE
    '2018-02-13', DATE '2018-02-16', DATE '2018-03-09', 216.87,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('39d987fd', '5a34d1ed', DATE
    '2017-07-09', DATE '2017-07-26', DATE '2017-08-01', 25.78,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('5e749466', '4d3a21d8', DATE
    '2017-05-16', DATE '2017-05-26', DATE '2017-06-07', 175.26,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('6a465704', '7cd1d2b5', DATE
    '2017-01-23', DATE '2017-02-15', DATE '2017-03-17', 44.60,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('ab0fd361', 'a3098322', DATE
    '2017-07-29', DATE '2017-08-10', DATE '2017-08-23', 123.27,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('d9880148', '47cfc27c', DATE
    '2017-05-16', DATE '2017-06-14', DATE '2017-06-06', 54.19,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('fd8004ec', '99c91b97', DATE
    '2018-06-03', DATE '2018-06-28', DATE '2018-06-14', 89.90,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('5143dfd7', '5c385d6d', DATE
    '2018-01-08', DATE '2018-01-31', DATE '2018-01-26', 112.45,
    'delivered');
```

```
INSERT INTO ecommerce_orders VALUES ('6377db57', 'd0a4f198', DATE
    '2017-03-11', DATE '2017-03-21', DATE '2017-04-05', 67.30,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('5337eaaa', '0ae79a56', DATE
    '2018-04-20', DATE '2018-04-27', DATE '2018-05-14', 310.00,
    'delivered');
INSERT INTO ecommerce_orders VALUES ('81b347cf', 'd4e99aaa', DATE
    '2017-09-10', DATE '2017-09-19', DATE '2017-10-02', 159.99,
    'returned');
INSERT INTO ecommerce_orders VALUES ('4203f3f6', 'e77aa1b8', DATE
    '2018-03-05', DATE '2018-03-14', DATE '2018-03-29', 78.50, 'returned');
INSERT INTO ecommerce_orders VALUES ('c840d1e5', '4d41ca3a', DATE
    '2017-12-12', DATE '2017-12-29', DATE '2017-12-27', 199.00,
    'returned');
INSERT INTO ecommerce_orders VALUES ('f9eccae5', '856deecf', DATE
    '2018-05-22', DATE '2018-06-01', DATE '2018-06-12', 245.75,
    'returned');
INSERT INTO ecommerce_orders VALUES ('e34eb88b', '647e24fd', DATE
    '2017-08-15', NULL, DATE '2017-09-08', 64.80, 'canceled');
INSERT INTO ecommerce_orders VALUES ('4e77032f', '801886d1', DATE
    '2018-02-27', NULL, DATE '2018-03-22', 38.40, 'canceled');
INSERT INTO ecommerce_orders VALUES ('21fc81af', 'a4157eab', DATE
    '2017-11-03', NULL, DATE '2017-11-30', 99.99, 'canceled');
INSERT INTO ecommerce_orders VALUES ('91a0f20d', '99ff292e', DATE
    '2018-08-20', NULL, DATE '2018-09-10', 132.60, 'shipped');
INSERT INTO ecommerce_orders VALUES ('3728bc7f', 'de118786', DATE
    '2018-08-25', NULL, DATE '2018-09-14', 57.35, 'shipped');
INSERT INTO ecommerce_orders VALUES ('8015a26a', 'f13904a0', DATE
    '2018-08-27', NULL, DATE '2018-09-18', 189.00, 'processing');
INSERT INTO ecommerce_orders VALUES ('301bc20d', '5f7478e5', DATE
    '2018-08-28', NULL, DATE '2018-09-20', 76.45, 'processing');
```

Note: in SQLite the DATE keyword is dropped and dates are written as plain text, e.g. '2017-10-02'.

Records Inserted

A total of 25 order records were inserted into the ECOMMERCE_ORDERS table.

SQL Query 1: Display All Orders

```
SELECT order_id, customer_id, order_date, delivery_date,
        order_amount, order_status
FROM
        ecommerce_or
ders ORDER  BY
order_date;
```

Output

order_id	customer_id	order_date	delivery_date	order_amount	order_status
6a465704	7cd1d2b5	2017-01-23	2017-02-15	44.60	delivered
6377db57	d0a4f198	2017-03-11	2017-03-21	67.30	delivered
5e749466	4d3a21d8	2017-05-16	2017-05-26	175.26	delivered
d9880148	47cfc27c	2017-05-16	2017-06-14	54.19	delivered
39d987fd	5a34d1ed	2017-07-09	2017-07-26	25.78	delivered
2d387ab9	9ab62b5e	2017-07-24	2017-08-07	86.22	delivered
ab0fd361	a3098322	2017-07-29	2017-08-10	123.27	delivered
e34eb88b	647e24fd	2017-08-15	NULL	64.80	canceled
81b347cf	d4e99aaa	2017-09-10	2017-09-19	159.99	returned
f1584b99	a9f7e979	2017-10-02	2017-10-10	141.90	delivered
21fc81af	a4157eab	2017-11-03	NULL	99.99	canceled
500dbb50	cb7524d7	2017-11-18	2017-12-02	259.83	delivered
c840d1e5	4d41ca3a	2017-12-12	2017-12-29	199.00	returned
5143dfd7	5c385d6d	2018-01-08	2018-01-31	112.45	delivered
46fa5a23	25ea1682	2018-02-13	2018-02-16	216.87	delivered
4e77032f	801886d1	2018-02-27	NULL	38.40	canceled
4203f3f6	e77aa1b8	2018-03-05	2018-03-14	78.50	returned
5337eaaa	0ae79a56	2018-04-20	2018-04-27	310.00	delivered
f9eccae5	856deecf	2018-05-22	2018-06-01	245.75	returned
fd8004ec	99c91b97	2018-06-03	2018-06-28	89.90	delivered
417c4fa3	0a3d7213	2018-08-08	2018-08-17	72.19	delivered
91a0f20d	99ff292e	2018-08-20	NULL	132.60	shipped
3728bc7f	de118786	2018-08-25	NULL	57.35	shipped
8015a26a	f13904a0	2018-08-27	NULL	189.00	processing
301bc20d	5f7478e5	2018-08-28	NULL	76.45	processing

SQL Query 2: Count of Orders under Each Order Status

```
SELECT order_status, COUNT(*) AS
total_orders FROM ecommerce_orders
GROUP BY order_status
ORDER BY total_orders DESC;
```

Output

order_status	total_orders
delivered	14

returned	4
canceled	3
processing	2
shipped	2

SQL Query 3: Returned Orders and Total Value of Returned Purchases

```
SELECT order_id, customer_id, order_date,
order_amount FROM ecommerce_orders
WHERE order_status = 'returned'
ORDER BY order_date;

SELECT COUNT(*) AS returned_orders,
       ROUND(SUM(order_amount), 2) AS
       returned_value
FROM ecommerce_orders
WHERE order_status = 'returned';
```

Output

order_id	customer_id	order_date	order_amount
81b347cf	d4e99aaa	2017-09-10	159.99
c840d1e5	4d41ca3a	2017-12-12	199.00
4203f3f6	e77aa1b8	2018-03-05	78.50
f9eccae5	856deecf	2018-05-22	245.75

returned_orders	returned_value
4	683.24

SQL Query 4: Average Delivery Duration for Completed Deliveries

```
-- Oracle
SELECT ROUND(AVG(delivery_date - order_date), 2) AS
avg_delivery_days FROM ecommerce_orders
WHERE order_status = 'delivered';

-- SQLite
SELECT ROUND(AVG(julianday(delivery_date) -
       julianday(order_date)), 2) AS avg_delivery_days
FROM ecommerce_orders
WHERE order_status = 'delivered';
```

Output

avg_delivery_da

ys
14.57

SQL Query 5: Total Order Value for Each Order Status

```
SELECT order_status, ROUND(SUM(order_amount), 2) AS
total_order_value FROM    ecommerce_orders
GROUP  BY order_status
ORDER  BY total_order_value DESC;
```

Output

order_status	total_order_value
delivered	1779.76
returned	683.24
processing	265.45
canceled	203.19
shipped	189.95

PL/SQL Procedure 1: Delivery Status and Delivery Duration of an Order

```
CREATE OR REPLACE PROCEDURE sp_order_delivery_info
(p_order_id IN VARCHAR2) IS
    v_cust
                ecommerce_orders.customer_id%T
    YPE; v_status
    ecommerce_orders.order_status%TYPE;
    v_odate
                ecommerce_orders.order_date%TY
    PE; v_ddate
                ecommerce_orders.delivery_date%
    TYPE;
BEGIN
    IF p_order_id IS NULL OR TRIM(p_order_id) IS NULL
       THEN RAISE_APPLICATION_ERROR(-20001, 'Order
       ID cannot be empty.');
    END IF;
    SELECT customer_id, order_status, order_date,
    delivery_date INTO       v_cust, v_status, v_odate,
    v_ddate
    FROM      ecommerce_orders
    WHERE  order_id = p_order_id;
    DBMS_OUTPUT.PUT_LINE('Order ID         : ' ||
    p_order_id); DBMS_OUTPUT.PUT_LINE('Customer
    ID                                    : ' || v_cust);
```

```
    DBMS_OUTPUT.PUT_LINE('Order Status    : ' ||
    v_status);
    DBMS_OUTPUT.PUT_LINE('Order Date: '||
    TO_CHAR(v_odate, 'DD-MON-YYYY'));
    IF v_ddate IS NULL THEN
        DBMS_OUTPUT.PUT_LINE('Delivery Date        : Not
        delivered yet'); DBMS_OUTPUT.PUT_LINE('Duration
        (days): N/A');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Delivery Date  : ' || TO_CHAR(v_ddate, 'DD-
        MON-YYYY')); DBMS_OUTPUT.PUT_LINE('Duration (days): ' || (v_ddate -
        v_odate));
   END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20002, 'No order found for ID ' ||
p_order_id); END sp_order_delivery_info;
/
```

Output

Results Explain Describe Saved SQL History

Procedure created.

0.03 seconds

Execution

```
SET
SERVEROUTPUT
ON; BEGIN
   sp_order_delivery_info('f1584b99')
 ; END;
 /
```

Output

Results Explain Describe Saved SQL History

Order ID : f1584b99
Customer ID : a9f7e979
Order Status : delivered
Order Date : 02-OCT-2017
Delivery Date : 10-OCT-2017
Duration (days): 8

Statement processed.

PL/SQL Function 2: Classify Delivery Performance

```
CREATE OR REPLACE FUNCTION fn_delivery_performance
(p_order_id IN VARCHAR2) RETURN VARCHAR2
IS
    v_ddate
    ecommerce_orders.delivery_date%TYPE;
    v_edate
    ecommerce_orders.estimated_date%TYPE;
BEGIN
    IF p_order_id IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001, 'Order ID cannot
    be empty.'); END IF;
    SELECT delivery_date,
    estimated_date INTO      v_ddate,
    v_edate
    FROM      ecommerce_orders
    WHERE  order_id =
    p_order_id; IF v_ddate IS
    NULL THEN
        RETURN 'Not
    Delivered'; ELSIF v_ddate
    <= v_edate THEN
        RETURN 'On
    Time'; ELSE
        RETURN
    'Delayed'; END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN

        RAISE_APPLICATION_ERROR(-20002, 'No order found for ID ' ||
p_order_id); END fn_delivery_performance;
/
```

Output

Results Explain Describe Saved SQL History

Function created.

0.02 seconds

Execution

```
SELECT order_id, order_status,
        fn_delivery_performance(order_id) AS performance
FROM          ecommerce_orders
WHERE  order_id IN ('f1584b99', 'd9880148', 'e34eb88b', '91a0f20d');
```

Output

Results Explain Describe Saved SQL History

ORDER_ID	ORDER_STATUS	PERFORMANCE
f1584b99	delivered	On Time
d9880148	delivered	Delayed
e34eb88b	canceled	Not Delivered
91a0f20d	shipped	Not Delivered

4 rows returned in 0.03 seconds CSV Export

PL/SQL Procedure 3: Returns and Delivery Performance Report

```
CREATE OR REPLACE PROCEDURE sp_returns_report (p_order_id IN
VARCHAR2 DEFAULT NULL) IS
    v_total     NUMBER  := 0;    v_ret_cnt  NUMBER  := 0;    v_ret_val
    NUMBER := 0; v_on       NUMBER := 0;  v_late     NUMBER := 0;
    v_nd          NUMBER  := 0;  v_exists       NUMBER;              v_perf
    VARCHAR2(20);
    v_status ecommerce_orders.order_status%TYPE;
    v_amount
    ecommerce_orders.order_amount%TYPE;
BEGIN
    IF p_order_id IS NOT NULL THEN              -- single-order
        mode SELECT COUNT(*) INTO v_exists FROM
        ecommerce_orders
        WHERE  order_id =
        p_order_id; IF v_exists = 0
        THEN
            RAISE_APPLICATION_ERROR(-20005, 'Invalid order ID: ' ||
        p_order_id); END IF;
        SELECT order_status, order_amount INTO v_status,
        v_amount FROM       ecommerce_orders WHERE
        order_id = p_order_id;
        DBMS_OUTPUT.PUT_LINE('Order ' || p_order_id || ' | ' || v_status ||
            ' | ' || v_amount || ' | ' || fn_delivery_performance(p_order_id));
        RETURN;
    END IF;
    FOR r IN (SELECT order_id, order_status, order_amount
                FROM
                        ecommerce_orders) LOOP     -- full report
        mode v_total := v_total + 1;
        v_perf  :=
        fn_delivery_performance(r.order_id); IF
```

```
        v_perf = 'On Time' THEN v_on := v_on + 1;
        ELSIF v_perf = 'Delayed' THEN v_late := v_late + 1;
        ELSE v_nd := v_nd + 1; END IF;
        IF r.order_status = 'returned' THEN
            v_ret_cnt := v_ret_cnt + 1;
            v_ret_val := v_ret_val + r.order_amount;
        END IF;
    END LOOP;
    IF v_total = 0 THEN
        RAISE_APPLICATION_ERROR(-20006, 'No data available to
    generate report.'); END IF;


    DBMS_OUTPUT.PUT_LINE('RETURNS AND DELIVERY PERFORMANCE
    REPORT');
    DBMS_OUTPUT.PUT_LINE('Total orders     : ' || v_total);
    DBMS_OUTPUT.PUT_LINE('Returned orders  : ' || v_ret_cnt);
    DBMS_OUTPUT.PUT_LINE('Value of returns : ' || v_ret_val);
    DBMS_OUTPUT.PUT_LINE('Return rate (%)  : ' ||
    ROUND(v_ret_cnt/v_total*100, 2));
    DBMS_OUTPUT.PUT_LINE('On Time: ' || v_on || '  Delayed: ' ||
                                v_late || '  Not Delivered: ' || v_nd);
    FOR r IN (SELECT order_id, order_amount FROM ecommerce_orders
                WHERE order_status = 'returned' ORDER BY order_date) LOOP
        DBMS_OUTPUT.PUT_LINE(RPAD(r.order_id, 12) || RPAD(r.order_amount, 10)
        ||
                                    fn_delivery_performance(r.order_id));
    END LOOP;
END sp_returns_report;
/
```

Output

Results Explain Describe Saved SQL History

Procedure created.

0.03 seconds

Execution

```
BEGIN
   sp_returns_report;
END;
/
```

Output

Results Explain Describe Saved SQL History

```
RETURNS AND DELIVERY PERFORMANCE REPORT
-------------------------------------------------------
Total orders     : 25
Returned orders  : 4
Value of returns : 683.24
Return rate (%)  : 16.00
Delivery performance (all orders):
On Time       : 14
Delayed       :  4
Not Delivered : 7
Returned orders:
ORDER ID    AMOUNT    PERFORMANCE
81b347cf    159.99    On Time
c840d1e5    199.00    Delayed
4203f3f6    78.50     On Time
f9eccae5    245.75    On Time
```

Exception Handling

Test Case 1 – Invalid Order ID (missing record)

```
BEGIN
   sp_order_delivery_info('INVALID99'
); EXCEPTION
   WHEN OTHERS THEN
     DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Error caught: ORA-20002: No order found for ID INVALID99

Statement processed.

0.00 seconds
```

Test Case 2 – NULL / Empty Order ID

```
BEGIN
   sp_order_delivery_info(NULL);
EXCEPTION
   WHEN OTHERS THEN
     DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

```
Results  Explain  Describe  Saved SQL  History

Error caught: ORA-20001: Order ID cannot be empty.

Statement processed.

0.00 seconds
```

Test Case 3 – Order Not Yet Delivered

```
BEGIN
   sp_order_delivery_info('91a0f20d');
   DBMS_OUTPUT.PUT_LINE('Performance     :' ||
fn_delivery_performance('91a0f20d')); END;
/
```

Output

```
Results  Explain  Describe  Saved SQL  History

Order ID       : 91a0f20d
Customer ID    : 99ff292e
Order Status   : shipped
Order Date     : 20-AUG-2018
Delivery Date  : Not delivered yet
Duration (days): N/A
Performance    : Not Delivered

Statement processed.

0.00 seconds
```

Test Case 4 – Function with Invalid Order ID

```
BEGIN
   DBMS_OUTPUT.PUT_LINE('Result: ' ||
fn_delivery_performance('ZZZ00000'));  EXCEPTION
   WHEN OTHERS THEN
     DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

```
Results  Explain  Describe  Saved SQL  History

Error caught: ORA-20002: No order found for ID ZZZ00000

Statement processed.

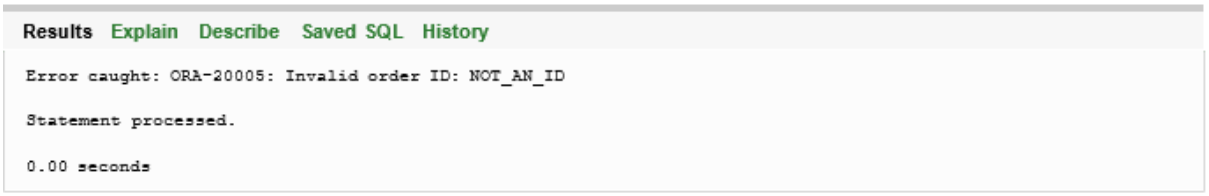
0.00 seconds
```

Test Case 5 – Report with Invalid Order ID

```
BEGIN
   sp_returns_report('NOT_AN_ID');
 EXCEPTION
```

```
    WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

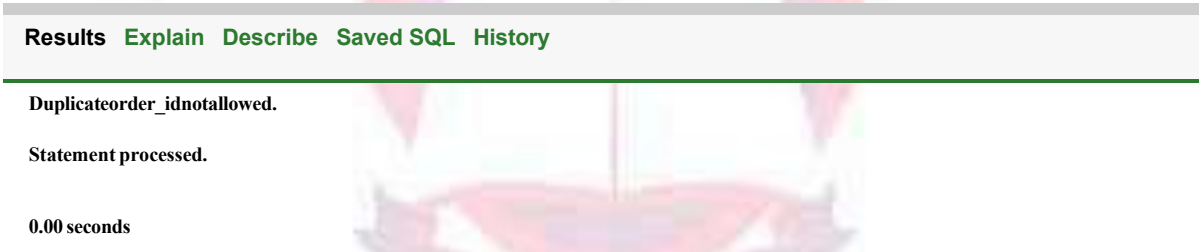
Output



Test Case 6 – Duplicate Order ID

```
BEGIN
    INSERT INTO ecommerce_orders
    VALUES ('f1584b99', 'cust0001', DATE '2018-09-01',
            NULL, DATE '2018-09-20', 50.00, 'processing');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
      DBMS_OUTPUT.PUT_LINE('Duplicate order_id not allowed.');
END;
/
```

Output



Test Case 7 – Invalid Amount (CHECK constraint)

```
BEGIN
    INSERT INTO ecommerce_orders
    VALUES ('neword01', 'cust0001', DATE '2018-09-01',
            NULL, DATE '2018-09-20', -10, 'processing');
EXCEPTION
    WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Invalid amount not allowed (SQLCODE -2290).

Statement processed.

0.00 seconds
```

Dataset Insights

- The dataset contains 25 orders with five statuses: delivered (14), returned (4), canceled (3), shipped (2), and processing (2).
- The average delivery duration of completed (delivered) orders is 14.57 days; the fastest delivery took 3 days and the slowest 29 days.
- 4 of the 18 orders that reached the customer arrived after the estimated date, and 14 arrived on time.
- Returned orders account for 4 of 25 orders (16%), with a total value of 683.24.
- Delivered orders generate the highest revenue (1779.76 of 3121.59 total order value).
- 7 orders (canceled, shipped, or processing) have no delivery date yet, which the classification function reports as Not Delivered.
- The delivery-performance function lets the business flag late deliveries and link them with returns.

Result

Thus, the Online Shopping Delivery and Returns Tracking system was successfully implemented using SQL and PL/SQL. The ECOMMERCE_ORDERS table was created, order records were inserted, SQL queries were executed successfully, PL/SQL procedures and functions were implemented, and exception handling was tested successfully.

BOOK LIST DATABASE

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating. Now, add three of your favorite books into the table.

Program:

```
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);
```

```
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);
```

```
INSERT INTO books VALUES(1,'English',9/10);
```

```
INSERT INTO books VALUES(2,'Maths',10/10);
```

```
INSERT INTO books VALUES(3,'Tamil',9/10);
```

```
SELECT*FROM books;
```



BOX OFFICE HITS DATABASE

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies. Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step.

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

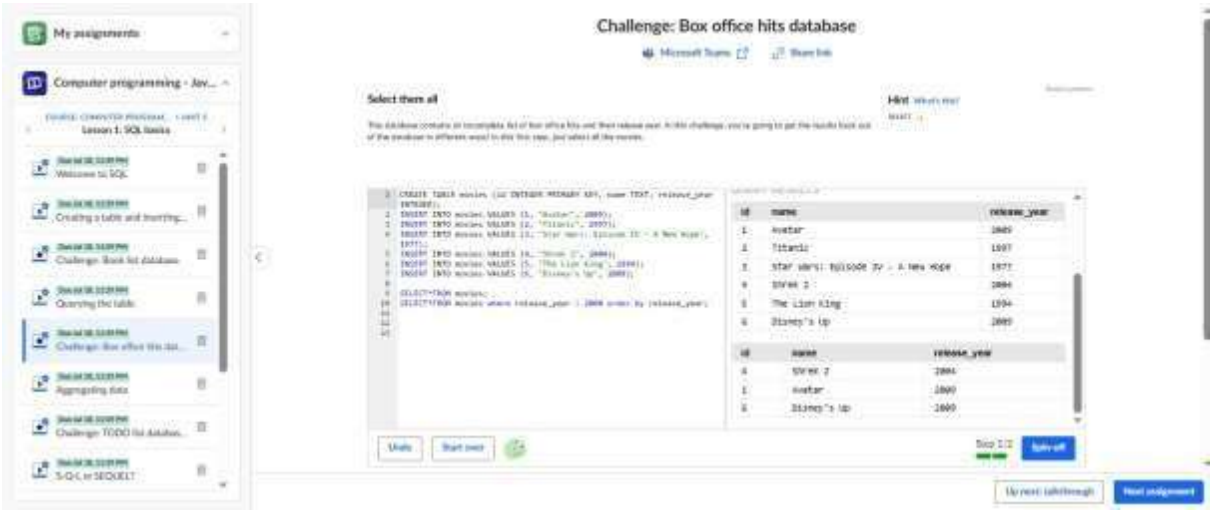
INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

SELECT*FROM movies;

SELECT*FROM movies where release_year > 2000 order by release_year;
```



TODO LISTDATABASE STATS

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take. Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

Program:

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

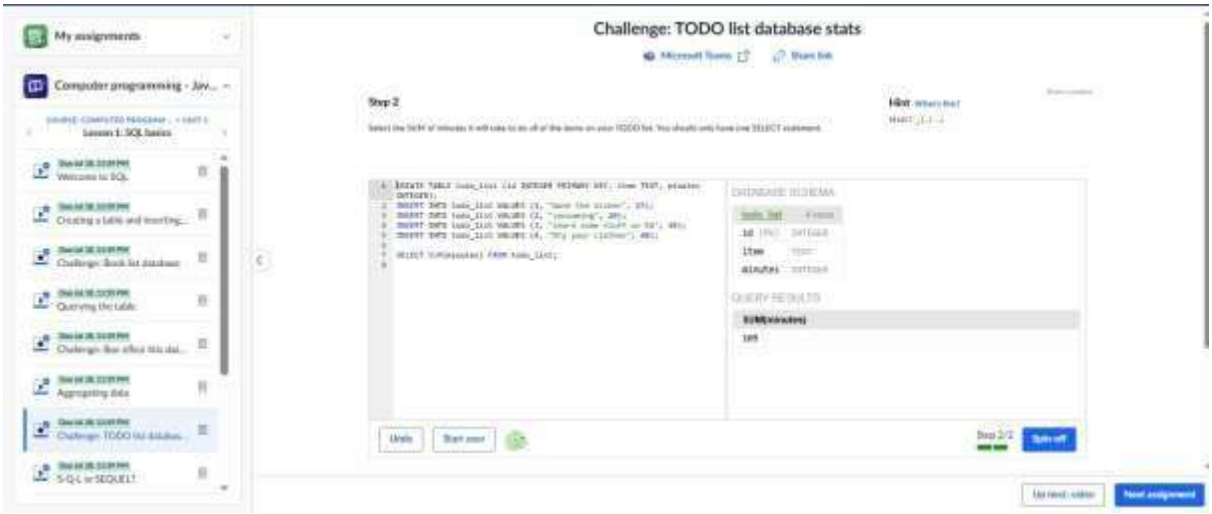
INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Dry your clothes", 40);

SELECT SUM(minutes) FROM todo_list;
```



PROJECT : DESIGN A STORE DATABASE

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

Program:

```
CREATE TABLE sneakers (
    id INTEGER PRIMARY KEY,
    brand TEXT,
    model TEXT,
    size INTEGER,
    color TEXT,
    price REAL
);
INSERT INTO sneakers VALUES (1, "Nike", "Air Max", 9, "Black", 120);
INSERT INTO sneakers VALUES (2, "Adidas", "Ultraboost", 10, "White", 180);
INSERT INTO sneakers VALUES (3, "Puma", "RS-X", 8, "Blue", 110);
INSERT INTO sneakers VALUES (4, "New Balance", "574", 9, "Gray", 95);
INSERT INTO sneakers VALUES (5, "Converse", "Chuck Taylor", 10, "Red", 60);
INSERT INTO sneakers VALUES (6, "Vans", "Old Skool", 8, "Black", 70);
INSERT INTO sneakers VALUES (7, "Nike", "Jordan 1", 11, "Red", 200);
INSERT INTO sneakers VALUES (8, "Adidas", "NMD", 9, "Black", 140);
INSERT INTO sneakers VALUES (9, "Puma", "Suede Classic", 10, "Green", 75);
INSERT INTO sneakers VALUES (10, "New Balance", "990", 11, "Gray", 185);
INSERT INTO sneakers VALUES (11, "Converse", "Run Star Hike", 8, "White", 115);
INSERT INTO sneakers VALUES (12, "Vans", "Sk8-Hi", 9, "Blue", 80);
```

```
INSERT INTO sneakers VALUES (13, "Nike", "Air Force 1", 10, "White", 130);
INSERT INTO sneakers VALUES (14, "Adidas", "Superstar", 8, "Black", 100);
INSERT INTO sneakers VALUES (15, "Puma", "Future Rider", 9, "Yellow", 90);
SELECT *
FROM sneakers
ORDER BY price DESC;
SELECT
    AVG(price) AS average_price,
    MAX(price) AS highest_price,
    MIN(price) AS lowest_price
FROM sneakers;
```

Output

DATABASE SCHEMA

sneakers 15 rows

id (PK) INTEGER
brand TEXT
model TEXT
size INTEGER
color TEXT
price REAL

QUERY RESULTS

id	brand	model	size	color	price
7	Nike	Jordan 1	11	Red	200
10	New Balance	990	11	Gray	185

id	brand	model	size	color	price
2	Adidas	Ultraboost	10	White	180
8	Adidas	NMD	9	Black	140
13	Nike	Air Force 1	10	White	130
1	Nike	Air Max	9	Black	120
11	Converse	Run Star Hike	8	White	115
3	Puma	RS-X	8	Blue	110
14	Adidas	Superstar	8	Black	100
4	New Balance	574	9	Gray	95
15	Puma	Future Rider	9	Yellow	90
12	Vans	Sk8-Hi	9	Blue	80
9	Puma	Suede Classic	10	Green	75
6	Vans	Old Skool	8	Black	70
5	Converse	Chuck Taylor	10	Red	60

average_price	highest_price	lowest_price
116.66666666666667	200	60



KARAOKE SONG SELECTOR

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles. Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990. People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', and released after 1990, and less than 4 minutes long.

Program:

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
 INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
 INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
```

VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);

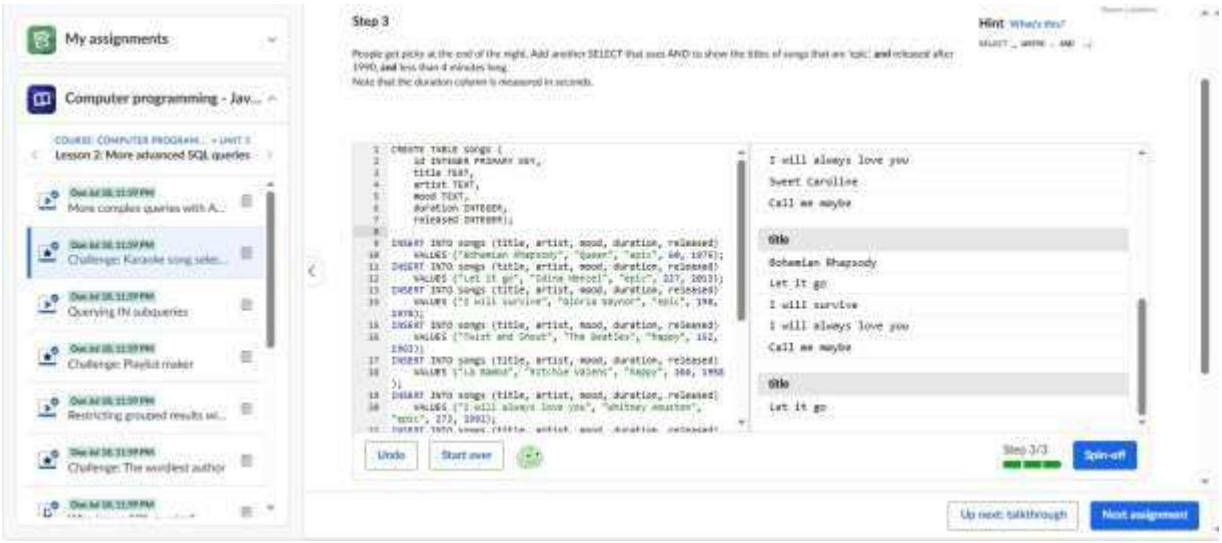
INSERT INTO songs (title, artist, mood, duration, released)

VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);

select title from songs;

select title from songs where mood = 'epic' or released > 1990;

select title from songs where mood = 'epic' and released > 1990 and duration < 240;



PLAYLIST MAKER

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'. Now you'll make a 'Pop' playlist. In preparation, select the name of all of the artists from the 'Pop' genre. To finish creating the 'Pop' playlist, add another query that will select the title of all the songs from the 'Pop' artists. It should use IN on a nested subquery that's based on your previous query.

Program:

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT);
INSERT INTO artists (name, country, genre)
   VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
```

```
    VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
 INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
 INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");
CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);
INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
```

```
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");
select title from songs where artist = 'Queen';
select name from artists where genre = 'Pop';
SELECT title
```

```
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = 'Pop'
);
```



THE WORDIEST AUTHOR

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column. Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

Program:

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
```

```
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
SELECT author,
       SUM(words) AS total_words
FROM books
```

```
GROUP BY author
HAVING SUM(words) > 1000000;
SELECT author,
    AVG(words) AS avg_words
FROM books
GROUP BY author
HAVING AVG(words) > 150000;
```



GRADEBOOK

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column. Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

Program:

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winstonia", 82, 0.9045);
SELECT name,
```

```
    number_grade,
    ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
SELECT COUNT(*),
  CASE
    WHEN number_grade > 90 THEN 'A'
    WHEN number_grade > 80 THEN 'B'
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
  END AS "letter_grade"
FROM student_grades
GROUP BY
  CASE
    WHEN number_grade > 90 THEN 'A'
    WHEN number_grade > 80 THEN 'B'
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
  END;
```



DATA DIG

What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?

Program:

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY,
    name TEXT,
    type TEXT,
    hp INTEGER,
    attack INTEGER
);
INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);
SELECT
    AVG(attack) AS avg_attack,
    MAX(attack) AS max_attack,
    MIN(attack) AS min_attack
FROM pokemon;
SELECT
```

```
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type;
SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 80;
SELECT
    name,
    attack,
    CASE
        WHEN attack >= 100 THEN "Very Strong"
        WHEN attack >= 80 THEN "Strong"
        ELSE "Average"
    END AS strength
FROM pokemon;
SELECT *
FROM pokemon
WHERE (type = "Fire" OR type = "Water")
AND attack > 80;
```

Output

DATABASE SCHEMA

pokemon 8 rows
id (PK) INTEGER
name TEXT
type TEXT
hp INTEGER
attack INTEGER

QUERY RESULTS

avg_attack	max_attack	min_attack
86.125	125	55

type	avg_attack
Electric	60
Fire	97
Grass	83.5
Water	104

type	avg_attack
Fire	97
Grass	83.5
Water	104

name	attack	strength
Pikachu	55	Average
Charizard	84	Strong
Blastoise	83	Strong
Venusaur	82	Strong
Jolteon	65	Average
Arcanine	110	Very Strong
Gyarados	125	Very Strong
Sceptile	85	Strong

id	name	type	hp	attack
2	Charizard	Fire	78	84
3	Blastoise	Water	79	83
6	Arcanine	Fire	90	110
7	Gyarados	Water	95	125



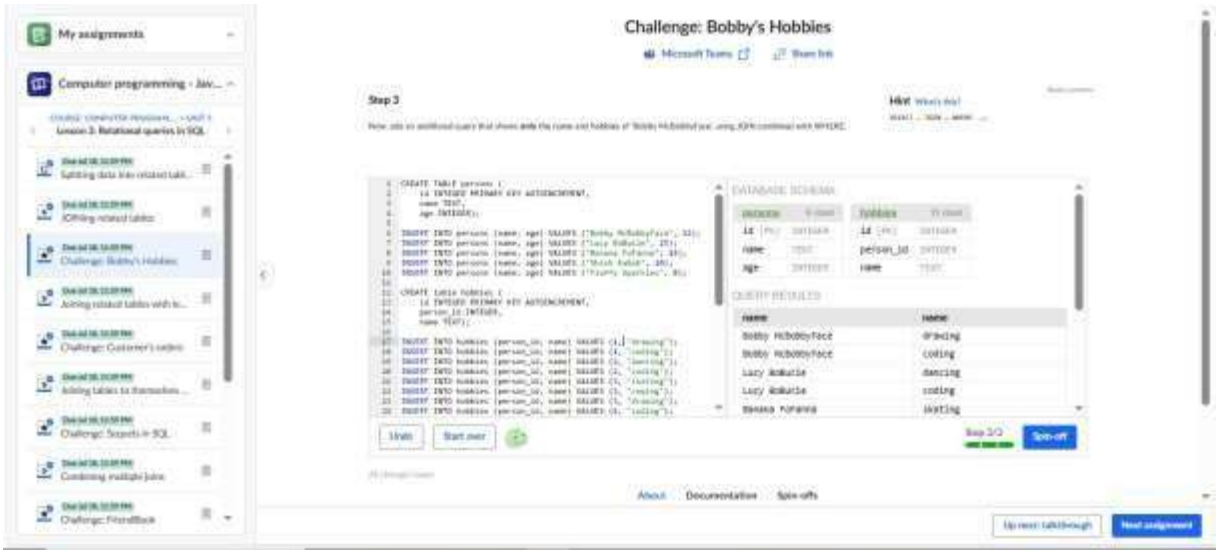
BOBBY'S HOBBIES

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person. Now, select the 2 tables with a join so that you can see each person's name next to their hobby. Now, add an additional query that shows only the name and hobbies of 'Bobby McBobbyFace', using JOIN combined with WHERE.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER);
INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
```

```
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Sam Star", 18);
INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name,
    hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```



CUSTOMER'S ORDERS

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY. Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.

Program:

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT);
INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);
 INSERT INTO orders (customer_id, item, price)
      VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);
SELECT customers.name,
```

```
    customers.email,
    orders.item,
    orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT customers.name,
    customers.email,
    SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```



SEQUELS IN SQL

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT,
   released INTEGER,
   sequel_id INTEGER);
INSERT INTO movies
   VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
   VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
   VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
   VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
   VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
SELECT m1.title,
     m2.title AS sequel_title
```

FROM movies AS m1

LEFT OUTER JOIN movies AS m2

ON m1.sequel_id = m2.id;



FRIENDBOOK

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies. Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT,
    age INTEGER);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
```

```
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
CREATE table friends (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person1_id INTEGER,
    person2_id INTEGER);
INSERT INTO friends (person1_id, person2_id)
    VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
    VALUES (2, 3);
SELECT persons.fullname,
        hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT p1.fullname AS person1,
        p2.fullname AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.id
JOIN persons AS p2
ON friends.person2_id = p2.id;
```



FAMOUS PEOPLE

Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?

Program:

```
CREATE TABLE authors (
    id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);
INSERT INTO authors VALUES (1, "J.K. Rowling", "United Kingdom");
INSERT INTO authors VALUES (2, "George Orwell", "United Kingdom");
INSERT INTO authors VALUES (3, "Mark Twain", "United States");
INSERT INTO authors VALUES (4, "Jane Austen", "United Kingdom");
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER
);
INSERT INTO books VALUES (1, "Harry Potter and the Sorcerer's Stone", 1);
INSERT INTO books VALUES (2, "Harry Potter and the Chamber of Secrets", 1);
INSERT INTO books VALUES (3, "1984", 2);
INSERT INTO books VALUES (4, "Animal Farm", 2);
INSERT INTO books VALUES (5, "The Adventures of Tom Sawyer", 3);
```

```
INSERT INTO books VALUES (6, "Pride and Prejudice", 4);
CREATE TABLE author_friends (
    id INTEGER PRIMARY KEY,
    author1_id INTEGER,
    author2_id INTEGER
);

INSERT INTO author_friends VALUES (1, 1, 2);
INSERT INTO author_friends VALUES (2, 3, 4);
SELECT authors.name AS author,
       books.title AS book
FROM authors
JOIN books
ON authors.id = books.author_id;
SELECT name,
       country
FROM authors;
SELECT a1.name AS author1,
       a2.name AS author2
FROM author_friends
JOIN authors AS a1
ON author_friends.author1_id = a1.id
JOIN authors AS a2
ON author_friends.author2_id = a2.id;
```

Output:

DATABASE SCHEMA

authors4 rows
id (PK)INTEGER
nameTEXT
countryTEXT

books6 rows
id (PK)INTEGER
titleTEXT
author_idINTEGER

author_friends2 rows
id (PK)INTEGER
author1_idINTEGER
author2_idINTEGER

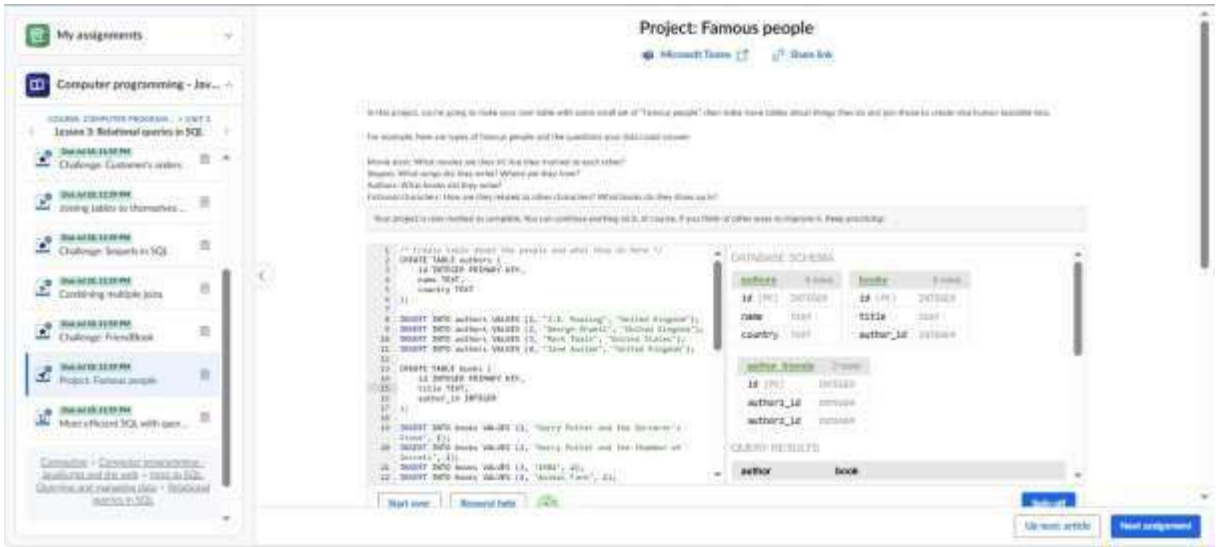
QUERY RESULTS

author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
J.K. Rowling	Harry Potter and the Chamber of Secrets
George Orwell	1984
George Orwell	Animal Farm

author	book
Mark Twain	The Adventures of Tom Sawyer
Jane Austen	Pride and Prejudice

name	country
J.K. Rowling	United Kingdom
George Orwell	United Kingdom
Mark Twain	United States
Jane Austen	United Kingdom

author1	author2
J.K. Rowling	George Orwell
Mark Twain	Jane Austen



DYNAMIC DOCUMENTS

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected. Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

Program:

```
CREATE table documents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    content TEXT,
    author TEXT);
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");
SELECT * FROM documents;
UPDATE documents
SET author = "Jackie Draper"
WHERE author = "Jackie Paper";
```

SELECT * FROM documents;

DELETE FROM documents

WHERE title = "Things I'm Afraid Of";

SELECT * FROM documents;



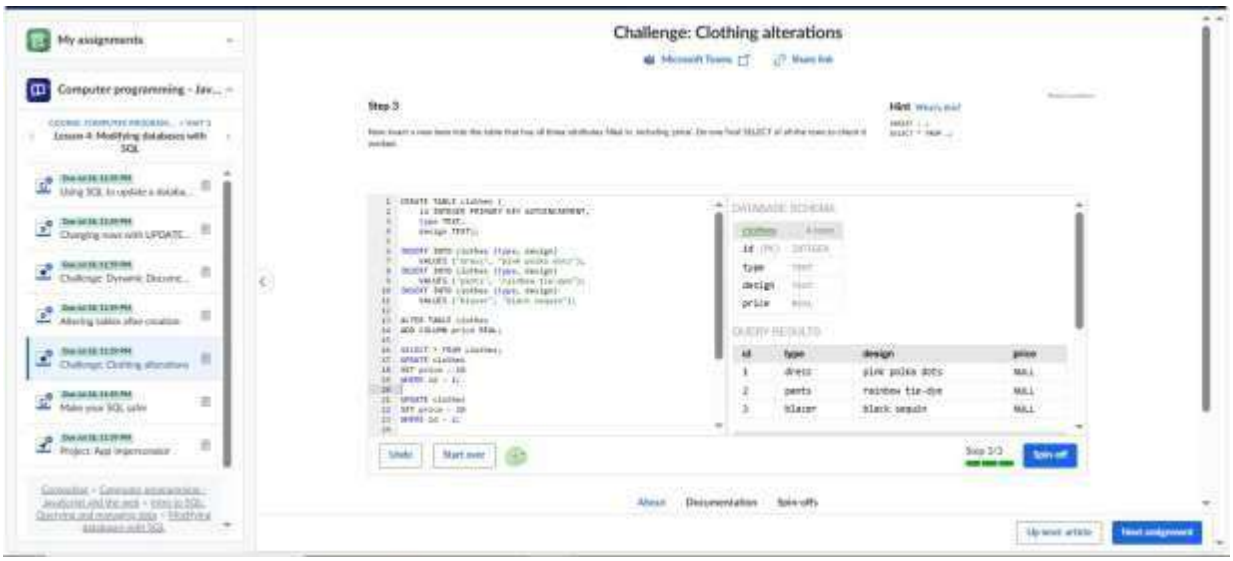
CLOTHING ALTERATIONS

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now. Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all the rows to check that it worked as expected. Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

Program:

```
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    type TEXT,
    design TEXT);
INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");
ALTER TABLE clothes
ADD COLUMN price REAL;
SELECT * FROM clothes;
UPDATE clothes
SET price = 10
WHERE id = 1;
UPDATE clothes
SET price = 20
WHERE id = 2;
```

```
UPDATE clothes
SET price = 30
WHERE id = 3;
SELECT * FROM clothes;
INSERT INTO clothes (type, design, price)
VALUES ("shirt", "blue stripes", 15);
SELECT * FROM clothes;
```



PROJECT: APP IMPERSONATO

CREATE a table to store the data.
INSERT a few example rows in the table.
Use an UPDATE to emulate what happens when you edit data in the app.
Use a DELETE to emulate what happens when you delete data in the app.

Program:

```
CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    content TEXT,
    likes INTEGER
);
INSERT INTO posts (username, content, likes)
VALUES ("alex", "Just finished my homework!", 12);
INSERT INTO posts (username, content, likes)
VALUES ("sam", "Playing my favorite game tonight.", 25);
INSERT INTO posts (username, content, likes)
VALUES ("taylor", "Had a great day at the beach.", 18);
SELECT * FROM posts;
UPDATE posts
SET content = "Just finished my homework and studied SQL!"
WHERE id = 1;
SELECT * FROM posts;
DELETE FROM posts
WHERE id = 2;
SELECT * FROM posts;
```

Output:

DATABASE SCHEMA

posts2 rows
id (PK)INTEGER
usernameTEXT
contentTEXT

posts2 rows
likesINTEGER

QUERY RESULTS

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
3	taylor	Had a great day at the beach.	18



1. Revising the Select Query I

Query all columns for all American cities in the **CITY** table with populations larger than `100000`. The **CountryCode** for America is `USA`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 100000;
```

Output:



Revising the Select Query II

Query the **NAME** field for all American cities in the **CITY** table with populations larger than `120000`. The *CountryCode* for America is `USA`.

The **CITY** table is described as follows:

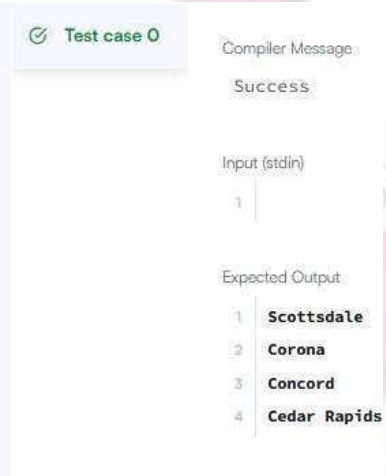
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT NAMEFROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 120000;
```

Output:



2. Select All

Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

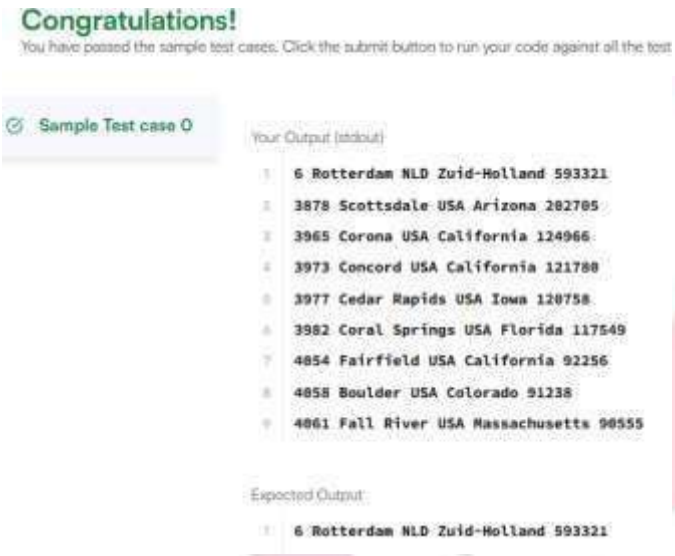
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY;
```

Output:



3. Select By ID

Query all columns for a city in **CITY** with the *ID* `1661`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE ID = 1661;
```

Output:



4. Japanese Cities' Attributes

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



5. Japanese Cities' Names

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT NAME FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



6. Weather Observation Station 1

Query a list of **CITY** and **STATE** from the **STATION** table.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Input:

```
SELECT CITY, STATE FROM STATION;
```

Output:



7. Weather Observation Station 3

Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE MOD(ID, 2) = 0 ORDER BY CITY;
```

Output:



8. Weather Observation Station 4

Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

For example, if there are three records in the table with **CITY** values 'New York', 'New York', 'Bengalaru', there are 2 different city names: 'New York' and 'Bengalaru'.

Input:

```
SELECT COUNT(CITY) - COUNT (DISTINCT City) FROM STATION ;
```

Output:



9. Weather Observation Station 5

Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Sample Input

For example, **CITY** has four entries: **DEF, ABC, PQRS** and **WXY**.

Sample Output

```
ABC 3
PQRS 4
```

Input:

```
SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY), CITY LIMIT 1;
SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY) DESC, CITY LIMIT
1;
```

Output:



Weather Observation Station 6

Query the list of *CITY* names starting with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[aeiouAEIOU]';
```

Output:



10. Weather Observation Station 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '[aeiouAEIOU]$';
```

Output:



11. Weather Observation Station 8

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP
'^[aeiouAEIOU].*[aeiouAEIOU]$';
```

Output:



12. Weather Observation Station 9

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[^aeiouAEIOU]';
```

Output:



13. Weather Observation Station 10

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE RIGHT(CITY, 1) NOT IN ('A', 'E', 'I',
'O', 'U') ORDER BY CITY;
```

Output:



14. Weather Observation Station 11

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

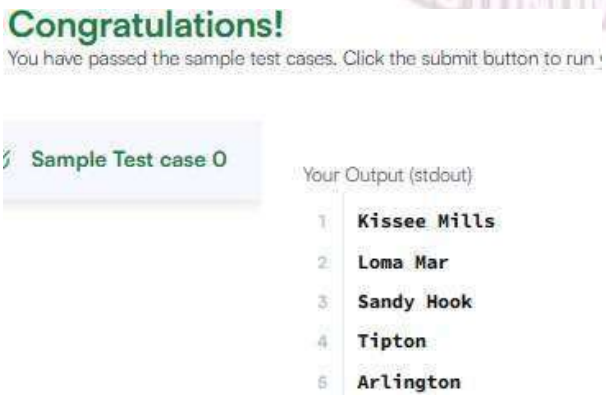
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY NOT REGEXP '^[aeiouAEIOU]' OR
CITY NOT REGEXP '[aeiouAEIOU]$';
```

Output:



16.Weather Observation Station 12

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitud

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[^aeiouAEIOU]' AND CITY
REGEXP '[^aeiouAEIOU]$';
```

Output:



16.Higher Than 75 Marks

Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

The **STUDENTS** table is described as follows:

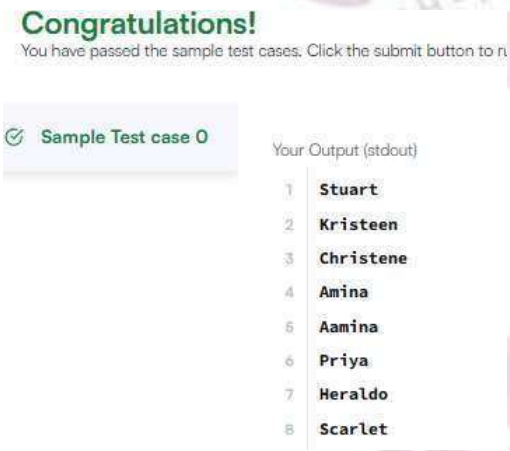
Column	Type
ID	Integer
Name	String
Marks	Integer

The *Name* column only contains uppercase (`A-Z`) and lowercase (`a-z`) letters.

Input:

```
SELECT NAME FROM STUDENTS WHERE MARKS > 75 ORDER BY RIGHT(NAME, 3), ID ASC;
```

Output:



15. Employee Names

Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Input:

```
SELECT name FROM Employee ORDER BY name;
```

Output:



16. Employee Salaries

Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than per month who have been employees for less than months. Sort your result by ascending *employee_id*.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Input:

```
SELECT name FROM Employee WHERE salary > 2000   AND months < 10 ORDER BY
employee_id ASC;
```

Output:



17. Type of Triangle

Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a triangle with sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The **TRIANGLES** table is described as follows:

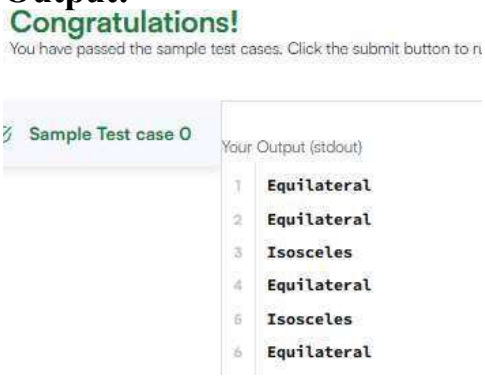
Column	Type
A	Integer
B	Integer
C	Integer

Each row in the table denotes the lengths of each of a triangle's three sides.

Input:

```
SELECT CASE  WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle
WHEN A = B AND B = C THEN 'Equilateral' WHEN A = B OR B = C OR A = C THEN
'Isosceles 'ELSE 'Scalene'END FROM TRIANGLES;
```

Output:



23. The PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: `AnActorName(A)`, `ADoctorName(D)`, `AProfessorName(P)`, and `ASingerName(S)`.
2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:

   `There are a total of [occupation_count] [occupation]s.`

   where `[occupation_count]` is the number of occurrences of an occupation in **OCCUPATIONS** and `[occupation]` is the *lowercase* occupation name. If more than one *Occupation* has the same `[occupation_count]`, they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Input:

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')') FROM OCCUPATIONS ORDER BY
Name ASC;
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ',
LOWER(Occupation), 's.') FROM OCCUPATIONS
```

```
GROUP BY Occupation
ORDER BY COUNT(Occupation) ASC, Occupation ASC;
```

Output:



24. Occupations

Pivot the *Occupation* column in **OCCUPATIONS** so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print **NULL** when there are no more names corresponding to an occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Input:

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
```

```
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn
    FROM OCCUPATIONS
) AS sub
GROUP BY rn;
```

Output:



25. Binary Tree Nodes

You are given a table, *BST*, containing two columns: *N* and *P*, where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	*Type*
N	*Integer*
P	*Integer*

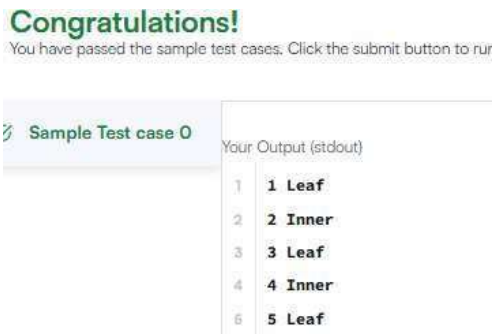
Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Input:

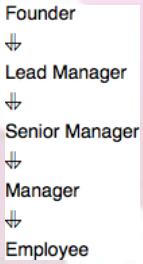
```
SELECT N, CASE WHEN P IS NULL THEN 'Root'
WHEN N IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Inner'
 ELSE 'Leaf' END FROM BST ORDER BY N;
```

Output:



26. New Companies

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

Note:

- The tables may contain duplicate records.
- The *company_code* is string, so the sorting should not be **numeric**. For example, if the *company_codes* are *C_1*, *C_2*, and *C_10*, then the ascending *company_codes* will be *C_1*, *C_10*, and *C_2*.

Input:

```
SELECT
    c.company_code,
    c.founder,
    COUNT(DISTINCT lm.lead_manager_code),
    COUNT(DISTINCT sm.senior_manager_code),
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
```

```
LEFT JOIN Lead_Manager lm ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm ON lm.lead_manager_code = sm.lead_manager_code
LEFT JOIN Manager m ON sm.senior_manager_code = m.senior_manager_code
LEFT JOIN Employee e ON m.manager_code = e.manager_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code ASC;
```

Output:



27. Revising Aggregations - The Count Function

Query a *count* of the number of cities in **CITY** having a *Population* larger than 100,000.

Input Format

The **CITY** table is described as follows:

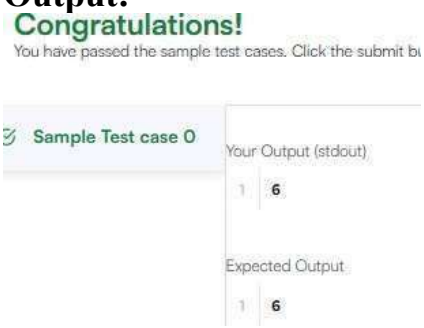
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT COUNT(*) FROM CITY WHERE POPULATION > 100000;
```

Output:



28. Revising Aggregations - Averages

Query the average population of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```

Output:



29. Average Population

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT FLOOR(AVG(POPULATION)) FROM CITY;
```

Output:



30. Japan Population

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



31. Population Density Difference

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```

Output:



32. The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

Note: *Salary* is per month.

Input:

```
SELECT CEIL(AVG(SALARY) - AVG(REPLACE(SALARY, '0', ''))) FROM EMPLOYEES;
```

Output:



33. Top Earners

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

Input:

```
SELECT (salary * months) AS earnings, COUNT(*) FROM Employee GROUP BY earnings ORDER BY earnings DESC LIMIT 1;
```

Output:



34. Weather Observation Station 2

Query the following two values from the **STATION** table:

1. The sum of all values in *LAT_N* rounded to a scale of decimal places.
2. The sum of all values in *LONG_W* rounded to a scale of decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2) FROM STATION;
```

Output:



35. Weather Observation Station 13

Query the sum of *Northern Latitudes* (*LAT_N*) from **STATION** having values greater than and less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SUM(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7880 AND LAT_N <
137.2345;
```

Output:



35. Weather Observation Station 14

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(MAX(LAT_N), 4) FROM STATION WHERE LAT_N < 137.2345;
```

Output:



36. Weather Observation Station 15

Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in **STATION** that is less than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N = (
SELECT MAX(LAT_N) FROM STATION WHERE LAT_N < 137.2345 );
```

Output:



37. Weather Observation Station 16

Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(MIN(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7780;
```

Output:



38. Weather Observation Station 17

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in **STATION** is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N = (
SELECT MIN(LAT_N) FROM STATION WHERE LAT_N > 38.7780
);
```

Output:



39. Weather Observation Station 18

Consider $P_1(a, b)$ and $P_2(c, d)$ to be two points on a 2D plane.

- a happens to equal the minimum value in Northern Latitude (LAT_N in **STATION**).
- b happens to equal the minimum value in Western Longitude (LONG_W in **STATION**).
- c happens to equal the maximum value in Northern Latitude (LAT_N in **STATION**).
- d happens to equal the maximum value in Western Longitude (LONG_W in **STATION**).

Query the Manhattan Distance between points P_1 and P_2 and round it to a scale of 4 decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Input:

```
SELECT ROUND(ABS(MAX(LAT_N) - MIN(LAT_N)) + ABS(MAX(LONG_W) - MIN(LONG_W)),4)
FROM STATION;
```

Output:



40. Weather Observation Station 19

Consider and to be two points on a 2D plane where are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in **STATION**.

Query the Euclidean Distance between points and and *format your answer* to display decimal digits.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SQRT(POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
POWER(MAX(LONG_W)MIN(LONG_W), 2)), 4) FROM STATION;
```

Output:



41. Weather Observation Station 20

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from **STATION** and round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:
```
SELECT ROUND(S.LAT_N, 4)
FROM STATION AS S
WHERE (SELECT COUNT(LAT_N) FROM STATION WHERE LAT_N < S.LAT_N) = (SELECT
COUNT(LAT_N) FROM STATION WHERE LAT_N > S.LAT_N);
```

Output:



42. Population Census

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT SUM(CITY.POPULATION)
 FROM CITY JOIN COUNTRY
```

```
    ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Asia';
```

Output:



43. African Cities

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT CITY.NAME FROM CITYINNER JOIN COUNTRYON CITY.CountryCode =
COUNTRY.Code WHERE COUNTRY.Continent = 'Africa';
```

Output:



44. Average Population of Each Continent

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))FROM CITY
INNER JOIN COUNTRY ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

Output:



45. The Report

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

Input:

```
SELECT
    CASE
        WHEN g.Grade < 8 THEN NULL
        ELSE s.Name
    END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
  ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
```

```
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```

Output:



46. Top Competitors

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Difficulty:* The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum score that can be achieved for a challenge at that difficulty level.

Column	Type
difficulty_level	Integer
score	Integer

- *Challenges:* The *challenge_id* is the id of the challenge, the *hacker_id* is the id of the hacker who created the challenge, and *difficulty_level* is the level of difficulty of the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer
difficulty_level	Integer

Input:

```
SELECT
    h.hacker_id,
    h.name
FROM submissions s
INNER JOIN hackers h
    ON s.hacker_id = h.hacker_id
INNER JOIN challenges c
    ON s.challenge_id = c.challenge_id
INNER JOIN difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```

Output:



47. Ollivander's Inventory

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

- *Wands_Property:* The *code* is the code of the wand, *age* is the age of the wand, and *is_evil* denotes whether the wand is good for the dark arts. If the value of *is_evil* is *0*, it means that the wand is not evil. The mapping between *code* and *age* is one-one, meaning that if there are two pairs, and , then and .

Column	Type
code	Integer
age	Integer
is_evil	Integer

Input:

```
SELECT
    w.id,
    p.age,
    w.coins_needed,
    w.power
FROM
    wands AS w
JOIN
```

```
    wands_property AS p ON w.code = p.code
WHERE
    p.is_evil = 0
    AND w.coins_needed = (
        SELECT
            MIN(w1.coins_needed)
        FROM
            wands AS w1
        JOIN
            wands_property AS p1 ON w1.code = p1.code
        WHERE
            p1.is_evil = 0
            AND w1.power = w.power
            AND p1.age = p.age
    )
ORDER BY
    w.power DESC,
    p.age DESC;
```

Output:



48. Challenges

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the

same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Challenges:* The *challenge_id* is the id of the challenge, and *hacker_id* is the id of the student who created the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer

Input:

```
SELECT H.HACKER_ID,
       H.NAME,
       COUNT(C.CHALLENGE_ID) AS C_COUNT
FROM HACKERS H
JOIN CHALLENGES C ON C.HACKER_ID = H.HACKER_ID
GROUP BY H.HACKER_ID, H.NAME
HAVING C_COUNT =
    (SELECT COUNT(C2.CHALLENGE_ID) AS C_MAX
     FROM CHALLENGES AS C2
     GROUP BY C2.HACKER_ID
     ORDER BY C_MAX DESC LIMIT 1)
OR C_COUNT IN
    (SELECT DISTINCT C_COMPARE AS C_UNIQUE
     FROM (SELECT H2.HACKER_ID,
                  H2.NAME,
                  COUNT(CHALLENGE_ID) AS C_COMPARE
           FROM HACKERS H2
           JOIN CHALLENGES C ON C.HACKER_ID = H2.HACKER_ID
           GROUP BY H2.HACKER_ID, H2.NAME) COUNTS
     GROUP BY C_COMPARE
     HAVING COUNT(C_COMPARE) = 1)
```

```
ORDER BY C_COUNT DESC, H.HACKER_ID;
```

Output:



49. Contest Leaderboard

You did such a great job helping Julia with her last oding contest challenge that she wants you to work on this one, too.The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

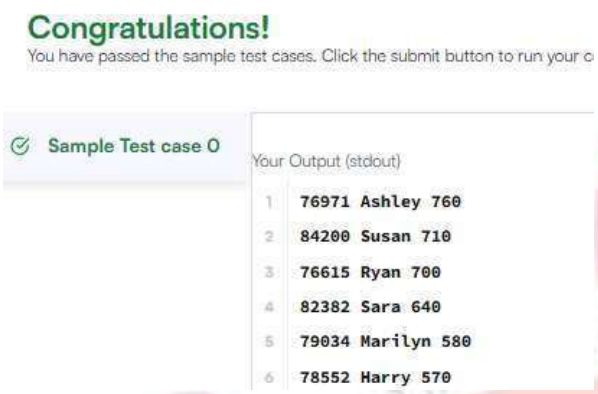
- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge for which the submission belongs to, and *score* is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Input:

```sql
SELECT X.hacker_id,
(SELECT H.NAME FROM HACKERS H
                    WHERE H.HACKER_ID = X.HACKER_ID) NAME,
SUM(X.SCORE) TOTAL_SCORE FROM
  (SELECT HACKER_ID, MAX(SCORE) SCORE FROM SUBMISSIONS S
   GROUP BY 1, S.CHALLENGE_ID) X
GROUP BY 1
HAVING TOTAL_SCORE > 0
ORDER BY TOTAL_SCORE DESC, HACKER_ID;
```

Output:



50. SQL Project Planning

You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

Sample Output

```
2015-10-28  2015-10-29
2015-10-30  2015-10-31
2015-10-13  2015-10-15
2015-10-01  2015-10-04
```

Input:

```
SELECT Start_Date, MIN(End_Date) AS Min_End_Date
FROM (
    SELECT Start_Date
    FROM Projects
    WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)
) AS a
JOIN (
    SELECT End_Date
    FROM Projects
    WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)
) AS b ON a.Start_Date < b.End_Date
GROUP BY Start_Date
ORDER BY DATEDIFF(MIN(End_Date), Start_Date) ASC, Start_Date ASC;
```

Output:



51. Placements

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

Sample Input

ID	Friend_ID
1	2
2	3
3	4
4	1

Friends

ID	Name
1	Ashley
2	Samantha
3	Julia
4	Scarlet

Students

ID	Salary
1	15.20
2	10.06
3	11.55
4	12.12

Packages

Sample Output

```
Samantha
Julia
Scarlet
```

Explanation

See the following table:

ID	1	2	3	4
Name	Ashley	Samantha	Julia	Scarlet
Salary	15.20	10.06	11.55	12.12
Friend ID	2	3	4	1
Friend Salary	10.06	11.55	12.12	15.20

Now,

- *Samantha's* best friend got offered a higher salary than her at 11.55
- *Julia's* best friend got offered a higher salary than her at 12.12
- *Scarlet's* best friend got offered a higher salary than her at 15.2
- *Ashley's* best friend did NOT get offered a higher salary than her

The name output, when ordered by the salary offered to their friends, will be:

- *San*
- *Juli*
- *Sca*

Input

```
SELEC
FROM S
JOIN
JOIN F
JOIN PACKAGES P2 ON F.FRIEND_ID = P2.ID
WHERE P2.SALARY > P1.SALARY
ORDER BY P2.SALARY;
```

Output:



52. Symmetric Pairs

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	*Type*
X	*Integer*
Y	*Integer*

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

Sample Input

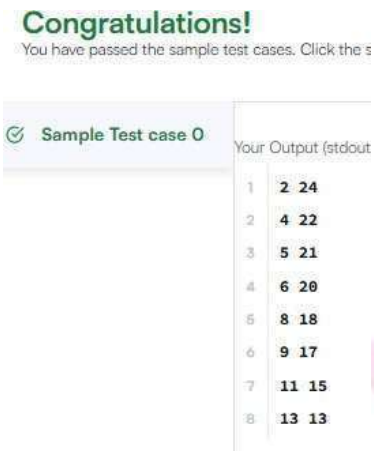
X	*Y*
20	*20*
20	*20*
20	*21*
23	*22*
22	*23*
21	*20*

Input:

```
SELECT f1.X, f1.Y FROM Functions f1
INNER JOIN Functions f2 ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
```

```
HAVING f1.X < f1.Y OR COUNT(f1.X) > 1
ORDER BY f1.X;
```

Output:



53. 15 Days of Learning SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Input:

```
SELECT SUBMISSION_DATE,
(SELECT COUNT(DISTINCT HACKER_ID)
```

```
 FROM SUBMISSIONS S2
 WHERE S2.SUBMISSION_DATE = S1.SUBMISSION_DATE AND
(SELECT COUNT(DISTINCT S3.SUBMISSION_DATE)
 FROM SUBMISSIONS S3 WHERE S3.HACKER_ID = S2.HACKER_ID AND S3.SUBMISSION_DATE
< S1.SUBMISSION_DATE) = DATEDIFF(S1.SUBMISSION_DATE , '2016-03-01')),
(SELECT HACKER_ID FROM SUBMISSIONS S2 WHERE S2.SUBMISSION_DATE =
S1.SUBMISSION_DATE
GROUP BY HACKER_ID ORDER BY COUNT(SUBMISSION_ID) DESC, HACKER_ID LIMIT 1) AS
TMP,
(SELECT NAME FROM HACKERS WHERE HACKER_ID = TMP)
FROM
(SELECT DISTINCT SUBMISSION_DATE FROM SUBMISSIONS) S1
GROUP BY SUBMISSION_DATE;
```

Output:



54. Draw The Triangle 1

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

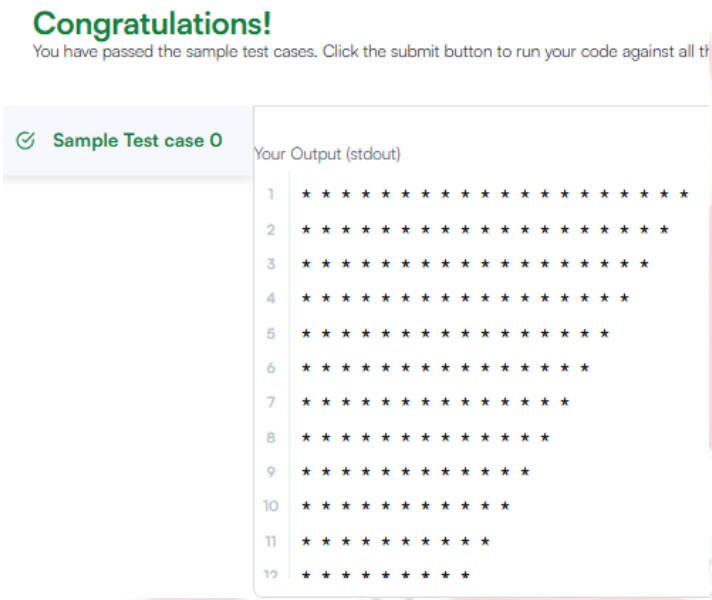
```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern *P(20)*.

Input:

```
SET @TEMP := 21;
SELECT REPEAT('* ', @TEMP := @TEMP - 1)
FROM INFORMATION_SCHEMA.TABLES;
```

Output:



55. Draw The Triangle 2

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

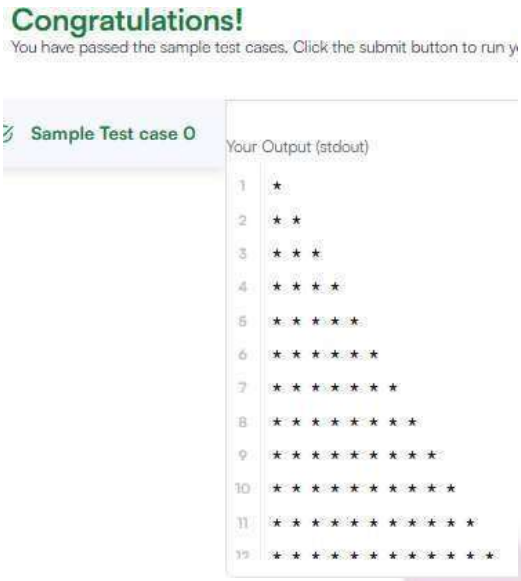
```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern *P(20)*.

Input:

```
SET @number = 0;
SELECT REPEAT('* ', @number := @number + 1)
FROM information_schema.tables
WHERE @number < 20;
```

Output:



56. Print Prime Numbers

Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

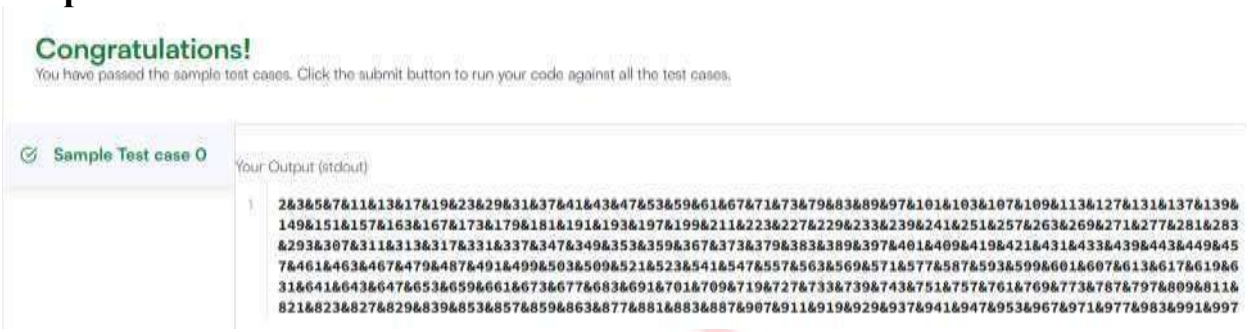
For example, the output for all prime numbers would be:

2&3&5&7

Input:

```
WITH RECURSIVE nums AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM nums p
WHERE NOT EXISTS (
    SELECT 1 FROM nums d WHERE d.n <= SQRT(p.n) AND d.n > 1 AND p.n % d.n = 0
);
```

Output:



57. Interviews

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Input:

```
SELECT t1.contest_id, t1.hacker_id, t1.name,
       SUM(ts) as ts, SUM(tas) as tas, SUM(tv) as tv, SUM(tuv) as tuv
FROM CONTESTS AS t1
LEFT JOIN COLLEGES AS t2
    ON t1.contest_id = t2.contest_id
LEFT JOIN CHALLENGES AS t3
    ON t2.college_id = t3.college_id
LEFT JOIN (SELECT challenge_id,
           SUM(total_views) AS tv, SUM(total_unique_views) as tuv
           FROM View_Stats
           GROUP BY challenge_id) AS t4
    ON t3.challenge_id = t4.challenge_id
LEFT JOIN (SELECT challenge_id,
           SUM(total_submissions) AS ts, SUM(total_accepted_submissions) as
tas
           FROM Submission_Stats
           GROUP BY challenge_id) as t5
    ON t3.challenge_id = t5.challenge_id
GROUP BY t1.contest_id, t1.hacker_id, t1.name
HAVING (ts + tas + tv + tuv > 0)
ORDER BY t1.contest_id;
```

Output:


Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all
Sample Test case 0
Your Output (stdout)
845 579 Rose 1987 580 1635 566
858 1053 Angela 703 160 1002 384
883 1055 Frank 1121 319 1217 338
1793 2655 Patrick 1337 360 1216 412
2374 2765 Lisa 2733 815 3368 984
2963 2845 Kimberly 4306 1221 3603 1184
3584 2873 Bonnie 2492 652 3019 954
4044 3067 Michael 1323 449 1722 528
4249 3116 Todd 1452 376 1767 463
4269 3256 Joe 1018 372 1766 530
4483 3386 Earl 1911 572 1644 477

Certificate of Accomplishment

SQL (Intermediate)

PRESENTED TO

Udhayavani M

The bearer of this certificate has passed the HackerRank skill certification test

Earned on: **03 Aug, 2026**

ID: B642BC14637A

Harishankaran K

CTO, HackerRank

Certificate of Accomplishment

SQL (Basic)

PRESENTED TO

Udhayavani M

The bearer of this certificate has passed the HackerRank skill certification test

Earned on: **02 Aug, 2026**

ID: 7E7AA9322BCE

Harishankaran K

CTO, HackerRank

Certificate of Accomplishment

SQL (Advanced)

PRESENTED TO

Udhayavani M

The bearer of this certificate has passed the HackerRank skill certification test

Earned on: **03 Aug, 2026**

ID: 769B12A664AC

Harishankaran K

CTO, HackerRank

CODECHEF

BASIC COMMANDS

1. ALL PRODUCTS

Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

select*from products;

OUTPUT

Your Output



product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture

✓ Excellent work!

2. HIGH PRICE OF PRODUCTS

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: **Products**

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

SELECT product_name, category FROM Products WHERE price > 100.00;

OUTPUT

Your Output

product_name	category
Laptop	Electronics
Desk Chair	Furniture
Smartphone	Electronics
Standing Desk	Furniture
Tablet	Electronics

✔ Excellent work!

3. AVERAGE SALARY

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

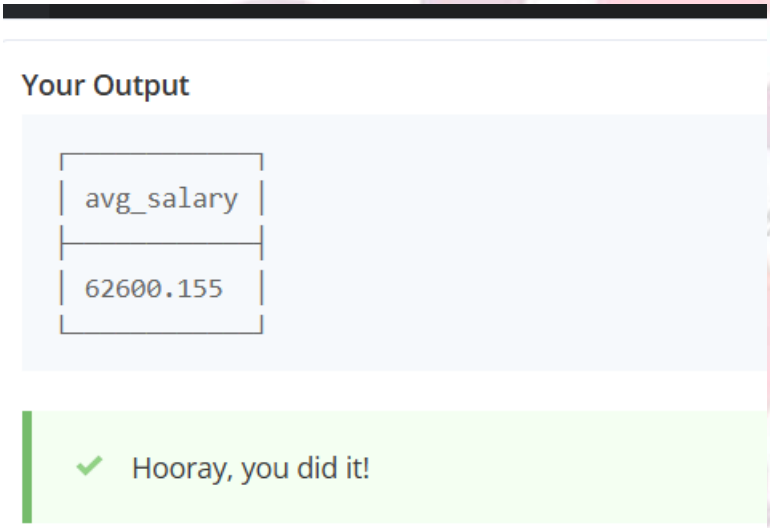
employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00

5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

SELECT AVG(salary) AS avg_salary FROM Works;

OUTPUT



4. LOCATE PEOPLE

Task

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

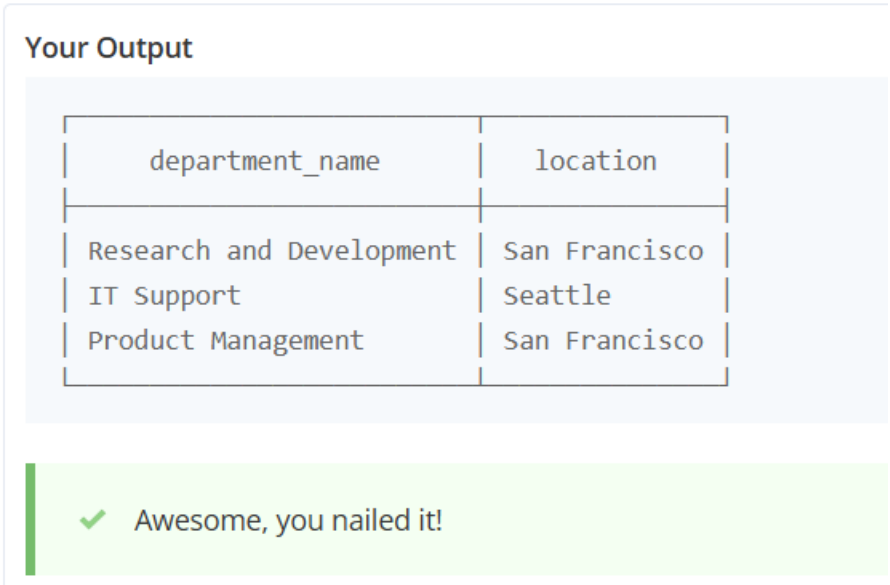
Table name: **departments**

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

INPUT

SELECT department_name, location FROM departments WHERE location LIKE 'S%';

OUTPUT



5. DISTINCT COMPANIES

Task

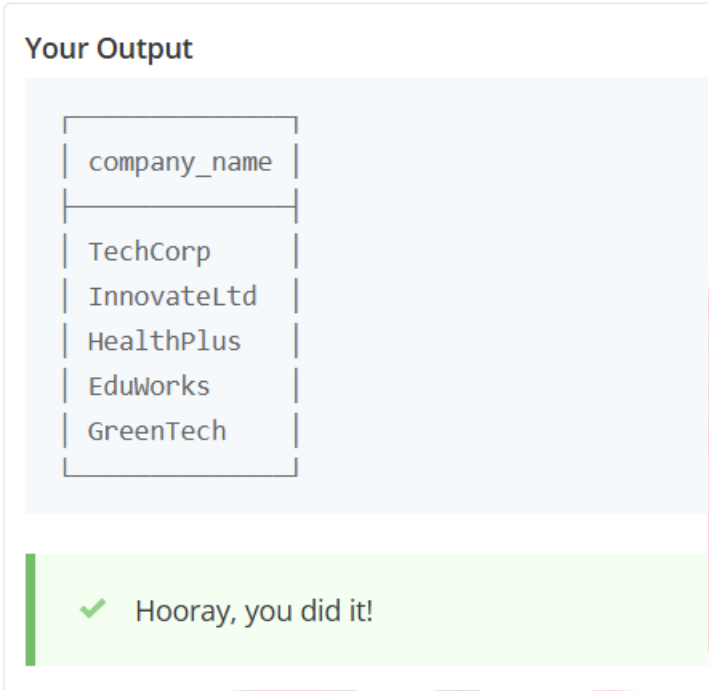
Write a query to select all the distinct companies (company_name) in the Works table.
Table name: **Works**

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

```
SELECT DISTINCT company_name FROM Works;
```

OUTPUT



6. FICTION COLLECTION SIZE

Task

Write a query to find the total count of books whose genre is **Fiction**.
Note: Output column name should be **fiction_count**.

Table name: **Books**

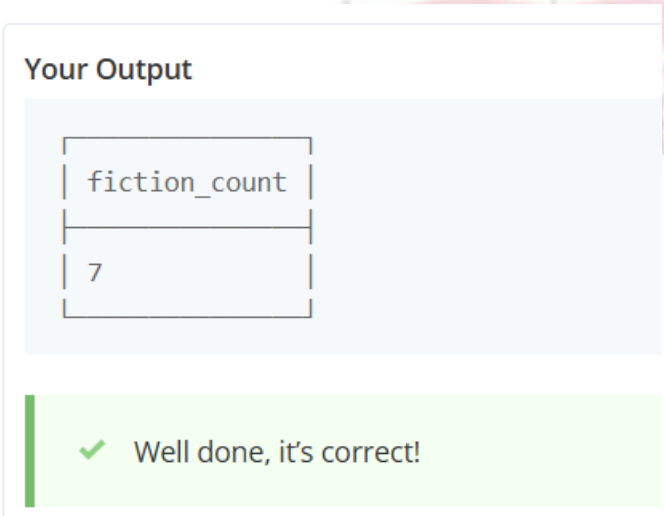
id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932

6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

INPUT

SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';

Output



7. LIST OF MOVIES WITH RATINGS

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: **Cinema**

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

INPUT

SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.

OUTPUT



8. HANDLING NULL VALUES

Task

Write a query to retrieve book_id, title, author and published_year of the books which have **NULL** rating for their books.

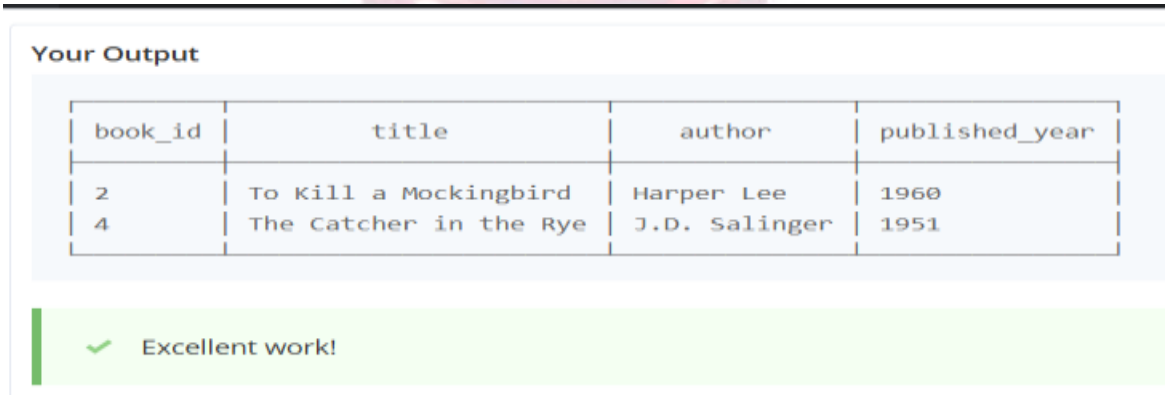
Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

INPUT

select book_id, title, author, published_year from library where rating is null;

OUTPUT



9. SALARY OF EMPLOYEES

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in **descending** order

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

```
SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time'
ORDER BY salary DESC;
```

OUTPUT

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

✓ You got it right!

10. DEPARTMENT OF EACH EMPLOYEE

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

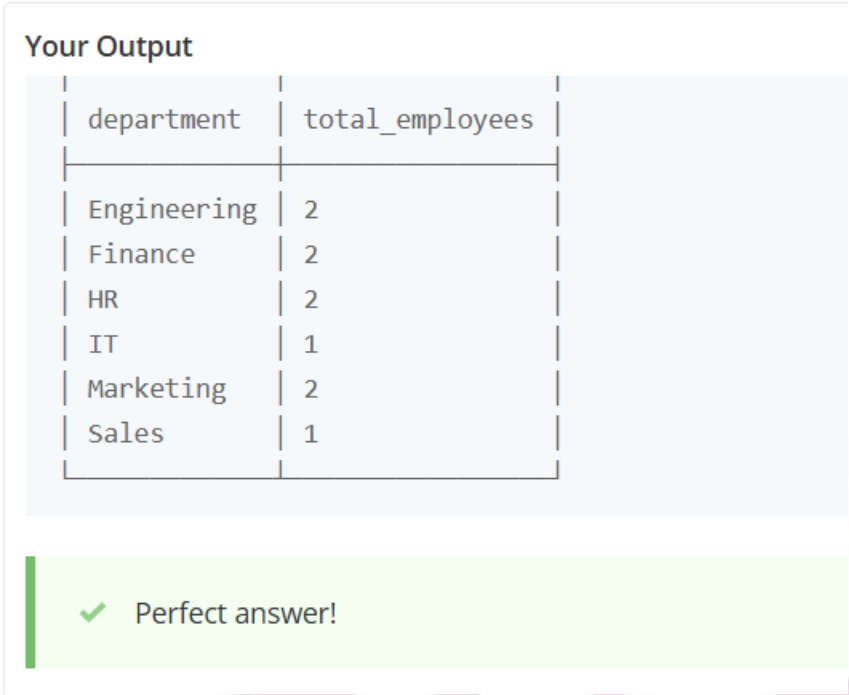
Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

```
select department, count(*) as total_employees from employees group by department;
```

OUTPUT



11. ARTICLE VIEWS

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in **ascending order**.

Table name: Views

article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times

107	2	Jane Smith	2	2	Global News

INPUT

select distinct author_id, author_name, publication_name from views where view_count = 0 order by author_id asc;

OUTPUT



12. PLAYER PERFORMANCE INSIGHTS

Task

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1

5	Eve	1100	6

Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

INPUT

select distinct player_name, score from players join matches where winner=player_name

order by score desc limit 3;

OUTPUT

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

✔ Awesome, you nailed it!

13. PLAYER DETAILS

Task

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **Players** and **Matches**.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

INPUT

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT

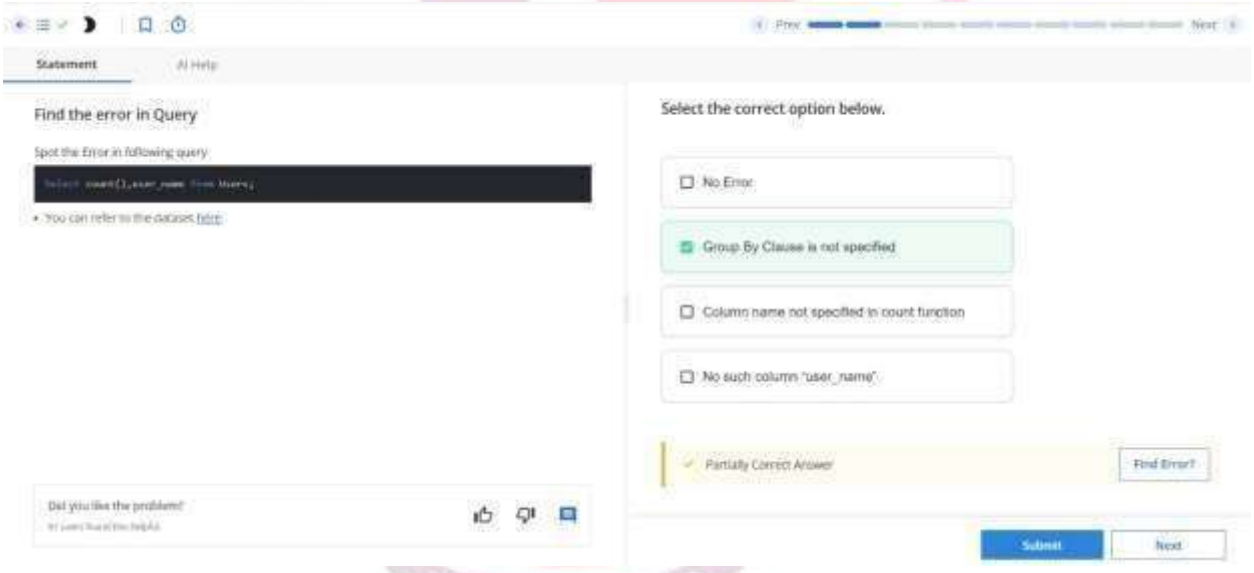
Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

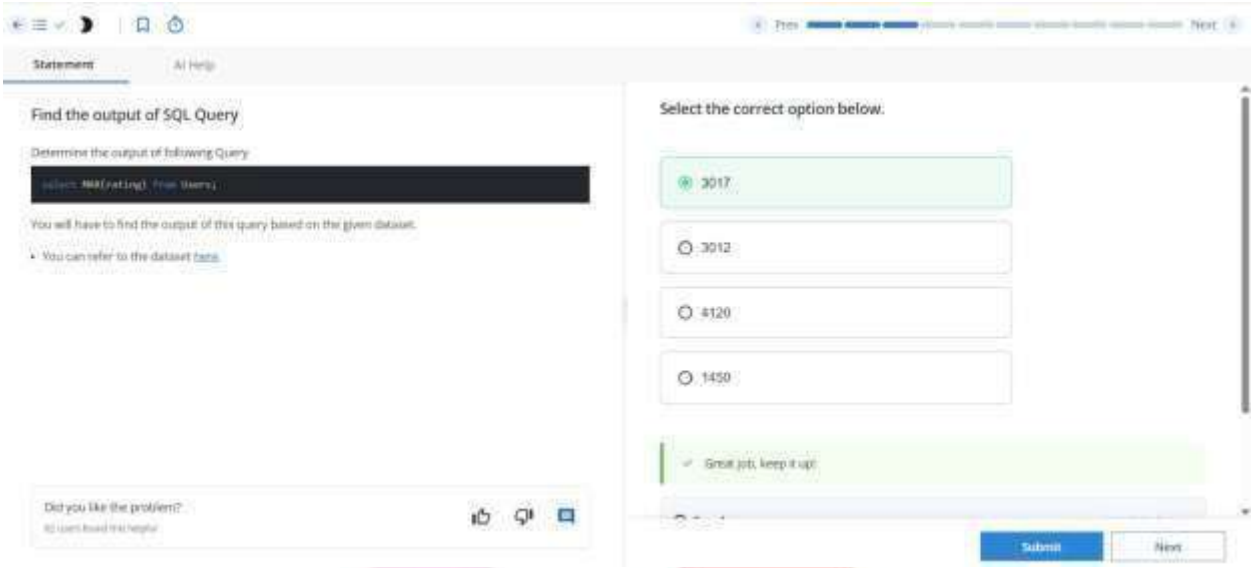
✔ Awesome, you nailed it!

CASE STUDY – CODECHEF DATABASE

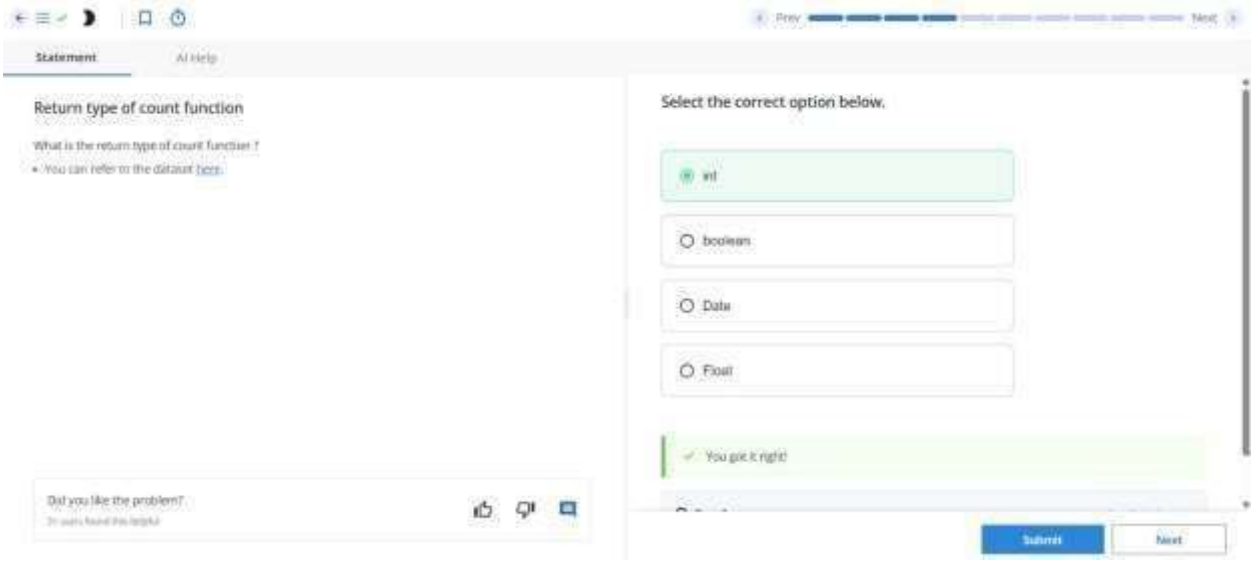
1.FIND THE ERROR IN QUERY



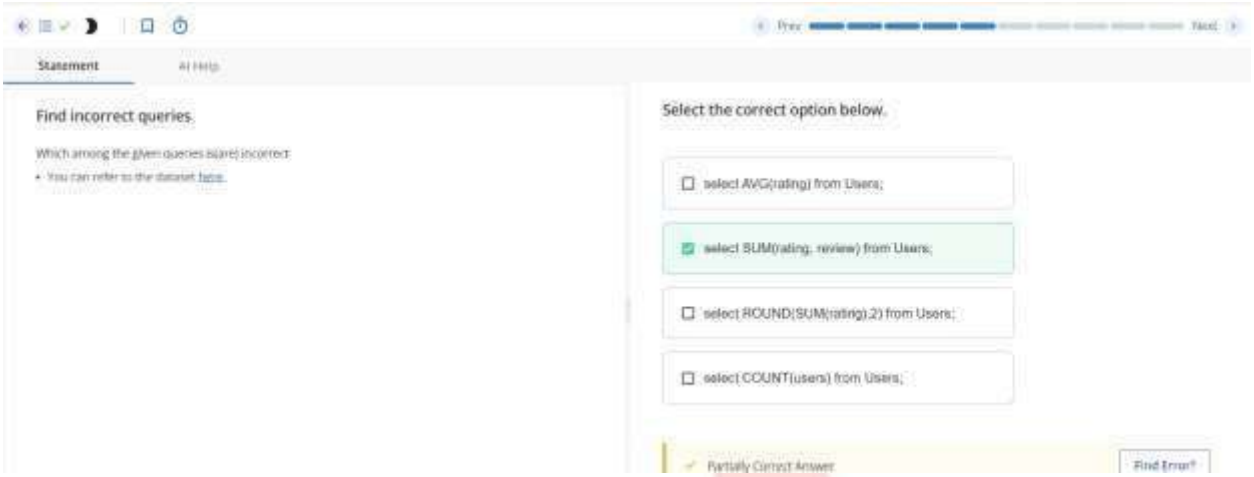
2. FIND THE OUTPUT OF SQL QUERY



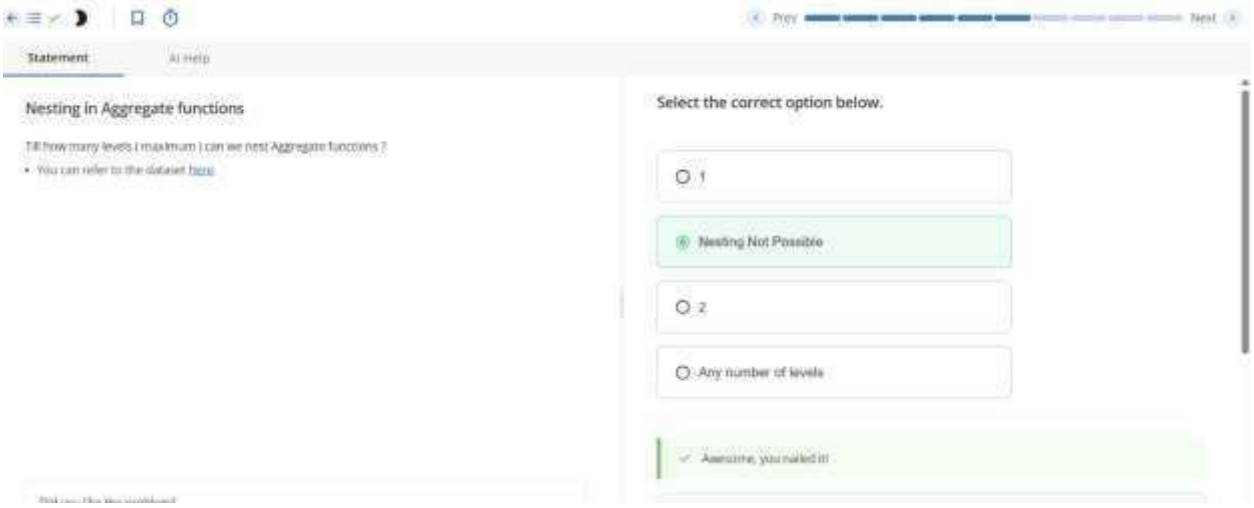
3. RETURN TYPE OF COUNT FUNCTION



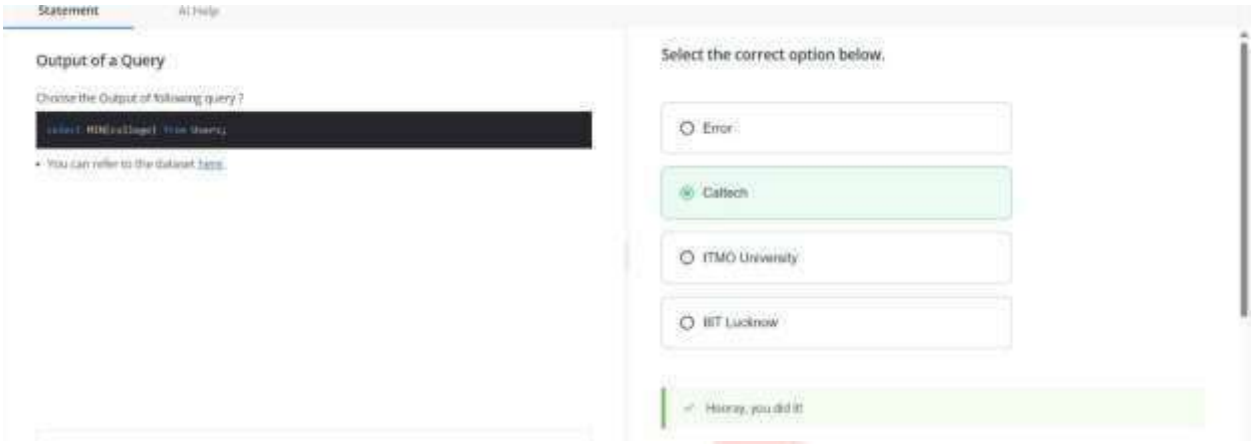
4. FIND INCORRECT QUERIES



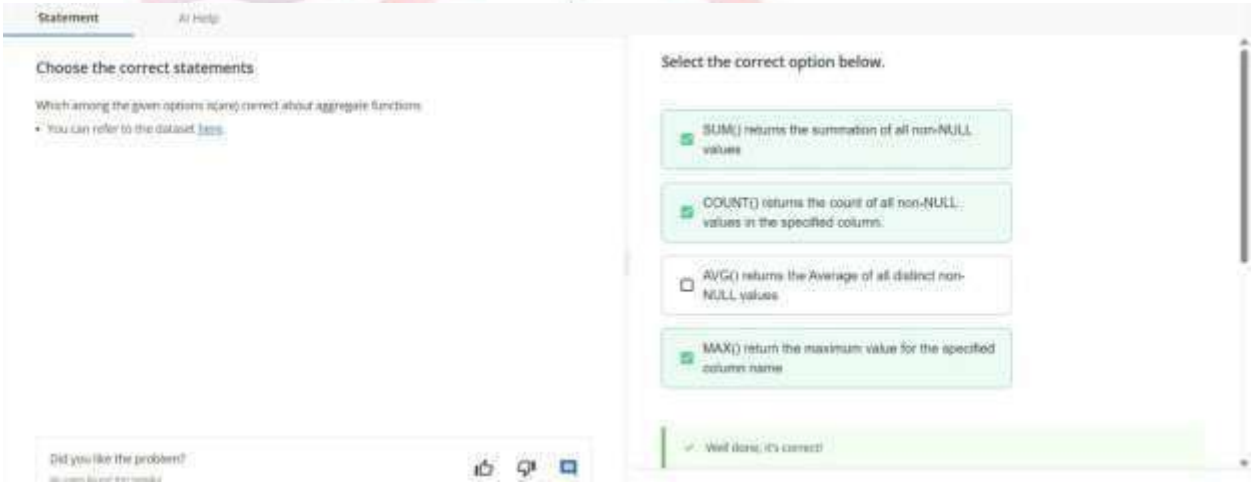
5. NESTED IN AGGREGATE FUNCTIONS



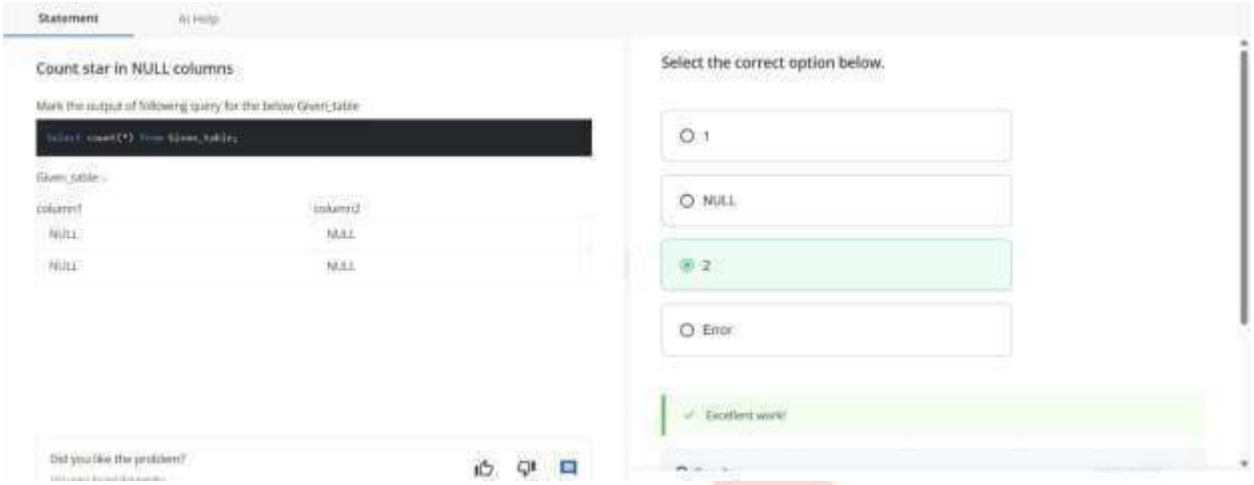
6. OUTPUT OF A QUERY



7. CHOOSE THE CORRECT STATEMENTS

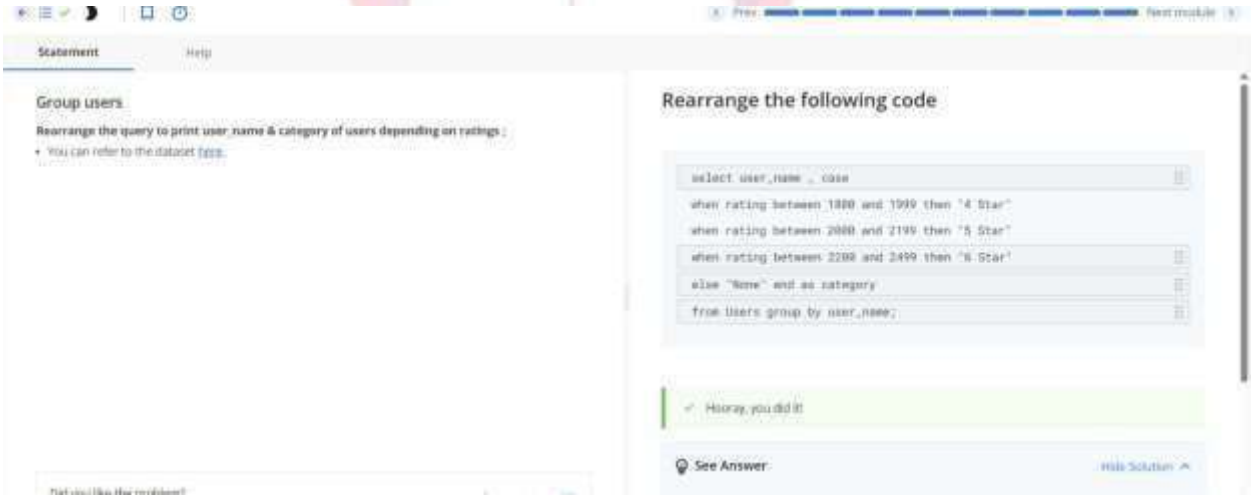


8. COUNT STAR IN NULL COLUMNS



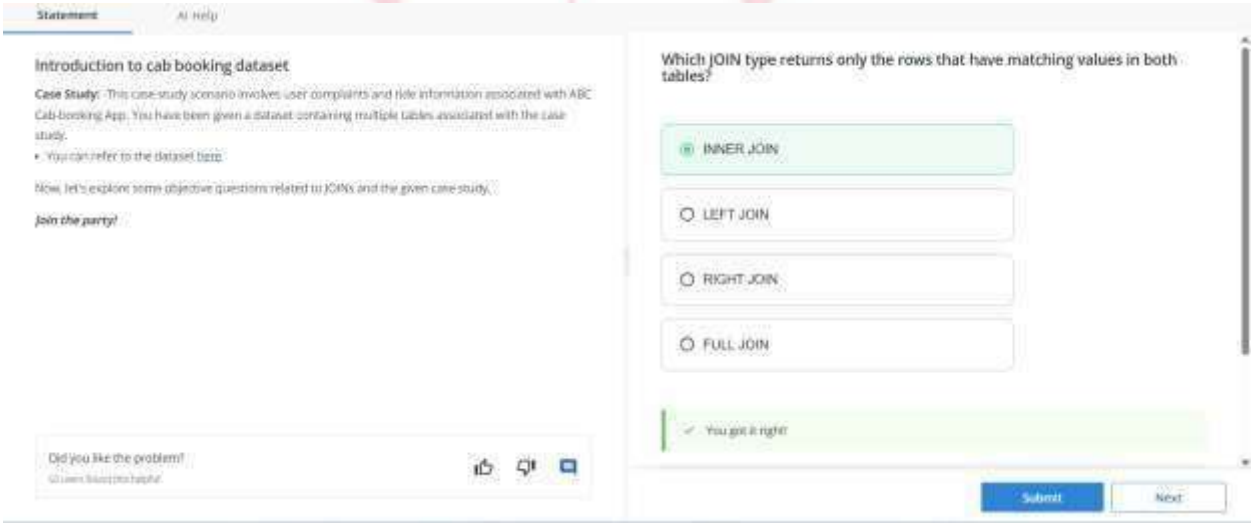
TRUTH-TOIL-TRIUMPH

9. GROUP USERS

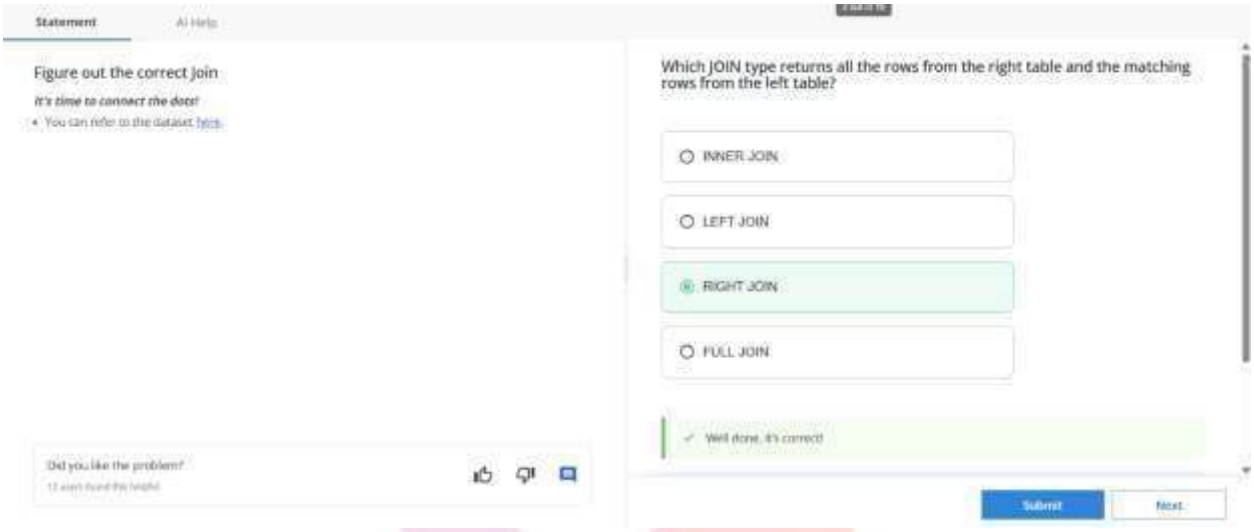


CASE STUDY – ABC CAB BOOKIN APP

1.INTRODUCTION TO CAB BOOKING DATASET

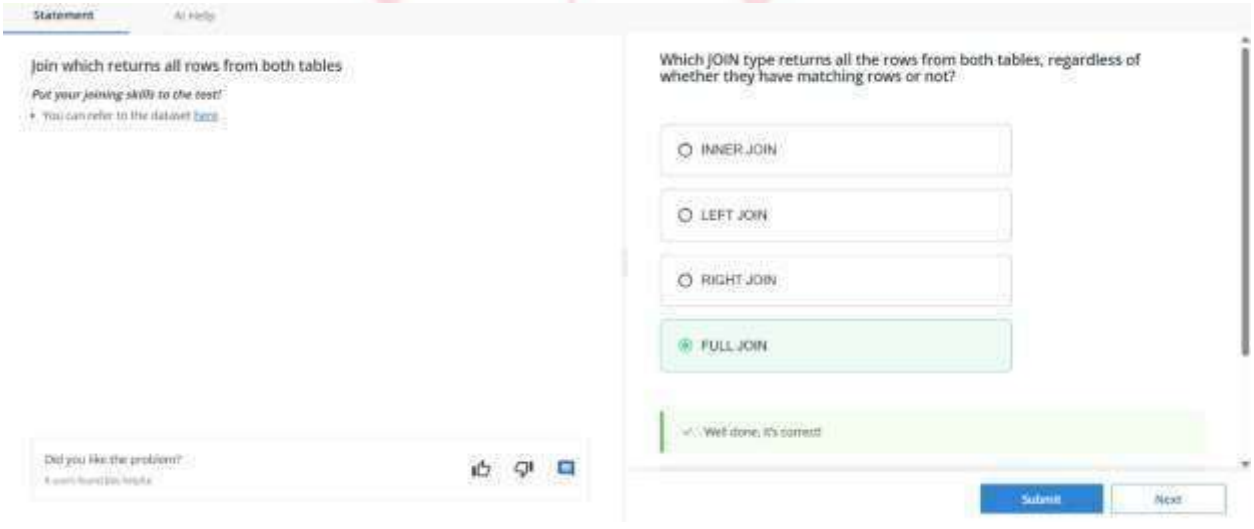


2. FIGURE OUT THE CORRECT JOIN

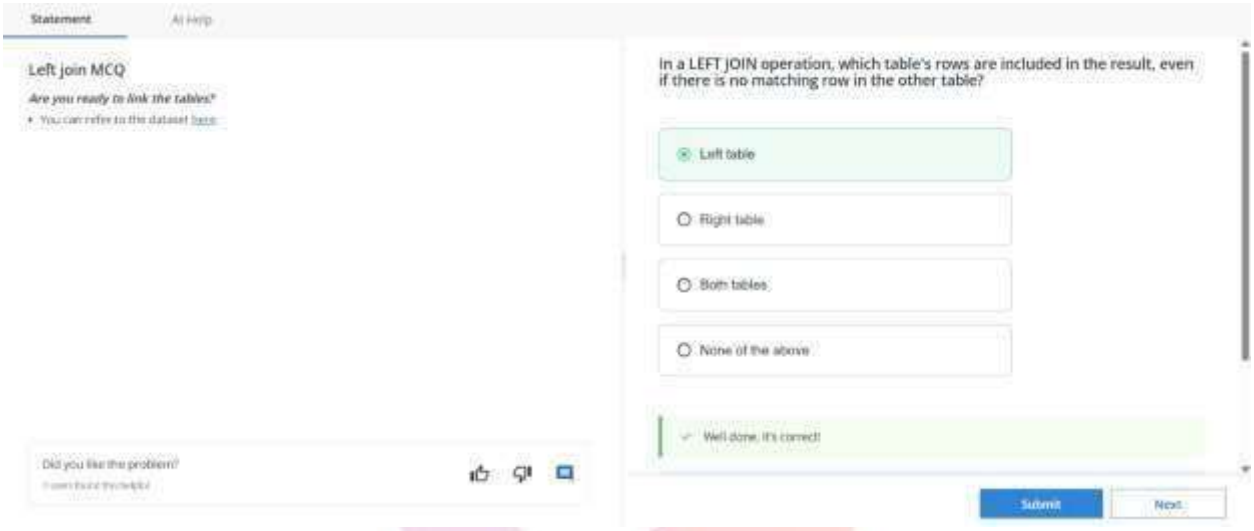


TRUTH-TOIL-TRIUMPH

3. JOIN WHICH RETURN ALL ROWS FROM BOTH TABLES

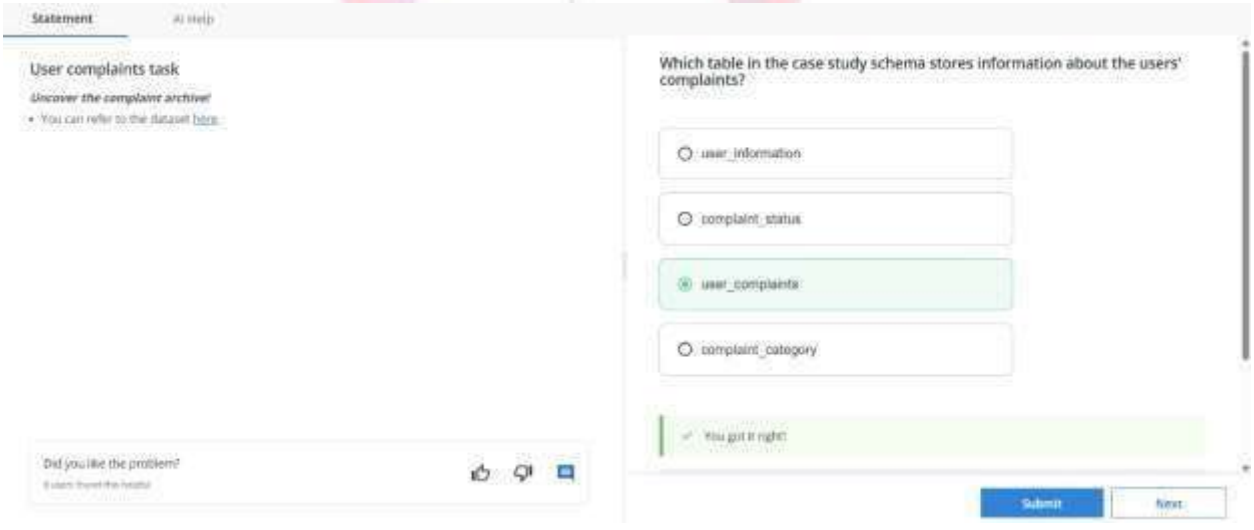


4. LEFT JOIN MCQ

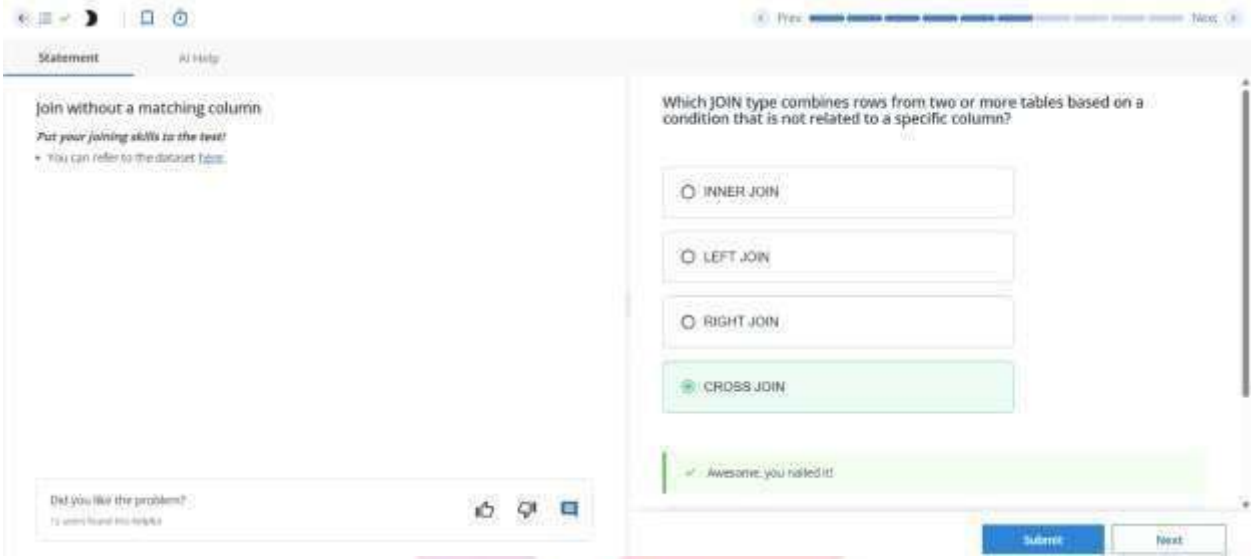


TRUTH-TOIL-TRIUMPH

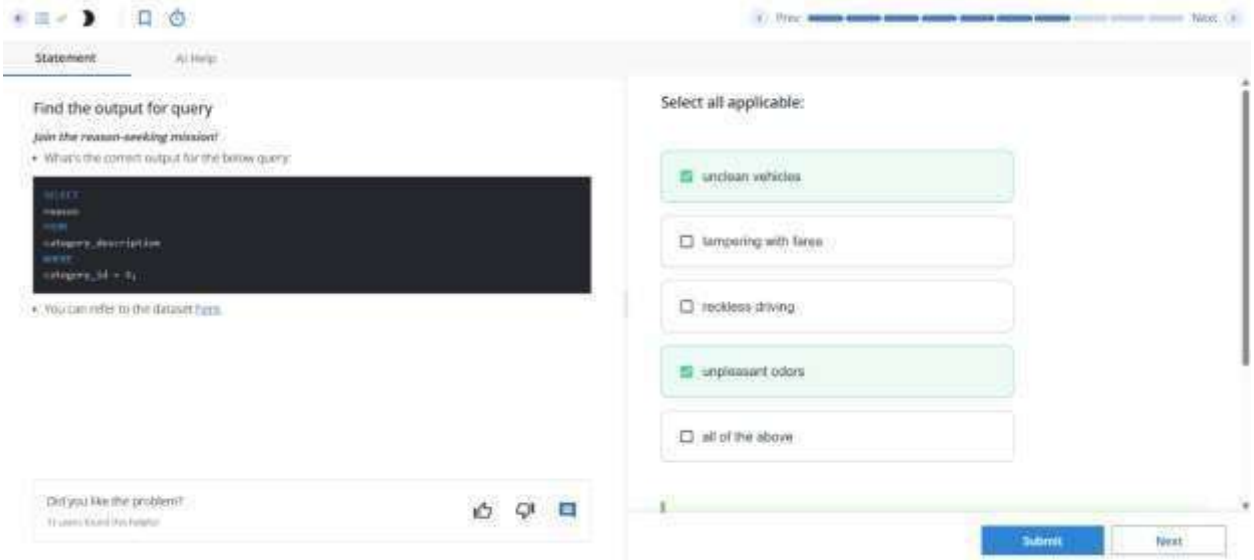
5. USER COMPLAINTS TASK



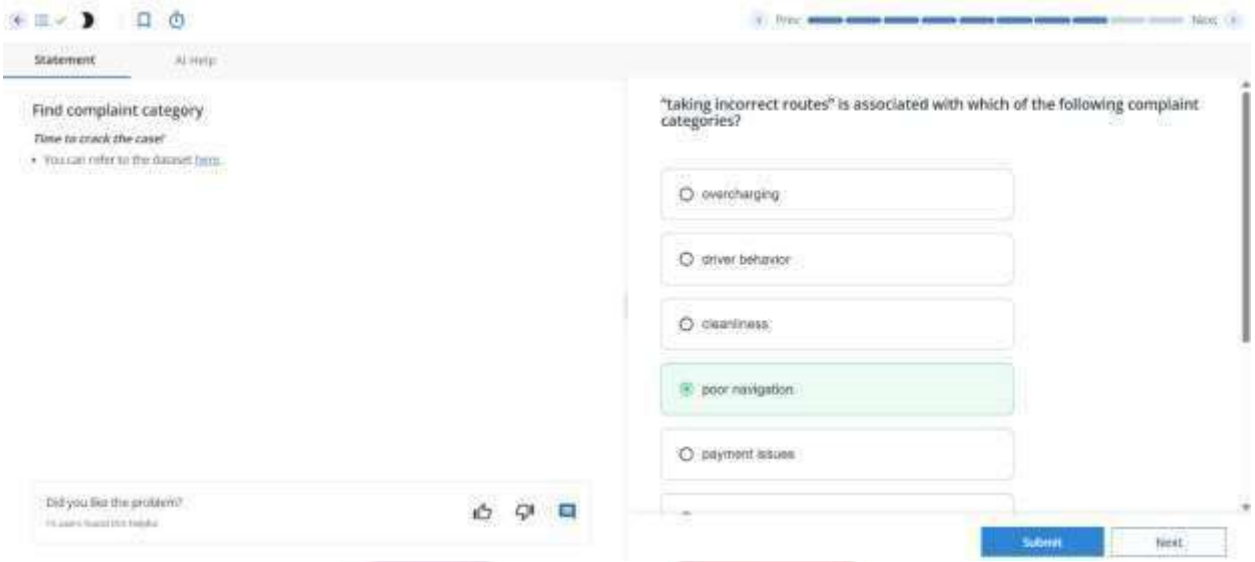
6. JOIN WITHOUT A MATCHING COLUMN



7. FIND THE OUTPUT FOR QUERY



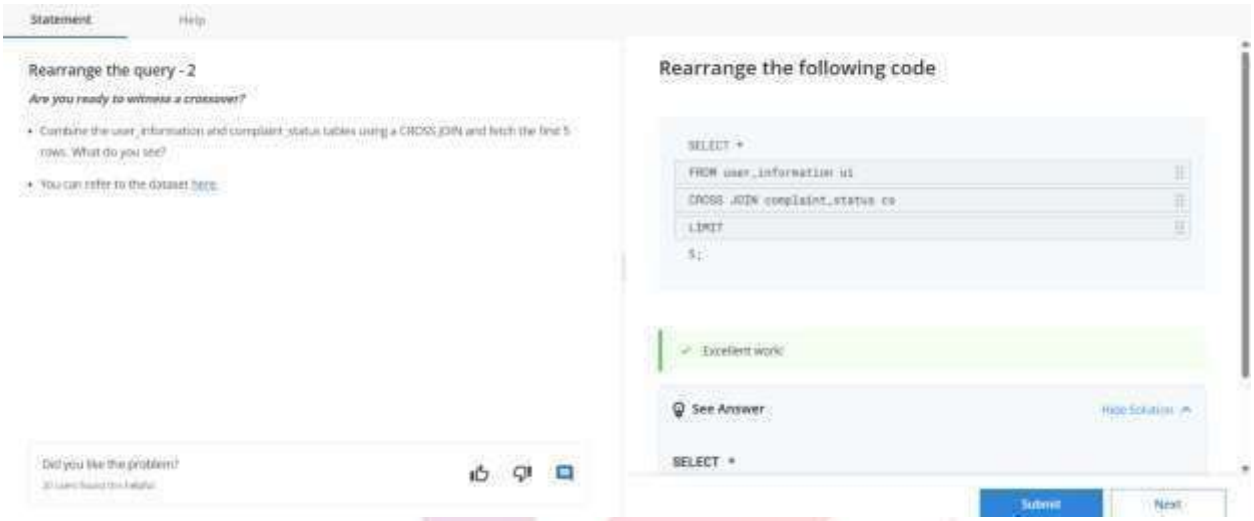
8. FIND COMPLAINT CATEGORY



9. REARRANGE THE QUERY - 1



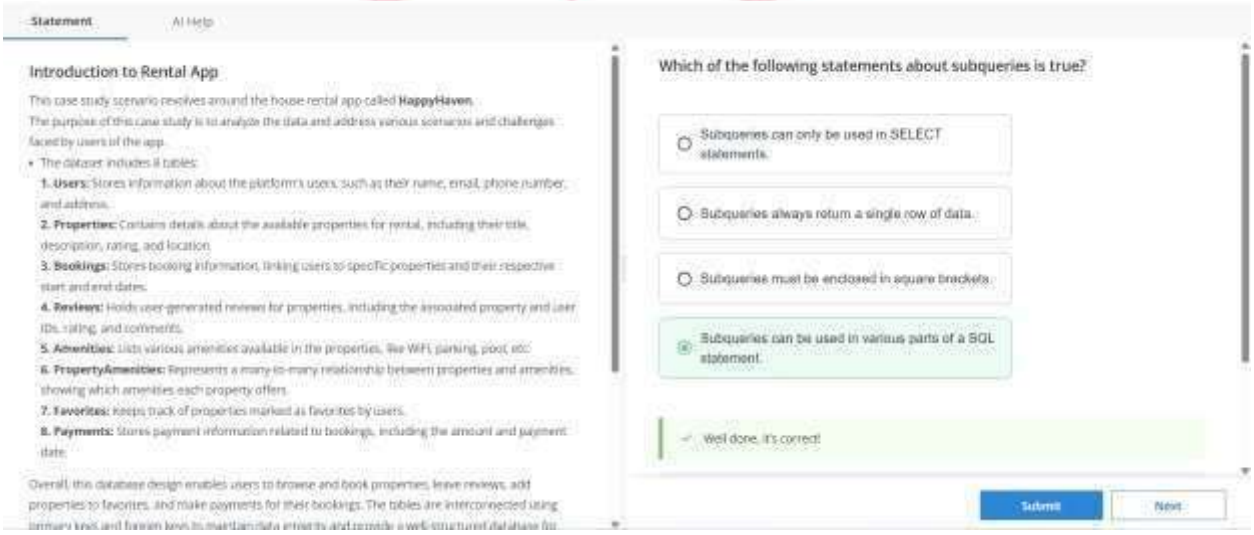
10. REARRANGE THE QUERY - 2



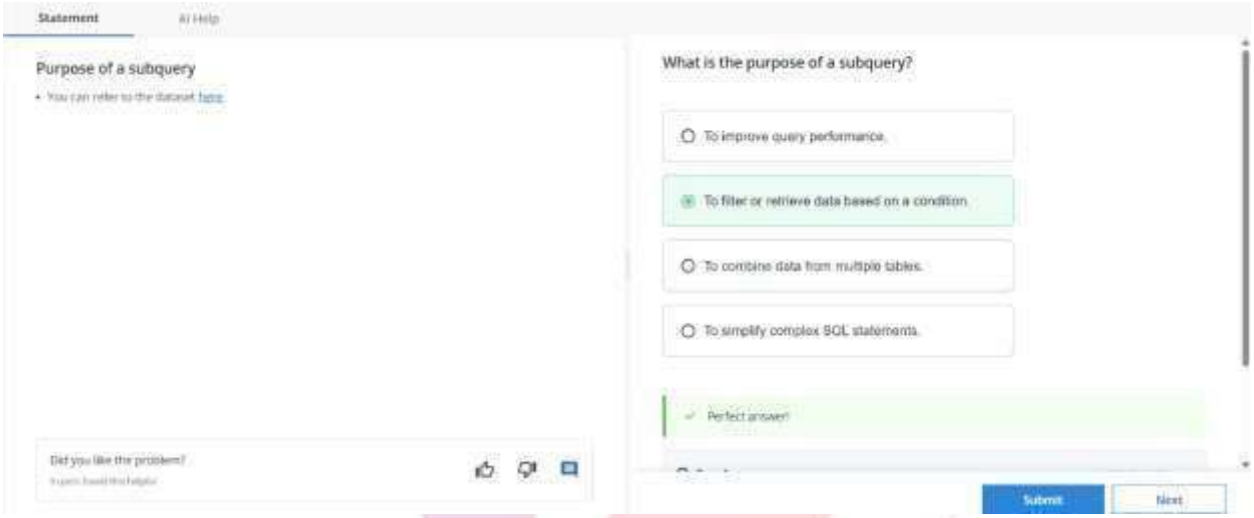
TRUTH-TOIL-TRIUMPH

CASE STUDY – HAPPY HAVEN HOUSE RENTAL APP

1.INTRODUCTION TO RENTAL APP

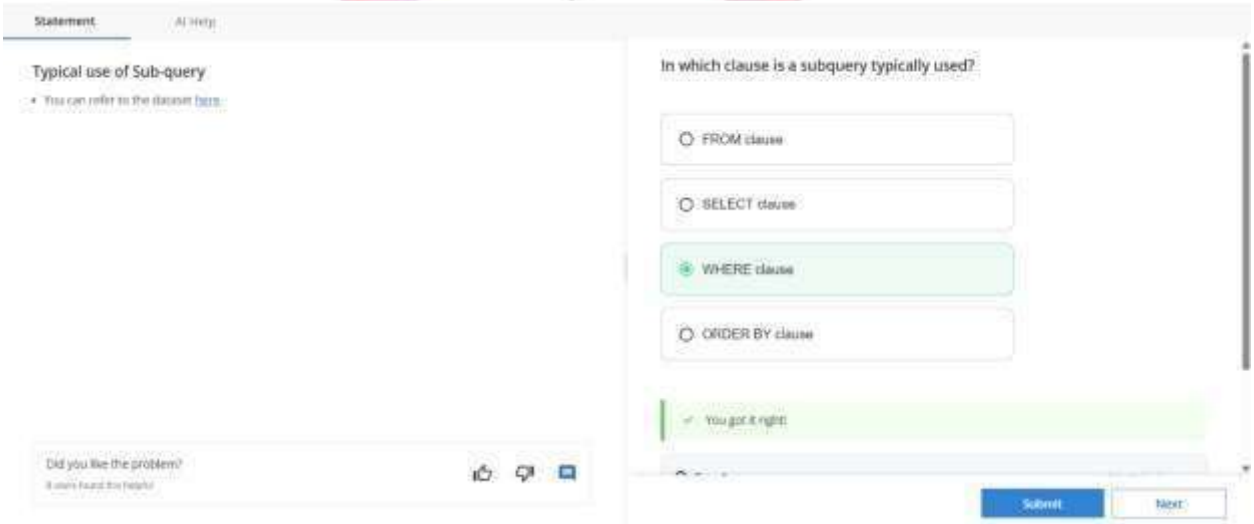


2. PURPOSE OF A SUBQUERY

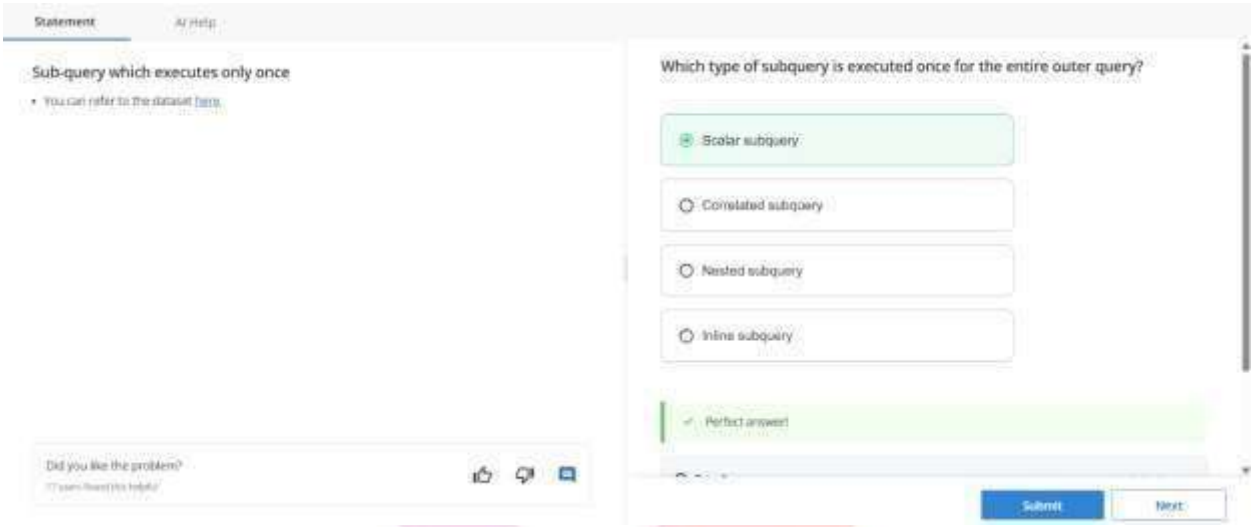


TRUTH-TOIL-TRIUMPH

3. TYPICAL USE OF SUB-QUERY

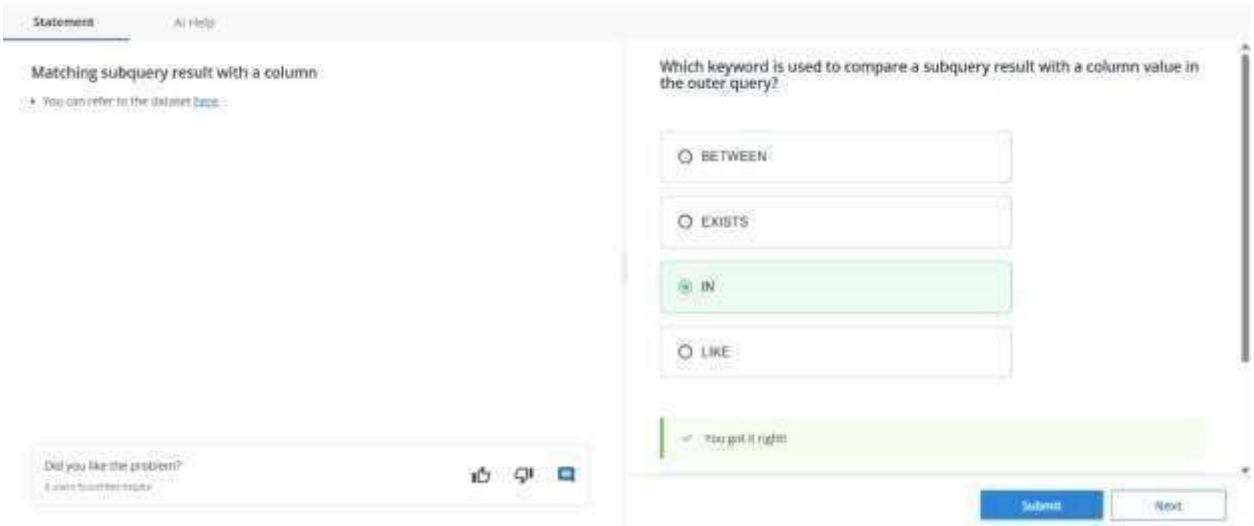


4. SUB-QUERY WHICH EXECUTES ONLY ONCE

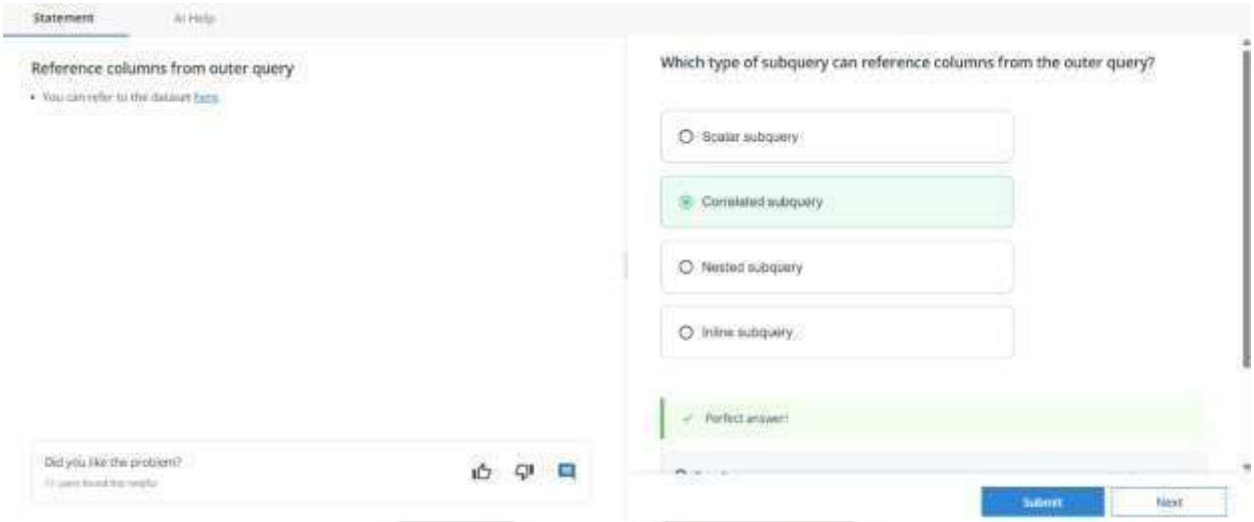


TRUTH-TOIL-TRIUMPH

5. MATCHING SUBQUERY RESULT WITH A COLUMN

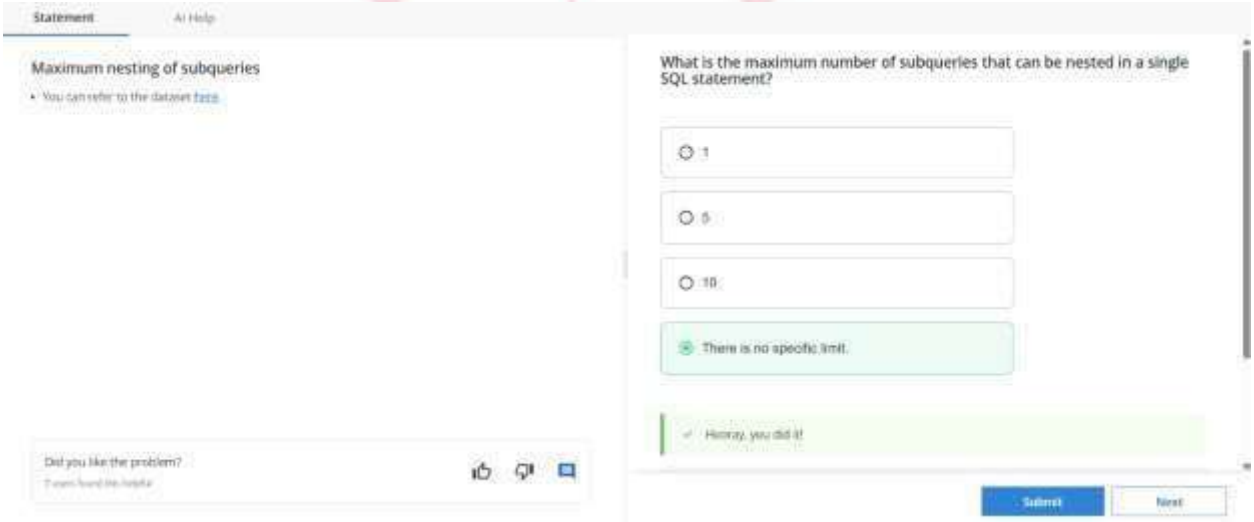


6. REFERENCE COLUMNS FROM OUTER QUERY

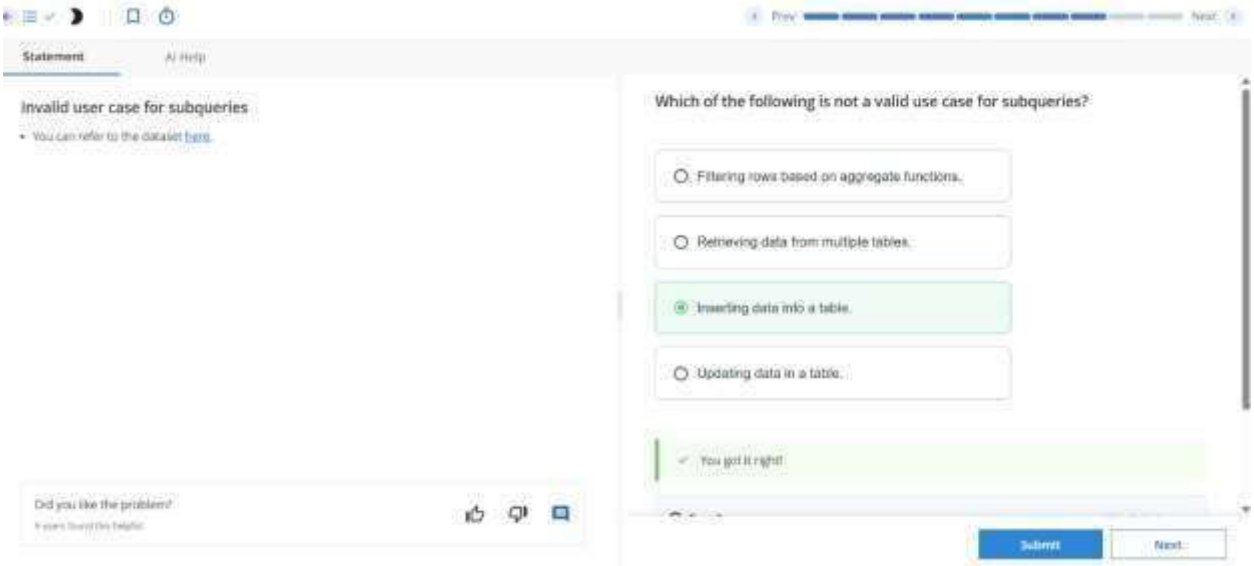


TRUTH-TOIL-TRIUMPH

7. MAXIMUM NESTING OF SUBQUERIES

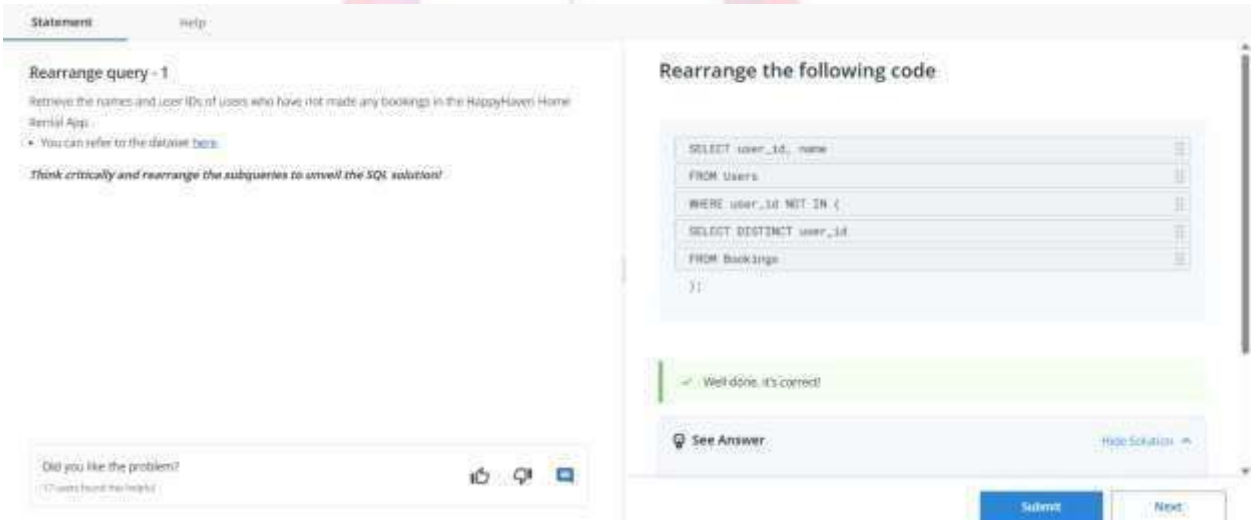


8. INVALID USER CASE FOR SUBQUERIES

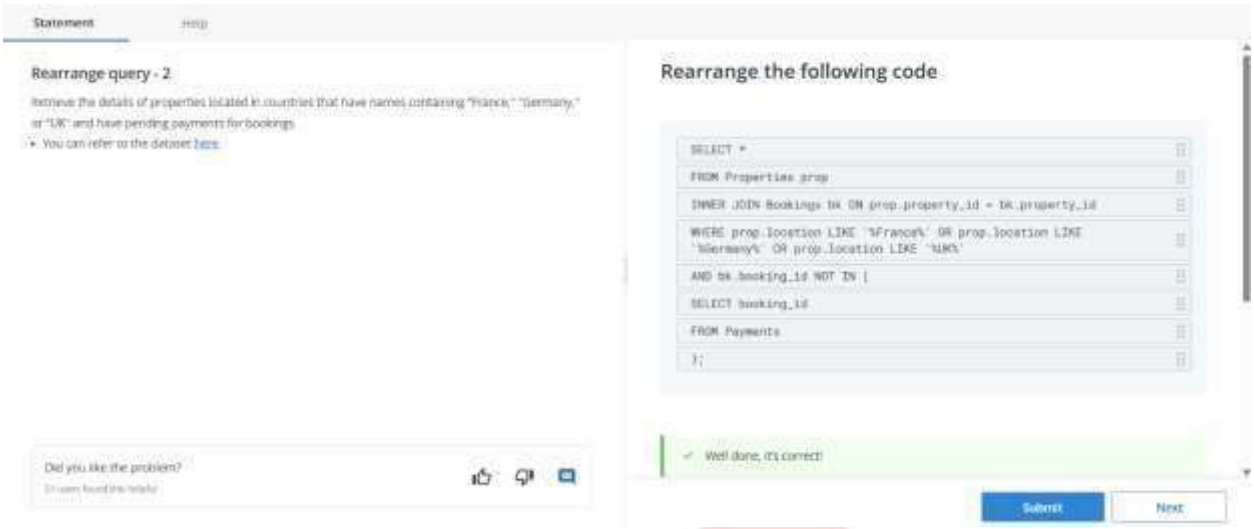


TRUTH-TOIL-TRIUMPH

9. REARRANGE QUERY-1



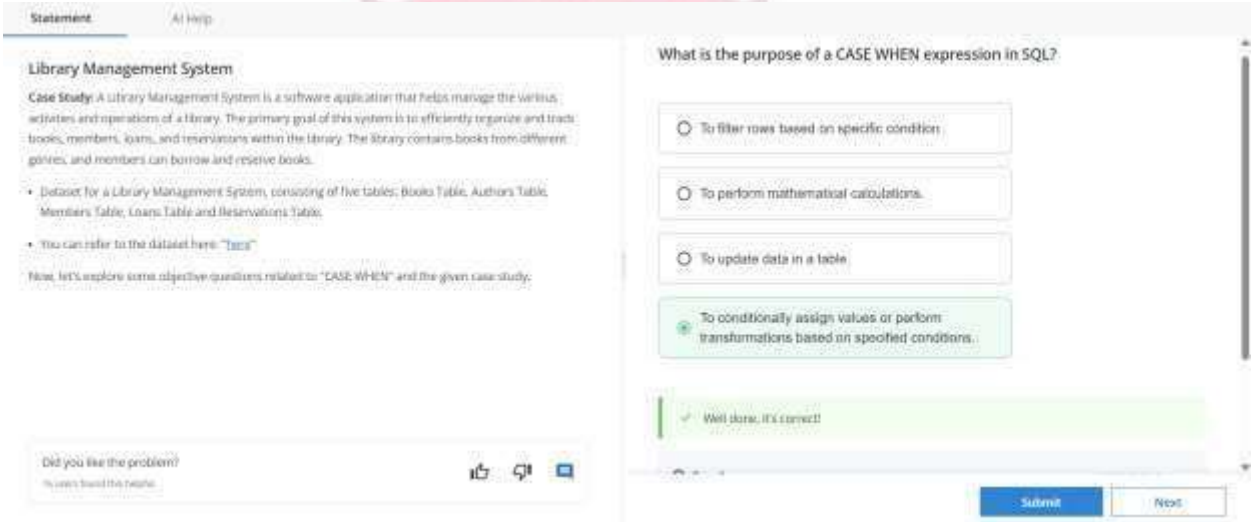
10. REARRANGE QUERY - 2



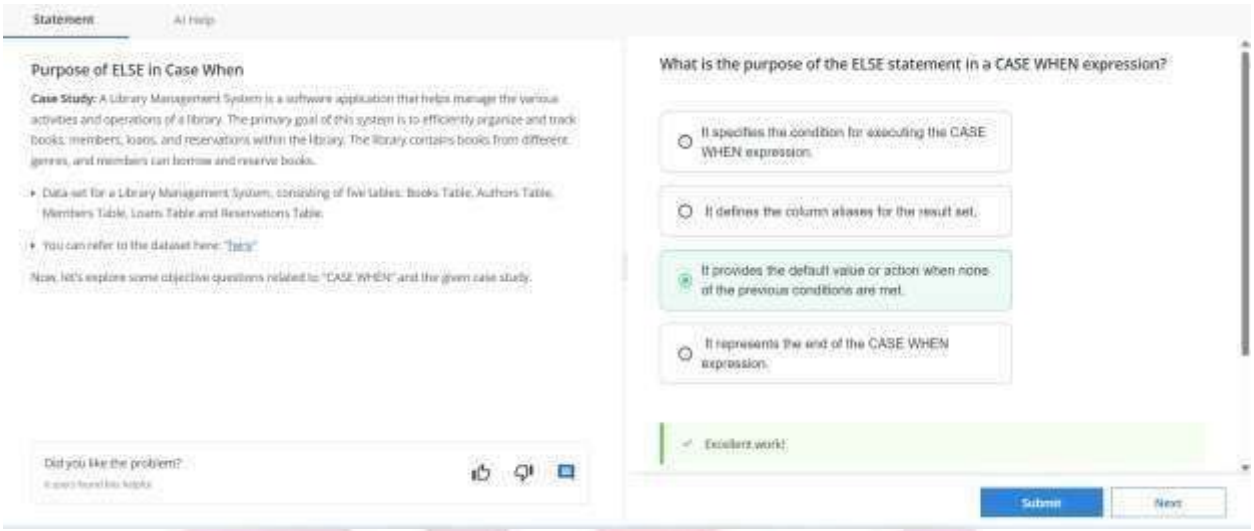
TRUTH-TOIL-TRIUMPH

CASE STUDY – LIBRARY MANAGEMENT SYSTEM

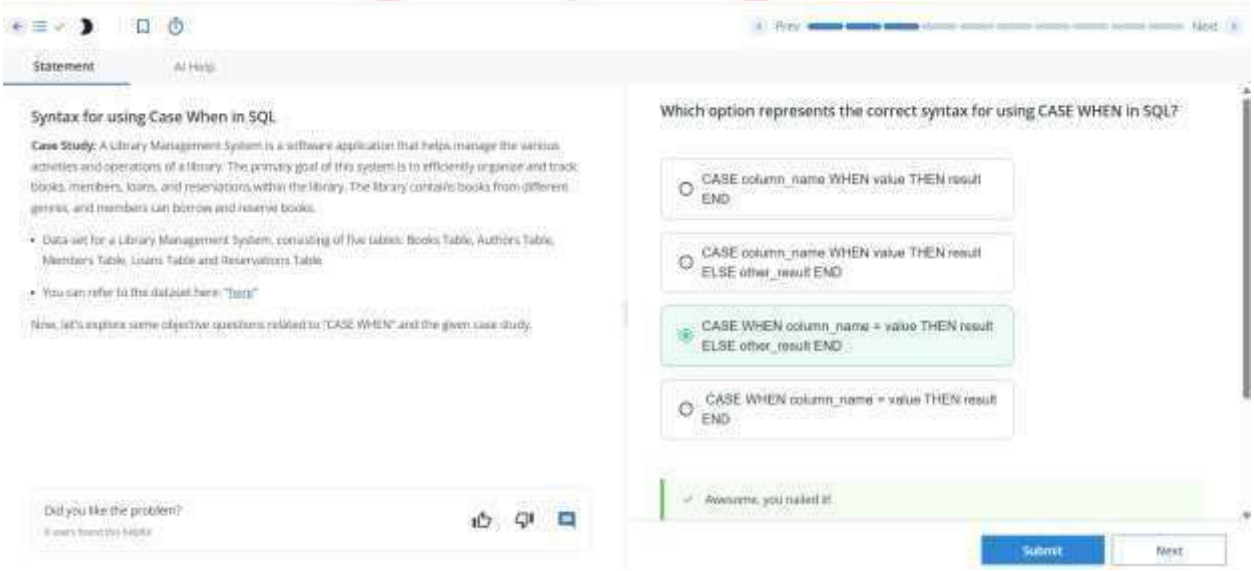
1. LIBRARY MANAGEMENT SYSTEM



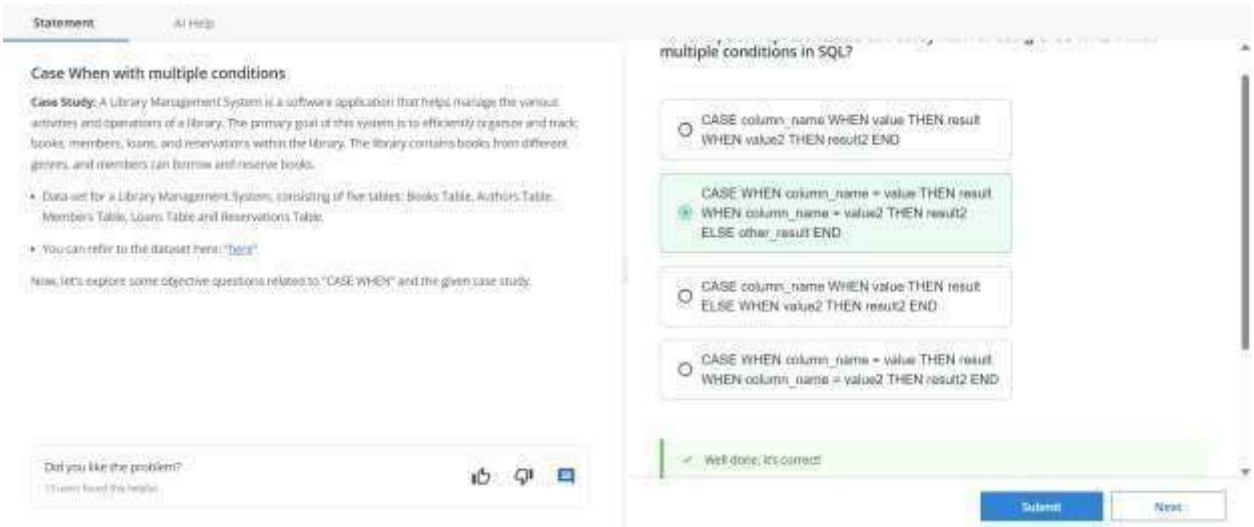
2. PURPOSE OF ELSE IN CASE WHEN



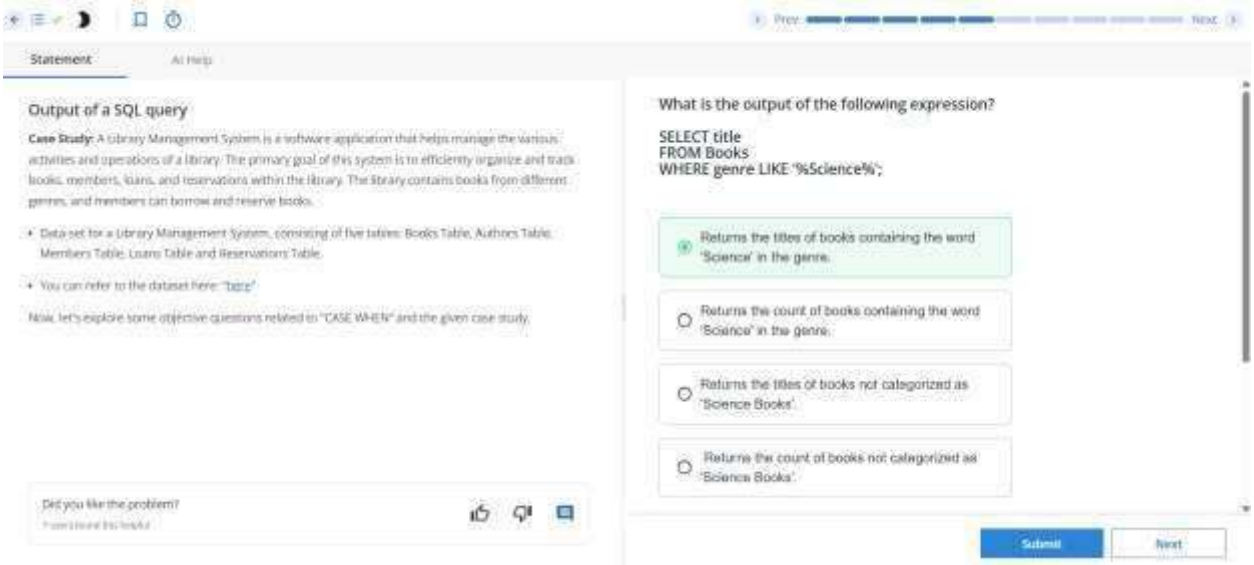
3. SYNTAX FOR USING CASE WHEN IN SQL



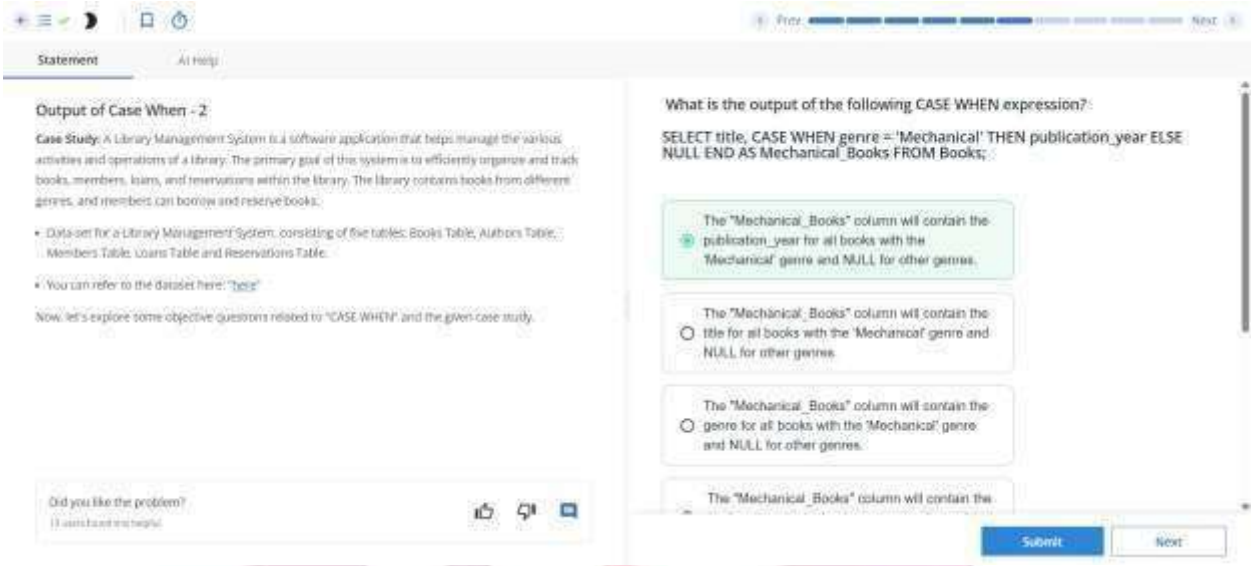
4. CASE WHEN WITH MULTIPLE CONDITIONS



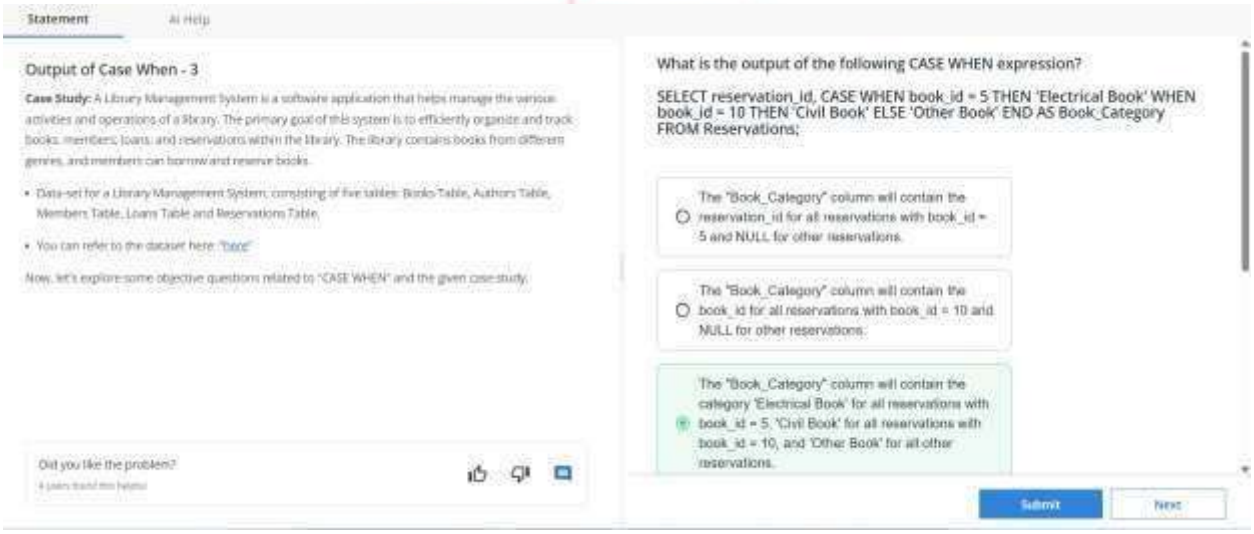
5. OUTPUT OF A SQL QUERY



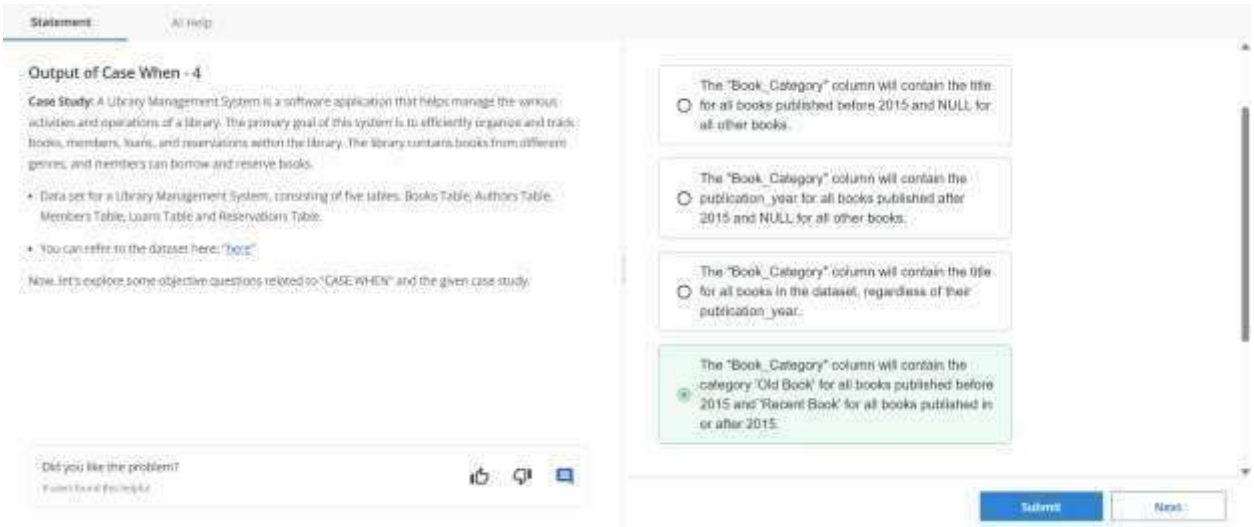
6. OUTPUT OF CASE WHEN - 2



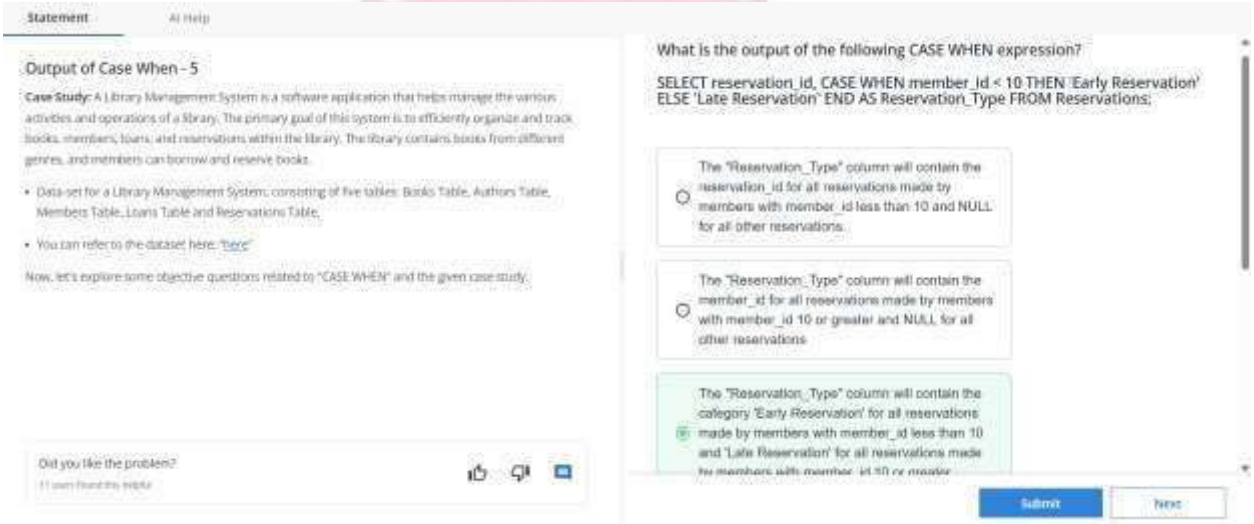
7. OUTPUT OF CASE WHEN - 3



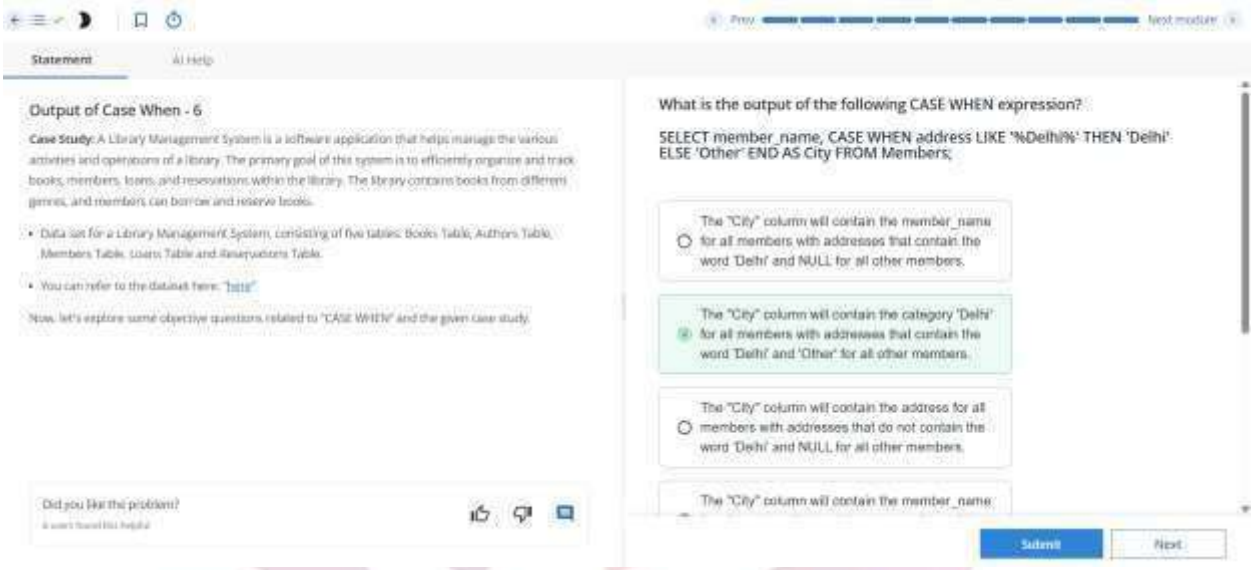
8. OUTPUT OF CASE WHEN - 4



9. OUTPUT OF CASE WHEN -5



10. OUTPUT OF CASE WHEN - 6



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Congratulations! You solved

50 Problems on CodeChef



Problem Solver Bronze Badge

CODECHEF

1. Consider the following tables CABHUB and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the names of all the white colored vehicles.

(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

(iv) To display the customer and the corresponding name of the vehicle hired by them.

(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

(iii) SELECT COUNT(*), Make FROM CABHUB;

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

PROGRAM

```
CREATE TABLE CABHUB2 (Vcode VARCHAR(10),VehicleName VARCHAR(30),Make VARCHAR(20),Color VARCHAR(15),Capacity INT,Charges INT);

INSERT INTO CABHUB2 VALUES('100', 'Innova', 'Toyota', 'WHITE', 7, 25);

INSERT INTO CABHUB2 VALUES('102', 'SX4', 'Suzuki', 'BLUE', 5, 20);

INSERT INTO CABHUB2 VALUES('104', 'C-Class', 'Mercedes', 'RED', 7, 35);

INSERT INTO CABHUB2 VALUES('105', 'A-Star', 'Suzuki', 'WHITE', 5, 40);

INSERT INTO CABHUB2 VALUES('109', 'Indigo', 'Tata', 'SILVER', 4, 50);

Select*from CABHUB2;

CREATE TABLE CUSTOMER1 (Code INT,CName VARCHAR(30),VCode VARCHAR(10));

INSERT INTO CUSTOMER1 VALUES(1, 'Ravi', '102');

INSERT INTO CUSTOMER1 VALUES(12, 'Kumar', '105');

INSERT INTO CUSTOMER1 VALUES(13, 'Priya', '109');

INSERT INTO CUSTOMER1 VALUES(14, 'Anitha', '104');

Select*from CUSTOMER1;
```

SELECT VehicleName FROM CABHUB2 WHERE Color = 'WHITE';

SELECT VehicleName, Make FROM CABHUB2 ORDER BY Capacity ASC;

SELECT VehicleName, Make, Capacity FROM CABHUB2 ORDER BY Capacity ASC;

SELECT MAX(Charges)FROM CABHUB2;

SELECT CUSTOMER1.CName, CABHUB2.VehicleName FROM CUSTOMER1 INNER JOIN CABHUB2 ON CUSTOMER1.Vcode = CABHUB2.Vcode;

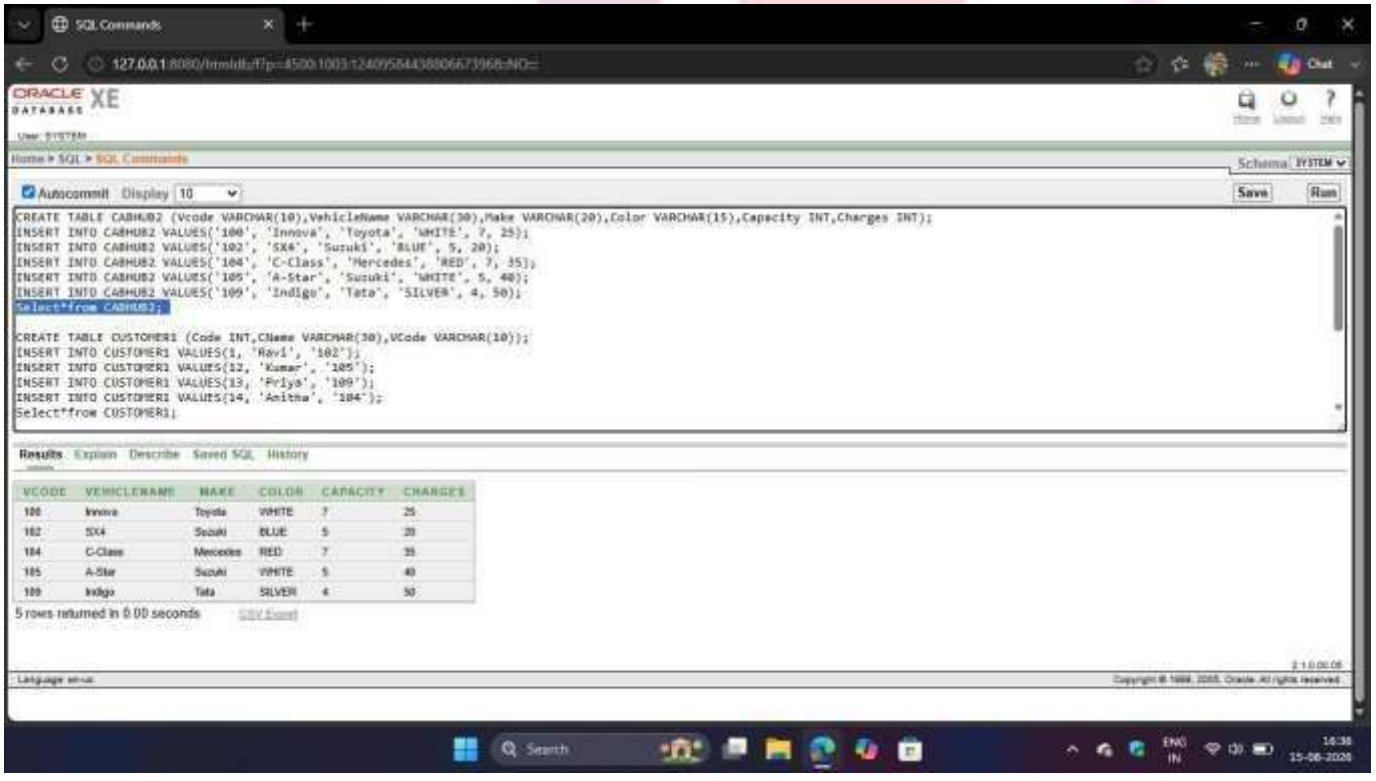
SELECT COUNT(DISTINCT Make)FROM CABHUB2;

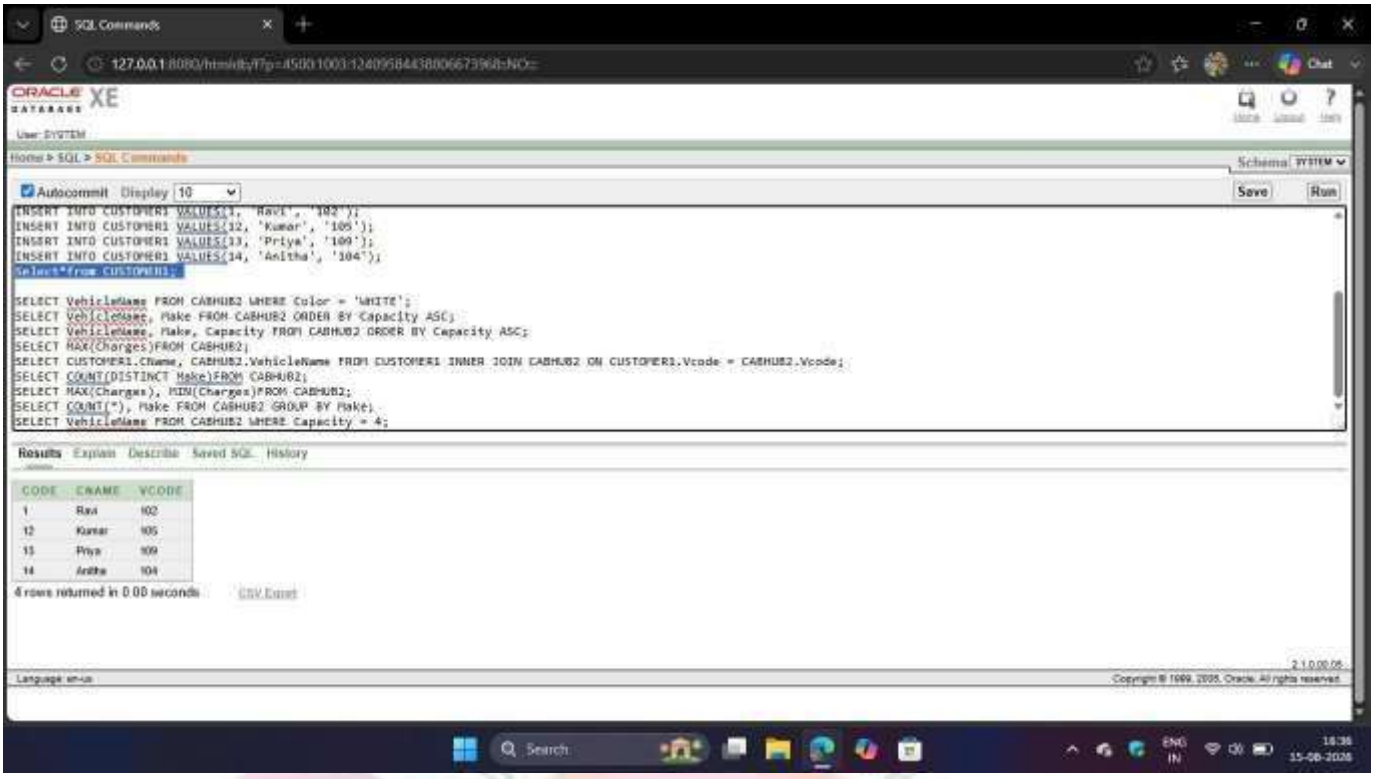
SELECT MAX(Charges), MIN(Charges)FROM CABHUB2;

SELECT COUNT(*), Make FROM CABHUB2 GROUP BY Make;

SELECT VehicleName FROM CABHUB2 WHERE Capacity = 4;

OUTPUT




SQL Commands
ORACLE XE
DATABASE
Autocommit
Display 10
Save
Run
INSERT INTO CUSTOMER1 VALUES(1, 'Ravi', '102');
INSERT INTO CUSTOMER1 VALUES(12, 'Kumar', '105');
INSERT INTO CUSTOMER1 VALUES(13, 'Priya', '109');
INSERT INTO CUSTOMER1 VALUES(14, 'Anitha', '104');
select*from CUSTOMER1;
SELECT VehicleName FROM CABHUB2 WHERE Color = 'WHITE';
SELECT VehicleName, Make FROM CABHUB2 ORDER BY Capacity ASC;
SELECT VehicleName, Make, Capacity FROM CABHUB2 ORDER BY Capacity ASC;
SELECT MAX(Charges)FROM CABHUB2;
SELECT CUSTOMER1.CName, CABHUB2.VehicleName FROM CUSTOMER1 INNER JOIN CABHUB2 ON CUSTOMER1.Vcode = CABHUB2.Vcode;
SELECT COUNT(DISTINCT Make)FROM CABHUB2;
SELECT MAX(Charges), MIN(Charges)FROM CABHUB2;
SELECT COUNT(*), Make FROM CABHUB2 GROUP BY Make;
SELECT VehicleName FROM CABHUB2 WHERE Capacity = 4;
Results
Explain
Describe
Saved SQL
History
CODE
CNAME
VCODE
1
Ravi
102
12
Kumar
105
13
Priya
109
14
Anitha
104
4 rows returned in 0.00 seconds
CSV Export
Language: en-us
Search

2. Given the following CO2_EMISSION relation, write SQL commands for questions (a) to (f).

(a) Display all the details of vehicles that use Petrol as the fuel type.

(b) Display the Company and Model of vehicles whose EngineSize is greater than 1.5 litres.

(c) Display the Company, Model, and CO2Emission in ascending order of CO2Emission.

(d) Display the details of vehicles whose CO2Emission is greater than 150 g/km.

(e) Count the number of vehicles that use Diesel as the fuel type.

(f) Insert the following record into the CO2_EMISSION table.108, 'MG', 'Hector', 'Petrol', 1.5, 160

(g) Write the output of the following SQL commands.

(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;

(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;

(iii) SELECTAVG(CO2Emission)FROMCO2_EMISSION WHERE FuelType='Petrol';

(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

PROGRAM

```
CREATE TABLE CO2_EMISSION SNo NUMBER(5), Company VARCHAR2(30),
 Model VARCHAR2(30),FuelType VARCHAR2(15),EngineSize NUMBER(3,1),
 CO2Emission NUMBER(5));
INSERT INTO CO2_EMISSION VALUES (101, 'Maruti', 'Swift', 'Petrol', 1.2, 120);
INSERT INTO CO2_EMISSION VALUES (102, 'Hyundai', 'Creta', 'Diesel', 1.5, 155);
INSERT INTO CO2_EMISSION VALUES (103, 'Honda', 'City', 'Petrol', 1.5, 140);
INSERT INTO CO2_EMISSION VALUES (104, 'Tata', 'Nexon', 'Diesel', 1.5, 160);
INSERT INTO CO2_EMISSION VALUES (105, 'Toyota', 'Innova', 'Diesel', 2.4, 180);
SELECT*FROM CO2_EMISSION;

SELECT *FROM CO2_EMISSION WHERE FuelType = 'Petrol';
SELECT Company, Model FROM CO2_EMISSION WHERE EngineSize > 1.5;
```

SELECT Company, Model, CO2Emission FROM CO2_EMISSION ORDER BY CO2Emission ASC;

SELECT * FROM CO2_EMISSION WHERE CO2Emission > 150;

SELECT COUNT(*)FROM CO2_EMISSION WHERE FuelType = 'Diesel';

INSERT INTO CO2_EMISSION VALUES (108, 'MG', 'Hector', 'Petrol', 1.5, 160);

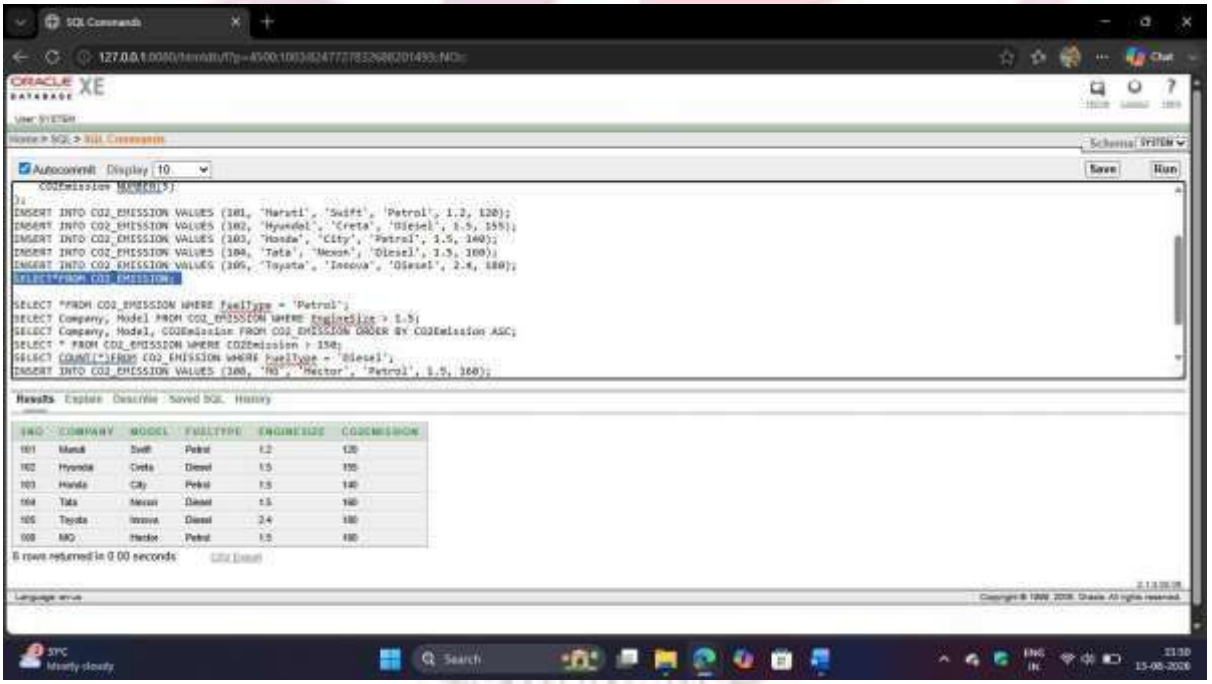
SELECT MAX(CO2Emission)FROM CO2_EMISSION;

SELECT MIN(EngineSize)FROM CO2_EMISSION;

SELECT AVG(CO2Emission)FROM CO2_EMISSION WHERE FuelType = 'Petrol';

SELECT COUNT(*)FROM CO2_EMISSION WHERE EngineSize >= 1.5;

OUTPUT



3. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables APPLICANTS and COURSES.

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

(b) To display the names of applicants, who are paying fee more than 30000.

(c) To display name of all applicants in ascending order of their joinyear.

(d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

(f) To display the applicant's name with their respective course's name from the tables APPLICANTS and COURSES.

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';

(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= 'M';

(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= 'A01' OR C_ID= 'A05';

(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT(*)=2;

PROGRAM

```
CREATE TABLE APPLICANTS ( NO NUMBER(10),NAME VARCHAR2(30),FEE NUMBER(10,2),GENDER CHAR(1),C_ID VARCHAR2(5),JOINYEAR NUMBER(4));

INSERT INTO APPLICANTS VALUES (1, 'Arun', 35000, 'M', 'A01', 2008);

INSERT INTO APPLICANTS VALUES (2, 'Priya', 32000, 'F', 'A05', 2009);

INSERT INTO APPLICANTS VALUES (3, 'Kumar', 28000, 'M', '02', 2011);

INSERT INTO APPLICANTS VALUES (4, 'Divya', 30000, 'F', '02', 2009);

INSERT INTO APPLICANTS VALUES (5, 'Rahul', 40000, 'M', 'A05', 2007);

INSERT INTO APPLICANTS VALUES (6, 'Anitha', 35000, 'F', 'A01', 2010);

SELECT*FROM APPLICANTS;
```

```
CREATE TABLE COURSESS (C_ID VARCHAR2(5),C_NAME VARCHAR2(30));

INSERT INTO COURSESS VALUES ('A01', 'Java');

INSERT INTO COURSESS VALUES ('A02', 'Python');

INSERT INTO COURSESS VALUES ('A05', 'Database');

INSERT INTO COURSESS VALUES ('A07', 'Web Designing');

INSERT INTO COURSESS VALUES ('A03', 'Cloud Computing');

SELECT*FROM COURSESS;

SELECT NAME, FEE, GENDER, JOINYEAR FROM APPLICANTS WHERE JOINYEAR
< 2010;

SELECT NAME FROM APPLICANTS WHERE FEE > 30000;

SELECT NAME FROM APPLICANTS ORDER BY JOINYEAR ASC;

SELECT JOINYEAR, COUNT(*) AS TOTAL_APPLICANTS FROM APPLICANTS
GROUP BY JOINYEAR ORDER BY JOINYEAR;

SELECT C_ID, COUNT(*) AS NO_OF_APPLICANTS FROM APPLICANTS GROUP BY
C_ID;

SELECT APPLICANTS.NAME, COURSES.C_NAME FROM APPLICANTS JOIN
COURSES ON APPLICANTS.C_ID = COURSES.C_ID;

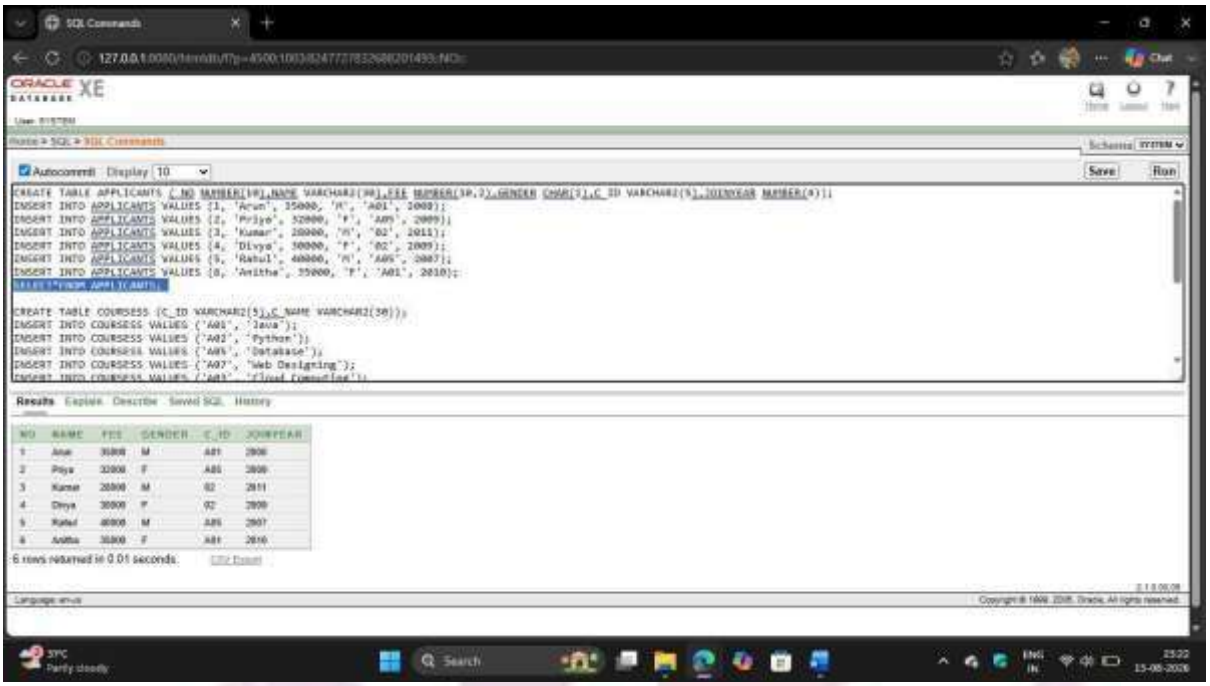
SELECT NAME, JOINYEAR FROM APPLICANTS WHERE GENDER = 'F' AND C_ID =
'02';

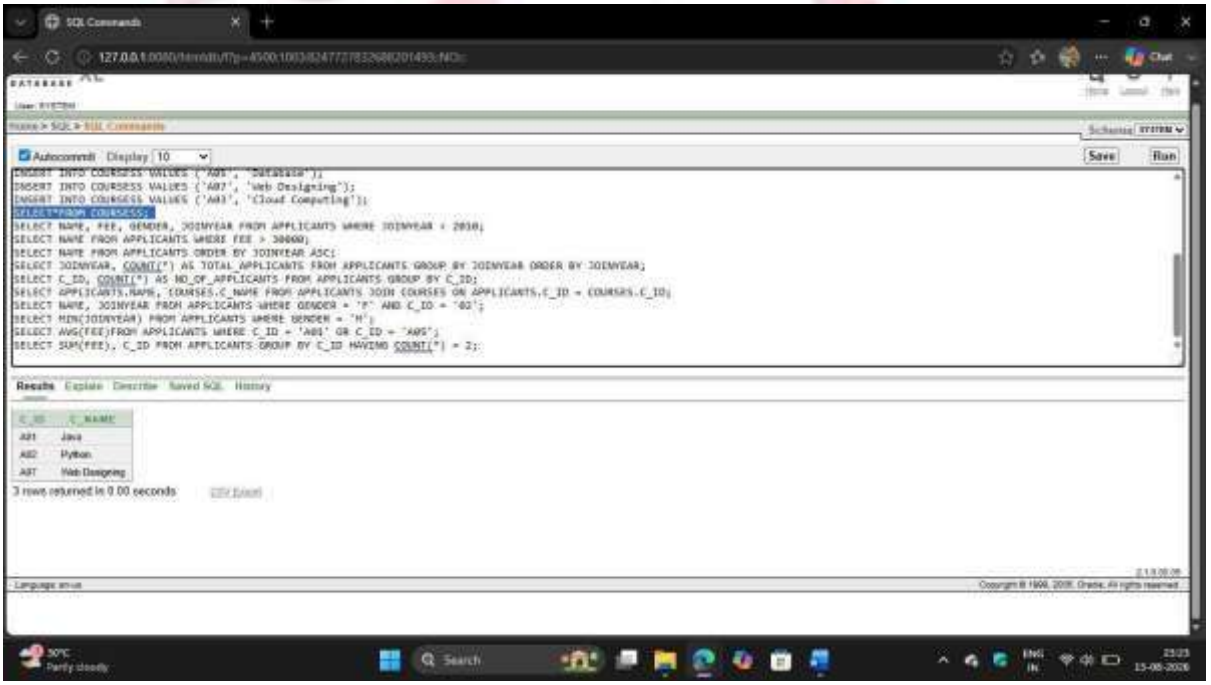
SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE GENDER = 'M';

SELECT AVG(FEE)FROM APPLICANTS WHERE C_ID = 'A01' OR C_ID = 'A05';

SELECT SUM(FEE), C_ID FROM APPLICANTS GROUP BY C_ID HAVING COUNT(*)
= 2;
```

OUTPUT


SQL Commands
ORACLE XE
CREATE TABLE APPLICANTS (C_NO NUMBER(10),NAME VARCHAR2(30),FEE NUMBER(10,2),GENDER CHAR(1),C_ID VARCHAR2(5),JOINYEAR NUMBER(4));
INSERT INTO APPLICANTS VALUES (1, 'Arun', 35000, 'M', 'A01', 2008);
INSERT INTO APPLICANTS VALUES (2, 'Priya', 32000, 'F', 'A05', 2009);
INSERT INTO APPLICANTS VALUES (3, 'Kumar', 28000, 'M', '02', 2011);
INSERT INTO APPLICANTS VALUES (4, 'Divya', 30000, 'F', '02', 2009);
INSERT INTO APPLICANTS VALUES (5, 'Rahul', 40000, 'M', 'A05', 2007);
INSERT INTO APPLICANTS VALUES (6, 'Anitha', 35000, 'F', 'A01', 2010);
SELECT*FROM APPLICANTS;
CREATE TABLE COURSESS (C_ID VARCHAR2(5),C_NAME VARCHAR2(30));
INSERT INTO COURSESS VALUES ('A01', 'Java');
INSERT INTO COURSESS VALUES ('A02', 'Python');
INSERT INTO COURSESS VALUES ('A05', 'Database');
INSERT INTO COURSESS VALUES ('A07', 'Web Designing');
Results
6 rows returned in 0.01 seconds


SQL Commands
INSERT INTO COURSESS VALUES ('A05', 'Database');
INSERT INTO COURSESS VALUES ('A07', 'Web Designing');
INSERT INTO COURSESS VALUES ('A03', 'Cloud Computing');
SELECT*FROM COURSESS;
SELECT NAME, FEE, GENDER, JOINYEAR FROM APPLICANTS WHERE JOINYEAR > 2010;
SELECT NAME FROM APPLICANTS WHERE FEE > 30000;
SELECT NAME FROM APPLICANTS ORDER BY JOINYEAR ASC;
SELECT JOINYEAR, COUNT(*) AS TOTAL_APPLICANTS FROM APPLICANTS GROUP BY JOINYEAR ORDER BY JOINYEAR;
SELECT C_ID, COUNT(*) AS NO_OF_APPLICANTS FROM APPLICANTS GROUP BY C_ID;
SELECT APPLICANTS.NAME, COURSES.C_NAME FROM APPLICANTS JOIN COURSES ON APPLICANTS.C_ID = COURSES.C_ID;
SELECT NAME, JOINYEAR FROM APPLICANTS WHERE GENDER = 'F' AND C_ID = '02';
SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE GENDER = 'M';
SELECT AVG(FEE)FROM APPLICANTS WHERE C_ID = 'A01' OR C_ID = 'A05';
SELECT SUM(FEE), C_ID FROM APPLICANTS GROUP BY C_ID HAVING COUNT(*) = 2;
Results
C_ID C_NAME
A01 Java
A02 Python
A07 Web Designing
3 rows returned in 0.00 seconds

4. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables PRODUCTS AND SUPPLIERS

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be

S01 2

S02 2

S03 1

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

(f) To display the name of the companies and the name of the products in descending order of company names.

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

PROGRAM

```
CREATE TABLE PRODUCTS2(PID NUMBER(5),SNAME VARCHAR2(30),QTY
NUMBER(5),PRICE NUMBER(10,2),COMPANY VARCHAR2(30), SUPCODE
VARCHAR2(5));
```

```
INSERT INTO PRODUCTS2 VALUES(101, 'DIGITAL CAMERA14X', 120, 12000,
'RENIX', 'S01');

INSERT INTO PRODUCTS2 VALUES(102, 'DIGITAL PAD 11i', 100, 22000, 'DIGI POP',
'S02');

INSERT INTO PRODUCTS2 VALUES(104, 'PEN DRIVE 16 GB', 500, 1100,
'STOREKING', 'S01');

INSERT INTO PRODUCTS2 VALUES(106, 'LED SCREEN', 70, 28000, 'DISEXPERTS',
'S02');

INSERT INTO PRODUCTS2 VALUES(105, 'CAR GPS SYSTEM', 60, 12000, 'MOVEON',
'S03');

SELECT*FROM PRODUCTS2;


CREATE TABLE SUPPLIERS2 (SUPCODE VARCHAR2(5),SNAME
VARCHAR2(30),CITY VARCHAR2(20));

INSERT INTO SUPPLIERS2 VALUES('S01', 'GET ALL INC', 'KOLKATA');

INSERT INTO SUPPLIERS2 VALUES('S03', 'EASY MARKET CORP', 'DELHI');

INSERT INTO SUPPLIERS2 VALUES('S02', 'DIGI BUSY GROUP', 'CHENNAI');

SELECT*FROM SUPPLIERS2;

SELECT *FROM PRODUCTS2 ORDER BY SNAME ASC;

SELECT SNAME, PRICE FROM PRODUCTS2 WHERE PRICE BETWEEN 10000 AND
15000;

SELECT SUPCODE, COUNT(*)FROM PRODUCTS2 GROUP BY SUPCODE;

SELECT PRICE, SNAME, QTY FROM PRODUCTS2 WHERE QTY > 100;

SELECT SNAME FROM SUPPLIERS2 WHERE CITY IN ('DELHI', 'CHENNAI');

SELECT COMPANY, SNAME FROM PRODUCTS2 ORDER BY COMPANY DESC;

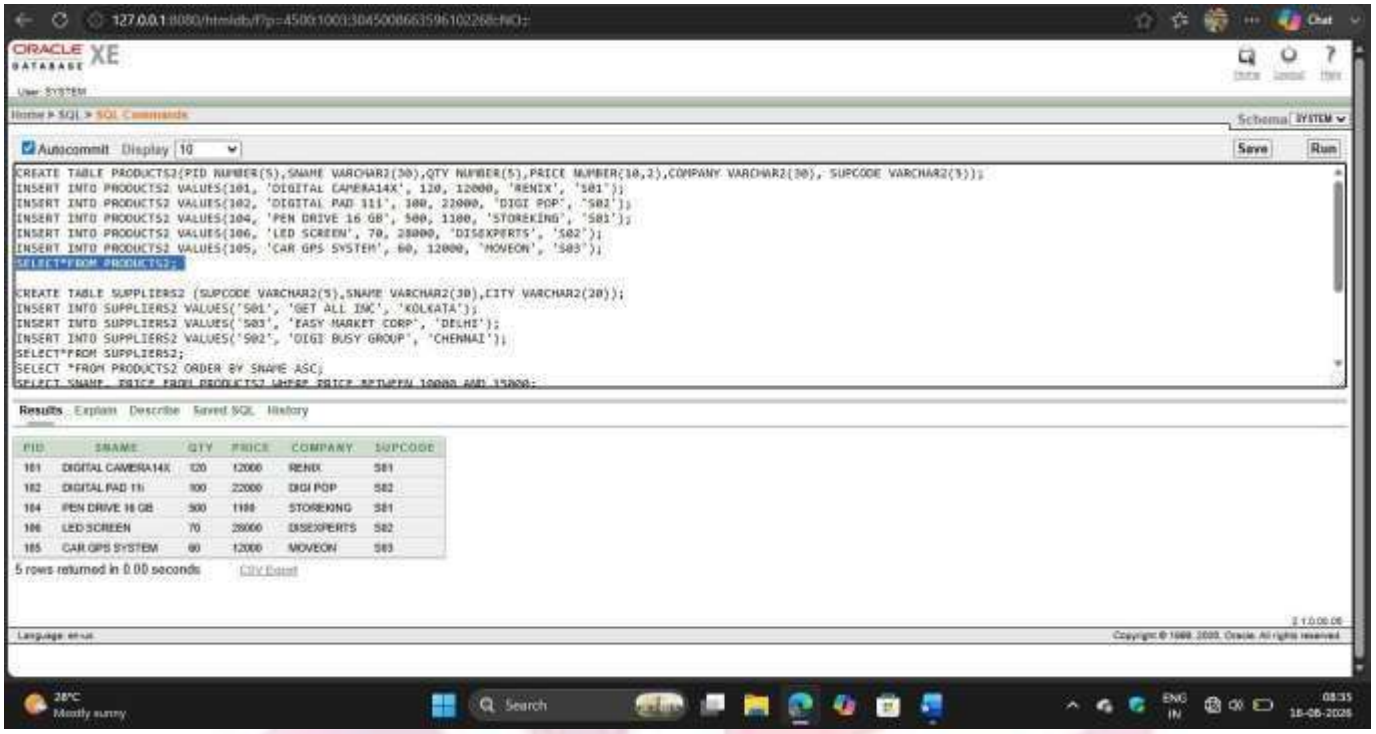
SELECT DISTINCT SUPCODE FROM PRODUCTS2;

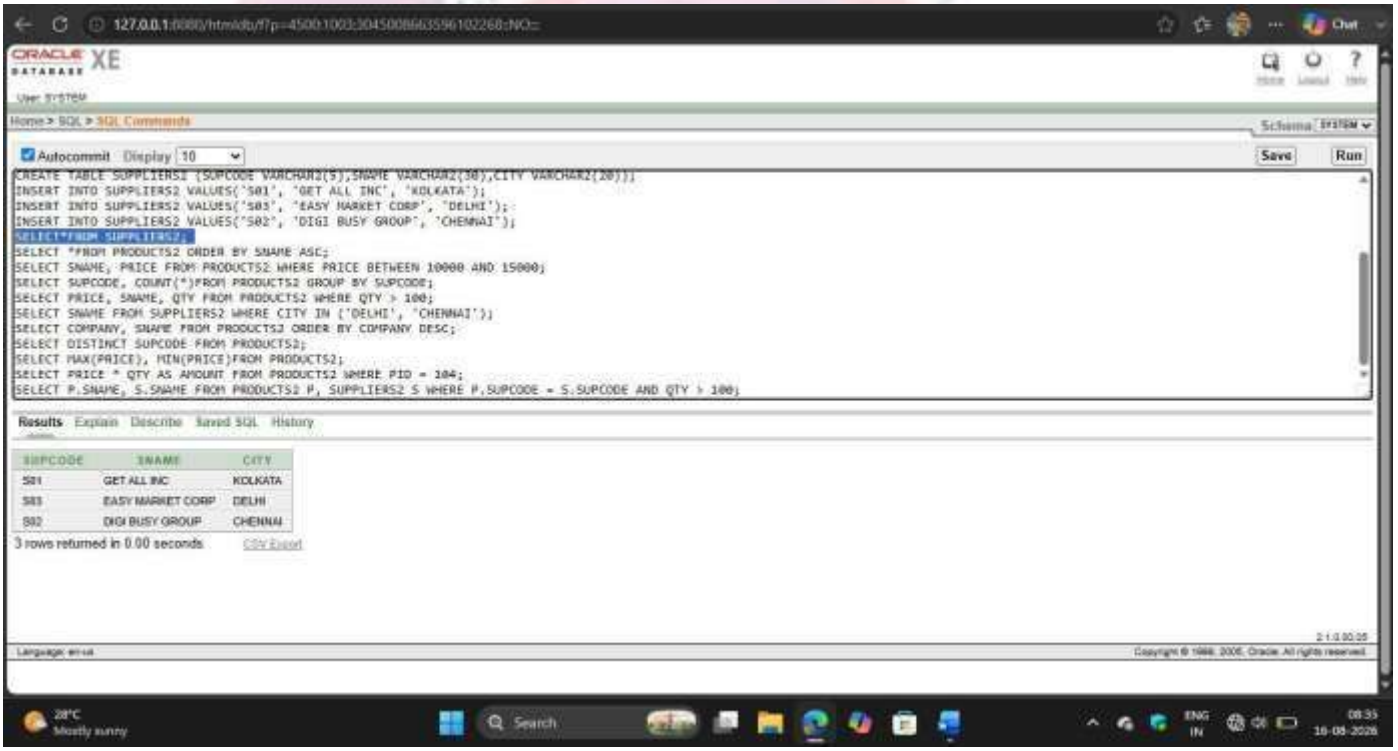
SELECT MAX(PRICE), MIN(PRICE)FROM PRODUCTS2;

SELECT PRICE * QTY AS AMOUNT FROM PRODUCTS2 WHERE PID = 104;

SELECT P.SNAME, S.SNAME FROM PRODUCTS2 P, SUPPLIERS2 S WHERE
P.SUPCODE = S.SUPCODE AND QTY > 100;
```

OUTPUT


ORACLE XE
DATABASE
Autocommit
Display 10
Save
Run
CREATE TABLE PRODUCTS2(PID NUMBER(5),SNAME VARCHAR2(30),QTY NUMBER(5),PRICE NUMBER(10,2),COMPANY VARCHAR2(30), SUPCODE VARCHAR2(5));
INSERT INTO PRODUCTS2 VALUES(101, 'DIGITAL CAMERA14X', 120, 12000, 'RENIX', '501');
INSERT INTO PRODUCTS2 VALUES(102, 'DIGITAL PAD 11i', 100, 22000, 'DIGI POP', '502');
INSERT INTO PRODUCTS2 VALUES(104, 'PEN DRIVE 16 GB', 500, 1100, 'STOREKING', '501');
INSERT INTO PRODUCTS2 VALUES(106, 'LED SCREEN', 70, 28000, 'DISEXPERTS', '502');
INSERT INTO PRODUCTS2 VALUES(105, 'CAR GPS SYSTEM', 60, 12000, 'MOVEON', '503');
SELECT*FROM PRODUCTS2;
CREATE TABLE SUPPLIERS2 (SUPCODE VARCHAR2(5),SNAME VARCHAR2(30),CITY VARCHAR2(20));
INSERT INTO SUPPLIERS2 VALUES('501', 'GET ALL INC', 'KOLKATA');
INSERT INTO SUPPLIERS2 VALUES('503', 'EASY MARKET CORP', 'DELHI');
INSERT INTO SUPPLIERS2 VALUES('502', 'DIGI BUSY GROUP', 'CHENNAI');
SELECT*FROM SUPPLIERS2;
SELECT *FROM PRODUCTS2 ORDER BY SNAME ASC;
Results Explain Describe Saved SQL History
PID SNAME QTY PRICE COMPANY SUPCODE
101 DIGITAL CAMERA14X 120 12000 RENIX 501
102 DIGITAL PAD 11i 100 22000 DIGI POP 502
104 PEN DRIVE 16 GB 500 1100 STOREKING 501
106 LED SCREEN 70 28000 DISEXPERTS 502
105 CAR GPS SYSTEM 60 12000 MOVEON 503
5 rows returned in 0.00 seconds
CSV Export


ORACLE XE
DATABASE
Autocommit
Display 10
Save
Run
CREATE TABLE SUPPLIERS2 (SUPCODE VARCHAR2(5),SNAME VARCHAR2(30),CITY VARCHAR2(20));
INSERT INTO SUPPLIERS2 VALUES('501', 'GET ALL INC', 'KOLKATA');
INSERT INTO SUPPLIERS2 VALUES('503', 'EASY MARKET CORP', 'DELHI');
INSERT INTO SUPPLIERS2 VALUES('502', 'DIGI BUSY GROUP', 'CHENNAI');
SELECT*FROM SUPPLIERS2;
SELECT *FROM PRODUCTS2 ORDER BY SNAME ASC;
SELECT SNAME, PRICE FROM PRODUCTS2 WHERE PRICE BETWEEN 10000 AND 15000;
SELECT SUPCODE, COUNT(*)FROM PRODUCTS2 GROUP BY SUPCODE;
SELECT PRICE, SNAME, QTY FROM PRODUCTS2 WHERE QTY > 100;
SELECT SNAME FROM SUPPLIERS2 WHERE CITY IN ('DELHI', 'CHENNAI');
SELECT COMPANY, SNAME FROM PRODUCTS2 ORDER BY COMPANY DESC;
SELECT DISTINCT SUPCODE FROM PRODUCTS2;
SELECT MAX(PRICE), MIN(PRICE)FROM PRODUCTS2;
SELECT PRICE * QTY AS AMOUNT FROM PRODUCTS2 WHERE PID = 104;
SELECT P.SNAME, S.SNAME FROM PRODUCTS2 P, SUPPLIERS2 S WHERE P.SUPCODE = S.SUPCODE AND QTY > 100;
Results Explain Describe Saved SQL History
SUPCODE SNAME CITY
501 GET ALL INC KOLKATA
503 EASY MARKET CORP DELHI
502 DIGI BUSY GROUP CHENNAI
3 rows returned in 0.00 seconds
CSV Export

5. Answer the questions (a) and (b) on the basis of the following tables SHOPPE and ACCESSORIES

(a) Write the SQL queries:

(i) To display Name and Price of all the accessories in ascending order of their Price.

(ii) To display Id and SName of all Shoppe in Nehru Place.

(iii) To display Minimum and Maximum Price of each Name of accessories.

(iv) To display Name, Price of all accessories and their respective SName where they are available.

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;

(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

(iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN (‘S02, ‘S03’);

PROGRAM

```
CREATE TABLE SHOPESS(S_ID NUMBER(10),S_NAME VARCHAR(20),AREA VARCHAR(20));

INSERT INTO SHOPESS VALUES(1001,'ABC COMPUTERONICS','CP');

INSERT INTO SHOPESS VALUES(1002,'ALL INFOTECH MEDIA','GK II');

INSERT INTO SHOPESS VALUES(1003,'TECH SHOPE','CP');

INSERT INTO SHOPESS VALUES(1004,'GREEKS TECHNO SOFT','NEHRU PLACE');

INSERT INTO SHOPESS VALUES(1005,'HITECH TECH STORE','NEHRU PLACE');

SELECT*FROM SHOPESS;


CREATE TABLE ACCESSORIES(NO NUMBER(10),NAME VARCHAR(20),PRICE NUMBER(20),ID NUMBER(10));

INSERT INTO ACCESSORIES VALUES(1,'MOTHER BOARD',1200,101);

INSERT INTO ACCESSORIES VALUES(2,'HARD DISK',5000,101);

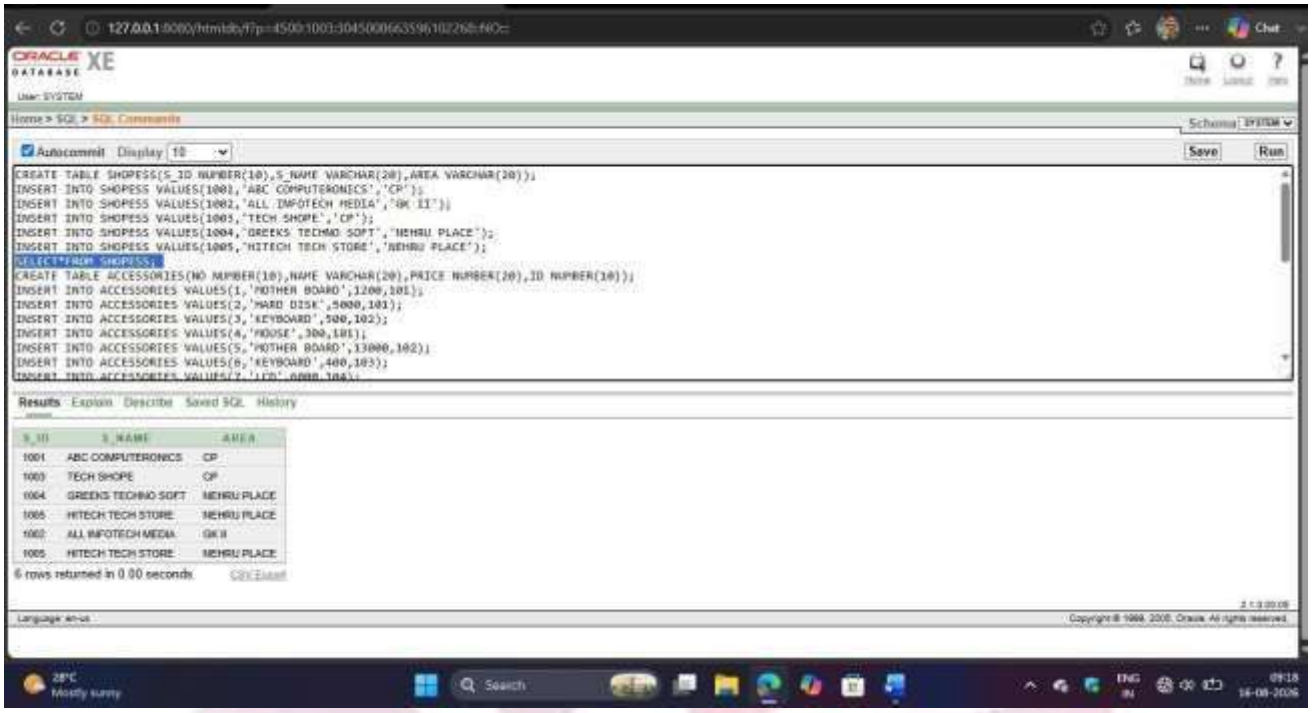
INSERT INTO ACCESSORIES VALUES(3,'KEYBOARD',500,102);

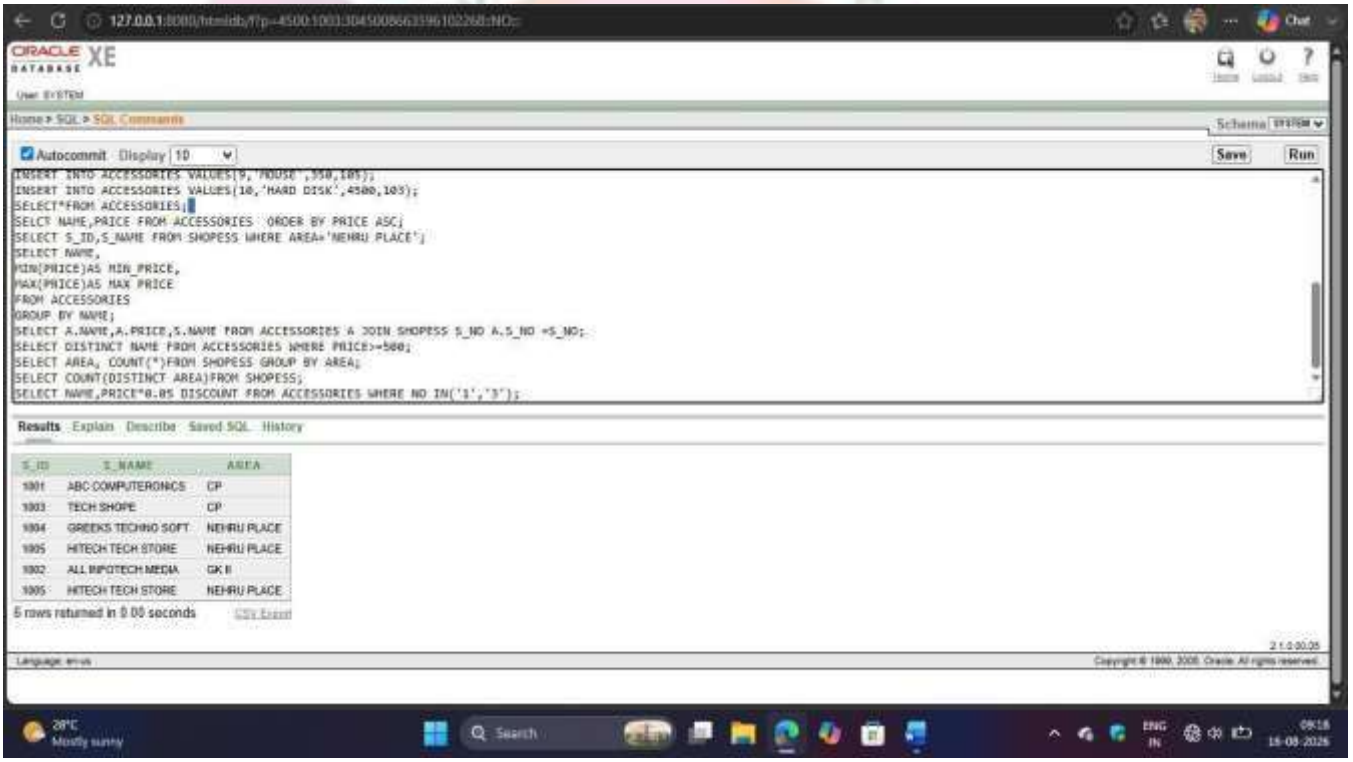
INSERT INTO ACCESSORIES VALUES(4,'MOUSE',300,101);
```

```
INSERT INTO ACCESSORIES VALUES(5,'MOTHER BOARD',13000,102);
INSERT INTO ACCESSORIES VALUES(6,'KEYBOARD',400,103);
INSERT INTO ACCESSORIES VALUES(7,'LCD',6000,104);
INSERT INTO ACCESSORIES VALUES(8,'LCD',5500,105);
INSERT INTO ACCESSORIES VALUES(9,'MOUSE',350,105);
INSERT INTO ACCESSORIES VALUES(10,'HARD DISK',4500,103);
SELECT*FROM ACCESSORIES;

SELCT NAME,PRICE FROM ACCESSORIES  ORDER BY PRICE ASC;
SELECT S_ID,S_NAME FROM SHOPESS WHERE AREA='NEHRU PLACE';
SELECT NAME,MIN(PRICE)AS MIN_PRICE, MAX(PRICE)AS MAX PRICE
FROM ACCESSORIES  GROUP BY NAME;
SELECT A.NAME,A.PRICE,S.NAME FROM ACCESSORIES A JOIN SHOPESS S_NO
A.S_NO =S_NO;
SELECT DISTINCT NAME FROM ACCESSORIES WHERE PRICE>=500;
SELECT AREA, COUNT(*)FROM SHOPESS GROUP BY AREA;
SELECT COUNT(DISTINCT AREA)FROM SHOPESS;
SELECT NAME,PRICE*0.05 DISCOUNT FROM ACCESSORIES WHERE NO IN('1','3');
```

OUTPUT


CREATE TABLE SHOPESS(S_ID NUMBER(10),S_NAME VARCHAR(20),AREA VARCHAR(20));
INSERT INTO SHOPESS VALUES(1001,'ABC COMPUTERONICS','CP');
INSERT INTO SHOPESS VALUES(1002,'ALL INFOTECH MEDIA','GK II');
INSERT INTO SHOPESS VALUES(1003,'TECH SHOPE','CP');
INSERT INTO SHOPESS VALUES(1004,'GREEKS TECHNO SOFT','NEHRU PLACE');
INSERT INTO SHOPESS VALUES(1005,'HITECH TECH STORE','NEHRU PLACE');
SELECT*FROM SHOPESS;
CREATE TABLE ACCESSORIES(NO NUMBER(10),NAME VARCHAR(20),PRICE NUMBER(20),ID NUMBER(10));
INSERT INTO ACCESSORIES VALUES(1,'MOTHER BOARD',1200,101);
INSERT INTO ACCESSORIES VALUES(2,'HARD DISK',5000,101);
INSERT INTO ACCESSORIES VALUES(3,'KEYBOARD',500,102);
INSERT INTO ACCESSORIES VALUES(4,'MOUSE',300,101);
INSERT INTO ACCESSORIES VALUES(5,'MOTHER BOARD',13000,102);
INSERT INTO ACCESSORIES VALUES(6,'KEYBOARD',400,103);
S_ID S_NAME AREA
1001 ABC COMPUTERONICS CP
1003 TECH SHOPE CP
1004 GREEKS TECHNO SOFT NEHRU PLACE
1005 HITECH TECH STORE NEHRU PLACE
1002 ALL INFOTECH MEDIA GK II
1005 HITECH TECH STORE NEHRU PLACE
6 rows returned in 0.00 seconds


INSERT INTO ACCESSORIES VALUES(9,'MOUSE',350,101);
INSERT INTO ACCESSORIES VALUES(10,'HARD DISK',4500,103);
SELECT*FROM ACCESSORIES;
SELCT NAME,PRICE FROM ACCESSORIES ORDER BY PRICE ASC;
SELECT S_ID,S_NAME FROM SHOPESS WHERE AREA='NEHRU PLACE';
SELECT NAME,
MIN(PRICE)AS MIN_PRICE,
MAX(PRICE)AS MAX PRICE
FROM ACCESSORIES
GROUP BY NAME;
SELECT A.NAME,A.PRICE,S.NAME FROM ACCESSORIES A JOIN SHOPESS S_NO A.S_NO =S_NO;
SELECT DISTINCT NAME FROM ACCESSORIES WHERE PRICE>=500;
SELECT AREA, COUNT(*)FROM SHOPESS GROUP BY AREA;
SELECT COUNT(DISTINCT AREA)FROM SHOPESS;
SELECT NAME,PRICE*0.05 DISCOUNT FROM ACCESSORIES WHERE NO IN('1','3');
S_ID S_NAME AREA
1001 ABC COMPUTERONICS CP
1003 TECH SHOPE CP
1004 GREEKS TECHNO SOFT NEHRU PLACE
1005 HITECH TECH STORE NEHRU PLACE
1002 ALL INFOTECH MEDIA GK II
1005 HITECH TECH STORE NEHRU PLACE
6 rows returned in 0.00 seconds

6. Write SQL commands for (a) to (f) and Write the outputs for (g) on the basis of table HOSPITAL

(a) To select all the information of patients of cardiology department.

(b) To list the names of female patients who are in ENT department.

(c) To list name of all patients with their date of admission in ascending order.

(d) To display Patient's Name, Charges, Age for only female patients.

(e) To count the number of patients with Age<30.

(f) To inset in a new row in the HOSPITAL table with the following data: 11, "Aftab", 24, "Surgery", {25/02/98}, 300, "M"

(g) Give the output of following SQL statements:

(i) Select COUNT (DISTINCT charges) from HOSPITAL;

(ii) Select MIN (Age) from HOSPITAL where Gender = "F"

(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT"

(iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

PROGRAM

```
CREATE TABLE HOSPITALL (NO NUMBER(3),NAME VARCHAR2(30),AGE
NUMBER(3),DEPARTMENT VARCHAR2(20),DATEOFADM DATE,CHARGES
NUMBER(10,2),GENDER CHAR(1));

INSERT INTO HOSPITALL VALUES(1, 'Ravi', 35, 'Cardiology',
TO_DATE('10/01/1998','DD/MM/YYYY'), 500, 'M');

INSERT INTO HOSPITALL VALUES(2, 'Priya', 25, 'ENT',
TO_DATE('15/03/1998','DD/MM/YYYY'), 350, 'F');

INSERT INTO HOSPITALL VALUES(3, 'Arun', 28, 'Cardiology',
TO_DATE('20/04/1998','DD/MM/YYYY'), 600, 'M');

INSERT INTO HOSPITALL VALUES(4, 'Meena', 22, 'ENT',
TO_DATE('05/05/1998','DD/MM/YYYY'), 400, 'F');

INSERT INTO HOSPITALL VALUES(5, 'Kumar', 45, 'Surgery',
TO_DATE('11/06/1998','DD/MM/YYYY'), 700, 'M');

INSERT INTO HOSPITALL VALUES(6, 'Anitha', 30, 'Cardiology',
TO_DATE('18/07/1998','DD/MM/YYYY'), 550, 'F');
```

```
INSERT INTO HOSPITALL VALUES(7, 'Vijay', 26, 'ENT',
TO_DATE('20/07/1998','DD/MM/YYYY'), 350, 'M');

INSERT INTO HOSPITALL VALUES(8, 'Divya', 29, 'Surgery',
TO_DATE('01/08/1998','DD/MM/YYYY'), 450, 'F');

INSERT INTO HOSPITALL VALUES(9, 'Suresh', 40, 'ENT',
TO_DATE('10/08/1998','DD/MM/YYYY'), 400, 'M');

INSERT INTO HOSPITALL VALUES(10, 'Lakshmi', 27, 'Cardiology',
TO_DATE('15/08/1998','DD/MM/YYYY'), 500, 'F');

SELECT*FROM HOSPITALL;


SELECT *FROM HOSPITALL WHERE DEPARTMENT = 'Cardiology';

SELECT NAME FROM HOSPITALL WHERE GENDER = 'F' AND DEPARTMENT =
'ENT';

SELECT NAME, DATEOFADM FROM HOSPITALL ORDER BY DATEOFADM ASC;

SELECT NAME, CHARGES, AGE FROM HOSPITALL WHERE GENDER = 'F';

SELECT COUNT(*)FROM HOSPITALL WHERE AGE < 30;

INSERT INTO HOSPITALL VALUES (11, 'Aftab', 24, 'Surgery',
TO_DATE('25/02/1998','DD/MM/YYYY'), 300, 'M');

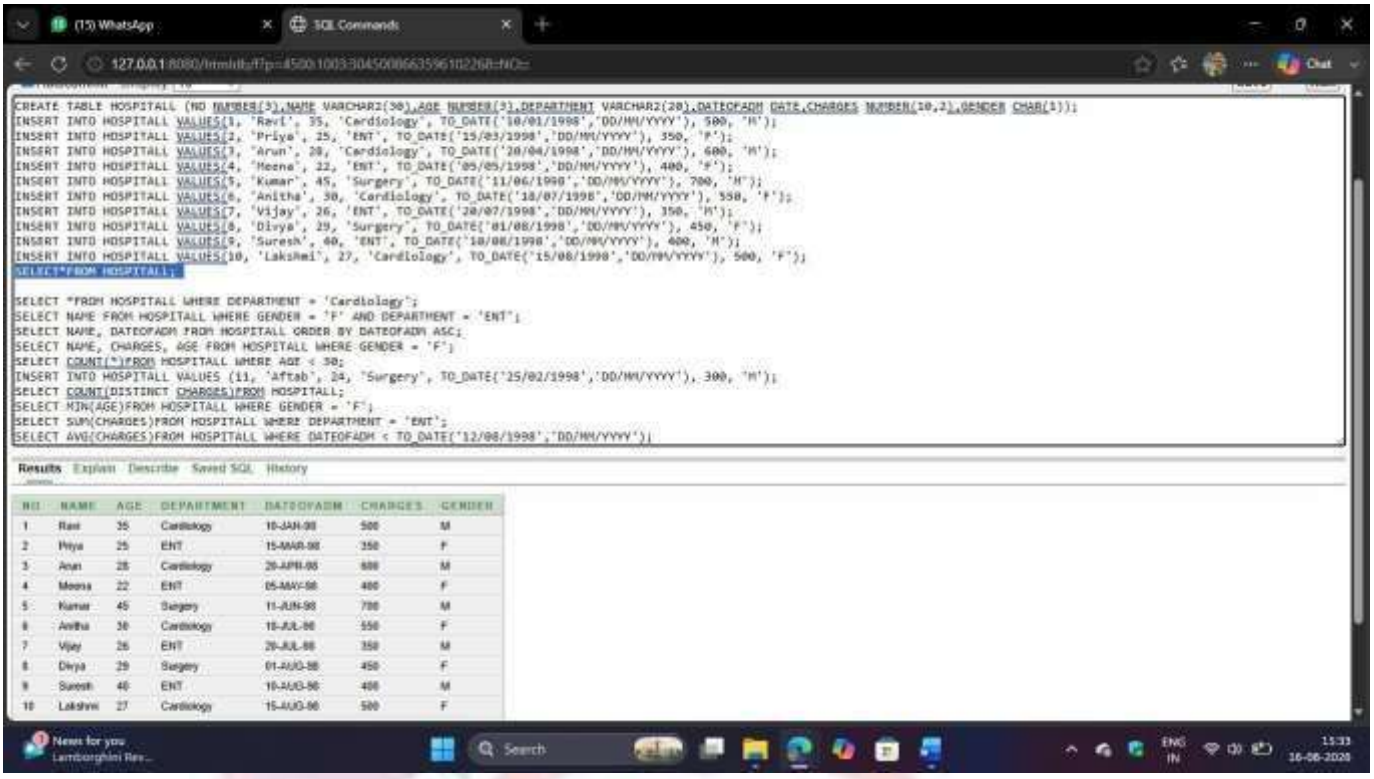
SELECT COUNT(DISTINCT CHARGES)FROM HOSPITALL;

SELECT MIN(AGE)FROM HOSPITALL WHERE GENDER = 'F';

SELECT SUM(CHARGES)FROM HOSPITALL WHERE DEPARTMENT = 'ENT';

SELECT AVG(CHARGES)FROM HOSPITALL WHERE DATEOFADM <
TO_DATE('12/08/1998','DD/MM/YYYY');
```

OUTPUT



AUTONOMOUS

7. Answer the questions (a) and (b) on the basis of the following

tables STORE and ITEM

(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

(ii) To display SNo and SName of all store location in CP.

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

(iv) To display IName, Price of all items and their respective SName where they are available.

(b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;

(iii) SELECT COUNT (DISTINCT Area) FROM STORE:

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

PROGRAM

```
CREATE TABLE STORE (SNo VARCHAR(5),SName VARCHAR(30),Area VARCHAR(20));

INSERT INTO STORE VALUES ('S01', 'ABC Store', 'CP');

INSERT INTO STORE VALUES ('S02', 'City Store', 'CP');

INSERT INTO STORE VALUES ('S03', 'Modern Store', 'Rohini');

INSERT INTO STORE VALUES ('S04', 'Super Store', 'Dwarka');

INSERT INTO STORE VALUES ('S05', 'Mega Store', 'CP');

Select*from STORE;

CREATE TABLE ITEM (INo VARCHAR(5),IName VARCHAR(30),Price DECIMAL(10,2),SNo VARCHAR(5));

INSERT INTO ITEM VALUES ('I01', 'Laptop', 55000, 'S01');

INSERT INTO ITEM VALUES ('I02', 'Mobile', 15000, 'S02');

INSERT INTO ITEM VALUES ('I03', 'Printer', 5000, 'S03');
```

INSERT INTO ITEM VALUES ('I04', 'Laptop', 60000, 'S04');

INSERT INTO ITEM VALUES ('I05', 'Tablet', 8000, 'S02');

INSERT INTO ITEM VALUES ('I06', 'Mobile', 12000, 'S05');

Select*from ITEM;

SELECT IName, Price FROM ITEM ORDER BY Price ASC;

SELECT SNo, SName FROM STORE WHERE Area = 'CP';

SELECT IName, MIN(Price), MAX(Price)FROM ITEM GROUP BY IName;

SELECT ITEM.IName, ITEM.Price, STORE.SName FROM ITEM INNER JOIN STORE ON ITEM.SNo = STORE.SNo;

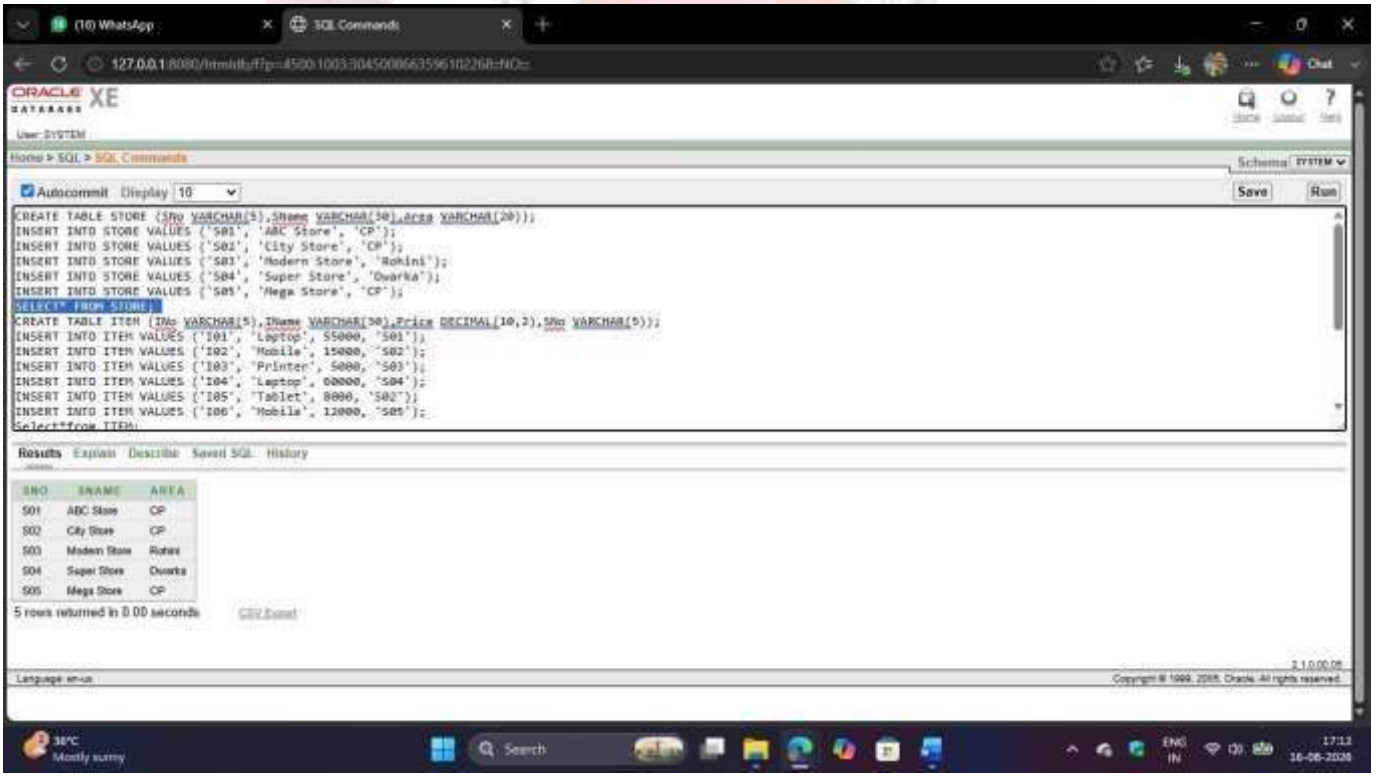
SELECT DISTINCT IName FROM ITEM WHERE Price >= 5000;

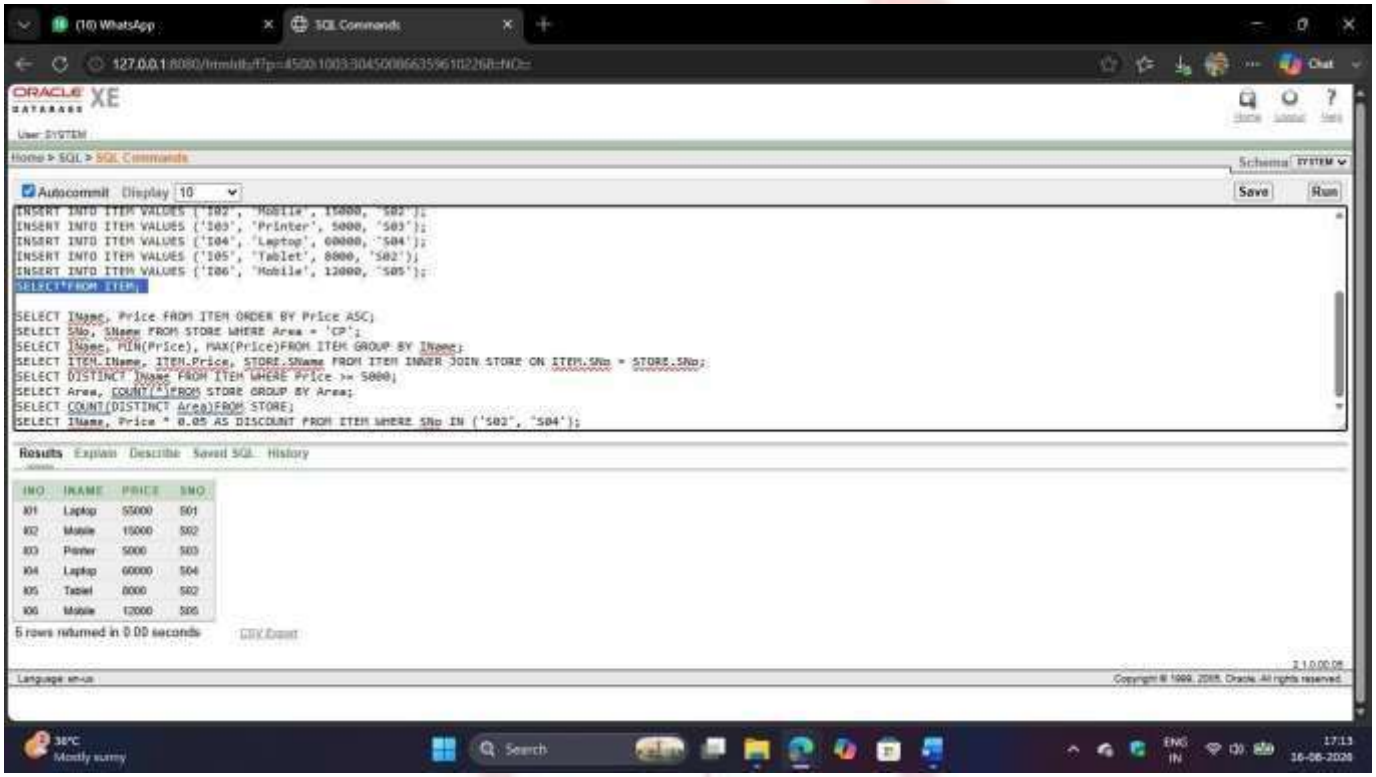
SELECT Area, COUNT(*)FROM STORE GROUP BY Area;

SELECT COUNT(DISTINCT Area)FROM STORE;

SELECT IName, Price * 0.05 AS DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S04');

OUTPUT




(10) WhatsApp
SQL Commands
127.0.0.1:8080/
ORACLE XE
DATABASE
User: SYSTEM
Home > SQL > SQL Commands
Schema: SYSTEM
Autocommit
Display 10
Save
Run
INSERT INTO ITEM VALUES ('I02', 'Mobile', 15000, 'S02');
INSERT INTO ITEM VALUES ('I03', 'Printer', 5000, 'S03');
INSERT INTO ITEM VALUES ('I04', 'Laptop', 60000, 'S04');
INSERT INTO ITEM VALUES ('I05', 'Tablet', 8000, 'S02');
INSERT INTO ITEM VALUES ('I06', 'Mobile', 12000, 'S05');
SELECT*FROM ITEM;
SELECT IName, Price FROM ITEM ORDER BY Price ASC;
SELECT SNo, SName FROM STORE WHERE Area = 'CP';
SELECT IName, MIN(Price), MAX(Price)FROM ITEM GROUP BY IName;
SELECT ITEM.IName, ITEM.Price, STORE.SName FROM ITEM INNER JOIN STORE ON ITEM.SNo = STORE.SNo;
SELECT DISTINCT IName FROM ITEM WHERE Price >= 5000;
SELECT Area, COUNT(*)FROM STORE GROUP BY Area;
SELECT COUNT(DISTINCT Area)FROM STORE;
SELECT IName, Price * 0.05 AS DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S04');
Results
Explain
Describe
Saved SQL
History
IMO INAME PRICE SNO
I01 Laptop 55000 S01
I02 Mobile 15000 S02
I03 Printer 5000 S03
I04 Laptop 60000 S04
I05 Tablet 8000 S02
I06 Mobile 12000 S05
6 rows returned in 0.00 seconds
CSV Export
Language: en-us
Copyright © 1999, 2005, Oracle. All rights reserved.
38°C
Mostly sunny
Search
ENG
IN
17:13
16-06-2026

8. Write SQL commands for (a) to (f) and write output for (g) on the basis of Teacher relation given below:

(a) To show all information about the teacher of history department

(b) To list the names of female teacher who are in Hindi department

(c) To list names of all teachers with their date of joining in ascending order.

(d) To display teacher's Name, Salary, Age for male teacher only

(e) To count the number of teachers with Age>23.

(f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"

(g) Give the output of following SQL statements:

(i) Select COUNT (distinct department) from TEACHER;

(II) Select MAX (Age) from TEACHER where Gender= "F"

(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};

(iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

PROGRAM

```
CREATE TABLE TEACHER (TNo INT,Name VARCHAR(30),Age INT,Department VARCHAR(20),DateOfJoin DATE,Salary INT,Gender CHAR(1));

INSERT INTO TEACHER VALUES(1, 'Anita', 25, 'History', TO_DATE('10/05/1994','DD/MM/YYYY'), 2500, 'F');

INSERT INTO TEACHER VALUES(2, 'Ravi', 28, 'Hindi', TO_DATE('15/03/1995','DD/MM/YYYY'), 3000, 'M');

INSERT INTO TEACHER VALUES(3, 'Priya', 24, 'Hindi', TO_DATE('20/01/1996','DD/MM/YYYY'), 2800, 'F');

INSERT INTO TEACHER VALUES(4, 'Kumar', 22, 'History', TO_DATE('12/06/1997','DD/MM/YYYY'), 2200, 'M');

INSERT INTO TEACHER VALUES(5, 'Meena', 30, 'Computer', TO_DATE('25/08/1993','DD/MM/YYYY'), 3500, 'F');

INSERT INTO TEACHER VALUES(6, 'Suresh', 27, 'English', TO_DATE('10/11/1995','DD/MM/YYYY'), 3200, 'M');
```

INSERT INTO TEACHER VALUES(7, 'Lakshmi', 23, 'Computer',
TO_DATE('18/05/1996','DD/MM/YYYY'), 2700, 'F');

INSERT INTO TEACHER VALUES(8, 'Arun', 26, 'English',
TO_DATE('05/12/1994','DD/MM/YYYY'), 2900, 'M');

SELECT *FROM TEACHER;

SELECT *FROM TEACHER WHERE Department = 'History';

SELECT*FROM TEACHER WHERE Gender = 'F' AND Department = 'Hindi';

SELECT Name, DateOfJoin FROM TEACHER ORDER BY DateOfJoin ASC;

SELECT Name, Salary, Age FROM TEACHER WHERE Gender = 'M';

SELECT COUNT(*)FROM TEACHER WHERE Age > 23;

INSERT INTO TEACHER VALUES (9, 'Raja', 26,
'Computer',TO_DATE('07/12/1999','DD/MM/YYYY'), 2300, 'M');

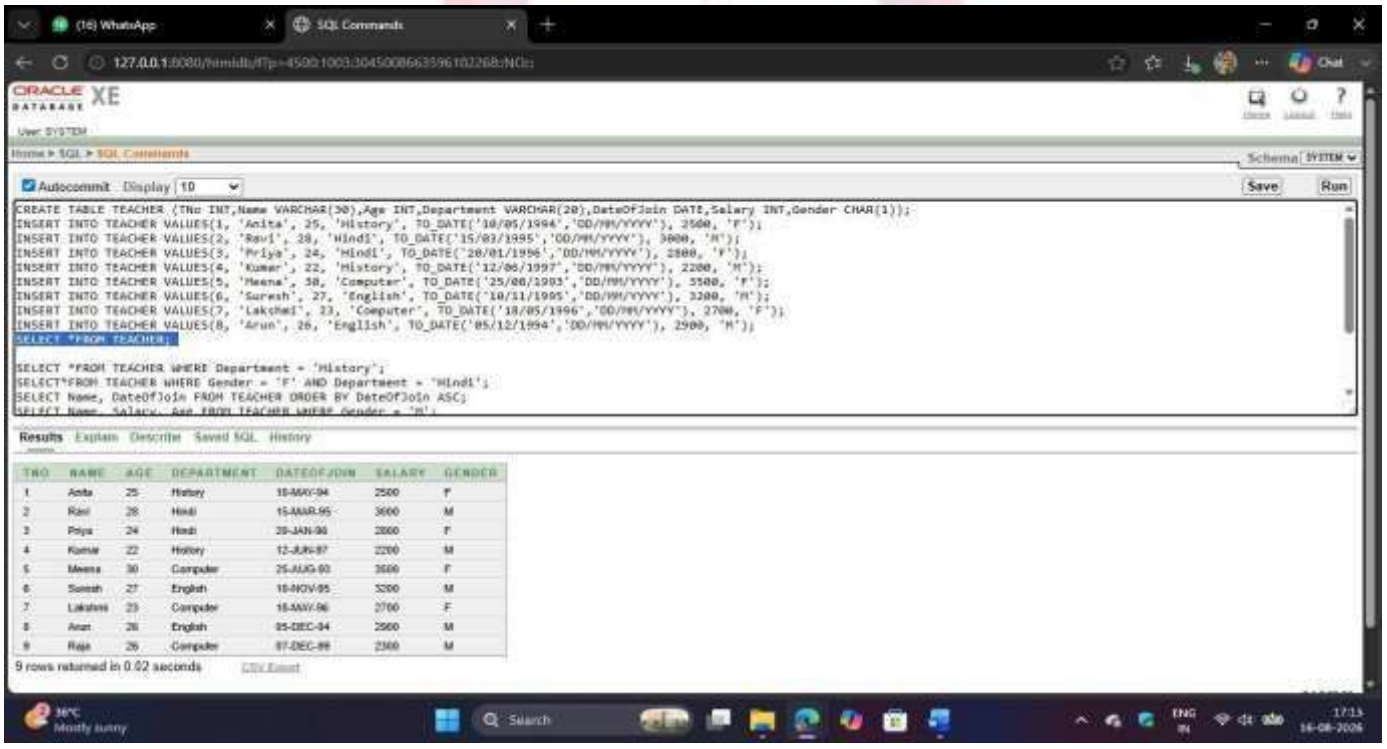
SELECT COUNT(DISTINCT Department)FROM TEACHER;

SELECT MAX(Age)FROM TEACHER WHERE Gender = 'F';

SELECT AVG(Salary)FROM TEACHER WHERE DateOfJoin < TO_DATE('12/07/1996',
'DD/MM/YYYY');

SELECT SUM(Salary)FROM TEACHER WHERE DateOfJoin < TO_DATE('12/07/1996',
'DD/MM/YYYY');

OUTPUT



9. Given the following tables for a database LIBRARY:

Write SQL queries for (a) to (f):

(a) To show Book name, Author name and Price of books of EPB publishers.

(b) To list the names of the books of Fiction Type.

(c) To display the names and price of the books in descending order of their price.

(d) To increase the price of all books of first publisher by 50.

(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).

(f) To insert a new row in the table Issued following the data: "F0002",4

(g) Give the output of the following queries based on the above tables:

(i) SELECT COUNT(DISTINCT Publishers) FROM Books;

(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;

(iv) SELECT COUNT (*) FROM Books;

PROGRAM

```
CREATE TABLE BOOKSS (BOOK_ID VARCHAR2(10),BOOK_NAME
VARCHAR2(40),AUTHOR_NAME VARCHAR2(40),PRICE
NUMBER(10,2),PUBLISHERS VARCHAR2(30),TYPE VARCHAR2(20),QUANTITY
NUMBER(5));

INSERT INTO BOOKSS VALUES('F0001', 'The Guide', 'R.K. Narayan', 450, 'EPB',
'Fiction', 10);

INSERT INTO BOOKSS VALUES('F0002', 'Malgudi Days', 'R.K. Narayan', 400, 'EPB',
'Fiction', 8);

INSERT INTO BOOKSS VALUES('F0003', 'Wings of Fire', 'A.P.J. Abdul Kalam', 550, 'PHI',
'Biography', 6);

INSERT INTO BOOKSS VALUES('F0004', 'The Alchemist', 'Paulo Coelho', 600, 'Penguin',
'Fiction', 4);

INSERT INTO BOOKSS VALUES('F0005', 'Discovery of India', 'Jawaharlal Nehru', 700,
'EPB', 'History', 3);
```

```
INSERT INTO BOOKSS VALUES('F0006', 'Ignited Minds', 'A.P.J. Abdul Kalam', 350, 'PHI', 'Biography', 7);

SELECT*FROM BOOKSS;


CREATE TABLE ISSUED (BOOK_ID VARCHAR2(10),QUANTITY NUMBER(5));

INSERT INTO ISSUED VALUES ('F0001', 2);

INSERT INTO ISSUED VALUES ('F0003', 1);

INSERT INTO ISSUED VALUES ('F0005', 3);

SELECT * FROM ISSUED;

SELECT BOOK_NAME, AUTHOR_NAME, PRICE FROM BOOKSS WHERE PUBLISHERS = 'EPB';

SELECT BOOK_NAME FROM BOOKSS WHERE TYPE = 'Fiction';

SELECT BOOK_NAME, PRICE FROM BOOKSS ORDER BY PRICE DESC;

UPDATE BOOKSS SET PRICE = PRICE + 50 WHERE PUBLISHERS = 'EPB';

SELECT B.BOOK_ID, B.BOOK_NAME, I.QUANTITY FROM BOOKSS B JOIN ISSUED I ON B.BOOK_ID = I.BOOK_ID;

INSERT INTO ISSUED VALUES ('F0002', 4);

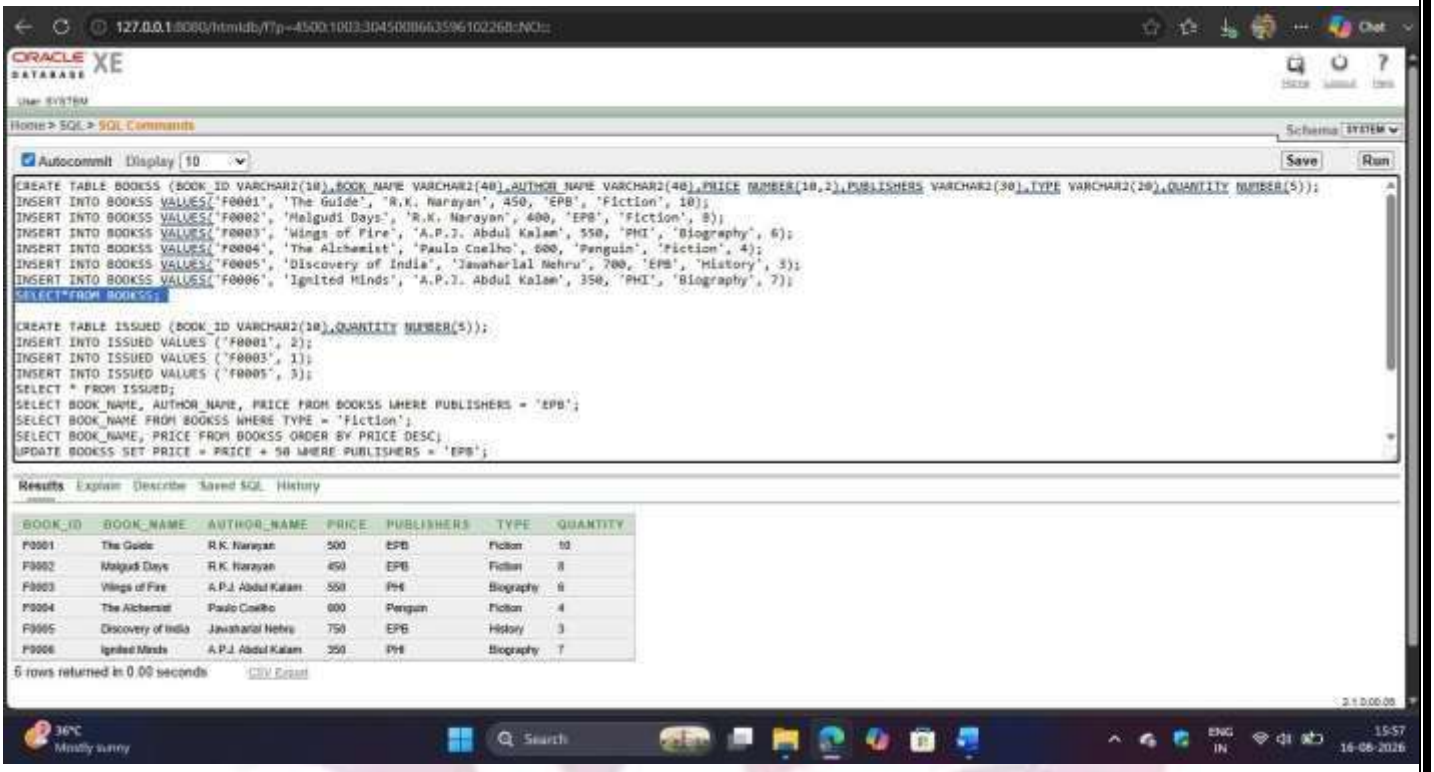
SELECT COUNT(DISTINCT PUBLISHERS)FROM BOOKSS;

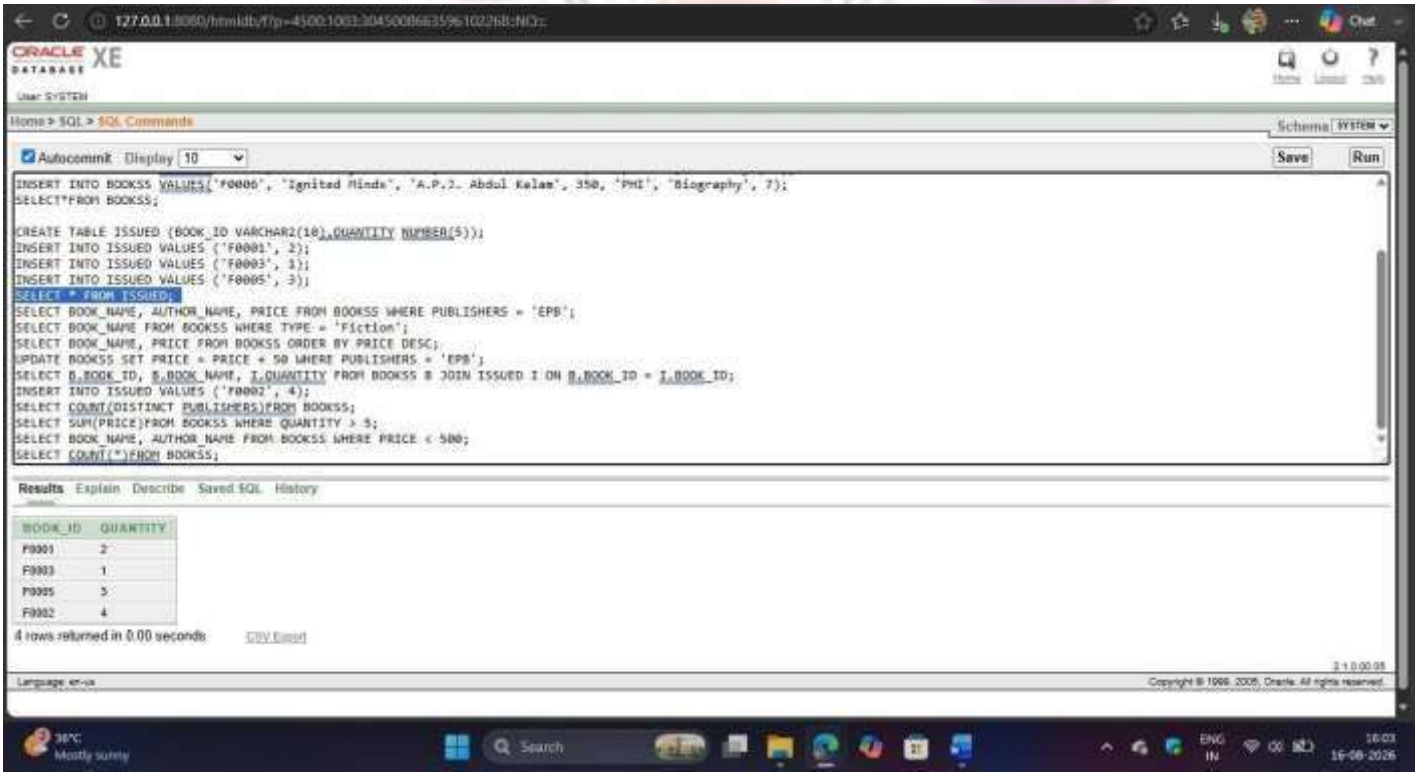
SELECT SUM(PRICE)FROM BOOKSS WHERE QUANTITY > 5;

SELECT BOOK_NAME, AUTHOR_NAME FROM BOOKSS WHERE PRICE < 500;

SELECT COUNT(*)FROM BOOKSS;
```

OUTPUT


ORACLE XE
DATABASE
User: SYSTEM
Home > SQL > SQL Commands
Schema: SYSTEM
Autocommit Display 10
Save
Run
CREATE TABLE BOOKSS (BOOK_ID VARCHAR2(10),BOOK_NAME VARCHAR2(40),AUTHOR_NAME VARCHAR2(40),PRICE NUMBER(10,2),PUBLISHERS VARCHAR2(30),TYPE VARCHAR2(20),QUANTITY NUMBER(5));
INSERT INTO BOOKSS VALUES('F0001', 'The Guide', 'R.K. Narayan', 450, 'EPB', 'Fiction', 10);
INSERT INTO BOOKSS VALUES('F0002', 'Malgudi Days', 'R.K. Narayan', 400, 'EPB', 'Fiction', 8);
INSERT INTO BOOKSS VALUES('F0003', 'Wings of Fire', 'A.P.J. Abdul Kalam', 550, 'PHI', 'Biography', 6);
INSERT INTO BOOKSS VALUES('F0004', 'The Alchemist', 'Paulo Coelho', 600, 'Penguin', 'Fiction', 4);
INSERT INTO BOOKSS VALUES('F0005', 'Discovery of India', 'Jawaharlal Nehru', 700, 'EPB', 'History', 3);
INSERT INTO BOOKSS VALUES('F0006', 'Ignited Minds', 'A.P.J. Abdul Kalam', 350, 'PHI', 'Biography', 7);
SELECT*FROM BOOKSS;
CREATE TABLE ISSUED (BOOK_ID VARCHAR2(10),QUANTITY NUMBER(5));
INSERT INTO ISSUED VALUES ('F0001', 2);
INSERT INTO ISSUED VALUES ('F0003', 1);
INSERT INTO ISSUED VALUES ('F0005', 3);
SELECT * FROM ISSUED;
SELECT BOOK_NAME, AUTHOR_NAME, PRICE FROM BOOKSS WHERE PUBLISHERS = 'EPB';
SELECT BOOK_NAME FROM BOOKSS WHERE TYPE = 'Fiction';
SELECT BOOK_NAME, PRICE FROM BOOKSS ORDER BY PRICE DESC;
UPDATE BOOKSS SET PRICE = PRICE + 50 WHERE PUBLISHERS = 'EPB';
Results Explain Describe Saved SQL History
BOOK_ID BOOK_NAME AUTHOR_NAME PRICE PUBLISHERS TYPE QUANTITY
F0001 The Guide R.K. Narayan 500 EPB Fiction 10
F0002 Malgudi Days R.K. Narayan 450 EPB Fiction 8
F0003 Wings of Fire A.P.J. Abdul Kalam 550 PHI Biography 6
F0004 The Alchemist Paulo Coelho 600 Penguin Fiction 4
F0005 Discovery of India Jawaharlal Nehru 750 EPB History 3
F0006 Ignited Minds A.P.J. Abdul Kalam 350 PHI Biography 7
6 rows returned in 0.00 seconds
CSV Export


ORACLE XE
DATABASE
User: SYSTEM
Home > SQL > SQL Commands
Schema: SYSTEM
Autocommit Display 10
Save
Run
INSERT INTO BOOKSS VALUES('F0006', 'Ignited Minds', 'A.P.J. Abdul Kalam', 350, 'PHI', 'Biography', 7);
SELECT*FROM BOOKSS;
CREATE TABLE ISSUED (BOOK_ID VARCHAR2(10),QUANTITY NUMBER(5));
INSERT INTO ISSUED VALUES ('F0001', 2);
INSERT INTO ISSUED VALUES ('F0003', 1);
INSERT INTO ISSUED VALUES ('F0005', 3);
SELECT * FROM ISSUED;
SELECT BOOK_NAME, AUTHOR_NAME, PRICE FROM BOOKSS WHERE PUBLISHERS = 'EPB';
SELECT BOOK_NAME FROM BOOKSS WHERE TYPE = 'Fiction';
SELECT BOOK_NAME, PRICE FROM BOOKSS ORDER BY PRICE DESC;
UPDATE BOOKSS SET PRICE = PRICE + 50 WHERE PUBLISHERS = 'EPB';
SELECT B.BOOK_ID, B.BOOK_NAME, I.QUANTITY FROM BOOKSS B JOIN ISSUED I ON B.BOOK_ID = I.BOOK_ID;
INSERT INTO ISSUED VALUES ('F0002', 4);
SELECT COUNT(DISTINCT PUBLISHERS)FROM BOOKSS;
SELECT SUM(PRICE)FROM BOOKSS WHERE QUANTITY > 5;
SELECT BOOK_NAME, AUTHOR_NAME FROM BOOKSS WHERE PRICE < 500;
SELECT COUNT(*)FROM BOOKSS;
Results Explain Describe Saved SQL History
BOOK_ID QUANTITY
F0001 2
F0003 1
F0005 3
F0002 4
4 rows returned in 0.00 seconds
CSV Export

10. Consider the following tables CARDEN and CUSTOMER and answer (a) and (b) parts of this question:
(a) Write SQL commands for the following statements:

(i) To display the name of all the SILVER colored cars.

(ii) To display name of car, make and capacity of cars in descending order of their sitting

capacity.

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

(iv) To display the customer name and the corresponding name of the cards hired by them.

(b) Give the output of the following SQL queries:i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

(iii) SELECT COUNT (*), Make FROM CARDEN;

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

PROGRAM

```
CREATE TABLE CARDEN (CarNo VARCHAR2(10),CarName VARCHAR2(30),Make VARCHAR2(20),Color VARCHAR2(15),Capacity NUMBER,Charges NUMBER);

INSERT INTO CARDEN VALUES('C01', 'Swift', 'Maruti', 'SILVER', 4, 1500);

INSERT INTO CARDEN VALUES('C02', 'Innova', 'Toyota', 'WHITE', 7, 2500);

INSERT INTO CARDEN VALUES('C03', 'Verna', 'Hyundai', 'BLACK', 5, 2000);

INSERT INTO CARDEN VALUES('C04', 'Ertiga', 'Maruti', 'SILVER', 7, 2200);

INSERT INTO CARDEN VALUES('C05', 'City', 'Honda', 'BLUE', 5, 2100);

INSERT INTO CARDEN VALUES('C06', 'Alto', 'Maruti', 'RED', 4, 1000);

SELECT*FROM CARDEN;

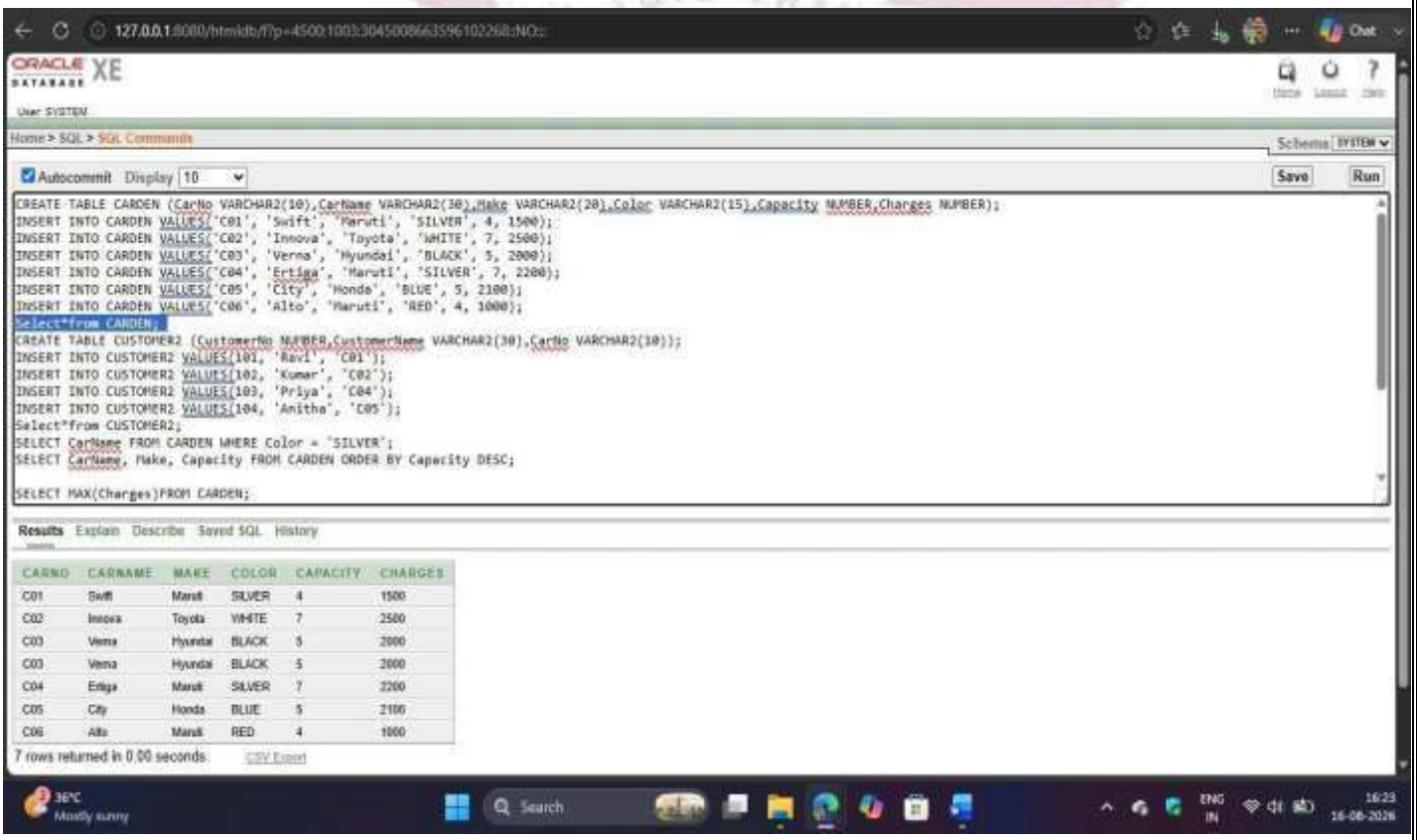

CREATE TABLE CUSTOMER2 (CustomerNo NUMBER,CustomerName VARCHAR2(30),CarNo VARCHAR2(10));

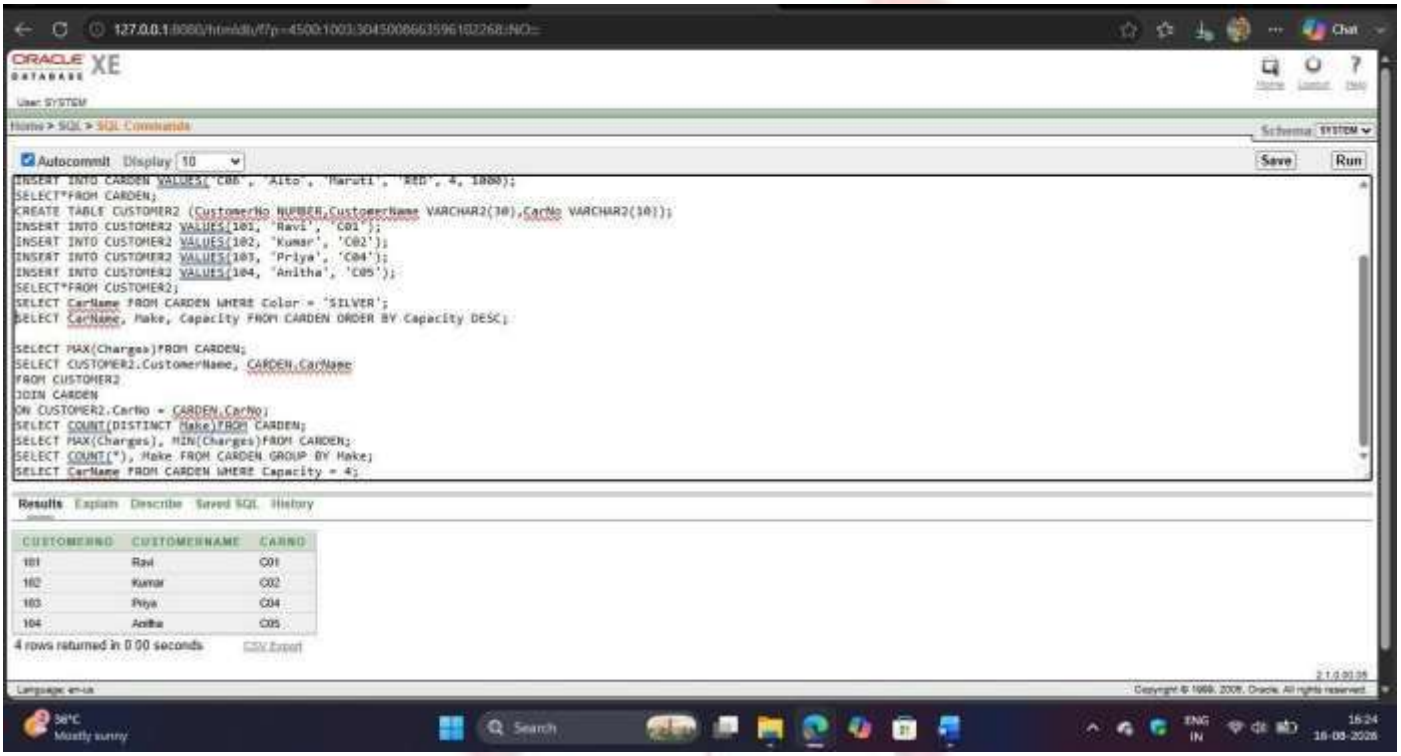
INSERT INTO CUSTOMER2 VALUES(101, 'Ravi', 'C01');

INSERT INTO CUSTOMER2 VALUES(102, 'Kumar', 'C02');
```

```
INSERT INTO CUSTOMER2 VALUES(103, 'Priya', 'C04');
INSERT INTO CUSTOMER2 VALUES(104, 'Anitha', 'C05');
SELECT*FROM CUSTOMER2;
SELECT CarName FROM CARDEN WHERE Color = 'SILVER';
SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;
SELECT MAX(Charges)FROM CARDEN;
SELECT CUSTOMER2.CustomerName, CARDEN.CarName FROM CUSTOMER2
JOIN CARDEN ON CUSTOMER2.CarNo = CARDEN.CarNo;
SELECT COUNT(DISTINCT Make)FROM CARDEN;
SELECT MAX(Charges), MIN(Charges)FROM CARDEN;
SELECT COUNT(*), Make FROM CARDEN GROUP BY Make;
SELECT CarName FROM CARDEN WHERE Capacity = 4;
```

OUTPUT




ORACLE XE
DATABASE
User: SYSTEM
Home > SQL > SQL Commands
Schema: SYSTEM
Autocommit Display 10
Save
Run
INSERT INTO CARDEN VALUES('C05', 'Alto', 'Maruti', 'RED', 4, 1000);
SELECT*FROM CARDEN;
CREATE TABLE CUSTOMER2 (CustomerNo NUMBER,CustomerName VARCHAR2(30),CarNo VARCHAR2(10));
INSERT INTO CUSTOMER2 VALUES(101, 'Ravi', 'C01');
INSERT INTO CUSTOMER2 VALUES(102, 'Kumar', 'C02');
INSERT INTO CUSTOMER2 VALUES(103, 'Priya', 'C04');
INSERT INTO CUSTOMER2 VALUES(104, 'Anitha', 'C05');
SELECT*FROM CUSTOMER2;
SELECT CarName FROM CARDEN WHERE Color = 'SILVER';
SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;
SELECT MAX(Charges)FROM CARDEN;
SELECT CUSTOMER2.CustomerName, CARDEN.CarName
FROM CUSTOMER2
JOIN CARDEN
ON CUSTOMER2.CarNo = CARDEN.CarNo;
SELECT COUNT(DISTINCT Make)FROM CARDEN;
SELECT MAX(Charges), MIN(Charges)FROM CARDEN;
SELECT COUNT(*), Make FROM CARDEN GROUP BY Make;
SELECT CarName FROM CARDEN WHERE Capacity = 4;
Results Explain Describe Saved SQL History
CUSTOMERNO CUSTOMERNAME CARNO
101 Ravi C01
102 Kumar C02
103 Priya C04
104 Anitha C05
4 rows returned in 0.00 seconds CSV Export
Copyright © 1999, 2005, Oracle. All rights reserved.
36°C
Mostly sunny
Search
ENG
IN
16:24
16-09-2026

11. Consider the following tables EMPLOYEE and SALGRADE and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB-2006' and '08-AUG-2009'.

(iv) TO add a new row in the EMPLOYEE table with the following data:

109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S

WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

PROGRAM

```
CREATE TABLE EMPLOYEE1 (ECODE NUMBER(5),NAME VARCHAR2(30),DESIGN VARCHAR2(30),SGRADE VARCHAR2(5),DOJ DATE,DOB DATE);

INSERT INTO EMPLOYEE1 VALUES(101, 'Amit Kumar', 'MANAGER', 'S03',TO_DATE('10-JAN-2005','DD-MON-YYYY'),TO_DATE('15-MAR-1975','DD-MON-YYYY'));

INSERT INTO EMPLOYEE1 VALUES(102, 'Ravi Sharma', 'CLERK', 'S02',TO_DATE('15-JUN-2007','DD-MON-YYYY'),TO_DATE('20-JUL-1980','DD-MON-YYYY'));

INSERT INTO EMPLOYEE1 VALUES(103, 'Priya Singh', 'ANALYST', 'S03',TO_DATE('20-MAR-2008','DD-MON-YYYY'),TO_DATE('10-DEC-1982','DD-MON-YYYY'));

INSERT INTO EMPLOYEE1 VALUES(104, 'Kumar Raj', 'CLERK', 'S01',TO_DATE('05-AUG-2009','DD-MON-YYYY'),TO_DATE('25-JAN-1985','DD-MON-YYYY'));
```

```
INSERT INTO EMPLOYEE1 VALUES(105, 'Anita Das', 'EXECUTIVE',
'S02',TO_DATE('12-MAY-2006','DD-MON-YYYY'),TO_DATE('18-NOV-1979','DD-MON-
YYYY'));

INSERT INTO EMPLOYEE1 VALUES(106, 'Vijay Roy', 'MANAGER',
'S03',TO_DATE('18-FEB-2008','DD-MON-YYYY'),TO_DATE('05-MAY-1978','DD-MON-
YYYY'));

SELECT*FROM EMPLOYEE1;


CREATE TABLE SALGRADE (SGRADE VARCHAR2(5),SALARY NUMBER(10,2),HRA
NUMBER(10,2));

INSERT INTO SALGRADE VALUES ('S01', 25000, 5000);

INSERT INTO SALGRADE VALUES ('S02', 35000, 7000);

INSERT INTO SALGRADE VALUES ('S03', 45000, 9000);

INSERT INTO SALGRADE VALUES ('S04', 55000, 11000);

SELECT*FROM SALGRADE;


SELECT *FROM EMPLOYEE1 ORDER BY DOJ DESC;

SELECT NAME, DESIGN FROM EMPLOYEE1 WHERE SGRADE IN ('S02', 'S03');

SELECT *FROM EMPLOYEE1 WHERE DOJ BETWEEN TO_DATE('09-FEB-2006','DD-
MON-YYYY') AND TO_DATE('08-AUG-2009','DD-MON-YYYY');

INSERT INTO EMPLOYEE1 VALUES (109,'Harish Roy','HEAD-IT','S02',TO_DATE('09-
SEP-2007','DD-MON-YYYY'),TO_DATE('21-APR-1983','DD-MON-YYYY'));

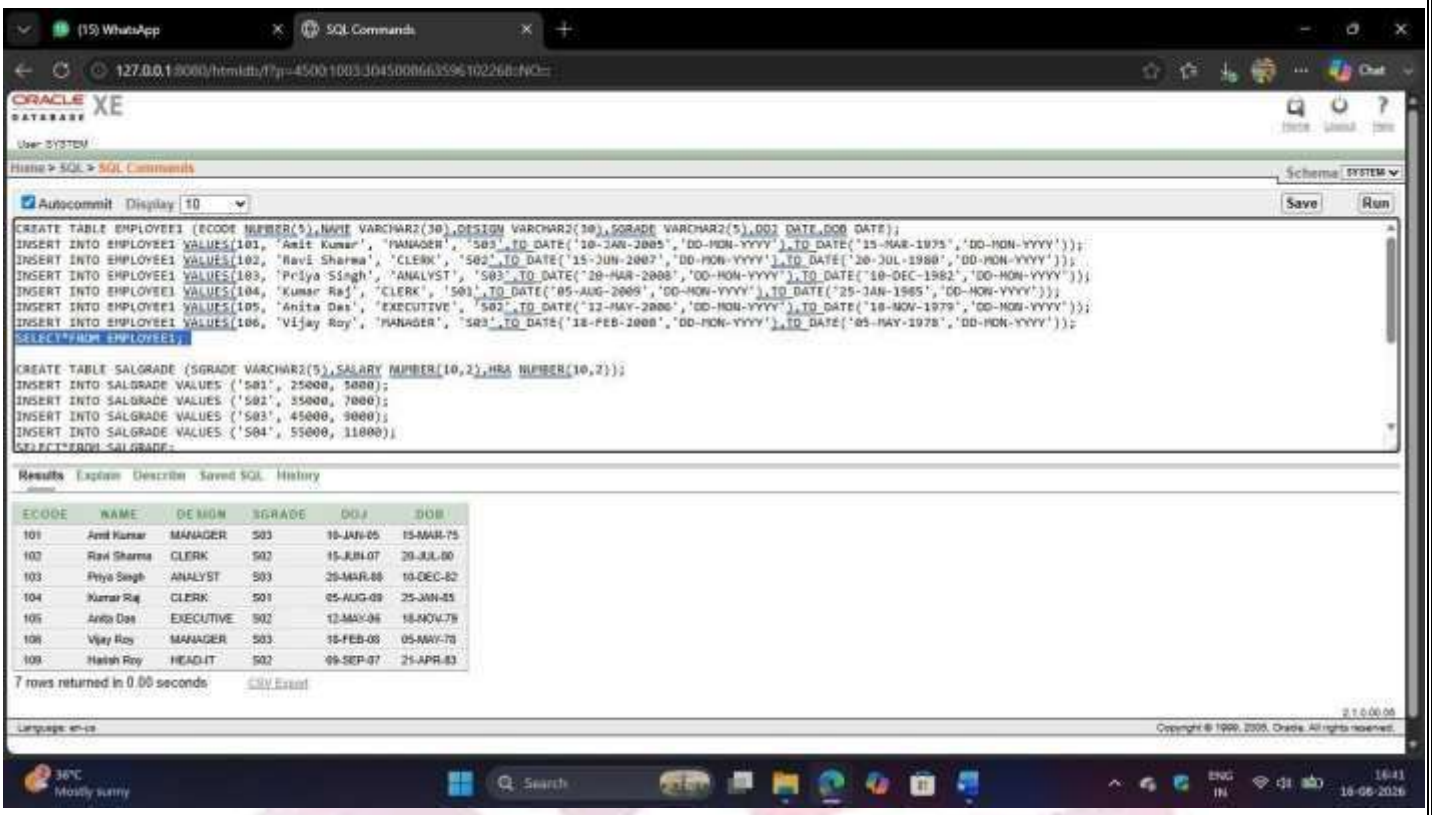
SELECT COUNT(SGRADE), SGRADE FROM EMPLOYEE1 GROUP BY SGRADE;

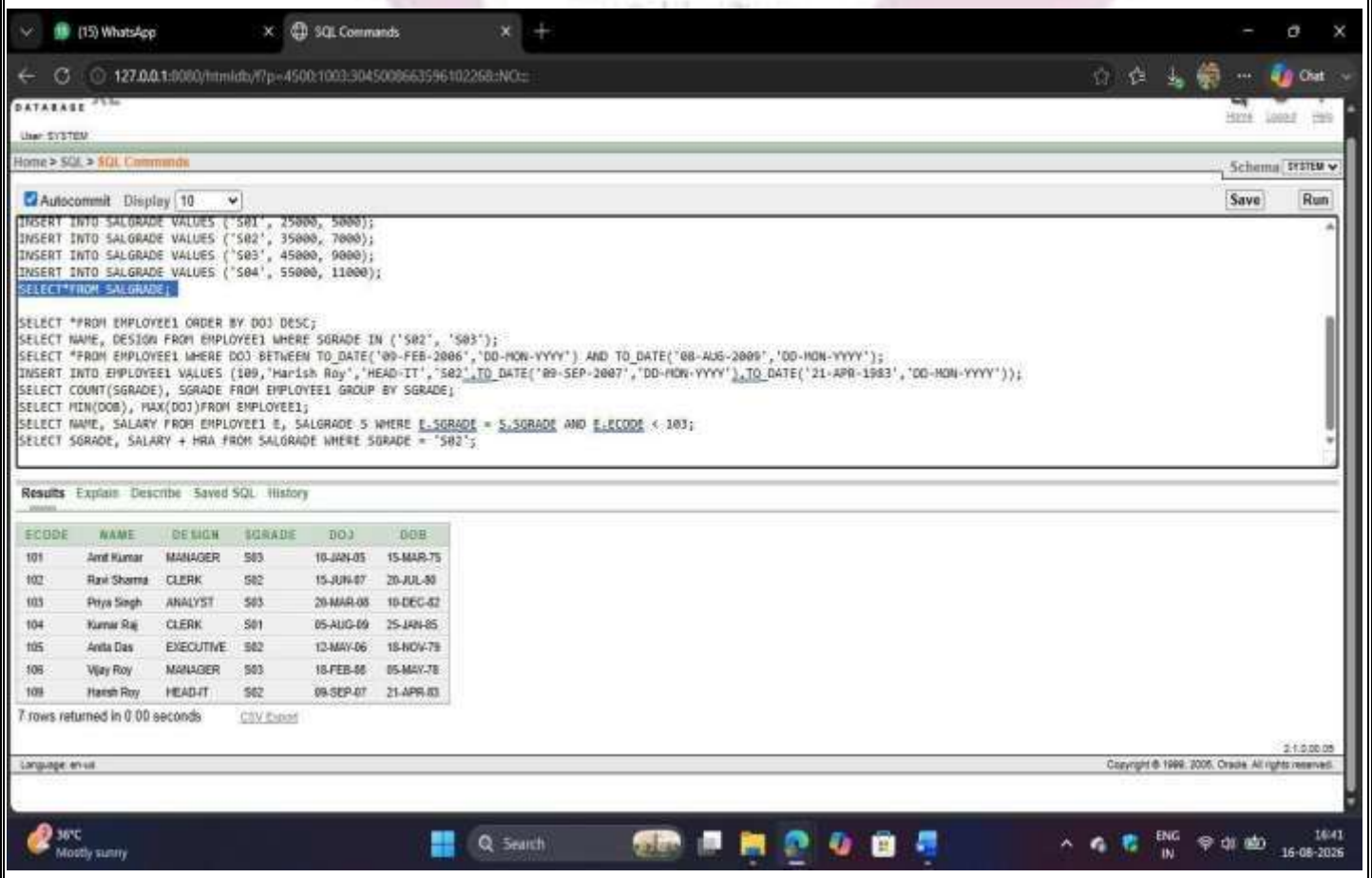
SELECT MIN(DOB), MAX(DOJ)FROM EMPLOYEE1;

SELECT NAME, SALARY FROM EMPLOYEE1 E, SALGRADE S WHERE E.SGRADE
= S.SGRADE AND E.ECODE < 103;

SELECT SGRADE, SALARY + HRA FROM SALGRADE WHERE SGRADE = 'S02';
```

OUTPUT


(15) WhatsApp
SQL Commands
127.0.0.1:8080/htmldb/f?p=4500:1003:3045008663596102268::NO::
ORACLE XE
DATABASE
User: SYSTEM
Home > SQL > SQL Commands
Schema: SYSTEM
Autocommit Display 10
Save
Run
CREATE TABLE EMPLOYEE1 (ECODE NUMBER(5),NAME VARCHAR2(30),DESIGN VARCHAR2(30),SGRADE VARCHAR2(5),DOJ DATE,DOB DATE);
INSERT INTO EMPLOYEE1 VALUES(101, 'Amit Kumar', 'MANAGER', 'S03',TO_DATE('10-JAN-2005','DD-MON-YYYY'),TO_DATE('15-MAR-1975','DD-MON-YYYY'));
INSERT INTO EMPLOYEE1 VALUES(102, 'Ravi Sharma', 'CLERK', 'S02',TO_DATE('15-JUN-2007','DD-MON-YYYY'),TO_DATE('20-JUL-1980','DD-MON-YYYY'));
INSERT INTO EMPLOYEE1 VALUES(103, 'Priya Singh', 'ANALYST', 'S03',TO_DATE('20-MAR-2008','DD-MON-YYYY'),TO_DATE('10-DEC-1982','DD-MON-YYYY'));
INSERT INTO EMPLOYEE1 VALUES(104, 'Kumar Raj', 'CLERK', 'S01',TO_DATE('05-AUG-2009','DD-MON-YYYY'),TO_DATE('25-JAN-1985','DD-MON-YYYY'));
INSERT INTO EMPLOYEE1 VALUES(105, 'Anita Das', 'EXECUTIVE', 'S02',TO_DATE('12-MAY-2006','DD-MON-YYYY'),TO_DATE('18-NOV-1979','DD-MON-YYYY'));
INSERT INTO EMPLOYEE1 VALUES(106, 'Vijay Roy', 'MANAGER', 'S03',TO_DATE('18-FEB-2008','DD-MON-YYYY'),TO_DATE('05-MAY-1978','DD-MON-YYYY'));
SELECT*FROM EMPLOYEE1;
CREATE TABLE SALGRADE (SGRADE VARCHAR2(5),SALARY NUMBER(10,2),HRA NUMBER(10,2));
INSERT INTO SALGRADE VALUES ('S01', 25000, 5000);
INSERT INTO SALGRADE VALUES ('S02', 35000, 7000);
INSERT INTO SALGRADE VALUES ('S03', 45000, 9000);
INSERT INTO SALGRADE VALUES ('S04', 55000, 11000);
SELECT*FROM SALGRADE;
Results Explain Describe Saved SQL History
ECODE NAME DESIGN SGRADE DOJ DOB
101 Amit Kumar MANAGER S03 10-JAN-05 15-MAR-75
102 Ravi Sharma CLERK S02 15-JUN-07 20-JUL-80
103 Priya Singh ANALYST S03 20-MAR-08 10-DEC-82
104 Kumar Raj CLERK S01 05-AUG-09 25-JAN-85
105 Anita Das EXECUTIVE S02 12-MAY-06 18-NOV-79
106 Vijay Roy MANAGER S03 18-FEB-08 05-MAY-78
109 Harish Roy HEAD-IT S02 09-SEP-07 21-APR-83
7 rows returned in 0.00 seconds
CSV Export
Language: en-us
Copyright © 1999, 2005, Oracle. All rights reserved.
36°C
Mostly sunny
Search
ENG
IN
16:41
16-08-2026


(15) WhatsApp
SQL Commands
127.0.0.1:8080/htmldb/f?p=4500:1003:3045008663596102268::NO::
DATABASE
User: SYSTEM
Home > SQL > SQL Commands
Schema: SYSTEM
Autocommit Display 10
Save
Run
INSERT INTO SALGRADE VALUES ('S01', 25000, 5000);
INSERT INTO SALGRADE VALUES ('S02', 35000, 7000);
INSERT INTO SALGRADE VALUES ('S03', 45000, 9000);
INSERT INTO SALGRADE VALUES ('S04', 55000, 11000);
SELECT*FROM SALGRADE;
SELECT *FROM EMPLOYEE1 ORDER BY DOJ DESC;
SELECT NAME, DESIGN FROM EMPLOYEE1 WHERE SGRADE IN ('S02', 'S03');
SELECT *FROM EMPLOYEE1 WHERE DOJ BETWEEN TO_DATE('09-FEB-2006','DD-MON-YYYY') AND TO_DATE('08-AUG-2009','DD-MON-YYYY');
INSERT INTO EMPLOYEE1 VALUES (109,'Harish Roy','HEAD-IT','S02',TO_DATE('09-SEP-2007','DD-MON-YYYY'),TO_DATE('21-APR-1983','DD-MON-YYYY'));
SELECT COUNT(SGRADE), SGRADE FROM EMPLOYEE1 GROUP BY SGRADE;
SELECT MIN(DOB), MAX(DOJ)FROM EMPLOYEE1;
SELECT NAME, SALARY FROM EMPLOYEE1 E, SALGRADE S WHERE E.SGRADE = S.SGRADE AND E.ECODE < 103;
SELECT SGRADE, SALARY + HRA FROM SALGRADE WHERE SGRADE = 'S02';
Results Explain Describe Saved SQL History
ECODE NAME DESIGN SGRADE DOJ DOB
101 Amit Kumar MANAGER S03 10-JAN-05 15-MAR-75
102 Ravi Sharma CLERK S02 15-JUN-07 20-JUL-80
103 Priya Singh ANALYST S03 20-MAR-08 10-DEC-82
104 Kumar Raj CLERK S01 05-AUG-09 25-JAN-85
105 Anita Das EXECUTIVE S02 12-MAY-06 18-NOV-79
106 Vijay Roy MANAGER S03 18-FEB-08 05-MAY-78
109 Harish Roy HEAD-IT S02 09-SEP-07 21-APR-83
7 rows returned in 0.00 seconds
CSV Export
Language: en-us
Copyright © 1999, 2005, Oracle. All rights reserved.
36°C
Mostly sunny
Search
ENG
IN
16:41
16-08-2026

12. Consider the following tables WORKER and PAYLAVEL and answer (a) and (b) parts of this question

(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

(iv) To add a new row with the following:

19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL = 'P003';

PROGRAM

```
CREATE TABLE WORKER (ECODE NUMBER(5),NAME VARCHAR2(30),DESIGN VARCHAR2(30),PLEVEL VARCHAR2(5),DOJ DATE,DOB DATE);

INSERT INTO WORKER VALUES(11, 'Arun Kumar', 'Manager', 'P002',TO_DATE('10-JAN-2007','DD-MON-YYYY'),TO_DATE('15-MAR-1985','DD-MON-YYYY'));

INSERT INTO WORKER VALUES(12, 'Ravi Sharma', 'Clerk', 'P001',TO_DATE('15-JUN-2008','DD-MON-YYYY'),TO_DATE('20-JUL-1986','DD-MON-YYYY'));

INSERT INTO WORKER VALUES(13, 'Priya Singh', 'Supervisor', 'P003',TO_DATE('20-MAR-2009','DD-MON-YYYY'),TO_DATE('10-DEC-1983','DD-MON-YYYY'));

INSERT INTO WORKER VALUES(14, 'Kumar Raj', 'Operator', 'P001',TO_DATE('05-AUG-2010','DD-MON-YYYY'),TO_DATE('25-JAN-1987','DD-MON-YYYY'));

INSERT INTO WORKER VALUES(15, 'Anita Das', 'Clerk', 'P002',TO_DATE('12-MAY-2006','DD-MON-YYYY'),TO_DATE('18-NOV-1985','DD-MON-YYYY'));

INSERT INTO WORKER VALUES(16, 'Vijay Roy', 'Manager', 'P003',TO_DATE('18-FEB-2008','DD-MON-YYYY'),TO_DATE('05-MAY-1984','DD-MON-YYYY'));
```

```
INSERT INTO WORKER VALUES(17, 'Meena Devi', 'Clerk', 'P002',TO_DATE('25-JUL-2009','DD-MON-YYYY'),TO_DATE('12-SEP-1986','DD-MON-YYYY'));

SELECT*FROM WORKER;


CREATE TABLE PAYLEVEL (PLEVEL VARCHAR2(5),PAY NUMBER(10,2),ALLOWANCE NUMBER(10,2));

INSERT INTO PAYLEVEL VALUES ('P001', 20000, 3000);

INSERT INTO PAYLEVEL VALUES ('P002', 30000, 5000);

INSERT INTO PAYLEVEL VALUES ('P003', 40000, 7000);

INSERT INTO PAYLEVEL VALUES ('P004', 50000, 9000);

SELECT*FROM PAYLEVEL;


SELECT *FROM WORKER ORDER BY DOB DESC;

SELECT NAME, DESIGN FROM WORKER WHERE PLEVEL IN ('P001', 'P002');

SELECT *FROM WORKER WHERE DOB BETWEEN TO_DATE('19-JAN-1984','DD-MON-YYYY') AND TO_DATE('18-JAN-1987','DD-MON-YYYY');

INSERT INTO WORKER VALUES (19,'Daya Kishore','Operator','P003',TO_DATE('19-JUN-2008','DD-MON-YYYY'),TO_DATE('11-JUL-1984','DD-MON-YYYY'));

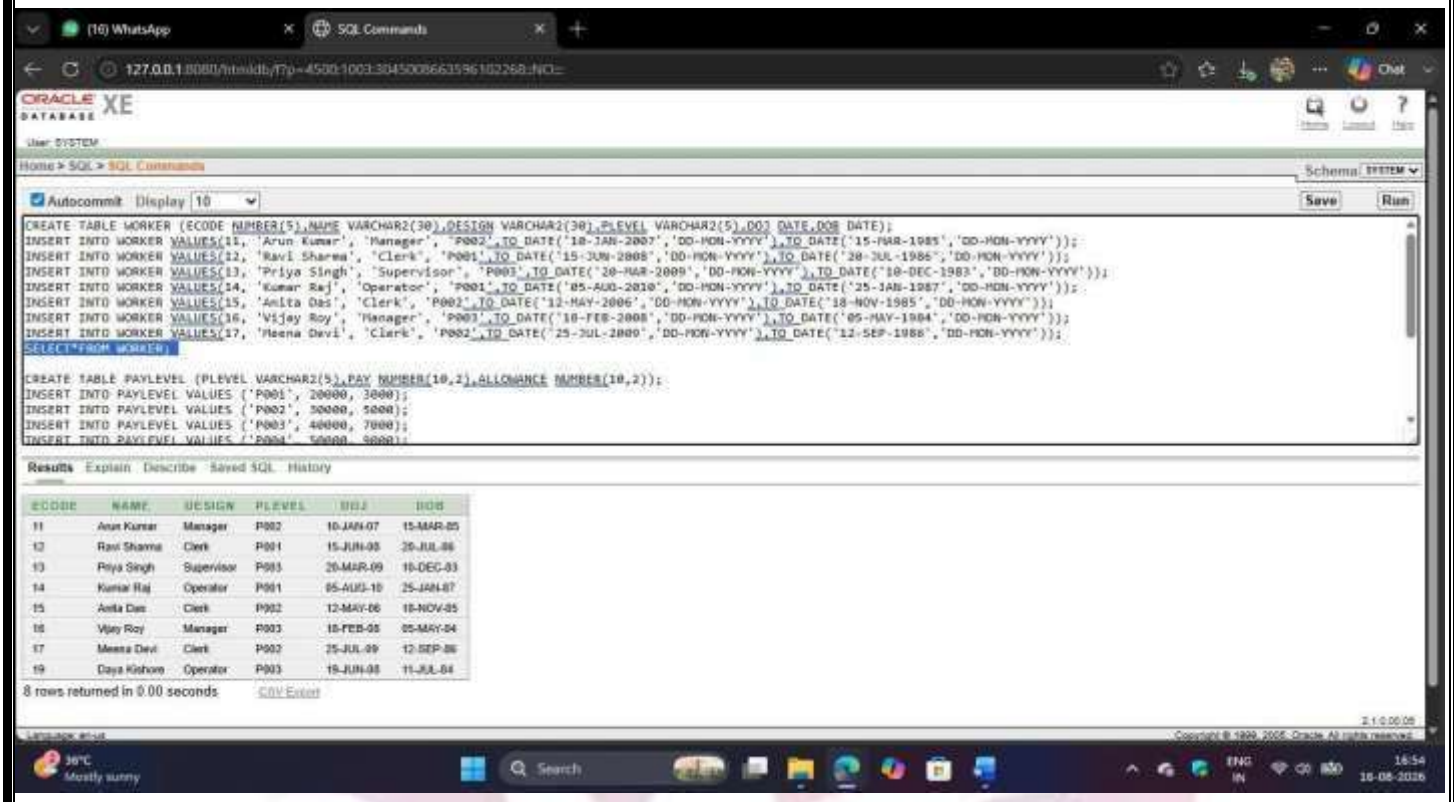
SELECT COUNT(PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

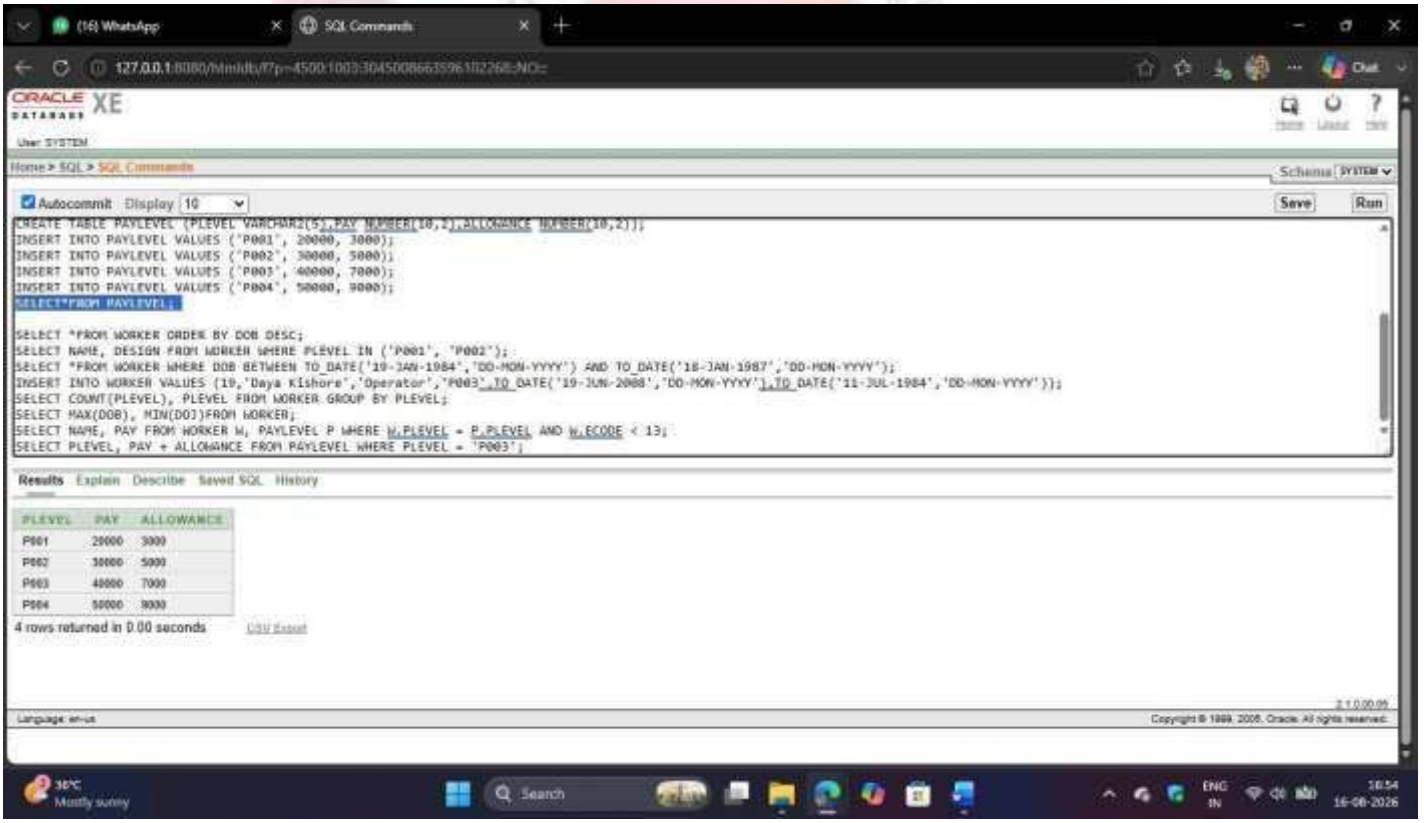
SELECT MAX(DOB), MIN(DOJ)FROM WORKER;

SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W.PLEVEL = P.PLEVEL AND W.ECODE < 13;

SELECT PLEVEL, PAY + ALLOWANCE FROM PAYLEVEL WHERE PLEVEL = 'P003';
```

OUTPUT


(16) WhatsApp
SQL Commands
ORACLE XE
DATABASE
User: SYSTEM
Home > SQL > SQL Commands
Schema: SYSTEM
Autocommit Display 10
Save
Run
CREATE TABLE WORKER (ECODE NUMBER(5),NAME VARCHAR2(30),DESIGN VARCHAR2(30),PLEVEL VARCHAR2(5),DOJ DATE,DOB DATE);
INSERT INTO WORKER VALUES(11, 'Arun Kumar', 'Manager', 'P002',TO_DATE('10-JAN-2007','DD-MON-YYYY'),TO_DATE('15-MAR-1985','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(12, 'Ravi Sharma', 'Clerk', 'P001',TO_DATE('15-JUN-2008','DD-MON-YYYY'),TO_DATE('20-JUL-1986','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(13, 'Priya Singh', 'Supervisor', 'P003',TO_DATE('20-MAR-2009','DD-MON-YYYY'),TO_DATE('10-DEC-1983','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(14, 'Kumar Raj', 'Operator', 'P001',TO_DATE('05-AUG-2010','DD-MON-YYYY'),TO_DATE('25-JAN-1987','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(15, 'Anita Das', 'Clerk', 'P002',TO_DATE('12-MAY-2006','DD-MON-YYYY'),TO_DATE('18-NOV-1985','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(16, 'Vijay Roy', 'Manager', 'P003',TO_DATE('10-FEB-2008','DD-MON-YYYY'),TO_DATE('05-MAY-1984','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(17, 'Meena Devi', 'Clerk', 'P002',TO_DATE('25-JUL-2009','DD-MON-YYYY'),TO_DATE('12-SEP-1988','DD-MON-YYYY'));
SELECT*FROM WORKER;
CREATE TABLE PAYLEVEL (PLEVEL VARCHAR2(5),PAY NUMBER(10,2),ALLOWANCE NUMBER(10,2));
INSERT INTO PAYLEVEL VALUES ('P001', 20000, 3000);
INSERT INTO PAYLEVEL VALUES ('P002', 30000, 5000);
INSERT INTO PAYLEVEL VALUES ('P003', 40000, 7000);
INSERT INTO PAYLEVEL VALUES ('P004', 50000, 9000);
Results Explain Describe Saved SQL History
ECODE NAME DESIGN PLEVEL DOJ DOB
11 Arun Kumar Manager P002 10-JAN-07 15-MAR-85
12 Ravi Sharma Clerk P001 15-JUN-08 20-JUL-86
13 Priya Singh Supervisor P003 20-MAR-09 10-DEC-83
14 Kumar Raj Operator P001 05-AUG-10 25-JAN-87
15 Anita Das Clerk P002 12-MAY-06 18-NOV-85
16 Vijay Roy Manager P003 10-FEB-08 05-MAY-84
17 Meena Devi Clerk P002 25-JUL-09 12-SEP-88
19 Daya Kishore Operator P003 19-JUN-08 11-JUL-84
8 rows returned in 0.00 seconds
CSV Export
Language: en-us
Copyright © 1999, 2005, Oracle. All rights reserved.
36°C
Mostly sunny
Search
ENG
IN
16:54
16-06-2026


(16) WhatsApp
SQL Commands
ORACLE XE
DATABASE
User: SYSTEM
Home > SQL > SQL Commands
Schema: SYSTEM
Autocommit Display 10
Save
Run
CREATE TABLE PAYLEVEL (PLEVEL VARCHAR2(5),PAY NUMBER(10,2),ALLOWANCE NUMBER(10,2));
INSERT INTO PAYLEVEL VALUES ('P001', 20000, 3000);
INSERT INTO PAYLEVEL VALUES ('P002', 30000, 5000);
INSERT INTO PAYLEVEL VALUES ('P003', 40000, 7000);
INSERT INTO PAYLEVEL VALUES ('P004', 50000, 9000);
SELECT*FROM PAYLEVEL;
SELECT *FROM WORKER ORDER BY DOB DESC;
SELECT NAME, DESIGN FROM WORKER WHERE PLEVEL IN ('P001', 'P002');
SELECT *FROM WORKER WHERE DOB BETWEEN TO_DATE('10-JAN-1984','DD-MON-YYYY') AND TO_DATE('18-JAN-1987','DD-MON-YYYY');
INSERT INTO WORKER VALUES (19,'Daya Kishore','Operator','P003',TO_DATE('19-JUN-2008','DD-MON-YYYY'),TO_DATE('11-JUL-1984','DD-MON-YYYY'));
SELECT COUNT(PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;
SELECT MAX(DOB), MIN(DOJ)FROM WORKER;
SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W.PLEVEL = P.PLEVEL AND W.ECODE < 13;
SELECT PLEVEL, PAY + ALLOWANCE FROM PAYLEVEL WHERE PLEVEL = 'P003';
Results Explain Describe Saved SQL History
PLEVEL PAY ALLOWANCE
P001 20000 3000
P002 30000 5000
P003 40000 7000
P004 50000 9000
4 rows returned in 0.00 seconds
CSV Export
Language: en-us
Copyright © 1999, 2005, Oracle. All rights reserved.
36°C
Mostly sunny
Search
ENG
IN
16:54
16-06-2026

13. Consider the following tables STORE and SUPPLIERS and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

(iii) To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

(iv) To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

(ii) SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

(iii) SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

(iv) SELECT MAX(LastBuy) FROM STORE;

PROGRAM

```
CREATE TABLE STORES (ItemNo NUMBER(5),Item VARCHAR2(30),Scode NUMBER(5),Qty NUMBER(5),Rate NUMBER(10,2),LastBuy DATE);

INSERT INTO STORES VALUES(2001, 'Pen', 21, 100, 10, TO_DATE('10-JAN-2020','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2002, 'Pencil', 22, 120, 8, TO_DATE('15-FEB-2020','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2003, 'Notebook', 23, 80, 25, TO_DATE('20-MAR-2020','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2004, 'Marker', 22, 150, 20, TO_DATE('05-APR-2020','DD-MON-YYYY'));

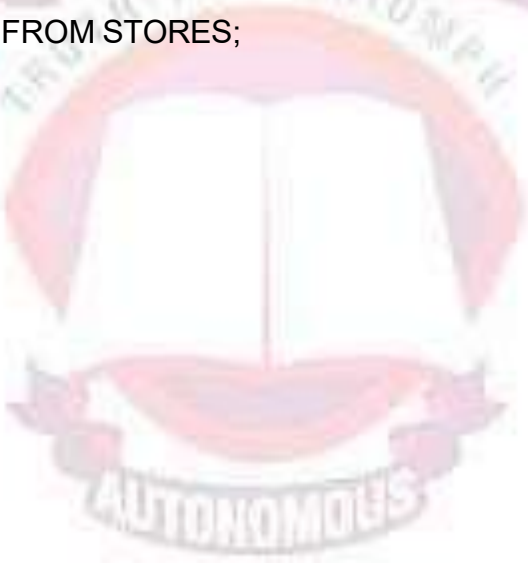
INSERT INTO STORES VALUES(2005, 'Eraser', 21, 130, 5, TO_DATE('10-MAY-2020','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2006, 'File', 23, 100, 30, TO_DATE('15-JUN-2020','DD-MON-YYYY'));
```

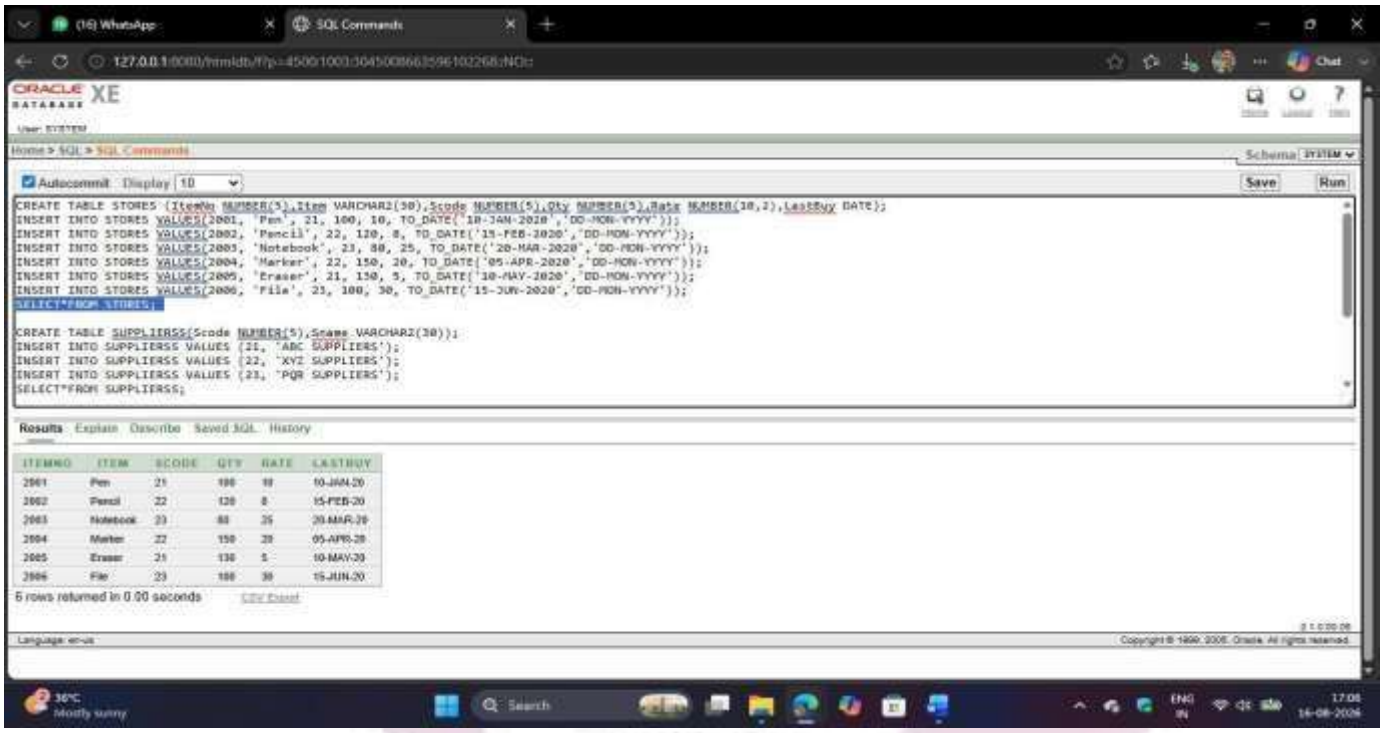
```
SELECT*FROM STORES;

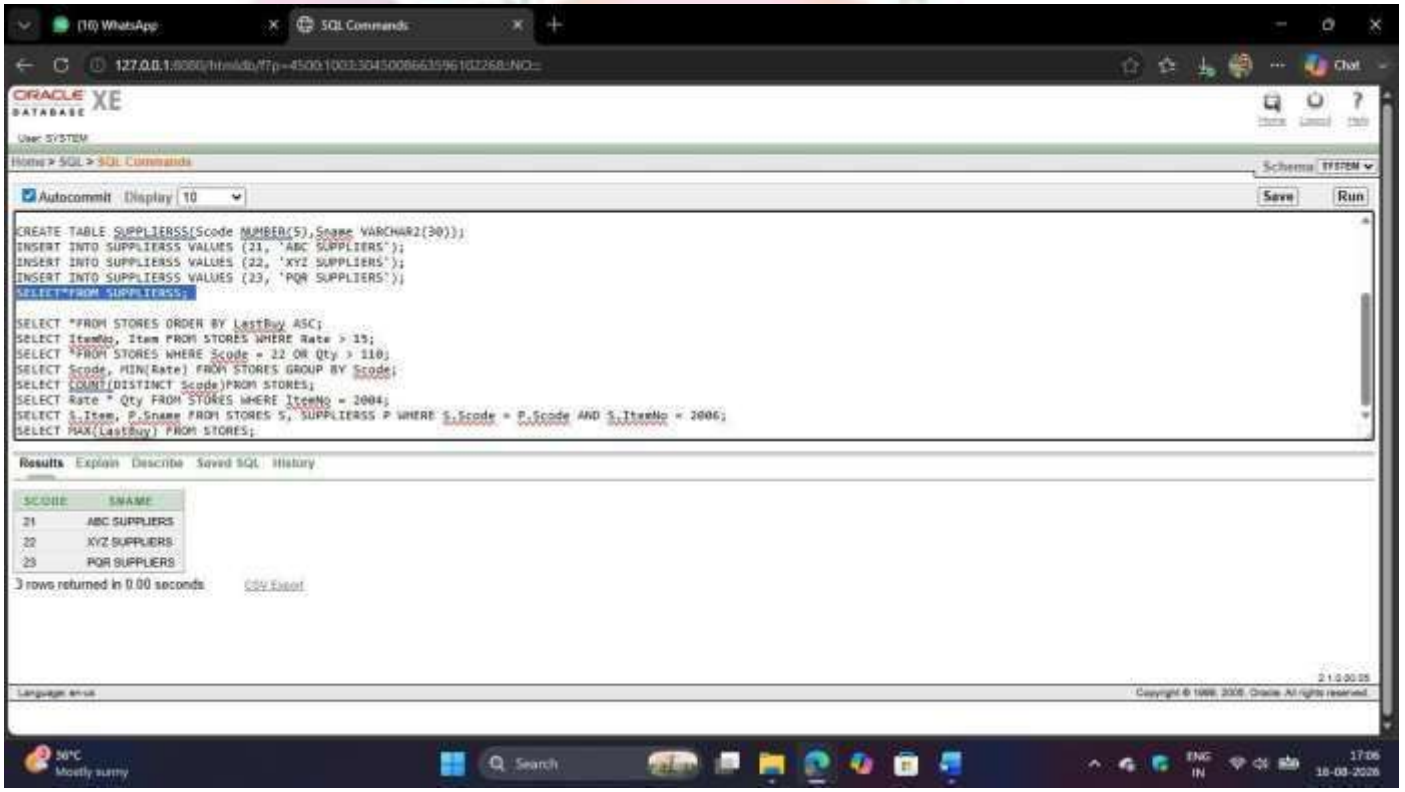

CREATE TABLE SUPPLIERSS(Scode NUMBER(5),Sname VARCHAR2(30));
INSERT INTO SUPPLIERSS VALUES (21, 'ABC SUPPLIERS');
INSERT INTO SUPPLIERSS VALUES (22, 'XYZ SUPPLIERS');
INSERT INTO SUPPLIERSS VALUES (23, 'PQR SUPPLIERS');
SELECT*FROM SUPPLIERSS;


SELECT *FROM STORES ORDER BY LastBuy ASC;
SELECT ItemNo, Item FROM STORES WHERE Rate > 15;
SELECT *FROM STORES WHERE Scode = 22 OR Qty > 110;
SELECT Scode, MIN(Rate) FROM STORES GROUP BY Scode;
SELECT COUNT(DISTINCT Scode)FROM STORES;
SELECT Rate * Qty FROM STORES WHERE ItemNo = 2004;
SELECT S.Item, P.Sname FROM STORES S, SUPPLIERSS P WHERE S.Scode =
P.Scode AND S.ItemNo = 2006;
SELECT MAX(LastBuy) FROM STORES;
```



OUTPUT


SQL Commands
ORACLE XE
CREATE TABLE STORES (ItemNo NUMBER(5),Item VARCHAR2(30),Scode NUMBER(5),Qty NUMBER(5),Rate NUMBER(10,2),LastBuy DATE);
INSERT INTO STORES VALUES(2001, 'Pen', 21, 100, 10, TO_DATE('10-JAN-2020','DD-MON-YYYY'));
INSERT INTO STORES VALUES(2002, 'Pencil', 22, 120, 8, TO_DATE('15-FEB-2020','DD-MON-YYYY'));
INSERT INTO STORES VALUES(2003, 'Notebook', 23, 80, 25, TO_DATE('20-MAR-2020','DD-MON-YYYY'));
INSERT INTO STORES VALUES(2004, 'Marker', 22, 150, 20, TO_DATE('05-APR-2020','DD-MON-YYYY'));
INSERT INTO STORES VALUES(2005, 'Eraser', 21, 130, 5, TO_DATE('10-MAY-2020','DD-MON-YYYY'));
INSERT INTO STORES VALUES(2006, 'File', 23, 100, 30, TO_DATE('15-JUN-2020','DD-MON-YYYY'));
SELECT*FROM STORES;
CREATE TABLE SUPPLIERSS(Scode NUMBER(5),Sname VARCHAR2(30));
INSERT INTO SUPPLIERSS VALUES (21, 'ABC SUPPLIERS');
INSERT INTO SUPPLIERSS VALUES (22, 'XYZ SUPPLIERS');
INSERT INTO SUPPLIERSS VALUES (23, 'PQR SUPPLIERS');
SELECT*FROM SUPPLIERSS;
Results
ITEMNO ITEM SCODE QTY RATE LASTBUY
2001 Pen 21 100 10 10-JAN-20
2002 Pencil 22 120 8 15-FEB-20
2003 Notebook 23 80 25 20-MAR-20
2004 Marker 22 150 20 05-APR-20
2005 Eraser 21 130 5 10-MAY-20
2006 File 23 100 30 15-JUN-20
6 rows returned in 0.00 seconds


SQL Commands
ORACLE XE
CREATE TABLE SUPPLIERSS(Scode NUMBER(5),Sname VARCHAR2(30));
INSERT INTO SUPPLIERSS VALUES (21, 'ABC SUPPLIERS');
INSERT INTO SUPPLIERSS VALUES (22, 'XYZ SUPPLIERS');
INSERT INTO SUPPLIERSS VALUES (23, 'PQR SUPPLIERS');
SELECT*FROM SUPPLIERSS;
SELECT *FROM STORES ORDER BY LastBuy ASC;
SELECT ItemNo, Item FROM STORES WHERE Rate > 15;
SELECT *FROM STORES WHERE Scode = 22 OR Qty > 110;
SELECT Scode, MIN(Rate) FROM STORES GROUP BY Scode;
SELECT COUNT(DISTINCT Scode)FROM STORES;
SELECT Rate * Qty FROM STORES WHERE ItemNo = 2004;
SELECT S.Item, P.Sname FROM STORES S, SUPPLIERSS P WHERE S.Scode = P.Scode AND S.ItemNo = 2006;
SELECT MAX(LastBuy) FROM STORES;
Results
SCODE SNAME
21 ABC SUPPLIERS
22 XYZ SUPPLIERS
23 PQR SUPPLIERS
3 rows returned in 0.00 seconds

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

14. Consider the following table GARMENT and FABRIC,

Write SQL commands for the statements (i) to (iv) and give outputs for the SQL queries (v) to (viii).

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

(ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

(vii) SELECT MAX (FCODE) FROM FABRIC;

(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

PROGRAM

```
CREATE TABLE GARMENTT (GCODE VARCHAR2(5),DESCRIPTION VARCHAR2(40),FCODE VARCHAR2(5),PRICE NUMBER(10,2),READYDATE DATE);

INSERT INTO GARMENTT VALUES('G01', 'Formal Shirt', 'F01', 1200,TO_DATE('10-DEC-2007','DD-MON-YYYY'));

INSERT INTO GARMENTT VALUES('G02', 'Silk Saree', 'F02', 2500,TO_DATE('15-JAN-2008','DD-MON-YYYY'));

INSERT INTO GARMENTT VALUES('G03', 'Woolen Coat', 'F03', 1800,TO_DATE('20-FEB-2008','DD-MON-YYYY'));

INSERT INTO GARMENTT VALUES('G04', 'Cotton Trouser', 'F01', 1300,TO_DATE('10-MAR-2008','DD-MON-YYYY'));

INSERT INTO GARMENTT VALUES('G05', 'Woolen Sweater', 'F03', 1400,TO_DATE('20-MAY-2008','DD-MON-YYYY'));

INSERT INTO GARMENTT VALUES('G06', 'Party Dress', 'F04', 1600,TO_DATE('20-JUN-2008','DD-MON-YYYY'));

SELECT*FROM GARMENTT;
```

```
CREATE TABLE FABRIC (FCODE VARCHAR2(5),TYPE VARCHAR2(30));

INSERT INTO FABRIC VALUES ('F01', 'Cotton');

INSERT INTO FABRIC VALUES ('F02', 'Silk');

INSERT INTO FABRIC VALUES ('F03', 'Wool');

INSERT INTO FABRIC VALUES ('F04', 'Polyester');

SELECT*FROM FABRIC;


SELECT GCODE, DESCRIPTION FROM GARMENTT ORDER BY GCODE DESC;

SELECT *FROM GARMENTT WHERE READYDATE BETWEEN TO_DATE('08-DEC-
2007','DD-MON-YYYY') AND TO_DATE('16-JUN-2008','DD-MON-YYYY');

SELECT AVG(PRICE)FROM GARMENTT WHERE FCODE = 'F03';

SELECT FCODE, MAX(PRICE), MIN(PRICE)FROM GARMENTT GROUP BY FCODE;

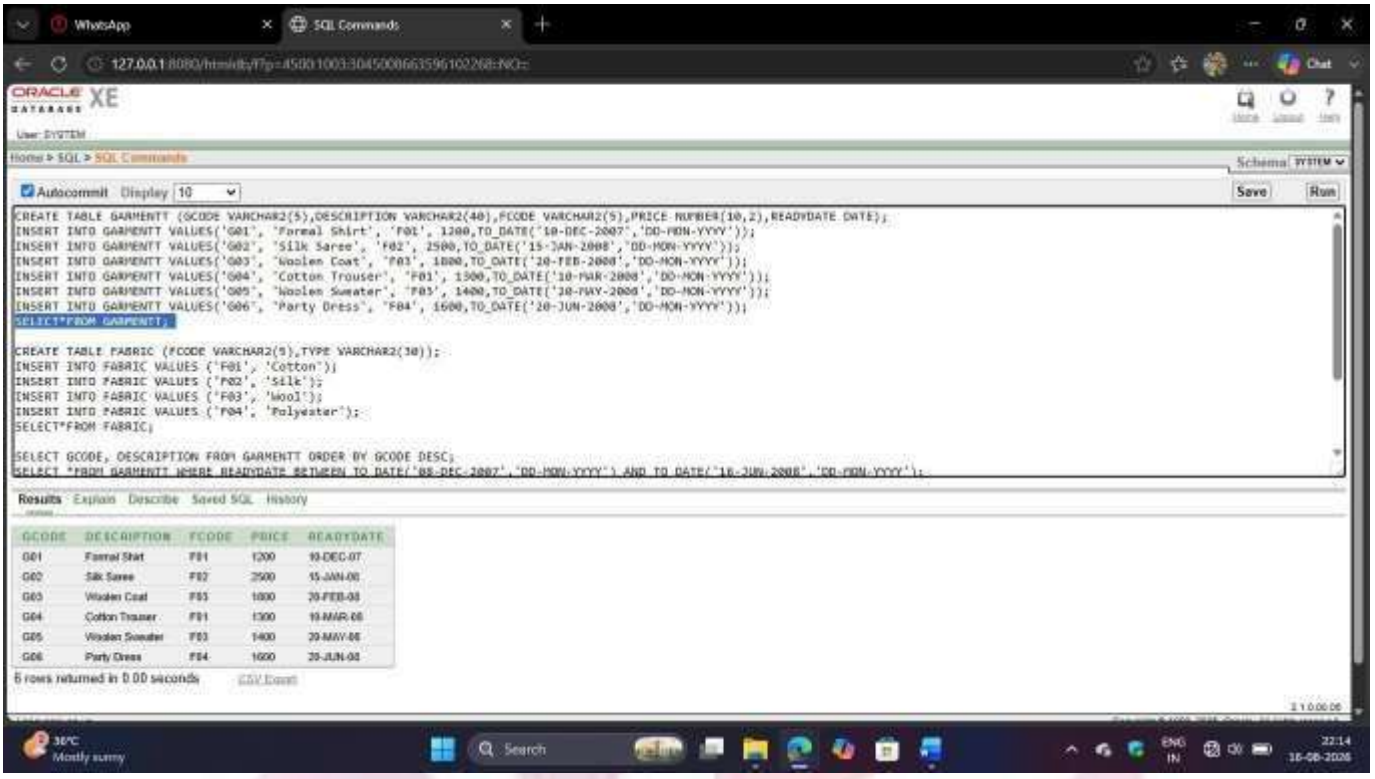
SELECT SUM(PRICE)FROM GARMENTT WHERE FCODE = 'F01';

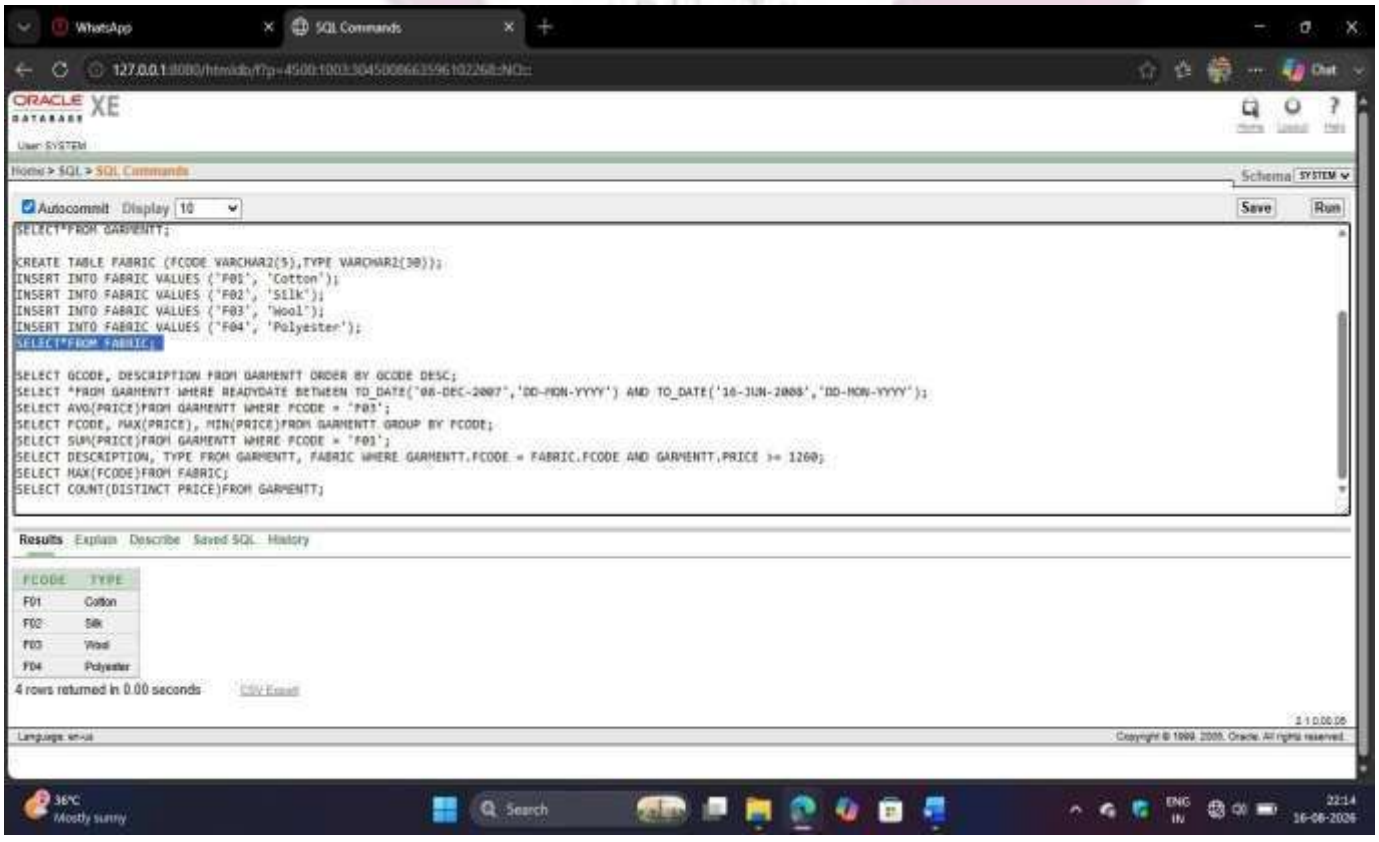
SELECT DESCRIPTION, TYPE FROM GARMENTT, FABRIC WHERE
GARMENTT.FCODE = FABRIC.FCODE AND GARMENTT.PRICE >= 1260;

SELECT MAX(FCODE)FROM FABRIC;

SELECT COUNT(DISTINCT PRICE)FROM GARMENTT;
```

OUTPUT


SQL Commands
CREATE TABLE GARMENTT (GCODE VARCHAR2(5),DESCRIPTION VARCHAR2(40),FCODE VARCHAR2(5),PRICE NUMBER(10,2),READYDATE DATE);
INSERT INTO GARMENTT VALUES('G01', 'Formal Shirt', 'F01', 1200,TO_DATE('10-DEC-2007','DD-MON-YYYY'));
INSERT INTO GARMENTT VALUES('G02', 'Silk Saree', 'F02', 2500,TO_DATE('15-JAN-2008','DD-MON-YYYY'));
INSERT INTO GARMENTT VALUES('G03', 'Woolen Coat', 'F03', 1800,TO_DATE('20-FEB-2008','DD-MON-YYYY'));
INSERT INTO GARMENTT VALUES('G04', 'Cotton Trouser', 'F01', 1300,TO_DATE('10-MAR-2008','DD-MON-YYYY'));
INSERT INTO GARMENTT VALUES('G05', 'Woolen Sweater', 'F03', 1400,TO_DATE('30-MAY-2008','DD-MON-YYYY'));
INSERT INTO GARMENTT VALUES('G06', 'Party Dress', 'F04', 1600,TO_DATE('20-JUN-2008','DD-MON-YYYY'));
SELECT*FROM GARMENTT;
CREATE TABLE FABRIC (FCODE VARCHAR2(5),TYPE VARCHAR2(30));
INSERT INTO FABRIC VALUES ('F01', 'Cotton');
INSERT INTO FABRIC VALUES ('F02', 'Silk');
INSERT INTO FABRIC VALUES ('F03', 'Wool');
INSERT INTO FABRIC VALUES ('F04', 'Polyester');
SELECT*FROM FABRIC;
SELECT GCODE, DESCRIPTION FROM GARMENTT ORDER BY GCODE DESC;
SELECT *FROM GARMENTT WHERE READYDATE BETWEEN TO_DATE('08-DEC-2007','DD-MON-YYYY') AND TO_DATE('16-JUN-2008','DD-MON-YYYY');
Results
GCODE DESCRIPTION FCODE PRICE READYDATE
G01 Formal Shirt F01 1200 10-DEC-07
G02 Silk Saree F02 2500 15-JAN-08
G03 Woolen Coat F03 1800 20-FEB-08
G04 Cotton Trouser F01 1300 10-MAR-08
G05 Woolen Sweater F03 1400 30-MAY-08
G06 Party Dress F04 1600 20-JUN-08
6 rows returned in 0.00 seconds


SQL Commands
SELECT*FROM GARMENTT;
CREATE TABLE FABRIC (FCODE VARCHAR2(5),TYPE VARCHAR2(30));
INSERT INTO FABRIC VALUES ('F01', 'Cotton');
INSERT INTO FABRIC VALUES ('F02', 'Silk');
INSERT INTO FABRIC VALUES ('F03', 'Wool');
INSERT INTO FABRIC VALUES ('F04', 'Polyester');
SELECT*FROM FABRIC;
SELECT GCODE, DESCRIPTION FROM GARMENTT ORDER BY GCODE DESC;
SELECT *FROM GARMENTT WHERE READYDATE BETWEEN TO_DATE('08-DEC-2007','DD-MON-YYYY') AND TO_DATE('16-JUN-2008','DD-MON-YYYY');
SELECT AVG(PRICE)FROM GARMENTT WHERE FCODE = 'F03';
SELECT FCODE, MAX(PRICE), MIN(PRICE)FROM GARMENTT GROUP BY FCODE;
SELECT SUM(PRICE)FROM GARMENTT WHERE FCODE = 'F01';
SELECT DESCRIPTION, TYPE FROM GARMENTT, FABRIC WHERE GARMENTT.FCODE = FABRIC.FCODE AND GARMENTT.PRICE >= 1260;
SELECT MAX(FCODE)FROM FABRIC;
SELECT COUNT(DISTINCT PRICE)FROM GARMENTT;
Results
FCODE TYPE
F01 Cotton
F02 Silk
F03 Wool
F04 Polyester
4 rows returned in 0.00 seconds

15. Given the following tables for a database FURNITURE : Write SQL command for (a) to (f) and write the outputs for (g) on the bases of tables **FURNITURE** AND **ARRIVALS.**

(a) To show all information about the baby cots from the FURNITURE table.

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

(e) To count the number of items, whose TYPE is "Sofa" from FURNITURE table.

(f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", Double bed", {25/03/03}, 25000, 30

(g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

(i) Select COUNT(distinct TYPE) from FURNITURE;

(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = "Baby cot";

(iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

PROGRAM

```
CREATE TABLE FURNITURE (ITEMCODE INT,ITEMNAME VARCHAR(30),TYPE
VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);

INSERT INTO FURNITURE VALUES(101, 'Baby cot', 'Baby cot',
TO_DATE('10/02/2002','DD/MM/YYYY'), 12000, 20);

INSERT INTO FURNITURE VALUES(102, 'Wooden Sofa', 'Sofa',
TO_DATE('15/01/2002','DD/MM/YYYY'), 18000, 30);

INSERT INTO FURNITURE VALUES(103, 'Double Bed', 'Bed',
TO_DATE('05/02/2002','DD/MM/YYYY'), 25000, 35);

INSERT INTO FURNITURE VALUES(104, 'Office Chair', 'Chair',
TO_DATE('20/01/2002','DD/MM/YYYY'), 8000, 15);

INSERT INTO FURNITURE VALUES(105, 'Baby Cot Deluxe', 'Baby cot',
TO_DATE('10/02/2002','DD/MM/YYYY'), 16000, 28);
```

```
SELECT*FROM FURNITURE;


CREATE TABLE ARRIVALS (ITEMCODE INT,ITEMNAME VARCHAR(30),TYPE
VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);

INSERT INTO ARRIVALS VALUES(11, 'Wooden Chair', 'Chair',
TO_DATE('10/01/2003','DD/MM/YYYY'), 9000, 15);

INSERT INTO ARRIVALS VALUES(12, 'Modern Sofa', 'Sofa',
TO_DATE('15/02/2003','DD/MM/YYYY'), 20000, 25);

INSERT INTO ARRIVALS VALUES(13, 'King Bed', 'Bed',
TO_DATE('20/02/2003','DD/MM/YYYY'), 30000, 20);

SELECT*FROM ARRIVALS;


SELECT *FROM FURNITURE WHERE TYPE = 'Baby cot';

SELECT ITEMNAME FROM FURNITURE WHERE PRICE > 15000;

SELECT ITEMNAME, TYPE FROM FURNITURE WHERE DATEOFSTOCK <
TO_DATE('28/04/2004','DD/MM/YYYY')ORDER BY ITEMNAME DESC;

SELECT ITEMNAME, DATEOFSTOCK FROM FURNITURE WHERE DISCOUNT > 25;

SELECT COUNT(*)FROM FURNITURE WHERE TYPE = 'Sofa';

INSERT INTO ARRIVALS VALUES (14, 'Velvet touch', 'Double bed',
TO_DATE('24/06/2007','DD/MM/YYYY'), 25000, 30);

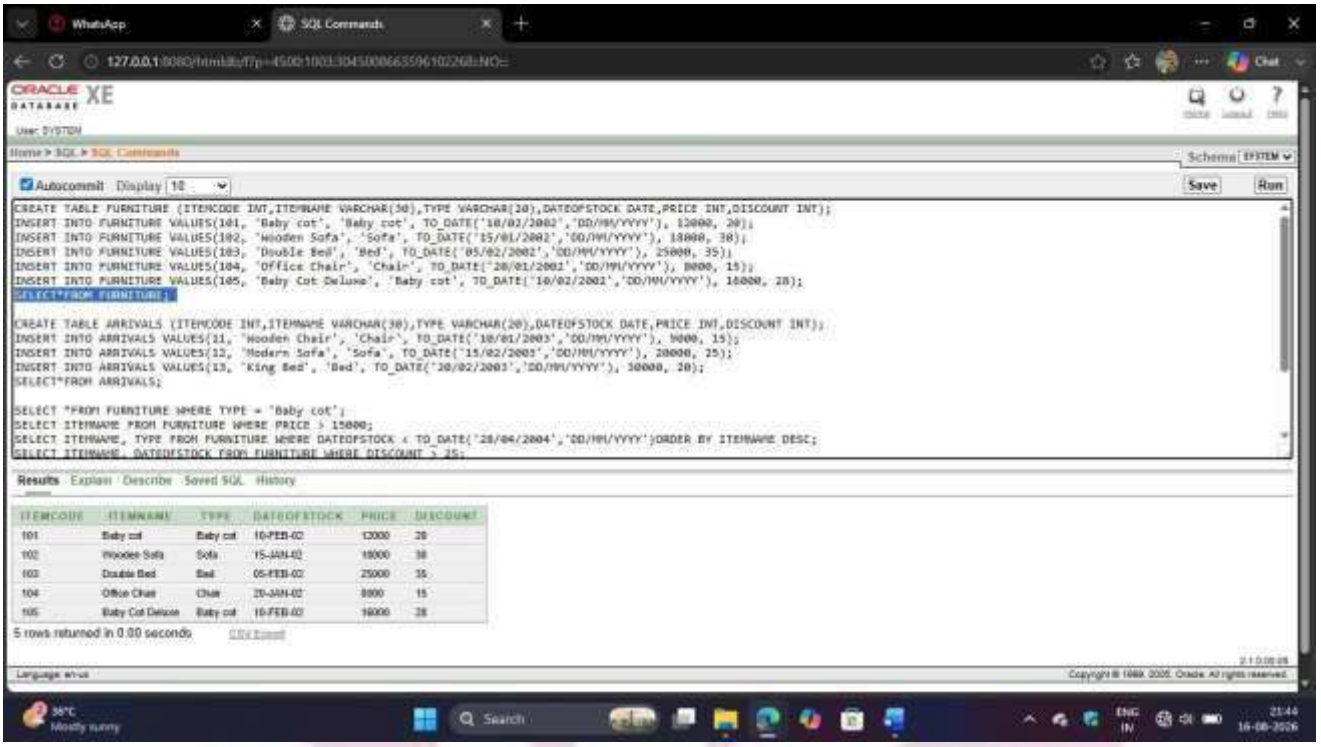
SELECT COUNT(DISTINCT TYPE)FROM FURNITURE;

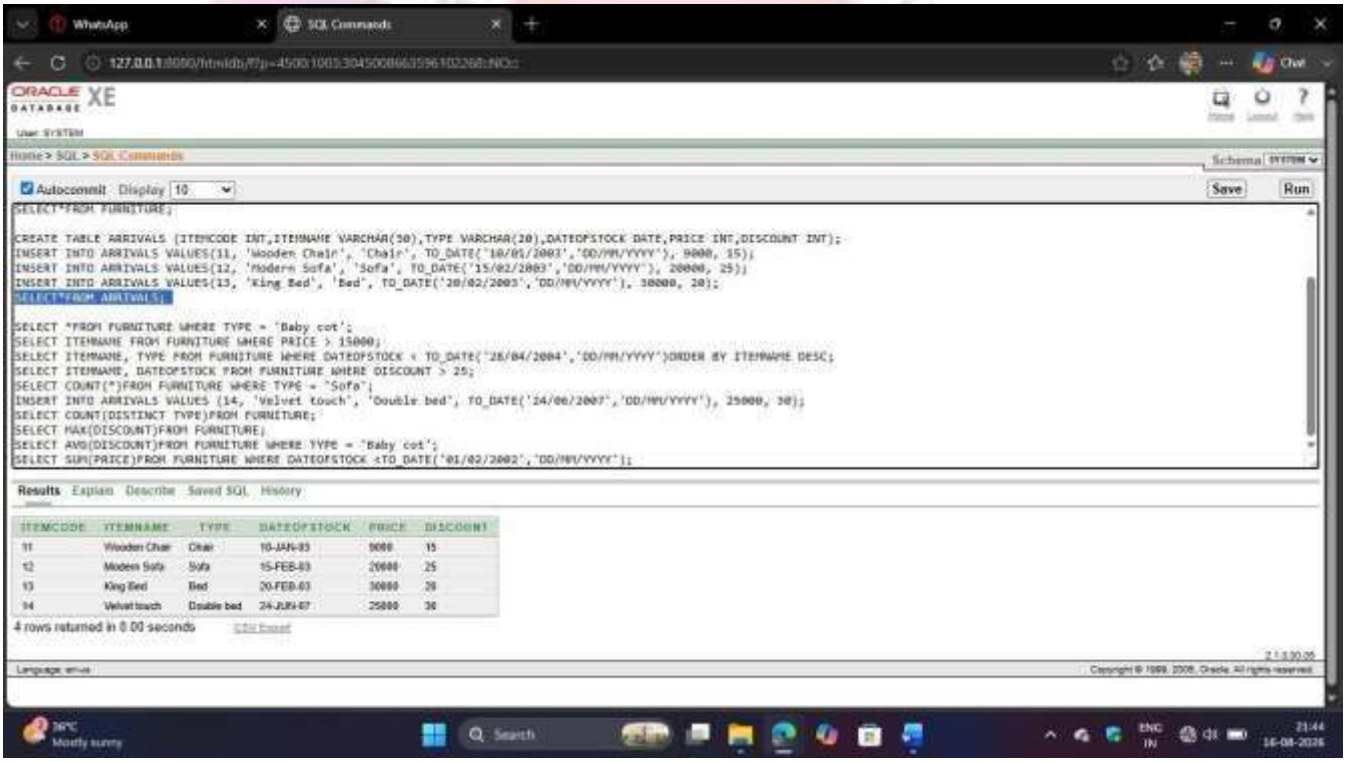
SELECT MAX(DISCOUNT)FROM FURNITURE;

SELECT AVG(DISCOUNT)FROM FURNITURE WHERE TYPE = 'Baby cot';

SELECT SUM(PRICE)FROM FURNITURE WHERE DATEOFSTOCK
<TO_DATE('01/02/2002','DD/MM/YYYY');
```

OUTPUT


SQL Commands
ORACLE XE
CREATE TABLE FURNITURE (ITEMCODE INT,ITEMNAME VARCHAR(30),TYPE VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);
INSERT INTO FURNITURE VALUES(101, 'Baby cot', 'Baby cot', TO_DATE('10/02/2002','DD/MM/YYYY'), 12000, 20);
INSERT INTO FURNITURE VALUES(102, 'Wooden Sofa', 'Sofa', TO_DATE('15/01/2002','DD/MM/YYYY'), 18000, 30);
INSERT INTO FURNITURE VALUES(103, 'Double Bed', 'Bed', TO_DATE('05/02/2002','DD/MM/YYYY'), 25000, 35);
INSERT INTO FURNITURE VALUES(104, 'Office Chair', 'Chair', TO_DATE('20/01/2002','DD/MM/YYYY'), 8000, 15);
INSERT INTO FURNITURE VALUES(105, 'Baby Cot Deluxe', 'Baby cot', TO_DATE('10/02/2002','DD/MM/YYYY'), 16000, 28);
SELECT*FROM FURNITURE;
CREATE TABLE ARRIVALS (ITEMCODE INT,ITEMNAME VARCHAR(30),TYPE VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);
INSERT INTO ARRIVALS VALUES(11, 'Wooden Chair', 'Chair', TO_DATE('10/01/2003','DD/MM/YYYY'), 9000, 15);
INSERT INTO ARRIVALS VALUES(12, 'Modern Sofa', 'Sofa', TO_DATE('15/02/2003','DD/MM/YYYY'), 20000, 25);
INSERT INTO ARRIVALS VALUES(13, 'King Bed', 'Bed', TO_DATE('20/02/2003','DD/MM/YYYY'), 30000, 20);
SELECT*FROM ARRIVALS;
SELECT *FROM FURNITURE WHERE TYPE = 'Baby cot';
SELECT ITEMNAME FROM FURNITURE WHERE PRICE > 15000;
SELECT ITEMNAME, TYPE FROM FURNITURE WHERE DATEOFSTOCK < TO_DATE('28/04/2004','DD/MM/YYYY')ORDER BY ITEMNAME DESC;
SELECT ITEMNAME, DATEOFSTOCK FROM FURNITURE WHERE DISCOUNT > 25;
Results
ITEMCODE ITEMNAME TYPE DATEOFSTOCK PRICE DISCOUNT
101 Baby cot Baby cot 10-FEB-02 12000 20
102 Wooden Sofa Sofa 15-JAN-02 18000 30
103 Double Bed Bed 05-FEB-02 25000 35
104 Office Chair Chair 20-JAN-02 8000 15
105 Baby Cot Deluxe Baby cot 10-FEB-02 16000 28
5 rows returned in 0.00 seconds


SQL Commands
ORACLE XE
SELECT*FROM FURNITURE;
CREATE TABLE ARRIVALS (ITEMCODE INT,ITEMNAME VARCHAR(30),TYPE VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);
INSERT INTO ARRIVALS VALUES(11, 'Wooden Chair', 'Chair', TO_DATE('10/01/2003','DD/MM/YYYY'), 9000, 15);
INSERT INTO ARRIVALS VALUES(12, 'Modern Sofa', 'Sofa', TO_DATE('15/02/2003','DD/MM/YYYY'), 20000, 25);
INSERT INTO ARRIVALS VALUES(13, 'King Bed', 'Bed', TO_DATE('20/02/2003','DD/MM/YYYY'), 30000, 20);
SELECT*FROM ARRIVALS;
SELECT *FROM FURNITURE WHERE TYPE = 'Baby cot';
SELECT ITEMNAME FROM FURNITURE WHERE PRICE > 15000;
SELECT ITEMNAME, TYPE FROM FURNITURE WHERE DATEOFSTOCK < TO_DATE('28/04/2004','DD/MM/YYYY')ORDER BY ITEMNAME DESC;
SELECT ITEMNAME, DATEOFSTOCK FROM FURNITURE WHERE DISCOUNT > 25;
SELECT COUNT(*)FROM FURNITURE WHERE TYPE = 'Sofa';
INSERT INTO ARRIVALS VALUES (14, 'Velvet touch', 'Double bed', TO_DATE('24/06/2007','DD/MM/YYYY'), 25000, 30);
SELECT COUNT(DISTINCT TYPE)FROM FURNITURE;
SELECT MAX(DISCOUNT)FROM FURNITURE;
SELECT AVG(DISCOUNT)FROM FURNITURE WHERE TYPE = 'Baby cot';
SELECT SUM(PRICE)FROM FURNITURE WHERE DATEOFSTOCK <TO_DATE('01/02/2002','DD/MM/YYYY');
Results
ITEMCODE ITEMNAME TYPE DATEOFSTOCK PRICE DISCOUNT
11 Wooden Chair Chair 10-JAN-03 9000 15
12 Modern Sofa Sofa 15-FEB-03 20000 25
13 King Bed Bed 20-FEB-03 30000 20
14 Velvet touch Double bed 24-JUN-07 25000 30
4 rows returned in 0.00 seconds

16. **Given the following COVID19 relation, write SQL commands for questions (a) to (f).**

a) Display all details of patients from **Tamil Nadu**.

b) Display the names of patients whose **Status** is **Recovered**.

c) Display **PatientName** and **Age** in ascending order of **Age**.

d) Display the names of patients who are **Vaccinated = 'Yes'**.

e) Count the number of patients whose status is **Active**.

f) Insert the following record into the **COVID19** table:

108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

(g) Write the output of the following SQL commands.

a. SELECT COUNT(*) FROM COVID19;

b. SELECT MAX(Age) FROM COVID19;

c. SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';

d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

PROGRAM

```
CREATE TABLE COVID19(PatientID INT,PatientName VARCHAR(30),State
VARCHAR(30),Age INT,Status VARCHAR(20),Vaccinated VARCHAR(5));

INSERT INTO COVID19 VALUES(101, 'Arun', 'Tamil Nadu', 25, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES(102, 'Priya', 'Kerala', 32, 'Active', 'Yes');

INSERT INTO COVID19 VALUES(103, 'Kumar', 'Tamil Nadu', 45, 'Active', 'No');

INSERT INTO COVID19 VALUES(104, 'Divya', 'Karnataka', 28, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES(105, 'Ravi', 'Tamil Nadu', 52, 'Recovered', 'Yes');

SELECT*FROM COVID19;


SELECT *FROM COVID19 WHERE State = 'Tamil Nadu';
```

```
SELECT PatientName FROM COVID19 WHERE Status = 'Recovered';

SELECT PatientName, Age FROM COVID19 ORDER BY Age ASC;

SELECT PatientName FROM COVID19 WHERE Vaccinated = 'Yes';

SELECT COUNT(*)FROM COVID19 WHERE Status = 'Active';

INSERT INTO COVID19 VALUES (108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');

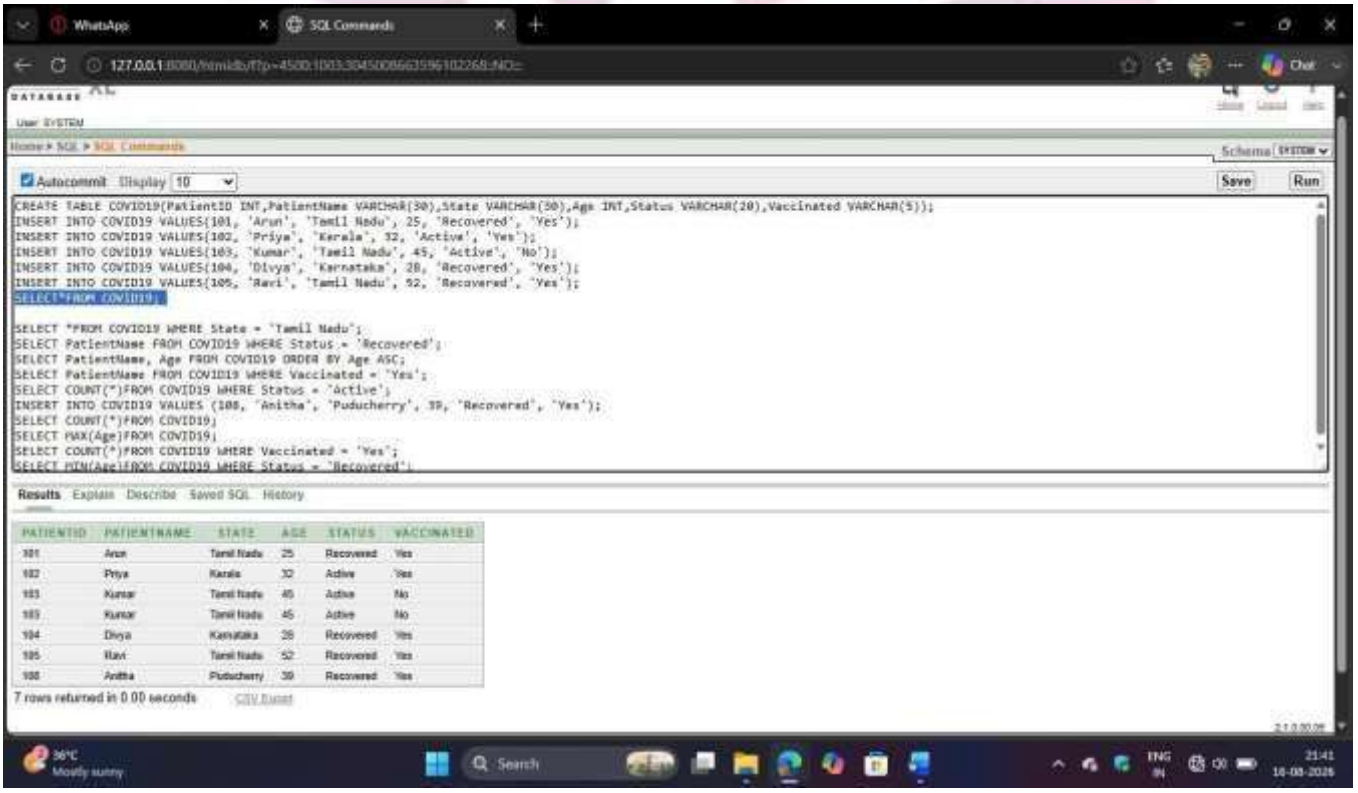
SELECT COUNT(*)FROM COVID19;

SELECT MAX(Age)FROM COVID19;

SELECT COUNT(*)FROM COVID19 WHERE Vaccinated = 'Yes';

SELECT MIN(Age)FROM COVID19 WHERE Status = 'Recovered';
```

OUTPUT



17. **Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).**

a) Display all houses located in **Chennai**.

b) Display the **Location** and **Price** of houses having **3 Bedrooms**.

c) Display **Location**, **AreaSqft**, and **Price** in descending order of **Price**.

d) Display the details of houses costing more than **₹70,00,000**.

e) Count the number of **Apartments**.

f) Insert the following record:

208, 'Vellore', 3, 1700, 7300000, 'Villa'

(g) Write the output of the following SQL commands.

(i) SELECT MAX(Price)FROM HOUSEPRICE;

(i) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

(ii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

(iii) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

PROGRAM

```
CREATE TABLE HOUSEPRICE (HouseID INT,Location VARCHAR(30),Bedrooms INT,AreaSqft INT,Price INT,HouseType VARCHAR(20));
INSERT INTO HOUSEPRICE VALUES(201, 'Chennai', 2, 1200, 5500000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES(202, 'Madurai', 3, 1500, 6500000, 'Villa');
INSERT INTO HOUSEPRICE VALUES(203, 'Chennai', 3, 1800, 7200000, 'Villa');
INSERT INTO HOUSEPRICE VALUES(204, 'Coimbatore', 2, 1100, 4800000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES(205, 'Trichy', 3, 1600, 6800000, 'Apartment');
SELECT*FROM HOUSEPRICE;

SELECT *FROM HOUSEPRICE WHERE Location = 'Chennai';
SELECT Location, Price FROM HOUSEPRICE WHERE Bedrooms = 3;
SELECT Location, AreaSqft, Price FROM HOUSEPRICE ORDER BY Price DESC;
SELECT *FROM HOUSEPRICE WHERE Price > 7000000;
```

```
SELECT  COUNT(*)FROM  HOUSEPRICE  WHERE HouseType =  'Apartment';

INSERT INTO HOUSEPRICE VALUES (208, 'Vellore', 3, 1700, 7300000, 'Villa');

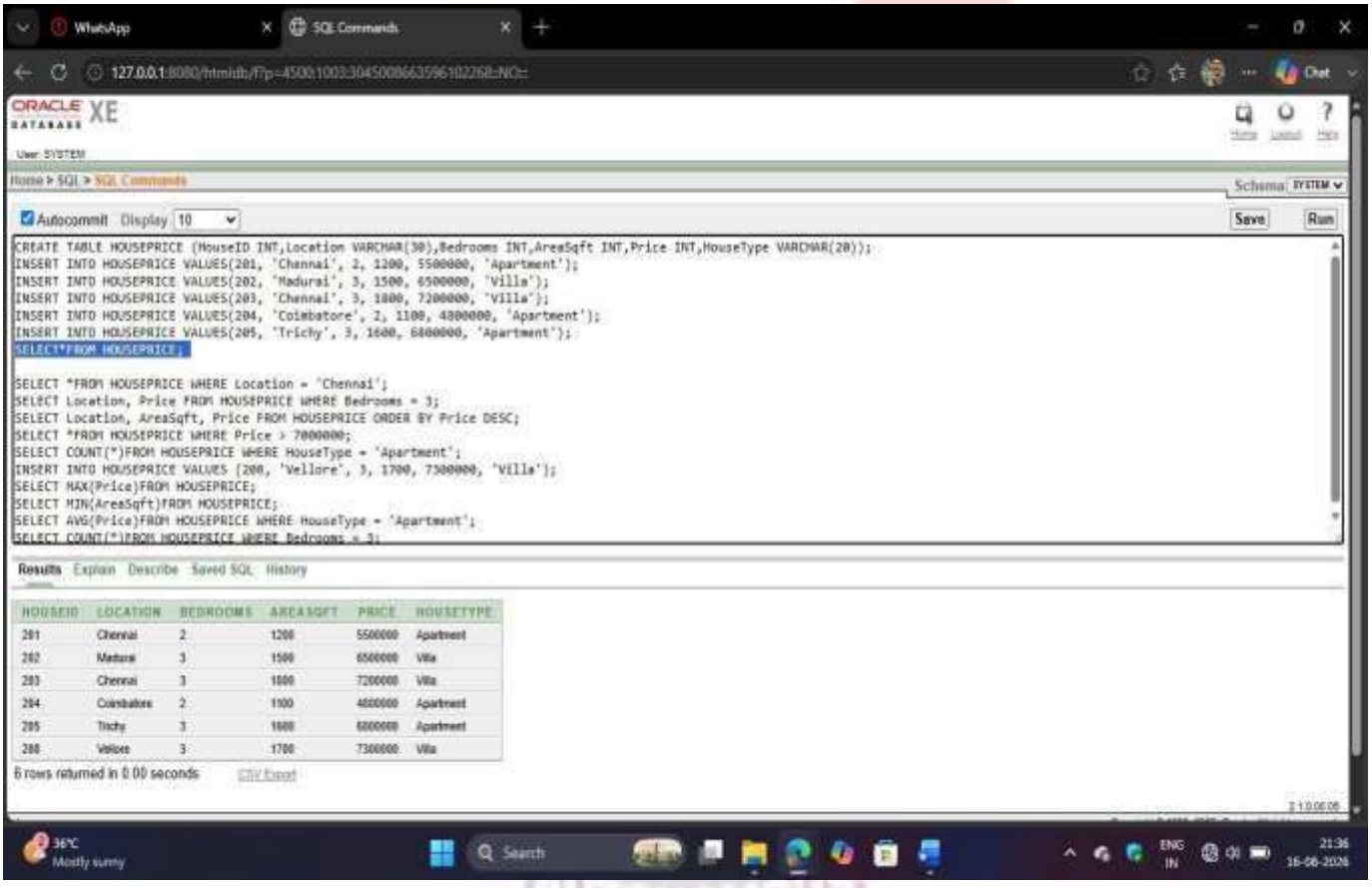
SELECT MAX(Price)FROM HOUSEPRICE;

SELECT  MIN(AreaSqft)FROM  HOUSEPRICE;

SELECT  AVG(Price)FROM HOUSEPRICE WHERE HouseType =  'Apartment';

SELECT  COUNT(*)FROM  HOUSEPRICE  WHERE  Bedrooms  =  3;
```

OUTPUT



18. **Given the following CANCERDATA relation, write SQL commands for questions (a) to (f).**

a) Display all patients diagnosed with **Breast Cancer**.

b) Display the names of patients whose **Stage** is **III**.

c) Display **PatientName** and **Age** in ascending order of **Age**.

d) Display the names of patients having **SurvivalYears greater than 5**.

e) Count the number of patients diagnosed with **Lung Cancer**.

f) Insert the following record:

308, 'Anand', 'Leukemia', 50, 'II', 7

(g) Write the output of the following SQL commands.

(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;

(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

PROGRAM

```
CREATE TABLE CANCERDATA (PatientID INT,PatientName
VARCHAR(30),CancerType VARCHAR(20),Age INT,Stage VARCHAR(5),SurvivalYears
INT);

INSERT INTO CANCERDATA VALUES(301, 'Arun', 'Breast', 45, 'II', 6);

INSERT INTO CANCERDATA VALUES(302, 'Priya', 'Lung', 55, 'III', 4);

INSERT INTO CANCERDATA VALUES(303, 'Kumar', 'Breast', 38, 'I', 8);

INSERT INTO CANCERDATA VALUES(304, 'Divya', 'Lung', 60, 'II', 3);

INSERT INTO CANCERDATA VALUES(305, 'Ravi', 'Prostate', 52, 'III', 7);

INSERT INTO CANCERDATA VALUES(306, 'Meena', 'Colon', 48, 'II', 5);

SELECT*FROM CANCERDATA;
```

```
SELECT *FROM CANCERDATA WHERE CancerType = 'Breast';

SELECT PatientName FROM CANCERDATA WHERE Stage = 'III';

SELECT PatientName, Age FROM CANCERDATA ORDER BY Age ASC;

SELECT PatientName FROM CANCERDATA WHERE SurvivalYears > 5;

SELECT COUNT(*) FROM CANCERDATA WHERE CancerType = 'Lung';

INSERT INTO CANCERDATA VALUES (308, 'Anand', 'Leukemia', 50, 'II', 7);

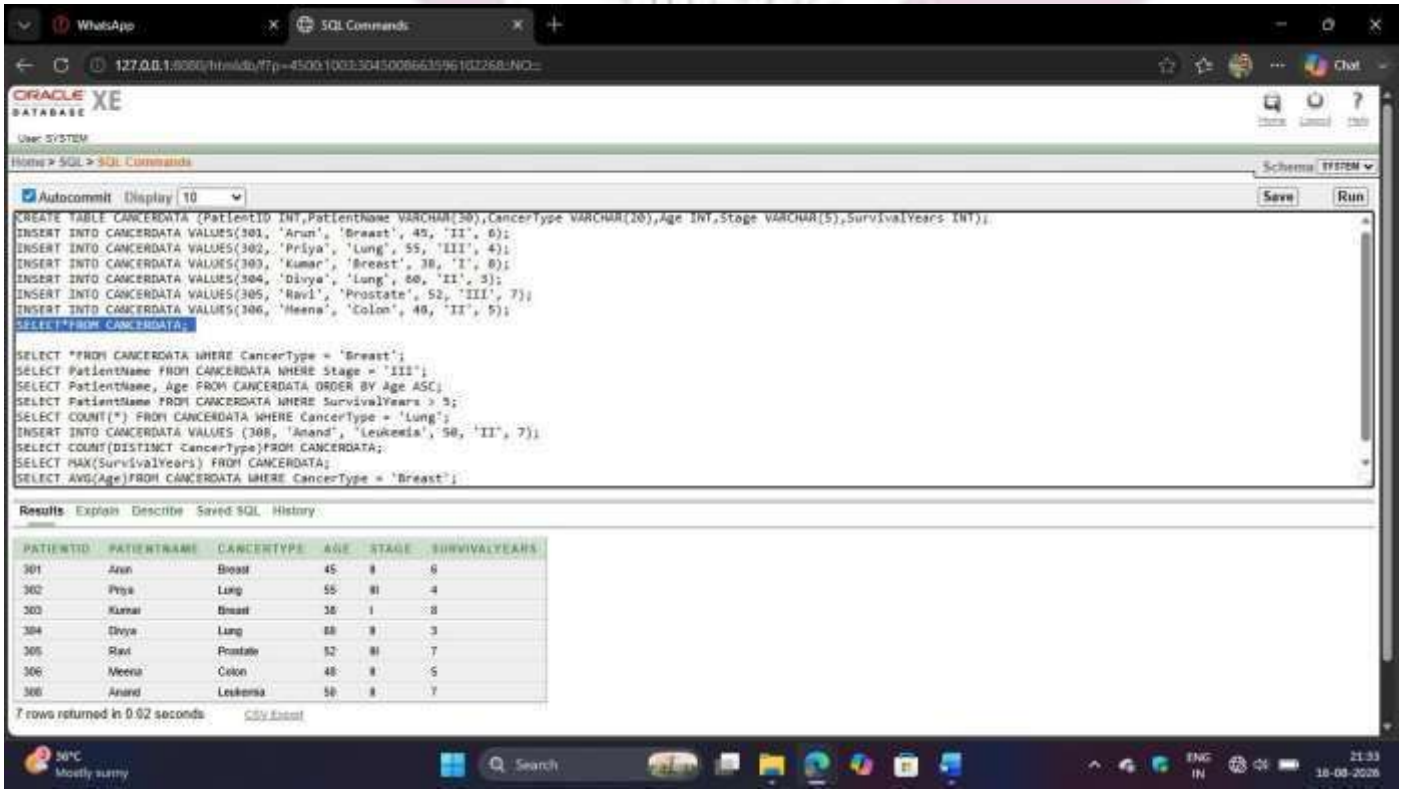
SELECT COUNT(DISTINCT CancerType)FROM CANCERDATA;

SELECT MAX(SurvivalYears) FROM CANCERDATA;

SELECT AVG(Age)FROM CANCERDATA WHERE CancerType = 'Breast';

SELECT MIN(Age)FROM CANCERDATA WHERE Stage = 'II';
```

OUTPUT



19. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables ITEMS and TRADERS

(a) To display the details of all the items in ascending order of item names (i.e. INAME).

(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

(c) To display the number of items, which are traded by each trader. The expected output of this query should be

T01 2

T02 2

T03 1

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

(f) To display the name of the companies and the bane of the items in descending order of company names.

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

(g3) SELE CT DISTINCT TCODE FROM ITEMS;

(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

PROGRAM

```
CREATE TABLE ITEMS (CODE INT,INAME VARCHAR(30),PRICE INT,QTY INT,TCODE VARCHAR(5),COMPANY VARCHAR(30));
INSERT INTO ITEMS VALUES (1001, 'LAPTOP', 25000, 100, 'T01', 'DELL');
INSERT INTO ITEMS VALUES (1002, 'MOBILE', 15000, 200, 'T01', 'SAMSUNG');
INSERT INTO ITEMS VALUES (1003, 'PRINTER', 12000, 80, 'T02', 'HP');
INSERT INTO ITEMS VALUES (1004, 'MONITOR', 18000, 160, 'T02', 'LG');
INSERT INTO ITEMS VALUES (1005, 'KEYBOARD', 5000, 250, 'T03', 'LOGITECH');
SELECT*FROM ITEMS;
```

```
CREATE TABLE TRADERS (TCODE VARCHAR(5),TNAME VARCHAR(30),CITY
VARCHAR(20));

INSERT INTO TRADERS VALUES ('T01', 'RAJESH', 'DELHI');

INSERT INTO TRADERS VALUES ('T02', 'SURESH', 'MUMBAI');

INSERT INTO TRADERS VALUES ('T03', 'RAMESH', 'CHENNAI');

SELECT*FROM TRADERS;


SELECT * FROM ITEMS ORDER BY INAME ASC;

SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;

SELECT TCODE, COUNT(*)FROM ITEMS GROUP BY TCODE;

SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY > 150;

SELECT TNAME FROM TRADERS WHERE CITY IN ('DELHI', 'MUMBAI');

SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;

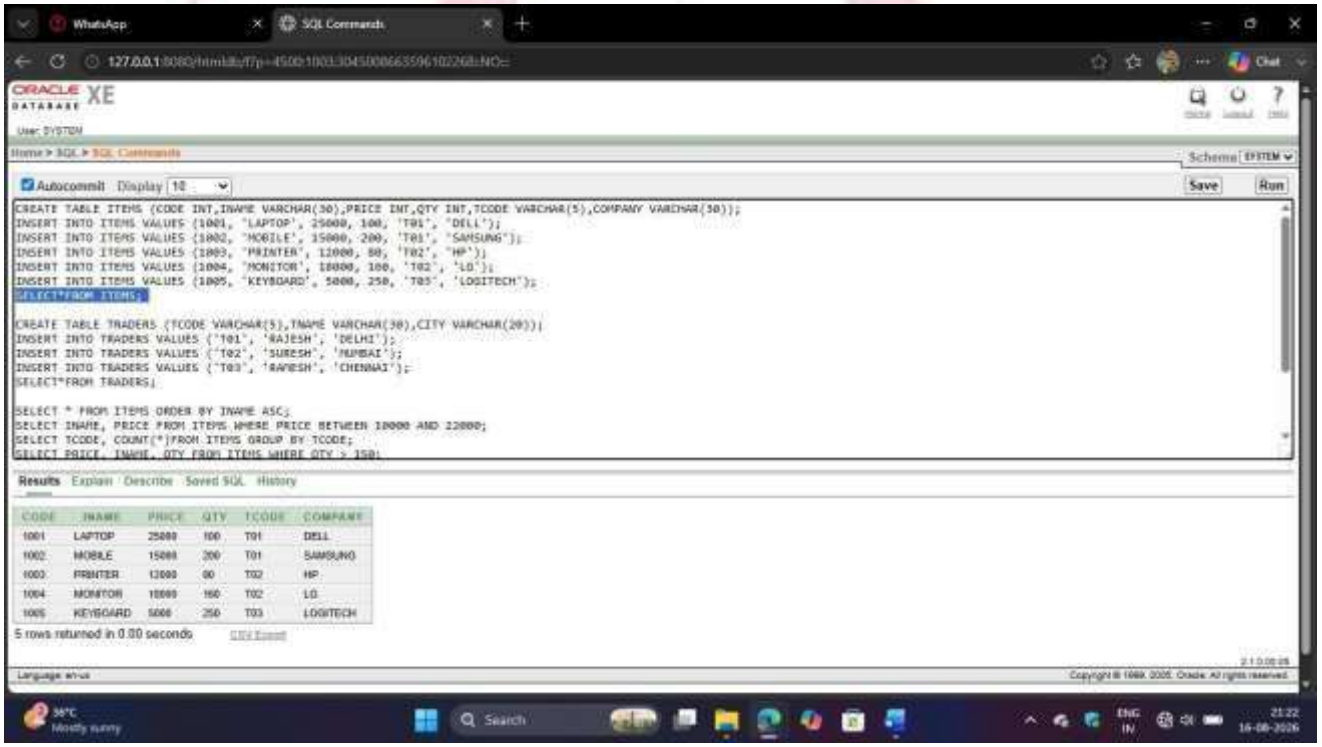
SELECT MAX(PRICE), MIN(PRICE)FROM ITEMS;

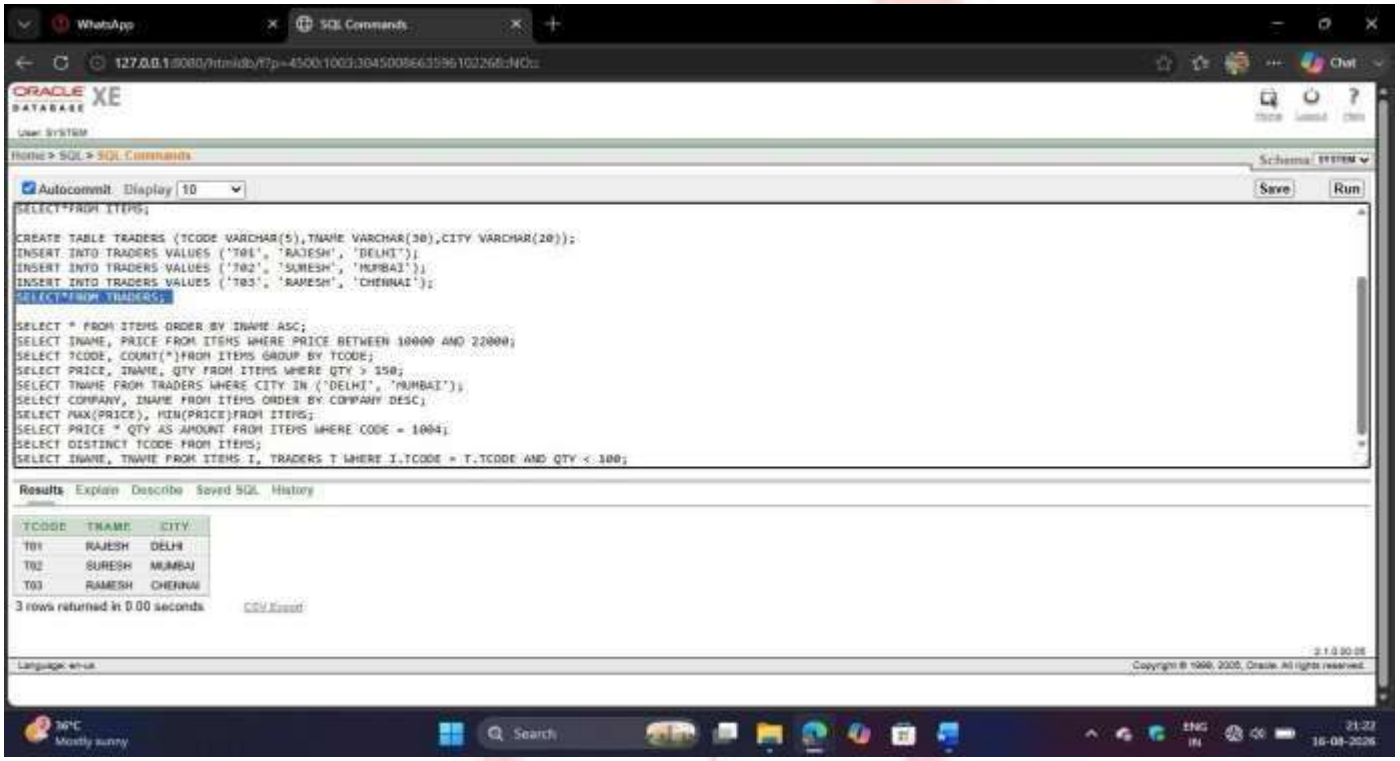
SELECT PRICE * QTY AS AMOUNT FROM ITEMS WHERE CODE = 1004;

SELECT DISTINCT TCODE FROM ITEMS;

SELECT INAME, TNAME FROM ITEMS I, TRADERS T WHERE I.TCODE = T.TCODE AND
QTY < 100;
```

OUTPUT




SQL Commands
ORACLE XE
Autocommit
Display 10
Save
Run
SELECT*FROM ITEMS;
CREATE TABLE TRADERS (TCODE VARCHAR(5),TNAME VARCHAR(30),CITY VARCHAR(20));
INSERT INTO TRADERS VALUES ('T01', 'RAJESH', 'DELHI');
INSERT INTO TRADERS VALUES ('T02', 'SUMESH', 'MUMBAI');
INSERT INTO TRADERS VALUES ('T03', 'RAMESH', 'CHENNAI');
SELECT*FROM TRADERS;
SELECT * FROM ITEMS ORDER BY INAME ASC;
SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;
SELECT TCODE, COUNT(*)FROM ITEMS GROUP BY TCODE;
SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY > 150;
SELECT TNAME FROM TRADERS WHERE CITY IN ('DELHI', 'MUMBAI');
SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;
SELECT MAX(PRICE), MIN(PRICE)FROM ITEMS;
SELECT PRICE * QTY AS AMOUNT FROM ITEMS WHERE CODE = 1004;
SELECT DISTINCT TCODE FROM ITEMS;
SELECT INAME, TNAME FROM ITEMS I, TRADERS T WHERE I.TCODE = T.TCODE AND QTY < 100;
Results Explain Describe Saved SQL History
TCODE TNAME CITY
T01 RAJESH DELHI
T02 SURESH MUMBAI
T03 RAMESH CHENNAI
3 rows returned in 0.00 seconds
CSV Export